

2026

2026 年全国大学生计算机系统能力大赛
操作系统设计赛（全国）
OS 内核实现赛道

内核技术手册

参赛作品：MyGO!!!! OS

学校：太原理工大学

团队名称：MyGO!!!!

参赛队员：屈世荣；张润泽；岳浩宇

指导教师：李欣

手册编写：张润泽

2026 年 5 月

前言

MyGO!!!! OS 是 2026 年全国大学生计算机系统能力大赛操作系统设计赛（OS 内核实现赛道）的参赛作品，采用 Rust 语言编写。本项目的核心目标是构建一个分层清晰、职责明确的内核工程结构，使得不同架构、不同固件协议、不同启动方式的适配工作能够在统一的框架下进行，而无需修改上层策略代码。

本手册的工程目的是为项目提供一份系统化的技术说明，帮助读者快速建立对内核整体图景的认知。同时，手册也为项目成员提供了统一的术语和设计约定。在多人协作的开发过程中，不同模块的实现者往往只熟悉自己负责的那一部分。手册明确了分层边界、模块契约和控制权的单向流动规则，使得每位成员都能够在不阅读全部源码的情况下，准确理解自己所在模块的前提条件、输出承诺和与其他模块的交互方式。这显著降低了协作摩擦，也使得代码评审和新功能开发可以在明确的架构原则下进行，而不依赖于个人经验或口头约定。

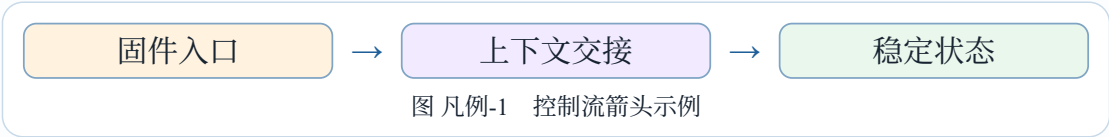
由于手册编写工作展开得较为匆忙，部分章节的叙述密度可能不均匀，个别图表和示例仍有待进一步打磨。我们会随着内核的持续开发逐步修订和补充手册内容，使其更准确地反映代码的实际结构和设计意图。读者在阅读过程中如发现表述不清或与实际实现不一致之处，欢迎提出反馈，帮助我们改进。

凡例

本技术手册采用统一的书写规则描述目录结构、控制权转移、数据布局、启动状态和硬件相关细节。凡例中的规则适用于正文所有章节，除非某个章节在局部明确给出了更严格的约束。凡例仅规定表达方式，不改变源代码中的模块边界、调用关系及实现事实。

正文中的等宽字体表示需要精确引用的技术对象。入口符号、函数名、结构体名、寄存器名、文件格式字段、命令行参数和路径片段均使用等宽字体。例如，`StartContext` 表示一个精确的结构体名称，`_start` 表示一个精确的入口符号，`docs/main.typ` 表示一个精确的路径片段。

图中的箭头用于表示控制权转移、依赖方向、数据流向或初始化顺序。箭头的含义由图题和相邻正文共同确定，不脱离上下文单独构成完整论证。图 凡例-1 展示了本手册最常见的控制流表示方式。



图表中的不同颜色用于提示职责类型。图 凡例-2 展示了本手册图表使用的颜色语义，相同的颜色在后继章节中保持相同的含义。

浅蓝色 表示平台无关的公共能力、接口抽象和通用基础设施。	浅橙色 表示当前正处于掌握控制权或执行关键启动动作的阶段。
浅绿色 表示已经完成稳定化、可被后续阶段直接消费的数据或运行状态。	浅紫色 表示阶段之间的一次性 handoff、上下文传递和控制权交接。

图 凡例-2 图表颜色语义

图表或代码标题的格式为“图/表/代码 章号-序号 名称”。图、表、代码及编号用于定位图表或代码，名称用于概括内容。表 凡例-1 给出了标题及正文引用的书写示例。

表 凡例-1 图表标号格式示例		
对象	标题格式	引用方式
图	图 1-1 项目分层结构	见图 1-1
表	表 1-1 职责边界	见表 1-1
代码	代码 1-1 进入内核的流程	见代码 1-1

参考手册的结构讲解部分不会出现真实代码。描述流程或算法时，正文优先使用流程图；当流程图不足以表达条件分支、循环、状态变化或错误处理路径时，正文将使用伪代码。伪代码仅用于说明控制流和数据依赖关系，不与任何具体源文件实现绑定。例如，在描述 trait 定义或泛型算法等与 Rust 语言紧密相关的设计时，可能会使用 Rust 伪代码；在描述系统调用分发、设备驱动接口等与 C 语言风格更贴近的设计时，使用 C 伪代码。两种伪代码风格服务于同一目的，即表达设计意图而非实现细节，读者应关注伪代码所传达的逻辑，而非其语言形式。

```
if (context.has_acpi) {
    parse_acpi_tables(context.acpi_root);
} else if (context.has_dtb) {
    parse_dtb_blob(context.dtb_blob);
} else {
    keep_platform_state_unparsed();
}
```

代码 范例-1 C 风格伪代码示例

Rust 风格伪代码示例如下所示：

```
trait CharDriver: Send + Sync {
    fn write(&self, buf: &[u8]) -> Result<usize, CharIoError>;
    fn read(&self, buf: &mut [u8]) -> Result<usize, CharIoError>;
    fn flush(&self) -> Result<(), CharIoError>;
    fn as_any(&self) -> &dyn Any;
}
```

代码 范例-1b Rust 风格伪代码示例

在描述二进制文件、固件表、镜像片段、启动参数块或内存转储时，正文使用 xxd 风格的十六进制输出。十六进制转储应根据偏移、十六进制字节序列和必要注释来理解，字段长度、对齐方式和字节序由相邻正文说明。

```
00002520 : 89 d8 48 8d 0d b8 21 00 00 99 f7 7c 24 14 85 d2 ..H...!...|$....
00002530 : 75 14 48 8d 0d a3 21 00 00 85 db 48 8d 05 9c 21 u.H...!...H...!
00002540 : 00 00 48 0f 44 c8 48 39 2d 43 3b 00 00 48 8d 05 ..H.D.H9-C;..H..
00002550 : 99 21 00 00 45 89 e0 48 8d 15 86 21 00 00 be 01 .!.E..H..!....
00002560 : 00 00 00 00 4c 89 ef 48 0f 45 d0 31 c0 e8 0f ee ff ...L..H.E.1....
00002570 : ff 85 c0 0f 88 6c 01 00 00 83 c3 01 48 63 c3 48 ....1.....Hc.H
00002580 : 3b 44 24 18 0f 8c 76 ff ff ff 80 7c 24 20 00 0f ;D$....v....|$....
00002590 : 85 6b ff ff ff 89 5c 24 38 4c 8b 64 24 28 83 7c .k....\ $8L.d$(.|
000025a0 : 24 38 00 74 0f 4c 89 ee 48 8d 3d 28 21 00 00 e8 $8.t.L..H.=(!...
000025b0 : 4c 12 00 00 4d 85 e4 0f 84 48 fe ff ff 4c 89 ee L..M....H...L..
000025c0 : 48 8d 3d f1 21 00 00 e8 34 12 00 00 e8 cf ed ff H.=.!...4.....
000025d0 : ff 41 0f b6 14 24 48 89 c3 48 8b 00 f6 44 50 01 .A...$H..H...DP.
```

代码 范例-2 xxd 风格十六进制示例

汇编代码是“不出现真实代码”规则的例外情况。在必须解释入口序列、寄存器约定、CSR 操作、TLB 相关指令或异常入口时，正文会直接写出真实架构汇编。

```
li.d    $t0, 0x9000000000000000
cswrwr  $t0, 0x180
jirl     $zero, $ra, 0
```

代码 凡例-3 LoongArch64 汇编示例

地址和长度的描述区分虚拟地址、物理地址和文件偏移。除非特别说明，固件表中的地址指固件提供的物理地址；内核访问时的地址指经过早期映射或内核映射转换后的虚拟地址；二进制结构说明中的偏移指文件或缓冲区内部的位置。表 凡例-2 给出了这些记号的用法示例。

表 凡例-2 地址和数值记号示例		
记号	示例	含义
虚拟地址	0x9000_0000_1000_0000	内核已经能够直接访问的映射地址。
物理地址	0x0000_0000_1000_0000	固件表、内存图或页表项描述的机器地址。
文件偏移	00000010:	二进制转储中相对于文件或缓冲区起点的位置。
容量单位	KiB、MiB、GiB	二进制容量单位；KB、MB、GB 表示十进制容量。

本手册使用“启动阶段”“运行期”“平台无关”“架构相关”“固件来源”和 handoff 等术语描述内核边界。表 凡例-3 给出了这些术语的固定含义，后文章节不再重复解释。

表 凡例-3 核心术语说明	
术语	含义
启动阶段	指进入 main 函数之前的控制权转移和基础设施建立过程。
运行期	指操作系统所需的各种基础能力建立之后的长期执行阶段。
平台无关	指不依赖具体指令集或特定固件入口的通用逻辑。
架构相关	指依赖具体处理器、指令集、异常模型或启动 ABI 等具体硬件架构的逻辑。
固件来源	指 ACPI 表、DTB、EFI 系统表或其他启动协议提供的平台信息。
handoff	指某一个阶段将一次性上下文交给下一阶段，并退出决策路径的过程。

正文中的“必须”“应该”“可以”和“不会”表达不同强度的工程约束。表 凡例-4 说明了这些约束词的含义，以避免将实现建议误读为硬性接口条件。

表 凡例-4 工程约束词说明

词语	含义
必须	违反该条件将导致接口失效、启动失败或语义不成立等设计缺陷。
应该	该规则为推荐边界，违反时需要给出明确理由。
可以	存在多种可接受的实现方案。
不会	当前设计明确排除了某一行为。

章节中的图表、伪代码、十六进制转储和汇编片段均服务于对结构的解释，它们不能替代对真实实现的验证。涉及启动路径、页表、固件解析和分配器状态的结论，最终仍以仓库源码、构建结果和目标平台运行日志为准。

目录

前言	I
凡例	II
目录	VI
第一章 分层结构与启动控制流	1
1.1 问题背景与设计目标	1
1.2 项目分层结构	2
1.3 启动链路概览	3
1.4 架构相关启动阶段	5
1.4.1 早期入口与预启动初始化	5
1.4.2 架构加载器	5
1.5 启动上下文	7
1.6 平台无关启动阶段	8
1.7 主入口与用户态交接	10
1.8 工程设计总结	10
第二章 内存管理子系统	13
2.1 设计目标与约束	13
2.2 总体分层结构	14
2.3 初始化顺序与锁顺序	15
2.4 启动分配与物理页管理	16
2.5 内核地址空间、Slab 分配器与内核堆	17
2.6 受管堆与精确回收	18
2.7 进程地址空间与缺页处理	19
2.8 架构注入契约	20
2.9 工程设计总结	21
第三章 设备模型与驱动框架	23
3.1 设计目标与约束	23
3.2 总体分层结构	24
3.3 设备能力核心抽象	25
3.4 设备能力多轨落实	26
3.4.1 字符设备能力	27
3.4.2 块设备能力	28
3.4.3 网络设备能力	28
3.4.4 RTC 设备能力	29
3.5 PnP 设备与驱动绑定	29
3.6 PnP 生命周期与热插拔顺序	30
3.7 用户态投影与设备文件	32
3.8 初始化顺序与典型路径	32
3.9 类型安全控制	33

3.10 工程设计总结	34
第四章 虚拟文件系统与 POSIX 兼容层	37
4.1 设计目标与约束	37
4.2 总体分层结构	38
4.3 索引节点与超级块	39
4.4 目录项与路径缓存	40
4.5 路径解析与挂载跨越	41
4.6 文件对象与能力冻结	43
4.7 文件描述符表与进程文件视图	44
4.8 挂载命名空间与超级块实例	45
4.9 文件系统驱动与特殊文件系统	46
4.10 VFS 上下文与跨子系统交接	46
4.11 工程设计总结	47
第五章 进程与线程管理	48
5.1 设计目标与约束	48
5.2 任务对象核心身份模型	49
5.3 PID 命名层、线程组与进程组	50
5.4 子任务与资源共享	51
5.5 执行上下文与架构交接	52
5.6 退出、僵尸状态与 wait 回收	53
5.7 等待队列	54
5.8 信号机制	55
5.9 凭据与能力	56
5.10 工程设计总结	57
第六章 调度系统	58
6.1 设计目标与约束	58
6.2 总体结构	59
6.3 调度策略与调度类	60
6.4 公平类与 EEVDF	61
6.5 运行队列与多调度类队列	62
6.6 重调度时机与上下文切换	63
6.7 唤醒、优先候选与系统调用后交接	64
6.8 CPU 亲和性、拓扑与负载均衡	65
6.9 空闲任务与初始化顺序	66
6.10 工程设计总结	67
第七章 并发与同步机制	68
7.1 设计目标与约束	68
7.2 自旋锁	69
7.3 可睡眠互斥锁	69
7.4 等待队列与丢失唤醒	70
7.5 原子状态与锁序控制	71
7.6 上下文类型与睡眠边界	72
7.7 锁粒度与所有权划分	73
7.8 内存序与可见性	74

7.9 超时等待、信号打断与取消路径	75
7.10 典型路径中的同步组合	77
7.11 调试、验证与故障收束	78
7.12 工程取舍与演进边界	79
7.13 工程设计总结	80
第八章 异常处理与系统调用接口	81
8.1 异常入口的职责	81
8.2 系统调用帧接口	82
8.3 表驱动分发	82
8.4 返回前收尾	83
8.5 异常、缺页与信号	83
8.6 入口分层与状态规范化	84
8.7 快速路径与完整路径	85
8.8 参数、返回值与错误码语义	86
8.9 用户访问窗口与可恢复异常	86
8.10 系统调用注册与分类组织	87
8.11 返回前安全点与异步事件	88
8.12 性能拆解与实现边界	89
8.13 故障路径与内核可信边界	90
8.14 兼容性与 ABI 视角	91
8.15 工程设计总结	92
第九章 用户态执行环境	93
9.1 ELF 装载流程	93
9.2 地址空间构建	94
9.3 用户栈与辅助向量	95
9.4 动态链接器与 vDSO	95
9.5 执行映像替换与任务状态交接	96
9.6 初始用户进程	96
9.7 装载请求与阶段化数据结构	96
9.8 ELF 校验与段权限	97
9.9 用户地址布局与保护区	98
9.10 用户栈 ABI	99
9.11 执行映像替换事务与失败回滚	100
9.12 文件描述符、凭据与进程身份	102
9.13 ELF 与缺页、mmap 的关系	102
9.14 兼容性与启动语义	103
9.15 安全性与资源约束	103
9.16 工程设计总结	104
第十章 信号与异步事件	105
10.1 信号状态模型	105
10.2 发送与权限检查	106
10.3 屏蔽、等待与信号动作	106
10.4 返回用户态前的投递	107
10.5 信号与系统调用重启	107

10.6 默认动作与任务状态转换	108
10.7 线程组选路与待处理队列	109
10.8 信号动作语义与用户处理函数	109
10.9 sigaltstack 与嵌套投递	110
10.10 信号等待与第七章的同步模型	111
10.11 系统调用重启的语义边界	112
10.12 作业控制与终端信号	112
10.13 兼容性与运行时语义	113
10.14 信号帧与 rt_sigreturn	113
10.15 权限、凭据与 pidfd 文件描述符	115
10.16 信号动作与进程接口的关系	115
10.17 工程设计总结	115
第十一章 日志、控制台与终端	117
11.1 全局控制台	117
11.2 日志输出与早期诊断	118
11.3 devtmpfs 文件系统与用户态终端入口	118
11.4 TTY 输入泵	118
11.5 控制字符与信号边界	119
11.6 启动阶段的输出链路	119
11.7 控制台注册、替换与生命周期	120
11.8 日志缓冲、级别与输出成本	121
11.9 devtmpfs 投影与标准输入输出	122
11.10 TTY 缓冲与控制接口	122
11.11 故障诊断与分层定位	124
11.12 虚拟化环境中的控制台约束	124
11.13 锁序与输出重入	125
11.14 用户态接口与运行组织	125
11.15 工程设计总结	126
第十二章 网络子系统	128
12.1 分层结构	128
12.2 接口管理与路由	129
12.3 协议栈轮询与主动推进	129
12.4 套接字与 VFS 边界	130
12.5 网络设备能力与控制路径	131
12.6 套接字生命周期	132
12.7 阻塞、非阻塞与就绪状态	132
12.8 路由、地址与接口状态	133
12.9 回环与主动推进	134
12.10 并发、锁序与资源回收	135
12.11 性能与演进视角	135
12.12 工程设计总结	136
后记：工程不足与未来展望	137
附录	141
A. 代码组织路径	141

B. 系统调用和 <code>errno</code> 表	142
C. 核心概念与代码句柄对照	156
名词索引	158
缩写词表	166
参考项目与致谢	169
参考项目	169
致谢	169

第一章 分层结构与启动控制流

我们在设计内核启动路径时面对的首要问题，是同一套内核需要同时运行在 LoongArch64 架构和 RISC-V64 架构上。两种平台的差异并不只体现在指令编码和寄存器名称上。固件交接方式不同，早期地址空间不同，内存描述来源不同，设备发现路径也不同。RISC-V64 架构的 QEMU 虚拟机直启路径由启动寄存器传入 DTB (Device Tree Blob, 设备树二进制) 物理地址。LoongArch64 架构的 ABI (Application Binary Interface, 应用二进制接口) 则传入 EFI (Extensible Firmware Interface, 可扩展固件接口) 标记、命令行地址和 EFI 系统表地址。当前 LoongArch64 架构加载器以 EFI 系统表为固件发现入口，优先从 EFI 配置表选择 ACPI (Advanced Configuration and Power Interface, 高级配置与电源接口)，缺省回退到 DTB。若平台无关层直接理解这些细节，启动代码会很快形成大量分支。若架构层只交出少量原始指针，后续子系统又无法判断这些指针的生命周期和有效性。

我们最终采用的边界是启动上下文 (StartContext)。架构相关层负责把固件和处理器现场整理成稳定的启动上下文。平台无关层只消费这个上下文，并按照统一顺序激活内存、设备、文件系统、控制台以及调度器。这个边界让启动过程形成清晰的信息流。底层代码吸收硬件差异。交接对象保留必要能力。高层代码围绕稳定数据和回调组织系统策略。

本章讨论这一结构的设计理由和关键控制流。第一部分说明分层结构如何约束依赖方向。第二部分说明从固件入口到 main 函数的启动链路。第三部分分析架构加载器如何构造启动上下文。第四部分说明平台无关启动阶段如何根据 DTB 或 ACPI 激活运行期子系统。最后一部分总结本章启动设计中的工程创新。

1.1 问题背景与设计目标

跨架构启动代码的难点不在于写出两个入口，而在于确定哪些差异应当留在架构层、哪些信息应当向上交给通用内核。启动早期没有完整堆分配器，没有统一设备模型，也没有稳定的日志输出。此时任何复杂依赖都可能放大故障面。启动后期又必须准备运行期系统。物理页分配器需要可用内存段。设备子系统需要 MMIO (Memory-Mapped Input/Output, 内存映射输入输出) 地址转换能力。控制台需要固件中的串口描述。调度器需要 VFS (Virtual File System, 虚拟文件系统) 根上下文和标准文件描述符。每一步都依赖前一步已经产出的稳定结果。

我们把启动路径的设计目标整理为四类：

表 1-1 启动路径的设计目标		
目标	含义	工程约束
依赖单向	架构相关层向上交付数据和能力，平台无关层不反向读取架构私有状态。	kernel 只能通过 hal 与 general 消费稳定接口。
交接稳定	固件表、命令行和内存图在移交前完成生命周期稳定化。	平台无关层不直接依赖 EFI 服务或启动寄存器。
能力显式	地址转换、堆映射和页表安装以回调形式交付。	高层代码不嵌入早期直映窗口和 MMIO 映射常量。
顺序可验证	每个启动阶段都由前置条件决定，失败路径可以定位到具体边界。	将建立电源控制和日志能力置于启动链路前端。

这四个目标共同决定了启动层的形态。我们不让平台无关层直接访问启动协议原始结构，也不让架构层继续参与运行期策略。架构层的任务是构造一个足够完整的交接对象。平台无关层的任务是验证这个对象，并把其中的数据转换为运行期状态。这个分界点越清晰，后续移植新平台时需要触碰的代码范围就越小。

1.2 项目分层结构

项目整体由五个层级组成。libs 模块提供可以复用的基础库和子系统部件。general 模块定义平台无关的数据结构、特征接口、固件视图以及启动上下文。arch 模块保存与具体指令集相关的入口、页表、异常处理和早期映射。hal 模块把不同架构的实现收敛成统一接口。kernel 模块负责策略编排和子系统集成。



这套分层的关键不在于目录名称，而在于依赖方向。这种单向依赖给启动路径带来两个直接收益：第一，平台差异被限制在 arch 模块和少量 hal 适配中；第二，平台无关层可以用相同的代码处理 DTB 路径和 ACPI 路径产出的结果。我们在调试启动问题时，也能先判断错误发生在交接之前还是交接之后。这个判断往往比单纯查看日志更重要，因为它直接决定是应当检查汇编入口、固件快照，还是检查通用子系统初始化顺序。

1.3 启动链路概览

内核启动是一个逐步降低不确定性的过程。固件交出控制权时，处理器只满足很小的执行条件。到 main 函数执行时，内核已经有运行期分配器、设备模型、VFS 上下文、控制台和调度器入口。中间的每个阶段都需要把上一阶段的原始状态转化成下一阶段可以消费的对象。



表 1-2 启动阶段职责划分

阶段	主要职责	交付结果
早期入口	建立最小地址空间, 安装异常入口, 切换临时栈, 清零 BSS 后保存启动参数。	可执行 Rust 代码的早期环境。
架构加载器	初始化早期日志和启动分配器, 复制固件表, 选择 DTB 或 ACPI, 准备地址转换和分配器回调。	启动上下文。
启动初始化	校验上下文, 根据固件来源进入 DTB 或 ACPI 路径, 激活内存、设备、VFS 和控制台等各子系统。	运行期内核环境。
主入口	注册运行期钩子, 初始化调度器, 安装系统调用表, 启动用户态初始进程。	可调度的用户态系统。

两个边界在这条链路中尤其重要。第一个边界是早期入口到架构加载器。这里要求代码已经有临时栈, BSS (Block Started by Symbol, 未初始化静态区) 状态也已知。第二个边界是架构加载器到 `__kernel_start_init` 入口。这里要求固件视图已经稳定, 地址转换函数已经可用, 内核镜像范围和可选内存图已经填入上下文。后续平台无关层只依赖这些结果。

1.4 架构相关启动阶段

架构相关启动阶段的核心职责，是把固件交来的初始现场转换为高级语言可以处理的状态。这个阶段直接面对控制寄存器、页表、异常入口、启动 ABI 和固件服务。我们把它拆成早期入口、预启动初始化以及架构加载器三段。

1.4.1 早期入口与预启动初始化

早期入口不能假设运行期基础设施已经存在。进入 `_start` 入口时，栈可能尚未切好，BSS 中可能存在旧值，页表状态也可能由固件或虚拟机决定。我们因此把入口动作限制在少数硬件相关操作上。先建立最小地址映射，再安装异常入口，然后跳到带临时栈的虚拟地址环境。随后 `pre_boot_init` 函数清零 BSS，并把启动参数写入静态存储。

```
fn arch_entry() -> ! {
    setup_minimal_mapping();
    install_early_exception_entry();
    clear_unknown_control_state();
    jump_to_virtualized_entry()
}

fn virtualized_entry(boot_args: BootArgs) -> ! {
    clear_return_boundary();
    switch_to_temporary_stack();
    prepare_arch_runtime_for_rust();
    pre_boot_init(boot_args);
    jump_to_arch_loader(boot_args)
}

fn pre_boot_init(boot_args: BootArgs) {
    clear_bss();
    save_boot_arguments(boot_args);
    init_boot_cpu_local_if_needed();
}
```

代码 1-1 早期入口与预启动初始化

这里最容易出错的是顺序。BSS 必须先清零，再写入启动参数。LoongArch64 架构会把 EFI 标记、命令行地址和系统表地址保存在静态原子变量中。RISC-V64 架构会保存启动硬件线程编号和 DTB 地址，并初始化启动硬件线程的每硬件线程数据。如果先保存参数再清零 BSS，刚写入的参数会被覆盖。这个错误在后续阶段表现为固件缺失，很难从故障点直接看出原因。

LoongArch64 架构入口还有一个实际约束。当前编译配置下，LLVM 编译器可能在发布模式生成 LSX 向量指令。我们在进入 Rust 语言代码前设置 `ELEN.FPE` 和 `ELEN.SXE` 位，确保早期 Rust 语言路径不会因为浮点或 LSX 状态不可用而陷入异常。这个细节说明早期环境并非只服务于汇编入口，它还要满足编译器生成代码的最低运行条件。

1.4.2 架构加载器

架构加载器位于架构相关启动阶段的末端。它已经可以执行较完整的 Rust 语言代码，但仍处在平台私有语境中。它的任务是建立早期诊断能力，复制固件数据，准备启动内存信息，并构造启动上下文。

```
fn kernel_arch_loader(raw: RawBootState) -> ! {
    install_exception_handlers();
    init_time_or_timer_source();
    bind_early_log_sink();
    init_boot_allocator();

    let firmware = snapshot_and_select_firmware(raw);
    let boot_map = snapshot_boot_memory_map(raw);
    let kernel_image = locate_kernel_image_range();

    let context = StartContext {
        boot: build_boot_info(raw),
        firmware,
        memory: StartMemory { kernel_image, boot_map },
        address: build_address_ops(),
        allocator: build_allocator_ops_if_supported(),
    };

    validate_context_before_handoff(&context);
    jump_to_kernel_start_init(&context)
}
```

代码 1-2 架构加载器核心流程

这一流程的顺序来自数据依赖。异常处理要早于复杂解析，否则早期错误会缺少可控出口。时间源或周期定时器要早于依赖时间戳的日志系统。启动分配器要在固件复制前可用，因为 DTB 和 ACPI 表都需要稳定的保存空间。固件选择要在启动上下文构造前完成，因为上下文中只能出现一种固件来源。

RISC-V64 架构的直启路径相对直接。启动寄存器提供 DTB 物理地址，加载器复制 DTB，记录 `StartBootProtocol::Direct` 枚举变体，并将启动内存图设为 `None`。后续 DTB 解析器从 `/memory` 节点取得 RAM（Random Access Memory，随机访问内存）信息。LoongArch64 架构的路径更复杂。加载器保存命令行，并以 EFI 系统表为固件发现入口。若配置表中存在 RSDP（Root System Description Pointer，根系统描述指针）且 EFI 内存映射可用，它会选择 ACPI 并复制 ACPI 表；否则在存在 FDT 时复制 DTB 视图。启动协议根据入口传入的 EFI 标记记录为 `Efi` 或 `Direct` 枚举变体，但当前实现仍要求能取得可用的 EFI 系统表来发现 ACPI 或 FDT。当前 ACPI 路径要求存在可用的启动内存映射，因为 ACPI 本身描述平台结构，却不提供可直接交给分配器的普通 RAM 列表。

这里还有一个生命周期约束。固件表指针不能简单透传给平台无关层。EFI 启动服务退出后，固件服务内存可能失效。DTB 所在物理页也可能在内存接管后被重新分配。加载器因此必须先复制需要的固件内容，再把稳定视图放入上下文。对于 DTB，这通常是一个连续二进制块的复制。对于 ACPI，加载器需要保存 RSDP，并维护 ACPI 表物理地址到复制后虚拟地址的映射。

1.5 启动上下文

启动上下文是架构层到平台无关层的唯一交接对象。它不是运行期数据库,也不是全局配置中心。它只在启动交接期间有效,负责描述已经稳定化的数据和必须由架构提供的能力。

```
struct StartContext {
    boot: StartBootInfo,
    firmware: StartFirmware,
    memory: StartMemory,
    address: StartAddressOps,
    allocator: Option<StartAllocatorOps>,
}

struct StartBootInfo {
    architecture: StartArchitecture,
    protocol: StartBootProtocol,
    boot_cpu_id: usize,
    command_line: Option<&'static [u8]>,
}

enum StartFirmware {
    Dtb(Dtb<'static>),
    Acpi(StartAcpiTables),
}

struct StartMemory {
    kernel_image: StartPhysRange,
    boot_map: StartMemoryMap,
}

enum StartMemoryMap {
    None,
    Regions(&'static [StartMemoryRegion]),
}

struct StartAddressOps {
    phys_to_virt: fn(usize) -> usize,
    virt_to_phys: fn(usize) -> usize,
    device_mmio_to_virt: fn(usize) -> usize,
}

struct StartAllocatorOps {
    kernel_heap_region: fn() -> VirtRange,
    map_kernel_heap_range: fn(usize, usize, usize, PagePolicy) -> bool,
    unmap_kernel_heap_range: fn(usize, usize) -> bool,
    init_kernel_page_table: fn(),
}
```

代码 1-3 启动上下文核心结构

这个结构的设计原则是传递已整理的事实和能力。boot 字段说明当前架构、启动协议、启动处理器标识和可选命令行。firmware 字段明确本次启动信任 DTB 还是 ACPI。memory 字段保存内核镜像占用范围和可选启动内存图。address 字段提供普通

物理地址、内核虚拟地址和设备 MMIO 地址之间的转换。allocator 字段提供内核堆区域、堆映射、解映射以及页表安装回调。

启动固件视图使用枚举而不是多个可选字段。这个选择可以把不变量前移。上下文中只能有一个固件来源。__kernel_start_init 入口只需要读取 firmware_source() 方法，随后分派到 DTB 或 ACPI 路径。平台无关层不会再次比较 EFI 表中是否还存在另一个候选固件，也不会解析失败后临时切换来源。启动日志和错误报告因此具有明确语义。

启动内存图的设计也体现了边界控制。DTB 直启系统常常在 DTB 的 /memory 节点中描述 RAM，此时独立启动内存图可以为空。EFI 或其它启动协议可能额外提供内存映射，此时加载器把协议私有描述符归一化为启动内存区域。平台无关层只看区域类型是否能在交接后使用，不理解 EFI 原始属性布局。ACPI 路径在上下文校验中要求 boot_map 字段至少包含可用区域，避免在没有普通 RAM 输入的情况下继续初始化分配器。

地址转换回调是上下文中最重要能力接口之一。LoongArch64 可以用 DMW 窗口把 MMIO 物理地址转换为非缓存虚拟地址。RISC-V64 可以把设备地址映射到专门的 MMIO 虚拟窗口。平台无关层只调用 device_mmio_to_virt 回调，不关心具体窗口常量。普通 RAM 的 phys_to_virt 回调与 virt_to_phys 回调也遵循同样原则。这样一来，分配器、固件解析器和设备初始化代码可以共享同一套地址能力。

StartContext::validate 方法是交接点上的防线。它检查架构标识、启动协议、内核镜像范围和内存图条目。对于 ACPI，它还检查 RSDP、复制表映射和可用启动内存段。我们把这些校验放在进入通用启动流程之前，原因很直接——如果上下文不满足基本不变量，后续错误会扩散到分配器、VFS 或设备注册阶段。尽早拒绝错误上下文，可以把问题定位在架构加载器一侧。

1.6 平台无关启动阶段

平台无关启动阶段从 __kernel_start_init 入口开始。这个入口接收启动上下文指针，先把空指针和基本不变量排除掉，再根据固件来源进入 dtb::kernel_start_init 函数或 acpi::kernel_start_init 函数。两个路径解析的固件格式不同，但目标一致。它们都要准备内存分配器，安装电源控制，建立 VFS 基础设施，激活设备子系统，注册控制台，并把必要的启动期 VFS 部件交给调度器。

```
unsafe extern "C" fn __kernel_start_init(ctx: *const StartContext) -> ! {
    let context = checked_ref(ctx);
    context.validate().expect("invalid StartContext");

    match context.firmware_source() {
        StartFirmwareSource::Dtb => dtb_kernel_start_init(context),
        StartFirmwareSource::Acpi => acpi_kernel_start_init(context),
    }

    main()
}
```

代码 1-4 平台无关启动初始化

DTB 路径先解析整棵设备树，建立节点索引，处理 `aliases`、`phandle`、`status`、`reg` 和 `ranges` 等设备树属性，然后产出 CPU、内存、保留区、串口、平台设备、PCIe 主桥和电源控制等标准描述。若启动上下文同时提供了独立内存图，DTB 解析出的内存段会与可用启动内存段做交叉过滤。这样可以避免固件描述之间不一致时把不可用页交给物理分配器。

ACPI 路径先通过复制后的 RSDP 和表映射建立 ACPI 表视图，再从 MADT (Multiple APIC Description Table, 多 APIC 描述表) 取得 CPU 数量，从 SPCR (Serial Port Console Redirection Table, 串口控制台重定向表) 和 ACPI 命名空间发现串口设备，从命名空间发现 `virtio-mmio` 设备，并解析电源控制寄存器。ACPI 路径的普通 RAM 输入来自启动内存区域列表。这与 DTB 路径不同，但在进入分配器之前，两者都会整理出同一种内存段列表。

内存分配器的激活顺序在两个路径中保持一致。首先绑定 `phys_to_virt` 回调与 `virt_to_phys` 回调。随后保留内核镜像范围。DTB 路径还会保留可能存在的外部 `initramfs` 初始内存文件系统范围。接着初始化物理页分配器。若架构提供启动分配器接口，再绑定内核堆映射回调，安装内核页表，初始化虚拟内存管理器，最后启用内核堆、Slab 分配器和全局分配器。

这个顺序不能随意交换。物理页分配器必须早于页表页申请。页表初始化必须早于内核堆映射的稳定使用。Slab 分配器依赖后备页和堆能力。全局分配器要放在最后，因为一旦切换完成，后续设备对象、VFS 索引节点和调度器对象都会使用运行期分配路径。我们在这一步之前保持启动分配器的职责有限，可以减少不可释放早期分配对系统长期内存状态的影响。

电源控制应当置于初始化的前期。分配器和页表初始化失败时，内核可能需要关机或重启。如果此时还没有安装平台电源控制，错误路径只能停机等待外部终止。DTB 和 ACPI 路径都会在激活分配器之前安装 `general::firmware::power` 电源控制模块，使内核 `panic` 异常处理和正常关机都能调用统一接口。

设备与 VFS 的启动顺序体现了设备模型边界。我们先注册 `tmpfs` 临时文件系统、`devtmpfs` 文件系统、`procfs` 文件系统和 `sysfs` 文件系统驱动，以及标准设备号策略和设备文件投影器，再创建 `devtmpfs` 文件系统超级块，并把设备能力投影机制连接到设备子系统。DTB 路径随后登记平台设备并扫描 PCIe 主桥。ACPI 路径当前登记从 ACPI 发现的串口和 `virtio-mmio` 设备。驱动探测成功后发布设备能力，`devtmpfs` 文件系统可以接收投影事件。DTB 路径在根文件系统确定后把这棵已经填充的 `devtmpfs` 文件系统挂载到 `/dev`，ACPI 路径则在 `tmpfs` 临时文件系统根建立后先挂载 `/dev`，再继续设备登记。具体内容将于第三章讨论。

根文件系统的选择在 DTB 路径中具有优先级。外部 `initramfs` 初始内存文件系统或内建 `initramfs` 初始内存文件系统优先。若不存在 `initramfs` 初始内存文件系统，再尝试从已经注册的块设备中挂载根盘。ACPI 路径当前采用 `tmpfs` 临时文件系统作为启动根。两条路径都会挂载 `devtmpfs` 文件系统到 `/dev`，并挂载 `/dev/shm` 与 `/sys`。需要注意的是，`procfs` 文件系统在核心文件系统阶段完成注册，但当前启动路径没有在这里自动挂载 `/proc`。两条路径最终都会创建 VFS 上下文、挂载命名空间、根目录和凭据，并通过 `sched::stash_boot_vfs_parts` 函数交给调度层。调度器随后给 PID 1 安装 VFS 上下文和文件描述符表。

控制台切换放在启动初始化后段。早期日志输出在架构加载器阶段就可以工作，运行期日志输出则依赖已经注册的字符设备。DTB 路径优先读取命令行中的 `console`

参数，然后回退到 `stdout-path` 属性。ACPI 路径优先读取命令行，再回退到 SPCR 指定或命名空间中发现的串口。找到控制台后，系统会注册控制台，绑定 `/dev/console`，并把日志输出端切换到正式字符设备。这个顺序可以让早期日志尽可能长地保留，同时在设备模型可用后切换到统一输出路径。

1.7 主入口与用户态交接

`main` 函数是启动阶段到运行阶段的交接点。执行到这里时，启动上下文已经被消费完毕。分配器、设备、VFS 和控制台已经处于运行期状态。`main` 函数不再接收启动上下文，也不重新解析固件。它只登记运行期钩子，启动调度器，并把 PID 1 送入用户态。

```
fn main() -> ! {
    register_vdso_tick_hook();
    register_net_poll_hook();

    let init = sched_boot_init();
    register_tty_poller();

    run_configured_startup_hooks();
    set_runtime_log_level();
    start_init_process(&init)
}
```

代码 1-5 主入口控制流

`sched::boot_init` 函数的注册顺序同样由依赖决定。它先通过 `hal` 模块注入上下文切换、时间、陷阱帧、内存切换和系统调用相关钩子。随后注册 `fork` 系统调用和 `clone` 系统调用需要的扩展复制钩子，再注册退出钩子和退出前钩子。退出前钩子要早于地址空间释放执行，因为 `robust futex` 清理机制和 `clear-child-tid` 机制需要在用户虚拟地址空间仍可访问时处理。接着注入用户进程镜像操作表、虚拟内存切换钩子和任务 CPU 状态发布钩子，最后创建 `init` 任务。

创建 `init` 任务后，调度层会取出启动阶段暂存的 VFS 部件。它构造 VFS 上下文和文件描述符表，安装标准输入、标准输出和标准错误，再把这些对象挂到 `init` 任务扩展槽中。之后为 CPU 0 创建空闲内核线程，并注册完整系统调用表。这个顺序保证用户态第一次进入内核时，文件系统、文件描述符和系统调用分发都已经就绪。

`start_init_process` 函数负责把 PID 1 替换为真实用户态镜像。它依次尝试 `/init`、`/sbin/init` 和 `/bin/init`。第一个可加载的用户镜像会获得用户地址空间、入口地址和初始栈。内核随后构造用户态陷阱帧，并通过架构相关返回路径进入用户态。此后系统进入常规调度阶段，启动上下文不再参与运行期控制流。

1.8 工程设计总结

本章围绕分层结构和启动控制流，说明了我们如何把架构差异、固件差异和运行期系统初始化组织在同一条单向链路中。启动路径的核心落在一组边界的配合上，而不只是某一个入口函数。早期入口保证代码能够安全进入 Rust 语言环境。架构加

载器稳定化固件信息，并构造启动上下文。平台无关层验证上下文，解析 DTB 或 ACPI，激活内存和设备。主入口完成调度器、系统调用和用户态 init 进程的交接。

工程设计具备以下创新：

第一是以启动上下文为中心建立一次性交接边界。回顾启动路径的早期设计，一个主要分歧在于平台无关层应当如何获得硬件信息。较直接的方案是让 kernel 模块在需要时调用 arch 模块查询函数。这个方案实现成本较低，但会把启动寄存器、固件表指针和早期映射状态长期暴露给高层。另一个方案是把所有固件信息提前解析成运行期全局变量。这个方向看似稳定，却容易让早期生命周期和运行期生命周期混在一起。我们最终选择启动上下文，让架构加载器在一个明确时刻完成数据收集、稳定化和能力封装。平台无关层只消费这个上下文，不反向读取架构私有状态。这个设计把启动交接变成了可验证的单次事务，也让支持新架构时的工作集中在上下文构造侧。

第二是把固件选择做成显式枚举和前置校验。DTB 与 ACPI 都能描述平台，但它们提供的信息范围不同，生命周期也不同。若上下文使用多个可选字段，平台无关层就必须在运行期维持组合不变量，例如 ACPI 表存在但内存图缺失，或者 DTB 和 ACPI 同时出现。我们把固件来源收敛为 `StartFirmware::Dtb` 枚举变体与 `StartFirmware::Acpi` 枚举变体。上下文中只能保存一种结果。`validate` 方法会在进入通用启动前检查该结果是否满足最低条件。ACPI 必须有复制后的 RSDP、表映射和可用内存段。DTB 路径则可以接受独立启动内存图为空，并由 `/memory` 节点提供 RAM 信息。这种设计减少了后续分派中的猜测，也让错误更早暴露在交接点。

第三是把地址转换和分配器能力作为显式回调交付。不同架构的早期地址空间差异很大。LoongArch64 可以依赖 DMW 形成直接窗口。RISC-V64 可以把 MMIO 放入专门虚拟窗口。若平台无关层直接写入这些常量，后续每增加一个架构都会扩散修改范围。我们在启动地址转换接口中明确区分普通 RAM 的 `phys_to_virt` 回调、内核地址反查的 `virt_to_phys` 回调，以及设备寄存器访问所需的 `device_mmio_to_virt` 回调。分配器需要的堆区域、堆映射和页表安装能力也通过启动分配器接口提供。这样，内存管理和设备初始化代码只依赖能力接口，具体映射策略仍由架构层掌握。

第四是把启动顺序设计成可解释的依赖链。启动初始化中很多顺序看似只是工程习惯，实际都对应明确约束。BSS 必须先清零再保存启动参数。电源控制必须早于分配器和页表等高风险操作。物理页分配器必须早于虚拟内存管理和 Slab 分配器。`devtmpfs` 文件系统需要早于设备扫描接收投影事件。控制台切换要晚于字符设备注册，以便早期日志保留尽可能久。我们把这些顺序固定在启动控制流中，并通过注释、伪代码和上下文校验说明原因。这样做的价值在调试阶段尤为明显。一次启动失败可以沿依赖链向前定位，而不是在多个全局状态之间反复猜测。

第五是让运行期交接保持最小状态继承。`main` 函数不再携带启动上下文，也不重新解释固件。它只登记运行期钩子，启动调度器，并让已有的 VFS、文件描述符和控制台状态进入 PID 1。调度层通过 `stash_boot_vfs_parts` 函数接收启动阶段准备好的 VFS 部件，再在 `boot_init` 函数中安装到 init 任务上。系统调用表在调度器就绪后注册，TTY 轮询器在空闲线程可用后启动。这个边界使启动阶段和运行阶段自然分离。启动代码完成一次性资源转换，运行期代码只面对已经稳定的对象。

从整体上看，第一章的分层与启动设计为后续章节提供了共同前提。第二章讨论的内存管理依赖启动阶段提供的可用物理内存和地址转换能力。第三章讨论的设备模型依赖启动阶段提供的固件解析结果、MMIO 转换和 `devtmpfs` 文件系统投影载体。

后续调度、系统调用和用户态加载也都建立在 `main` 之后的运行期状态之上。我们把这些依赖收束在清晰的启动链路中, 是为了让每个子系统都能在自己的边界内演化, 同时保持整个内核的启动行为可推理、可验证和可移植。

第二章 内存管理子系统

在第一章中, 我们把启动链路划分为架构相关阶段和平台无关阶段, 并说明了启动上下文如何交付固件视图、内存图、地址转换函数和分配器回调。本章讨论这些信息如何被进一步转化为运行期内存管理能力。内存管理是内核最早激活的核心子系统之一。设备对象、VFS 索引节点、任务结构、页表页和用户地址空间都依赖它。若这一层的边界不清晰, 后续子系统的错误会被误判为设备错误、文件系统错误或调度错误, 实际根因却可能是分配路径已经破坏了内存所有权。

我们面对的内存问题并非单一问题。启动阶段需要一个依赖极少的临时分配器。运行期需要管理不连续的物理页段。内核堆需要把物理页映射到可用虚拟地址。小对象需要低延迟复用。大对象需要页级对齐和完整回滚。用户态进程还需要独立的地址空间、缺页处理、共享映射和写时复制。把这些需求压进一个通用分配器会让接口变得含混, 也会让锁顺序难以验证。

2.1 设计目标与约束

内存管理子系统需要同时满足四类约束。第一类约束来自启动顺序。早期代码不能依赖运行期堆, 运行期分配器又需要早期分配器提供元数据空间。第二类约束来自硬件内存布局。可用 RAM 可能分散在多个物理段, 中间夹有内核镜像、固件保留区、设备 MMIO 和 `initramfs` 初始内存文件系统。第三类约束来自分配模式。8 字节对象和 8 MiB 对象不应走完全相同的路径。第四类约束来自用户态安全。用户页表、用户指针和缺页异常必须被明确隔离, 不能让内核普通内存访问路径承担不可信地址的风险。

我们将设计目标整理为表 2-1。

表 2-1 内存管理子系统的设计目标		
目标	含义	工程约束
启动自举	从启动分配器逐步过渡到完整运行期分配器。	正式接管后必须封存启动分配器, 并回收可归还的整页尾部。
物理页可靠	按固件内存段和保留区建立保留段信息的伙伴页分配器。	不可把内核镜像、元数据页和设备区误交给普通分配路径。
分配路径分工	小对象走 Slab 分配器, 大对象走内核堆, 受管对象走受管堆。	各路径共享统计、审计和注册表, 但热路径保持短。
用户空间隔离	用户虚拟地址空间管理 VMA、常驻页、COW 和共享后备。	架构层只注入页表、布局、用户访问和缺页解码操作。

这些目标决定了内存管理不能只围绕一种算法展开。启动分配器解决的是自举问题。伙伴页分配器解决物理页所有权问题。内核地址空间解决虚拟地址与物理页绑

定问题。Slab 分配器和内核堆解决对象分配路径问题。受管堆提供可选的受管对象能力。用户虚拟地址空间面向用户态地址空间，处理 VMA、缺页和页表。每一层都只暴露自己能够稳定承诺的接口。

2.2 总体分层结构

allocator 是分层内核分配器的总入口。它保存启动分配器 (BootAllocator)、伙伴页分配器 (BuddyAllocator)、内核地址空间 (KernelAddressSpace)、内核堆 (KernelHeap)、Slab 分配器 (SlabAllocator)、受管分配器 (ManagedAllocator)、元数据分配器 (MetadataAllocator) 和分配注册表 (AllocationRegistry)。这些对象共同构成一条从原始启动区间到运行期分配 API 的链路。



这条分层链路有两个关键性质。其一是所有权向上转化。启动分配器只拥有一段启动期连续区间。伙伴页分配器接管可用物理页。地址空间层将物理页转化为带虚拟地址的后备映射范围 (BackedRange)。Slab 分配器、内核堆和受管堆再把这些范围解释为对象或缓存。其二是策略向上集中。伙伴页分配器不理解对象大小。地址空间层不理解 Slab 尺寸类别。Slab 分配器不理解用户态 VMA。用户地址空间策略集中在用户虚拟地址空间，不会散落到具体架构页表实现中。

分配器还维护统一的能力查询、统计、审计和回收入口。capabilities() 方法暴露 API 版本、页大小、小对象上限、CPU 上限和受管 GC 能力。stats() 方法汇总启动分配器、虚拟内存分配域、Slab 分配器、内核堆和受管堆状态。audit() 方法可以扫描物理页、虚拟地址、Slab 缓存、内核堆缓存和受管对象。reclaim() 方法与 reclaim_caches() 方法用于主动释放缓存页。这些入口让内存管理具备运行期可观测性，而不需要外部模块直接读取内部结构。

2.3 初始化顺序与锁顺序

内存管理的初始化顺序由第一章中的启动链路触发。架构加载器先调用 `init_boot` 函数，建立最早期的线性分配能力。平台无关启动阶段解析固件后，调用 `bind_address_translation` 函数注入 `phys_to_virt` 回调和 `virt_to_phys` 回调。随后 `init_phys` 函数根据可用内存段和保留区初始化伙伴页分配器。架构层提供内核堆回调后，启动代码调用 `init_kernel_page_table` 函数，再进入 `init_vmem` 函数。最后依次初始化内核堆和 Slab 分配器，并通过 `activate_global` 函数切换全局分配器。

```
fn init_allocator(ctx: &StartContext, memory: &[MemorySegment], reserved:
&[(usize, usize)]) {
    KERNEL_ALLOCATOR.bind_address_translation(ctx.address.phys_to_virt,
ctx.address.virt_to_phys);

    KERNEL_ALLOCATOR.init_phys(memory, reserved)?;

    if let Some(ops) = ctx.allocator {
        KERNEL_ALLOCATOR.bind_kernel_heap_ops(
            ops.kernel_heap_region,
            ops.map_kernel_heap_range,
            ops.unmap_kernel_heap_range,
        );
        (ops.init_kernel_page_table)();
        KERNEL_ALLOCATOR.init_vmem(reserved)?;
        KERNEL_ALLOCATOR.init_kheap();
        KERNEL_ALLOCATOR.init_slab(cpu_count);
        KERNEL_ALLOCATOR.activate_global()?;
    }
}
```

代码 2-1 分配器初始化顺序

这个顺序有明确原因。`init_phys` 函数需要启动分配器提供初始化元数据，也需要 `phys_to_virt` 回调把物理元数据页转换为可访问地址。`init_vmem` 函数需要伙伴页分配器已经就绪，因为内核地址空间会建立直接映射区和内核分配域，并可能申请元数据页。元数据分配器只有在 `init_vmem` 成功并释放伙伴页分配器锁后，才能切到动态路径。源码中特意保留了这个顺序，避免虚拟内存分配域初始化期间持有物理页锁，又因元数据扩容重入物理页锁。

`activate_global` 函数是启动期和运行期的分界。它会检查启动分配器、物理页层、虚拟内存分配域、内核堆和 Slab 分配器是否已经全部就绪，初始化分配注册表，然后封存启动分配器。启动分配器封存后会取出当前游标之后的完整页尾部，通过 `virt_to_phys` 回调转成物理地址，再交还伙伴页分配器。包含已用字节的部分页不会释放，因为其中可能仍保存早期元数据。

锁顺序是分配器能稳定组合的另一个基础。当前源码中明确规定从 `init_lock` 初始化锁开始，随后依次是虚拟内存分配域、元数据、物理页、登记表分片、Slab 全局状态、Slab 每 CPU 缓存、内核堆、受管 GC 和精确根注册表。这个顺序有两个实际含义。第一，任何路径都不能反序获取锁。第二，调用架构层映射回调和可能触发分配的函数前，应先释放分配器内部锁。

表 2-2 分配器锁顺序与约束

区域	锁顺序	约束原因
初始化	init_lock 最先获取, 正常运行期尽量不持有。	避免初始化阶段和运行期分配交错。
地址空间与元数据	虚拟内存分配域早于元数据, 元数据早于物理页。	元数据动态扩容可能需要物理页, 不能在持有物理页锁时反向进入虚拟内存分配域。
对象分配	分配注册表早于 Slab 全局状态, Slab 全局状态早于每 CPU 缓存, 随后才进入内核堆。	小对象缓存和注册表记账需要一致的所有权顺序。
受管对象	受管 GC 早于精确根注册表。	GC 扫描需要稳定根集合, 根注册表不能反向调用分配热路径。

锁顺序使分层结构具备可验证性。我们在后续优化单个分配路径时, 只要没有破坏这个顺序, 就不会把局部优化扩散成全局死锁风险。

2.4 启动分配与物理页管理

启动分配器是最早期的线性递增分配器。它内部只保存起始地址、结束地址、当前游标、initialized 标记和 sealed 标记。分配时按 layout 参数的对齐要求向前推进游标。实现使用 CAS 循环更新 pos 字段, 因此即便在早期多核状态尚未完整建立时, 也不会因为普通非原子写入产生游标回退。

```
fn boot_alloc(layout: Layout) -> *mut u8 {
    if !initialized() || sealed() {
        return null_mut();
    }

    loop {
        let pos = load_pos();
        let aligned = align_up(pos, layout.align());
        let next = aligned + layout.pad_to_align().size().max(1);
        if next > heap_end() {
            return null_mut();
        }
        if compare_exchange_pos(pos, next).is_ok() {
            return aligned as *mut u8;
        }
    }
}
```

代码 2-2 启动期线性分配

启动分配器的关键设计落在生命周期边界上, 分配速度只是次要目标。它不支持释放。正式分配器接管之后, 它会进入封存状态, 后续 `alloc()` 方法必须失败。这个选择阻止了运行期代码继续从启动期池中分配对象。封存阶段还会把未用完整页尾部交给伙伴页分配器。这样早期预留容量不会永久浪费, 已经存放启动元数据的部分页也不会被错误复用。

伙伴页分配器是运行期物理页管理的核心。当前实现保留内存段信息。它支持多个物理内存段, 不假设整机 RAM 是连续区间。每个段保留来源于固件解析的边界, 初始化时会排除内核镜像、保留区和分配器元数据页。伙伴页分配器还区分默认空闲链表和低端 DMA 空闲链表, 并提供精确物理放置和大页对齐相关能力。

伙伴页分配器的经典分裂合并仍然保留。分配时根据请求页数计算阶数, 并从对应或更高阶数的空闲链表获取空闲块。高阶块会被分裂为两个低阶伙伴。释放时检查伙伴是否空闲, 满足条件就向上合并。当前实现还引入 0 阶延迟合并水位。频繁分配和释放 4 KiB 页时, 立即合并会导致下一次分配又重新分裂。保留少量 0 阶热页可以减少分裂和合并抖动。当空闲比例降到阈值以下时, 延迟合并会被禁用, 优先恢复高阶连续空间。

物理页层还提供审计和回收统计。伙伴分配器统计 (BuddyStats) 记录总页数、已分配页、空闲页、保留页、元数据页、内存段数量、阶数统计、分裂次数、合并次数和分配失败次数。伙伴分配器审计 (BuddyAudit) 会扫描内存段、哈希节点和空闲链表, 检查链表循环、节点失效和页记账不一致。这些数据对定位内存泄漏和错误释放非常关键。

2.5 内核地址空间、Slab 分配器与内核堆

内核地址空间位于伙伴页分配器和对象分配器之间。它解决的问题是拿到物理页后映射到哪里。它内部有三个分配域。DirectMap 分配域记录直接映射区。Kernel 分配域服务普通内核堆和大对象。Managed 分配域服务受管堆。一次带物理后备的分配会经历三步。先在分配域中保留虚拟地址, 再向伙伴页分配器申请物理页, 最后调用架构层提供的 `map_kernel_heap_range` 回调建立映射。释放时按相反方向撤销映射、归还物理页和释放虚拟区间。

这个层次把三个不同的操作隔离开。上层 Slab 分配器和内核堆不需要理解页表。下层伙伴页分配器不需要理解虚拟地址布局。架构层只提供映射和解映射回调, 不进入分配器内部锁。后备映射范围是这层交给上层的核心对象, 包含分配域、虚拟地址、物理地址、大小和阶数。

Slab 分配器处理高频小对象。当前尺寸类别覆盖 8 到 2048 字节。每个尺寸类别维护自己的 Slab 链表和 `SlabNode` 空闲链表。一个 Slab 由一批页作为后备映射范围, 再切分为固定大小槽位。槽位状态通过 `alloc_bitmap` 字段和 `cache_bitmap` 字段记录。对象可能处于空闲、已分配或缓存中。热路径优先访问每 CPU 缓存。缓存命中时, 只需要在本地缓存锁内弹出一个槽位。缓存失配时再进入 Slab 全局状态, 通过位图和 `next_free_hint` 字段查找可用槽。

```
fn alloc_small(layout: Layout) -> Option<usize> {
    let class = size_class_for(layout)?;
    if let Some(entry) = per_cpu_cache[class].pop() {
```



```

        mark_cached_as_allocated(entry);
        return Some(entry.ptr);
    }

    let mut state = slab_state.lock();
    if let Some(slot) = state.find_free_slot(class) {
        return Some(slot.ptr);
    }

    let range = address_space.alloc_kernel_backed_range(class.slab_order)?;
    state.add_slab(class, range);
    state.find_free_slot(class).map(|slot| slot.ptr)
}

```

代码 2-3 小对象分配路径

释放路径同样优先回到每 CPU 缓存。缓存未滿时，对象不会立即归还 Slab 全局状态。缓存满时会排出一批对象，再交给全局状态处理。这个策略减少了锁竞争，也降低了跨核共享压力。空 Slab 数量超过保留水位时，Slab 分配器会回收后备映射范围并复用 SlabNode 元数据。

内核堆处理 Slab 分配器不适合承载的大对象。它不直接操作页表，也不自己管理物理页算法，而是组合内核地址空间与伙伴页分配器。对于基础页和两页块，内核堆维护阶范围缓存。缓存命中时，可以直接复用已经具备虚拟地址和物理后备的范围。缓存满时，新释放范围会替换最旧范围，被淘汰者立即回到底层。这个环形队列避免满桶时移动大量槽位，也让近期释放后立即分配的模式走短路径。

Slab 分配器和内核堆共同服务 GlobalAlloc 全局分配器接口。小请求进入 Slab 分配器，大请求进入内核堆，所有成功分配都会进入分配注册表。登记表记录指针、大小、对齐、分配域、类别和后备信息。释放时先查询登记表，确认所有权，再按记录回到对应分配路径。这个设计让非法释放和重复释放更容易被发现，也为统计和审计提供统一入口。

2.6 受管堆与精确回收

受管堆是分配器中可选的受管对象区域。它不替代普通内核堆，而是为需要 GC (Garbage Collection, 垃圾回收) 语义的对象提供独立路径。启用时，内核地址空间会为受管分配域分配一段连续虚拟区域，并由受管分配器初始化 GC 控制结构。默认轻量配置是 2 MiB，较大的配置可以使用 64 MiB 级别。

受管堆的核心原则是精确优先。每个受管对象必须携带跟踪描述符 (TraceDescriptor)，其中记录对象大小、对齐、强引用偏移和弱引用偏移。GC 不做保守栈扫描，而是依赖根帧、根句柄、动态精确根槽和自动根提供者。这样可以避免把普通整数误判为指针，也使移动对象、弱引用和终结器更容易保持一致。

GC 路径包含新生代回收、全量回收和显式压力回收。当前实现中存在新生代复制、对象晋升、老年代标记清除和碎片感知压缩决策等机制。受管分配器还提供弱引用、终结器、固定对象、脏卡、记忆集对象和安全点相关状态。外部子系统可以通过精确根注册表注册稳定根槽，也可以安装自动根提供者。

这套受管路径被放在分配器的上层，原因是它需要普通分配能力和地址空间能力已经就绪。它的锁顺序也靠后，避免 GC 在扫描或回收时反向进入物理页热路径。

受管堆的价值在于为复杂对象生命周期提供一种可审计的选择，而不是让内核所有对象都进入 GC。

2.7 进程地址空间与缺页处理

用户虚拟地址空间是用户态地址空间的顶层对象。它位于平台无关层，负责把 VMA 集合、常驻页映射、用户页表句柄、brk 游标、mmap 游标和 mlock 状态组织在一起。架构层只提供不透明的页表根句柄 (PgDHandle) 和一组函数指针。用户虚拟地址空间不知道 LoongArch64 或 RISC-V64 如何编码页表项，也不直接操作硬件寄存器。

VMA 集合由虚拟内存区域集合 (VmaSet) 管理，底层使用有序结构支持按地址查找、范围覆盖检查、插入、拆分和合并。pages 映射记录已经常驻的页。每个常驻页由常驻页对象 (ResidentPage) 表示，可能是匿名页、共享匿名页、私有文件页、共享文件页或直接映射页。共享文件页和共享匿名页通过全局弱引用表复用常驻页，避免同一共享后备在多个地址空间中被重复读入。

map_anon 函数和 map_file 函数只注册 VMA，不立即分配所有物理页。首次访问时，架构陷入处理器调用平台无关层的 dispatch_page_fault 缺页分派入口。这个入口先用缺页解码接口从陷阱帧中取出缺页类型、缺页地址和来源特权级。若缺页来自内核态用户访问路径，会先尝试让当前用户虚拟地址空间执行延迟补页，再用 __ex_table 异常表修复项把不可修复的用户指针错误转换为可返回的错误。若缺页来自用户态，则交给当前任务的 VmSpace::handle_fault 方法。

```
fn dispatch_page_fault(tf: TrapFramePtr) -> FaultOutcome {
    let kind = fault_decode.fault_kind(tf);
    let addr = fault_decode.fault_addr(tf);

    if !fault_decode.fault_from_user(tf) {
        if let Some(vm) = current_task_vm_space()
            && vm.handle_fault(addr, kind) == FaultOutcome::Fixed
        {
            return FaultOutcome::Fixed;
        }
        return if fault_decode.try_fixup_kernel_access(tf) {
            FaultOutcome::Fixed
        } else {
            FaultOutcome::Kernel(UncaughtKernelAccess)
        };
    }

    current_task_vm_space()
        .map(|vm| vm.handle_fault(addr, kind))
        .unwrap_or(FaultOutcome::Kernel(NoVmSpace))
}
```

代码 2-4 缺页处理分派

VmSpace::handle_fault 方法先定位缺页所在页和 VMA。若找不到 VMA，会尝试按向下增长规则扩展栈。若权限不满足，返回 segv 枚举变体。若页已经常驻，写缺页可能触发 COW、共享脏页跟踪标记或权限修复。若页尚未常驻，则根据后备信息分配或获取常驻页。匿名页分配零页。共享匿名页通过共享标识和偏移查找共享页。

文件映射按文件偏移读取页。直接映射页直接使用指定物理地址。完成后通过 `UserPgdOps::map` 方法建立用户页表映射。

`fork` 系统调用路径体现了用户虚拟地址空间的策略边界。`fork` 系统调用复制 VMA 元数据，已常驻页根据私有 COW 或共享语义重建页表。父进程中原本可写的私有页会被保护为 COW。子进程映射同一个常驻页，也以 COW 权限进入。第一次写入时，缺页处理器复制物理页并替换当前进程映射。共享文件页、共享匿名页、System V 共享内存和直接映射页保持共享语义。这个策略全部位于平台无关层，架构页表代码只执行映射、解映射、保护和 TLB 失效。

用户指针访问通过用户访问接口集中封装。系统调用路径和装载器调用 `copy_from_user` 函数、`copy_to_user` 函数或 `copy_cstr_from_user` 函数。这些安全包装把内核切片和用户地址传给架构实现。架构侧使用异常表处理访问失败。上层只看到 `Result<_, UserAccessError>` 类型表达式，从而把用户态非法指针归约为标准错误，而不会变成不可恢复的内核异常。

2.8 架构注入契约

内存管理的跨架构复用依赖四组操作接口。用户地址布局接口描述用户页大小、`brk` 基址、`mmap` 区间、默认栈、PIE（Position Independent Executable，位置无关可执行文件）基址、解释器基址和 `vDSO` 基址。用户页表接口提供用户页表的新建、销毁、映射、解映射、保护、`fork` 克隆、激活和 TLB 失效。用户访问接口提供用户内存复制和用户字符串长度读取。缺页解码接口从陷阱帧中提取缺页类型、缺页地址、来源特权级，并尝试进行内核态用户访问修复。

表 2-3 用户地址空间的架构注入接口		
接口	提供内容	设计边界
UserVmLayoutOps	用户页大小、堆、 <code>mmap</code> 、栈、PIE、解释器和 <code>vDSO</code> 布局。	平台无关层消费布局，不固化具体虚拟地址常量。
UserPgdOps	PGD 创建、释放、映射、解映射、保护、激活和 TLB 失效。	页表格式保留在架构层。
UserAccessOps	<code>copy_from_user</code> 、 <code>copy_to_user</code> 和 <code>strnlen_user</code> 。	用户指针异常由架构层修复，系统调用只处理错误码。
FaultDecodeOps	缺页类型、缺页地址、是否来自用户态以及内核访问修复。	陷阱帧结构不进入平台无关层。

这些接口采用 `AtomicPtr` 原子指针加 `Release/Acquire` 内存序的注册模式。架构层在调度器和用户态内存初始化前完成注册。平台无关层调用时只读取函数表，不保存架构私有结构。这个模式与调度器的架构钩子类似。它让 `LoongArch64` 和 `RISC-V64` 可以共用同一套用户虚拟地址空间策略，同时保留各自的页表格式和异常恢复实现。

2.9 工程设计总结

本章讨论的内存管理子系统,从启动期的线性分配一直延伸到用户态缺页处理。它的主线是把不同阶段的内存问题放在不同层次中处理。启动期关注能否可靠自举,物理页层关注所有权和碎片,地址空间层关注虚拟地址与物理后备的绑定,对象分配层关注热路径延迟和回收,用户地址空间层关注隔离、共享和异常恢复。

内存管理子系统具备以下创新:

第一是把启动自举和运行期分配做成可验证的单向交接。回顾整个分配器的启动过程,最容易混淆的点在于启动分配器是否只是一个临时实现。它确实简单,但它不是可以被随意绕过的临时工具。它负责为伙伴页分配器、元数据分配器和早期固件解析提供最初的分配能力。正式分配器接管后,它必须被封存。剩余完整页尾部也必须按所有权移交给伙伴页分配器。我们曾经可以选择让启动分配器永久保留一段私有池,后续在内存压力下再按需使用。这个方案会让早期内存与运行期内存之间出现长期双重所有权。当前设计把交接固定在 `activate_global` 函数,使启动期内存的生命周期有明确终点。这个边界让内存统计更可信,也避免运行期对象继续落入不可释放的启动区间。

第二是物理页管理保留了固件段信息和运行期审计能力。很多教学化的伙伴系统实现会把所有 RAM 展平为一个连续数组。真实平台上的可用内存通常并不连续,还夹杂内核镜像、固件保留区和设备窗口。我们在伙伴页分配器中保留内存段视图,并围绕内存区域、阶空闲链表、哈希节点和保留范围建立所有权管理。0 阶延迟合并用于降低高频页分配的分裂和合并抖动,低空闲比例时又会退回积极合并。更重要的是,伙伴页分配器提供可扫描的审计结果。它能检查链表循环、节点损坏和记账不一致。这个设计让物理页管理不只是一个分配算法,也成为可以长期观测和验证的内核基础设施。

第三是把虚拟地址、物理页和页表操作拆成三个相互约束的边界。内核地址空间不拥有物理页算法,也不理解对象大小。它只负责在分配域中保留虚拟地址,向伙伴页分配器请求物理后备,并调用架构注入的映射回调。这个结构看似多了一层,实际解决了两个长期问题。其一,Slab 分配器和内核堆不需要携带页表细节。其二,架构层不需要进入分配器锁内部。尤其是在内核堆使用独立高半区窗口时,直接映射区只承担物理跨度视图和统计角色,真实可分配性由内核分配域与伙伴页分配器共同决定。这种边界处理减少了启动期脆弱的保留区切分逻辑,也让后续替换页表实现时不影响对象分配策略。

第四是小对象和大对象采用不同热路径,同时通过分配注册表统一所有权。Slab 分配器面向 2048 字节以内的小对象,使用尺寸类别、位图、每 CPU 缓存和批量补货减少锁竞争。内核堆面向页级对象和大对象,使用范围缓存复用已经具备虚拟地址与物理后备的范围。二者的算法目标不同,但分配结果都会进入分配注册表。释放时必须先通过登记表确认指针来源,再回到对应后端。这个组合让热路径保持专门化,同时让错误释放、重复释放和统计审计具有统一入口。我们没有把所有对象都交给 Slab 分配器,也没有让大对象路径绕过记账。这个取舍保证了性能和所有权检查可以同时存在。

第五是用户地址空间策略集中在用户虚拟地址空间,架构层只实现机械动作。用户虚拟地址空间管理 VMA 集合、常驻页、共享后备、fork 系统调用的 COW、mremap 系统调用、mprotect 系统调用、缺页提交和文件页回写。架构层通过用户页表接口、

用户访问接口和缺页解码接口提供页表操作、用户指针访问和缺页解码。这样做的结果是，LoongArch64 和 RISC-V64 可以共用同一套地址空间语义。页表格式、陷阱帧和异常表恢复仍保留在各自架构内部。这个边界对系统调用稳定性很重要。用户指针错误会通过用户访问错误返回给系统调用，而不是在平台无关层变成无法恢复的内核异常。

第六是内存管理提供了可选的精确受管堆，同时没有削弱普通内核分配路径。受管堆使用跟踪描述符、根帧、根句柄、动态精确根槽和自动根提供者构造精确根集合。它支持弱引用、终结器、对象晋升、脏卡和多种回收入口。与此同时，普通 Box 类型、内核对象、小对象和大对象仍然走 Slab 分配器或内核堆。受管堆作为可选能力存在，使需要复杂生命周期的对象可以使用 GC 语义，也避免把整个内核强制纳入受管运行时。这种设计给后续扩展留下空间，同时保持基础分配路径的可预测性。

从整体看，内存管理子系统的价值不只在于若干算法的组合。更关键的是每个算法都被放在了合适的边界内。启动分配器只负责早期自举。伙伴页分配器只负责物理页。地址空间层只负责虚拟范围和映射协调。Slab 分配器、内核堆和受管堆各自服务不同对象生命周期。用户虚拟地址空间把用户地址空间策略从架构细节中分离出来。边界清晰之后，性能优化和正确性检查才能分别推进，而不会相互牵扯。

第三章 设备模型与驱动框架

在第二章中,内存管理子系统围绕物理页和虚拟地址空间展开,并进一步讨论了堆分配与 Slab 缓存。那一章的核心问题,是如何在相对同构的内存资源之上建立稳定的分配与回收机制。本章问题的性质发生了根本性变化。设备抽象面对的是硬件异构性。串口使用字节流,块设备使用可寻址块,网卡使用数据包,RTC 设备则围绕时间和中断工作。PCI 主桥与 IRQ 控制器又承担平台基础设施的角色。它们的访问单位不同,控制命令不同,生命周期也不同。中断、DMA、MMIO 和固件描述对每类设备的约束也不相同。若设备模型过早收敛到单一 I/O 接口,很多设备的真实能力会被压缩。若完全放开,每个驱动自行维护注册、命名、热插拔和用户态投影,全局生命周期又会难以推理。

我们在设备模型中采用了以设备功能接口 (DeviceFunction) 为中心的多轨结构。这里的设备能力表示驱动向内核开放出来的能力,并不等同于某一种固定设备文件。字符设备、块设备、网络设备和 RTC 设备都是设备能力的具体落实。UART 设备、VirtIO 块设备、VirtIO 网络设备与 LS7A RTC 设备等驱动,则继续把这些设备能力落实到具体硬件上。这样一来,设备模型形成了从抽象到具体的层次关系:最上层关注开放能力和生命周期;中间层保留各类设备的类型化语义;最下层处理总线、寄存器、中断和队列等硬件细节。

本章主要说明这一结构的设计目标、核心机制和关键路径。为了保持叙述边界清晰,设备管理被拆成四个问题来讨论。第一,硬件如何被发现并绑定驱动。第二,驱动如何把能力登记到全局设备能力注册表。第三,各类设备能力如何保留自己的类型化接口。第四,设备能力如何被投影到 devtmpfs 文件系统、sysfs 文件系统和 procfs 文件系统这样的用户可见视图中。这四个问题之间存在明确的信息流。底层硬件事实向上被整理为 PnP 设备。PnP 设备经驱动探测发布设备能力。设备能力再被观察者投影为用户态名字空间中的节点。

3.1 设计目标与约束

设备模型需要同时满足三类约束。第一类约束来自硬件。不同设备对资源的依赖差异很大:串口可能只需要 MMIO 和中断;块设备通常需要队列、DMA 或轮询推进机制;网卡需要与协议栈建立接口关系;RTC 需要同时表达时间读写、告警和周期中断。第二类约束来自内核内部。文件系统希望直接拿到块设备对象,网络协议栈希望直接拿到网络接口对象,VFS 希望把部分设备映射为文件节点。第三类约束来自用户态兼容。用户态仍然按照 /dev、设备号、ioctl 系统调用和 sysfs 文件系统属性等传统方式观察设备。设备模型必须提供这些视图,同时避免让这些视图反向决定底层设备身份。

我们将设计目标整理为以下几点。

表 3-1 设备模型的设计目标

目标	含义	工程约束
生命周期统一	不同总线和不同设备类型都进入统一的发现、探测、移除、失效流程。	驱动移除时必须先阻止新访问,再排空旧请求,最后释放硬件资源。
能力表达开放	字符、块、网络、RTC 等设备能力通过同一个设备能力注册表发布。	核心注册表只理解类别、名称和生命周期入口,不嵌入每类设备的 I/O 细节。
类型语义保留	每类设备能力保留自己的类型化对象和控制请求。	网络设备不被强制映射为字符流,块设备不丢失异步提交语义。
用户视图解耦	/dev、sysfs 文件系统、procfs 文件系统观察设备能力事件后生成视图。	投影失败只能影响用户可见性,不能反向破坏底层探测事务。

这些目标决定了设备模型不能以传统的字符设备和块设备作为根抽象。字符和块仍然重要,但它们只是两类已经落实的设备能力。网络设备和 RTC 设备也需要同等地位。未来可能加入的输入设备、图形设备和声音设备,也应当能够在不修改 PnP 核心与 devtmpfs 文件系统核心的情况下接入。我们因此把设备管理的根概念从“设备文件类型”上移到“开放能力类型”,用设备能力表达所有已发布能力的共同边界。

3.2 总体分层结构

设备模型可以分为五层。发现层把 DTB、ACPI 和 PCI 等来源中的硬件事实整理出来。PnP 层负责设备身份、驱动匹配、资源归属、状态转换和移除顺序。设备能力层维护全局开放能力注册表。类型化设备层保存字符、块、网络和 RTC 等具体 I/O 语义。用户态投影层把设备能力映射为 /dev 节点、sysfs 条目和 procfs 诊断视图。



这条信息流具有两个重要性质。其一是单向依赖。PnP 层不调用 devtmpfs 文件系统。设备能力注册表不依赖 sysfs 文件系统或 procfs 文件系统。用户态投影只是观察设备能力事件。其二是事务边界清晰。驱动探测的成功条件只覆盖硬件初始化、类型化对象构造和设备能力注册表注册。/dev 节点创建属于后续投影动作。这样可以避免一个用户态视图错误破坏底层设备生命周期，也可以允许设备先于 devtmpfs 文件系统挂载完成注册。

我们在实现中故意让注册表保存 `Arc<dyn DeviceFunction>` 类型表达式。这意味着 PnP 核心不需要知道字符设备对象、块设备对象、网络设备对象或 RTC 设备对象的内部结构，只需要对所有设备能力调用同一组生命周期方法。需要具体类型的上层模块通过 `as_any` 方法和 `function_as` 方法显式恢复类型。这种显式恢复机制使通用路径保持稳定，也让专用路径在代码层面暴露自己的类型依赖。

3.3 设备能力核心抽象

设备能力是设备模型中最小的开放能力接口。它不定义读写方法，也不定义 `ioctl` 命令，只定义设备能力类别、内部名称、生命周期标记、I/O 排空和类型恢复。这样设计的关键原因，是读写语义并非所有设备共有。字符设备适合流式读写，块设备适合异步块请求，网络设备适合数据包队列，RTC 设备适合时间与告警控制。把这些语义放入同一个特征接口，会使核心接口不断膨胀。

```

trait DeviceFunction: Send + Sync {
    fn class_id(&self) -> DeviceClassId;
    fn dev_name(&self) -> &str;
    fn mark_gone(&self);
    fn drain_io(&self) {}
    fn as_any(&self) -> &dyn core::any::Any;
}
  
```

代码 3-1 设备能力核心接口

`class_id` 方法返回设备能力的类别标识。当前内置类别包含 `char`、`block` 和 `rtc`，网络设备在网络子系统中通过 `DeviceClassId::new("net")` 注册为独立类别。`dev_name` 方法返回设备能力注册表的内部名称，它与 `/dev` 下的文件名没有必然对应关系。`mark_gone` 方法用于热插拔移除阶段，调用后旧句柄仍可能因为引用计数继续存在，但新 I/O 应当尽快返回设备不可用。`drain_io` 方法用于排空已经提交的异步 I/O，字符设备和 RTC 这类没有异步请求队列的设备能力可以使用默认空实现。`as_any` 方法是类型恢复入口，新设备类型通过它接入通用注册表。

设备能力注册表使用 `class_id + dev_name` 作为唯一键。这个选择比单纯使用名称更稳健，因为不同类别的设备可以拥有相同的自然编号，例如 `net0`、`rtc0`、`uart0` 的命名空间本来就属于不同类别。它也比在注册表中维护多个具体类型列表更开放，因为新增设备能力类别时不需要修改核心结构。

```
fn register_function(func: Arc<dyn DeviceFunction>) -> Result<(),
FunctionRegistryError> {
    let key = (func.class_id(), func.dev_name());
    let mut list = FUNCTIONS.lock();

    if list.iter().any(|existing| {
        existing.class_id() == key.0 && existing.dev_name() == key.1
    }) {
        return Err(FunctionRegistryError::NameExists);
    }

    list.try_push(func.clone());
    drop(list);

    publish_function_event(DeviceFunctionEventKind::Registered, func);
    Ok(())
}
```

代码 3-2 设备能力注册的核心流程

这里有一个容易忽略的细节。事件发布必须发生在注册表锁释放之后。`devtmpfs` 文件系统、`sysfs` 文件系统和 `procfs` 文件系统都可能在回调中读取设备能力快照、创建索引节点或记录投影状态。如果注册表在回调过程中仍然持锁，观察者一旦回到设备核心查询状态，就可能形成锁顺序反转。我们因此选择了快照式遍历和锁外回调。这个选择增加了少量复制成本，但换来了更清晰的并发边界。

3.4 设备能力多轨落实

设备能力多轨结构的核心，是把通用生命周期和类型化 I/O 分开。所有设备能力都有类别、名称和移除入口，只有具体类别才拥有读写、控制、队列、统计等语义。表 3-2 列出了当前已经落实的主要轨道。

表 3-2 设备能力轨道与类型化对象

轨道	类型化对象	核心语义	主要消费者
字符	CharDevice	非阻塞读写、刷新、poll 系统调用、TTY 控制、控制台语义。	devtmpfs 文件系统、TTY 层、控制台和通用字符设备适配器。
块	BlockDevice	Bio 请求异步提交、几何信息、能力限制、同步等待适配。	文件系统、块设备挂载路径、devtmpfs 块设备节点。
网络	net::NetDevice	接口编号、链路状态、MTU、MAC、收发统计和协议栈轮询。	网络协议栈、套接字层、网络控制接口。
RTC	RtcDevice	时间读写、告警配置、周期中断、RTC ioctl 兼容。	时间子系统、devtmpfs RTC 适配器、用户态 RTC 程序。

3.4.1 字符设备能力

字符设备能力面向顺序流式设备。串口、控制台、TTY、随机数设备都属于这一类。字符设备的关键设计点，是把非阻塞 I/O、阻塞全量写、刷新和 poll 系统调用分开。控制台输出需要尽量避免长时间阻塞，TTY 输入需要支持等待队列和事件唤醒，用户态文件接口又需要把返回值和错误码映射到 POSIX 语义。把这些语义混在一个方法里会增加调用方判断成本，因此我们在字符设备驱动接口中保留了相对清晰的分工。

```

trait CharDriver: Send + Sync {
    fn write(&self, buf: &[u8]) -> Result<usize, CharIoError>;
    fn read(&self, buf: &mut [u8]) -> Result<usize, CharIoError>;
    fn poll_read(&self) -> bool;
    fn poll_add_waiter(&self, task: &Arc<Task>, read: bool, write: bool) ->
    bool;
    fn poll_remove_waiter(&self, task: &Arc<Task>);
    fn flush(&self) -> Result<(), CharIoError>;
    fn control(&self, req: CharControlRequest) ->
    Result<CharControlResponse, ControlError>;
    fn as_any(&self) -> &dyn Any;
}

```

代码 3-3 字符设备驱动接口

字符设备对象自身保存状态位。设备仍处于 active 状态时，读写请求会转发到底层驱动。进入 gone 状态后，新请求返回 Unavailable。这里没有要求旧句柄立即失效，因为 devtmpfs 文件系统索引节点、TTY 对象或正在进行的系统调用都可能持有 Arc 强引用。我们采用“先标记，后回收”的顺序，使对象内存由引用计数自然收束，底层硬件访问由状态检查提前阻断。

字符设备能力与 /dev 字符设备文件之间存在一层适配。字符设备能力发布的是内核内部能力，devtmpfs 投影器才决定它是否生成 /dev/console、/dev/uart0 或其它

串口节点。这样同一个字符设备可以被投影成多个用户可见入口,也可以先被内核作为启动控制台使用,稍后再补齐用户态节点。

3.4.2 块设备能力

块设备能力面向可寻址 I/O。它的自然请求单位是 Bio 请求,其中包含操作类型、块范围、缓冲区和完成状态。驱动接收 Bio 请求后将请求放入硬件队列或软件队列,随后返回。完成路径再写回结果并唤醒等待方。同步文件读写通过 `submit_bio_wait` 函数建立在异步请求之上。

```
fn submit_bio_wait(dev: &BlockDevice, range: BlockRange, buffer: BioBuffer)
-> BioResult {
    let completion = BioCompletion::new();
    let bio = Bio::new(BioOp::Read, range, buffer, completion.clone());

    dev.queue_bio(bio)?;
    loop {
        if let Some(result) = completion.try_take() {
            return result;
        }
        dev.drain();
        scheduler::yield_now();
    }
}
```

代码 3-4 块设备同步等待适配

这个同步适配保留了底层异步语义。文件系统和块设备文件可以获得阻塞式接口,驱动仍然按队列模型工作。热插拔移除时, `BlockFunction::drain_io` 方法会调用 `BlockDevice::drain` 方法,尽量推进已经提交的请求完成。这样 PnP 核心不需要理解 Bio 请求的内部字段,也能在移除阶段给块 I/O 留出收束窗口。

块设备还维护几何信息、限制信息、能力标志和 I/O 统计。几何信息描述逻辑块大小、物理块大小和容量,限制信息描述最大请求大小、对齐要求和丢弃范围限制,能力标志描述只读、刷新、丢弃、写零等操作是否可用。这些信息只对块轨道有意义,因此保存在块设备对象内部,由文件系统和块适配层按需读取。

3.4.3 网络设备能力

网络设备能力的主要消费者是协议栈。它的核心数据单位是数据包,控制语义围绕接口状态展开。网络设备能力发布网络设备对象,同时实现网络控制请求,如查询接口编号、链路状态、MAC 地址、MTU、收发统计,或设置管理启用状态。网络设备可以不投影为 `/dev` 节点,因为它已经通过协议栈和套接字层进入用户态。

```
fn control_net_device(dev: &Arc<NetDevice>, req: NetControlRequest)
-> Result<NetControlResponse, ControlError>
{
    if !dev.is_active() {
        return Err(ControlError::NoDevice);
    }

    match req {
```

```

        NetControlRequest::GetInterfaceId =>
Ok(NetControlResponse::U32(dev.id().raw())),
        NetControlRequest::GetMacAddress =>
Ok(NetControlResponse::MacAddress(dev.driver().mac_address())),
        NetControlRequest::GetMtu =>
Ok(NetControlResponse::Usize(dev.mtu())),
        NetControlRequest::SetMtu { mtu } => {
            dev.set_mtu(mtu)?;
            Ok(NetControlResponse::Done)
        }
        NetControlRequest::SetAdminUp { up } => {
            net::stack().set_iface_admin_up(dev.id(), up)?;
            Ok(NetControlResponse::Done)
        }
        _ => query_driver_or_stack(dev, req),
    }
}

```

代码 3-5 网络设备能力的控制请求分发

这里体现了设备能力多轨结构的一个实际价值。网卡驱动只需要把硬件收发能力落实为网络设备对象和网络设备能力，协议栈直接消费类型化对象。VFS 兼容层只在确有需要时参与，例如用户态查询接口属性或修改 MTU。核心设备模型不要求网卡先伪装成字符设备文件。

3.4.4 RTC 设备能力

RTC 设备能力把时间设备从字符设备历史接口中拆出。用户态仍可通过 `/dev/rtc0` 和 `ioctl` 系统调用访问 RTC 设备，但内核内部使用 RTC 设备对象表达时间读写、告警配置、周期中断和功能查询。RtcFunction 注册为 `DeviceClassId::RTC` 类别，VFS 投影器再生成 `DevNodeSpec::Custom` 规范和必要的符号链接。

RTC 设备的实现说明了兼容层与核心层的边界。我们可以保留用户态 ABI 的历史形式，同时让内核内部保持类型化结构。这样新增 RTC 驱动时，只需要实现 `RtcDriver` 驱动接口和注册 `RtcFunction` 设备能力，不需要在 `devtmpfs` 文件系统核心或系统调用层增加硬件类型特判。

3.5 PnP 设备与驱动绑定

PnP 层负责管理硬件实体。它保存硬件身份、总线信息、状态机、父子拓扑、已发布设备能力、已拥有资源和绑定驱动。驱动通过驱动工厂创建实例，再通过 PnP 驱动接口参与匹配和探测。匹配策略先考虑总线归属，再考虑优先级。同级同优先级的多个驱动同时匹配时，PnP 核心返回歧义错误，避免注册顺序成为隐藏策略。

```

trait PnpDriver: Send + Sync {
    fn name(&self) -> &'static str;
    fn bus_type(&self) -> PnpBusType;
    fn priority(&self) -> PnpDriverPriority;
    fn matches(&self, dev: &PnpDevice) -> bool;
    fn probe(&self, dev: &Arc<PnpDevice>) -> Result<(), PnpError>;
}

```

```
fn remove(&self, dev: &Arc<PnpDevice>);
}
```

代码 3-6 PnP 驱动接口

驱动探测的典型流程可以分为五步。第一步，从 PnP 设备对象中读取硬件身份和资源描述。第二步，申请或映射 MMIO、中断、DMA 等资源。第三步，初始化硬件并构造驱动私有对象。第四步，创建类型化设备和对应设备能力。第五步，调用 `PnpDevice::register_function` 方法完成事务式注册。任何一步失败都应回滚已完成的资源申请，或者把资源交给 PnP 的已拥有资源机制统一释放。

```
fn probe(dev: &Arc<PnpDevice>) -> Result<(), PnpError> {
    let resources = dev.platform_resources()?;
    let mmio = map_mmio(resources.mmio)?;
    let irq = request_irq(resources.irq)?;
    dev.own_resource(PnpOwnedResource::Irq(irq.clone()))?;

    let driver = Arc::new(ConcreteDriver::new(mmio, irq)?);
    let typed = Arc::new(TypedDevice::new(driver)?);
    let func: Arc<dyn DeviceFunction> =
        Arc::new(ConcreteFunction::new("device0", typed));

    dev.register_function(func)?;
    Ok(())
}
```

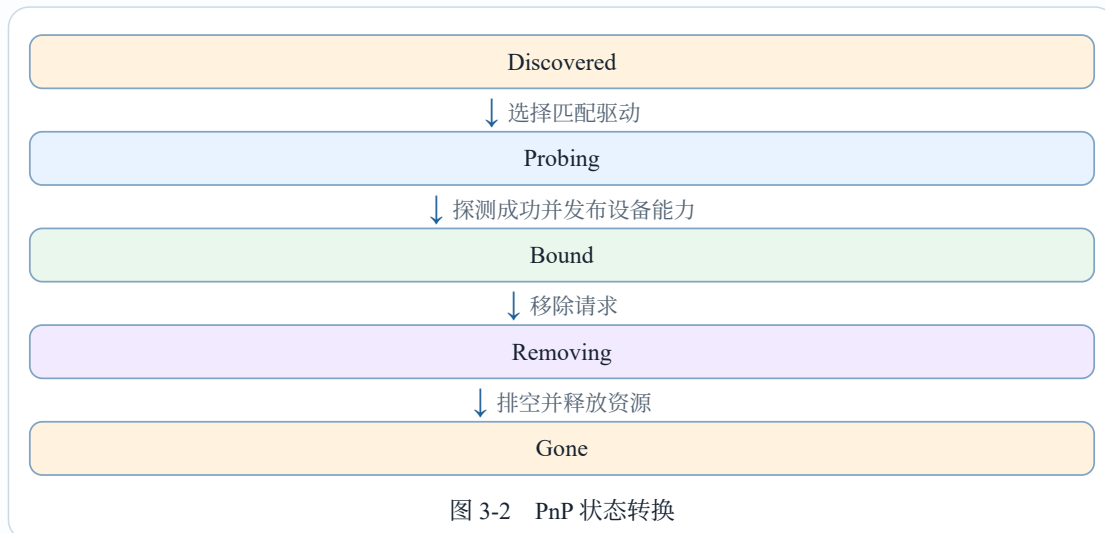
代码 3-7 探测与设备能力注册事务

`register_function` 方法的事务边界只覆盖 PnP 设备自身和全局设备能力注册表。它会先把设备能力挂到 PnP 设备，再尝试推入全局注册表。如果全局注册失败，已经挂接到 PnP 设备的设备能力会被撤销并标记失效。`devtmpfs` 文件系统、`sysfs` 文件系统和 `procfs` 文件系统的投影结果不进入这个事务。投影失败会记录为用户视图失败，底层硬件不因此回滚。

驱动依赖尚未满足时，探测过程可以返回延迟依赖。这个机制主要用于中断控制器、MSI 控制器、系统控制器和 PCI 主桥等基础资源的初始化顺序。我们记录缺失的依赖键，并在对应资源登记完成后重试受影响设备。事后回顾，这个机制比简单的全局重扫更容易定位启动问题，因为日志能够指出阻塞探测过程的具体资源。

3.6 PnP 生命周期与热插拔顺序

PnP 设备状态机约束设备在不同阶段允许执行的操作。`Discovered` 状态表示设备已发现但未绑定驱动。`Probing` 状态表示正在尝试绑定。`Bound` 状态表示驱动已绑定并可能发布设备能力。`Removing` 状态表示移除流程已开始，此时新的绑定和新的 I/O 都应被阻止。`Gone` 状态表示设备生命周期结束。



移除流程必须固定顺序。我们先把设备状态切到 `Removing`，阻止新的绑定。随后递归移除子设备，避免父设备资源提前释放。接着对已经发布的设备能力调用 `mark_gone` 方法，让新 I/O 尽快失败。之后调用 `drain_io` 方法，推动已经提交的请求完成。只有在访问路径和在途请求都被收束之后，驱动的 `remove` 方法才能关闭硬件，PnP 核心才能释放已拥有资源并注销设备能力。

```

fn remove_device(dev: &Arc<PnpDevice>) {
    dev.transition(PnpState::Removing)?;

    for child in dev.children_snapshot() {
        remove_device(&child);
    }

    for func in dev.functions_snapshot() {
        func.mark_gone();
    }

    for func in dev.functions_snapshot() {
        func.drain_io();
    }

    if let Some(driver) = dev.bound_driver() {
        driver.remove(dev);
    }

    dev.release_owned_resources_lifo();
    dev.unregister_functions();
    dev.transition(PnpState::Gone)?;
}
  
```

代码 3-8 PnP 移除流程

这个顺序的安全性来自明确的依赖关系。`mark_gone` 方法处理新访问，`drain_io` 方法处理旧请求，`driver.remove` 方法处理硬件关闭，资源释放处理内核对象归还。若先释放资源，旧句柄仍可能访问已经归还的 MMIO 或 IRQ 句柄。若先关闭硬件，块设

备队列中仍可能存在没有完成的请求。我们把这些阶段固定下来,使每个驱动不必单独证明同一组竞态条件。

3.7 用户态投影与设备文件

设备能力注册表发布的是内核内部能力,用户态看到的是经过投影的名字空间。devtmpfs 文件系统订阅设备能力事件,调用 VFS 设备文件层的投影器生成设备节点集合 (DevNodeSet),再根据设备节点规范创建索引节点。sysfs 文件系统和 procfs 文件系统也通过同一套投影快照生成诊断信息。这样用户态视图集中在 VFS 设备文件层解释,devtmpfs 文件系统核心不需要直接理解 RTC 设备、loop 控制设备接口或其它专用节点。

```
type DeviceFileProjectorBuild =
    fn(&dyn DeviceFunction) -> Result<Option<DevNodeSet>, VfsError>;

struct DeviceFileProjector {
    owner: &'static str,
    name: &'static str,
    build: DeviceFileProjectorBuild,
}

fn devnodes_for_function(func: &dyn DeviceFunction) ->
Result<Option<DevNodeSet>, VfsError> {
    for projector in projector_snapshot()? {
        if let Some(nodes) = (projector.build)(func)? {
            return Ok(Some(nodes));
        }
    }
    Ok(None)
}
```

代码 3-9 设备文件投影器

投影状态独立于底层生命周期。一个设备能力注册成功后,投影可能处于待处理、已绑定、失败或已解绑。待处理表示事件已被观察到但节点尚未完成绑定。已绑定表示节点已经进入 devtmpfs 文件系统。失败表示节点创建失败,错误码会被记录并暴露给诊断视图。已解绑表示设备能力注销后节点已解除。这个状态属于用户 ABI 视图,不参与 PnP 探测的成败判断。

设备节点规范的载荷直接携带打开节点时需要的类型化对象。字符节点携带字符设备对象,块节点携带块设备强引用,自定义节点携带类型化端点。devtmpfs 文件系统创建索引节点后,open 路径可以直接恢复对象引用,不需要再通过主次设备号查全局表。设备号仍然存在,但它属于 POSIX 兼容视图,主要服务于 stat 系统调用、mknod 系统调用和 sysfs 展示。

3.8 初始化顺序与典型路径

设备初始化依赖 VFS、分配器、固件解析和地址转换能力。启动早期先注册核心文件系统和设备文件投影器,再创建 devtmpfs 文件系统超级块,并安装设备能力投影订阅。随后设备初始化上下文把 device_mmio_to_virt 回调、中断域、DMA 入口、

实时时钟钩子等能力交给驱动注册过程。最后固件或总线扫描创建 PnP 设备，驱动工厂尝试绑定，探测成功后发布设备能力。

DTB 路径和 ACPI 路径的发现来源不同，后续设备模型保持一致。DTB 路径从设备树节点中解析平台设备、中断控制器、PCI 主桥等结构。ACPI 路径从 RSDP、MADT、SPCR、命名空间等来源中整理串口、virtio-mmio 设备和平台资源。两条路径最终都转化为 PnP 设备和设备能力发布事件。这样固件格式差异被限制在发现层，生命周期和投影规则保持统一。

表 3-3 设备初始化的关键顺序

阶段	主要动作	顺序原因
VFS 准备	注册 tmpfs、devtmpfs、procfs、sysfs 和设备文件投影器。	投影规则需要先于设备能力事件订阅安装。
投影订阅	创建 devtmpfs 文件系统超级块，订阅设备能力注册表事件。	已经注册或后续注册的设备能力都能被统一投影。
驱动上下文	注入 MMIO 映射、中断域、DMA、RTC 钩子等能力。	驱动探测需要这些平台能力才能初始化硬件。
设备发现	从 DTB、ACPI、PCI 等来源创建 PnP 设备并尝试绑定驱动。	此时分配器、VFS 投影和平台能力均已可用。

以 VirtIO 块设备为例，固件或 PCI 扫描创建 PnP 设备并记录 MMIO 或 BAR 资源。PnP 核心选择 VirtIO 块驱动并进入探测流程。驱动映射寄存器，协商设备特性，初始化虚拟队列，构造块设备对象，再发布块设备能力。设备能力注册表发布 Registered 事件后，devtmpfs 投影器生成块设备节点。根文件系统选择逻辑可以在后续阶段从已注册块设备中寻找根盘。

3.9 类型安全控制

设备控制命令是设备模型中容易失控的部分。传统 ioctl 系统调用使用整数命令码和原始指针，编译器无法检查命令与参数类型是否匹配。我们在内核内部更倾向类型化控制，将请求、响应和错误类型绑定在特征接口的关联类型上。用户态仍可通过 ioctl 系统调用进入系统，但无类型命令只停留在 VFS 兼容层，核心设备模型使用结构化请求。

```
trait DriverControl {
    type Request;
    type Response;
    type Error;

    fn control(&self, req: Self::Request) -> Result<Self::Response, Self::Error>;
}
```

代码 3-10 类型安全控制接口

类型化控制的价值在于把接口错误前移。网络设备的 `SetMtu` 请求、RTC 设备的 `ReadTime` 请求、字符设备的 TTY 控制，都可以在内核内部用枚举和结构体表达。调用方构造请求时已经受到类型系统约束，驱动返回的响应也有明确类型。对于通用路径无法表达的专用能力，调用方可以通过 `function_as::<T>` 方法恢复具体设备能力类型，然后调用对应类型化控制。这个入口要显式使用，避免核心特征接口不断加入可选方法。

3.10 工程设计总结

在工程推进的过程中，我们通过多种方式实现了竞态条件和各种边界条件的规避。例如，设备注册和移除最容易出现的问题，通常不在正常路径，而在错误回滚和热插拔边界。我们在 `PnpDevice::register_function` 方法中把事务边界限定在 PnP 设备和全局设备能力注册表之间。驱动构造好类型化设备能力后，先挂入 PnP 设备，再注册到全局注册表。如果全局注册失败，已经挂入 PnP 设备的设备能力会被撤销并标记失效。这样可以避免 PnP 设备认为自己已经暴露能力，但全局注册表中不存在对应记录。移除路径也被拆成固定阶段。先进入 `Removing` 状态，递归移除子设备。再对设备能力调用 `mark_gone` 方法，阻止新的 I/O。随后调用 `drain_io` 方法，尽量收束旧请求。之后才调用驱动 `remove` 方法并释放已拥有资源。这个顺序对应明确的依赖关系。`mark_gone` 方法处理新访问。`drain_io` 方法处理在途请求。`remove` 方法处理硬件关闭。资源释放处理内核对象归还。我们把这些动作固化到框架中，驱动作者不需要在每个驱动里重新证明同一组竞态条件。

我们还通过类型化控制降低了设备专用命令的风险。传统 `ioctl` 系统调用的优点是兼容范围广，问题是命令码和参数指针之间缺少编译期约束。对于简单字符设备，这个问题尚可通通过少量分支控制。对于网络设备、RTC 设备和块设备这类带有复杂状态的对象，无类型命令很容易扩散到驱动内部。我们采用类型化控制，把请求、响应和错误类型绑定在具体设备轨道中。网络设备的 `SetMtu` 请求、RTC 设备的读写时间和告警配置、字符设备的 TTY 控制，都可以在内核内部用枚举和结构体表达。用户态仍然通过 `ioctl` 系统调用进入系统，但解析和兼容停留在 VFS 适配层。设备核心只接收结构化请求。这个设计还有一个附加收益，即专用能力的入口更加显式。通用路径面向设备能力。专用路径通过 `function_as::<T>` 方法恢复具体设备能力类型。调用方必须明确声明自己依赖哪个设备轨道，核心特征接口因此保持稳定。

除此之外，我们还把用户态投影集中在 VFS 设备文件层。早期设备模型很容易把 `/dev` 节点创建逻辑分散到不同驱动或不同文件系统中。短期实现较快，长期会造成投影策略不一致。我们把投影规则收束到设备文件投影器和设备节点规范。投影器根据类型化设备能力生成节点集合。设备节点规范携带打开节点所需的类型化载荷。`devtmpfs` 文件系统只负责创建索引节点和维护目录树。`sysfs` 文件系统与 `procfs` 文件系统使用同一套投影快照生成诊断信息，避免各自向下转换底层设备能力。设备号也被放在投影侧处理。它服务于 POSIX 兼容和用户态观察，不参与内核内部的设备查找。这样一来，用户态 ABI 的演进被限制在 VFS 设备文件层，底层设备能力生命周期不受设备号、节点名或符号链接策略影响。

设备管理子系统充分利用了 Rust 语言的各种核心优势，将传统 POSIX 系统极其容易触发的各种漏洞在编译期就直接规避，大大提高了设备管理子系统的稳定性与驱动中逻辑漏洞的可复现性。对于我们的项目来说，要编写一个驱动程序，只需要

实现对应的特征接口并放进注册表，在 PnP 层就可以通过自动认领的机制自动启用设备及其驱动，甚至可以通过设备能力机制做到自定义设备类型，这大幅增加了设备管理子系统的可拓展性与驱动开发的便捷性，大大便利了社区对操作系统本身生态的完善工作以及设备驱动程序的维护工作。

设备管理子系统具备以下创新：

第一是通过分层抽象与实现机制，把抽象和具体的辩证关系落实到设备模型中。这看似是一个悬空的哲学命题，但在工程实现中，我们发现它对应着非常实际的约束。设备抽象过早统一，会损失硬件能力的真实语义。设备抽象过度贴近每个驱动，又会让生命周期、资源归属和用户态视图分散到各处。在项目早期，我们曾经考虑过接近 POSIX 传统设备模型的路径。这个路径把字符设备和块设备作为基础分类，再用设备号和 `/dev` 节点串联用户态访问。它的兼容语义清晰，VFS 适配也直接。问题出现在网络设备、RTC 设备和 loop 控制设备进入系统之后。网络设备的自然消费者是协议栈。RTC 设备的自然语义是时间、告警和中断。loop 控制设备更接近控制端点。若全部压入字符设备，大量语义会通过 `ioctl` 系统调用旁路表达，内核内部也会被用户态兼容形式牵引。我们也尝试过更极端的类型化对象路径，让每类设备都直接暴露给各自子系统。这个方向保留了能力差异，却分散了全局生命周期。最终我们选择收敛到设备能力。它只抽象所有设备必须共享的内容，例如类别、内部名称、失效标记、I/O 排空入口和类型恢复入口。真实 I/O 语义则留给类型化对象。抽象并非脱离具体的理想模型，具体也不是抽象的被动实例化。两者在工程推进中持续相互塑造。抽象从具体硬件差异中提取共性，具体则依循抽象框架有序展开。

第二是设备管理子系统通过设备能力机制实现了前所未有的高度可拓展性。这里的可拓展性不只体现为可以多注册一个驱动。更重要的含义是新增设备类型时，核心生命周期规则无需重写。我们把拓展点拆成三个层次。第一层是设备类别标识和设备能力，用于让新设备能力进入统一注册表。第二层是类型化对象，用于保存新设备自己的 I/O 语义和控制请求。第三层是 VFS 投影器或类型化消费者，用于决定这类设备能力是否需要用户态文件节点，也可以决定它是否由网络栈、时间子系统等内核模块直接消费。这个结构使网络设备、RTC 设备和 loop 控制设备这类非传统的或设备本身不属于传统 POSIX 建模形式的字符块设备能够沿着同一条路径接入。以 RTC 设备为例，底层驱动发布 RTC 设备能力。devtmpfs 文件系统通过投影器生成 `/dev/rtc*` 兼容入口。时间相关操作仍保留在 RTC 设备对象的类型化控制中。新增设备能力轨道时，PnP 核心仍然只处理探测、移除、失效和资源释放。设备能力注册表仍然只处理类别、名称和事件。devtmpfs 文件系统核心仍然只消费设备节点规范。这种扩展方式把变化限制在新增类型周围，避免修改全局主流程。

第三是硬件生命周期、开放能力和用户态命名空间三者通过 POSIX 兼容层的接入、底层设备抽象的无设备号设计以及零成本抽象三位一体结构实现分离。这个设计最初来自一个实际问题。底层设备探测成功以后，是否必须等 `/dev` 节点也创建成功，才能认为设备注册成功。若把两者绑在一起，devtmpfs 文件系统的命名冲突或临时内存不足就会导致硬件探测回滚。若完全分开，又需要能够诊断用户态节点缺失的原因。当前设计把 PnP、设备能力注册表和投影层分成三个阶段。PnP 设备表达硬件身份、资源归属和驱动绑定。设备能力注册表表达驱动向内核开放的能力。devtmpfs 文件系统、sysfs 文件系统和 procfs 文件系统表达用户态可见视图。设备能力注册成功后，底层设备已经可以被内核内部消费。投影失败只记录在投影状态中，供 procfs 文件系统和 sysfs 文件系统诊断。这个边界让启动顺序更宽松。设备可以早于 `/dev` 挂

载完成注册。`devtmpfs` 文件系统可以在订阅安装后补齐已有设备能力。用户态名字空间也可以在后续挂载阶段接入。单向依赖因此得到保持，设备核心不需要反向依赖 VFS。

从工程实践看，这些创新共同支撑了设备子系统的长期演化能力。设备数量增加时，核心生命周期规则保持稳定。新增设备能力轨道时，已有字符和块设备路径不需要重写。用户态 ABI 需要补充时，修改集中在投影层。单向依赖避免了循环依赖。类型化接口降低了控制命令的误用概率。分阶段移除让热插拔和错误回滚具备清晰的分析边界。设备管理层的可扩展性并不是单个特征接口带来的结果，而是抽象边界、事务边界和投影边界共同作用的结果。

第四章 虚拟文件系统与 POSIX 兼容层

在第三章中, 设备模型把硬件能力整理为设备能力, 并通过投影层把一部分设备能力落实到 `/dev`、`sysfs` 文件系统和 `procfs` 文件系统等用户可见视图中。到了本章, 问题从设备能力的发布转向了用户态接口的统一。用户程序不应直接理解块设备队列、字符设备缓冲区、管道环形队列或进程状态表。它需要的是一套稳定的 POSIX 文件接口。虚拟文件系统承担这个边界, 把多种异构资源整理为路径、文件描述符和文件操作。

VFS 面对的异构性不亚于设备模型。磁盘文件有持久化数据和目录层级。`tmpfs` 内存文件系统的文件完全位于内存中。`procfs` 文件系统的文件在读取时动态生成文本。`devtmpfs` 文件系统节点承接设备能力的用户态投影。管道没有目录中的长期身份, 却仍然需要一个文件对象来表达读端和写端。若 VFS 只关注普通文件, 系统调用层就会被迫为每类资源开辟专用路径。若 VFS 试图把所有资源压成相同的读写语义, 又会损失设备控制、挂载边界、共享偏移和 `unlink-but-open` 等关键语义。

我们在 VFS 中采用的核心思路, 是把稳定 ABI 与内部对象模型分开。路径解析、权限检查和文件描述符管理由 VFS 统一处理。具体文件系统和设备适配器只实现自己负责的对象语义。打开动作是二者的交接点。它把一个路径名转换为带有访问能力的文件对象, 并把后续操作限制在这个已冻结的能力边界内。这样做既能满足 POSIX 的兼容要求, 也能让 `tmpfs` 内存文件系统、`devtmpfs` 文件系统、`procfs` 文件系统和 `pipefs` 管道文件系统按各自的内部规律演化。

4.1 设计目标与约束

VFS 的设计目标可以分为四类。第一是命名空间稳定。路径名必须能在目录树、挂载点和符号链接之间得到确定解析, 并且要遵守进程可见根与当前工作目录。第二是对象身份稳定。一个文件可以被重命名, 可以存在多个硬链接, 也可以在删除后继续被已打开的文件描述符访问。第三是能力边界稳定。打开时通过权限检查后, 后续读写不能再被路径替换影响。第四是扩展边界稳定。新增文件系统或设备节点适配器时, 不应修改系统调用层和文件描述符表。

表 4-1 VFS 的设计目标

目标	设计含义	关键约束
命名与对象分离	目录项描述路径分量和父子关系, 索引节点描述文件对象本体。	<code>rename</code> 、 <code>link</code> 和 <code>unlink</code> 都不能破坏已打开文件的对象身份。
打开能力冻结	文件对象在 <code>open</code> 时保存访问模式和凭据快照。	后续 I/O 只检查文件对象能力, 避免路径 TOCTOU 窗口。

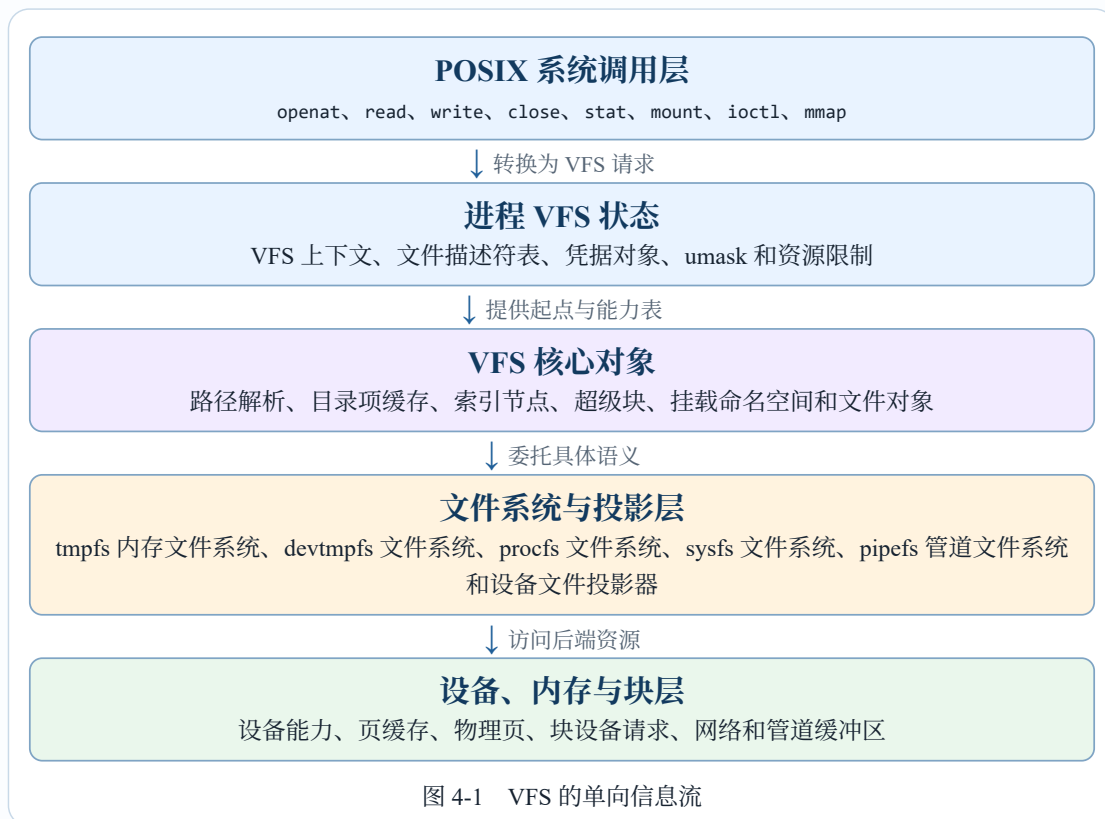
续表 4-1 VFS 的设计目标

目标	设计含义	关键约束
挂载边界显式	挂载命名空间维护挂载索引, 路径解析跨越挂载点时切换挂载对象。	.. 必须同时处理目录父节点和挂载父节点, 且不能越过进程根。
驱动扩展收敛	文件系统实现索引节点操作集 (InodeOps) 与文件操作集 (FileOps), 设备节点通过投影器注入节点载荷。	系统调用层只消费 VFS 通用接口, 不下沉到具体文件系统或设备类型。

这些目标中, 最容易被低估的是打开能力冻结。POSIX 程序常常先打开路径, 再把文件描述符交给另一个线程或子进程。此后这个描述符应当代表一次已经完成的授权, 而不是每次读写都重新追踪路径。路径可以被其他任务重命名或替换, 文件权限也可以改变。若读写时重新解析路径, 程序会得到不稳定的对象。若读写时重新检查路径权限, 攻击者就可能在检查和使用之间替换目录项。我们把权限检查和路径解析集中在打开阶段, 后续调用只面对文件对象。

4.2 总体分层结构

VFS 被拆成五个层次。最上层是 POSIX 系统调用翻译层, 它把 `openat`、`read`、`write`、`stat`、`mount` 和 `ioctl` 等调用转换为 VFS 请求。第二层是 VFS 上下文和文件描述符表, 它保存进程可见根、当前目录、凭据、`umask` 和已打开文件。第三层是路径解析、目录项缓存、索引节点管理和挂载命名空间。第四层是具体文件系统驱动。第五层是设备、内存和块层后端。



这条信息流保持单向依赖。系统调用层依赖 VFS。VFS 依赖文件系统驱动和设备投影。具体设备不反向依赖系统调用层。devtmpfs 文件系统会观察第三章中的设备能力注册表，但设备探测是否成功不由 /dev 节点创建结果决定。这个边界使启动顺序更宽松。设备可以先注册设备能力，devtmpfs 文件系统在挂载后再补齐节点。VFS 也可以在没有某类特殊文件系统时继续管理普通文件。

4.3 索引节点与超级块

索引节点 (Inode) 表达文件系统对象的身份。它包含跨文件系统唯一的索引节点编号 (InodeId)、对象类型、设备号、块大小和指向索引节点操作集 (InodeOps) 的操作表。可变元数据集中在索引节点元数据 (InodeMeta) 中，包括大小、权限、所有者、时间戳和硬链接计数。大小和链接计数同时保存为原子镜像，供热路径快速读取。超级块 (Superblock) 表达一次文件系统实例，它保存文件系统类型、实例标识、设备号、根索引节点、根目录项和索引节点缓存。

```

struct InodeId {
    fs_id: u32,
    ino: u64,
}

struct InodeMeta {
    size: u64,
    nlink: u32,
    mode: u16,
    uid: u32,
  
```

```

        gid: u32,
        atime: Timespec,
        mtime: Timespec,
        ctime: Timespec,
        blocks: u64,
    }

    struct Inode {
        id: InodeId,
        kind: InodeKind,
        rdev: DevId,
        blksize: u32,
        meta: Spinlock<InodeMeta>,
        cached_size: AtomicU64,
        cached_nlink: AtomicU32,
        lifecycle: AtomicU8,
        ops: Arc<dyn InodeOps>,
        superblock: Weak<Superblock>,
    }

    struct Superblock {
        fs_type: FsType,
        fs_id: u32,
        dev_id: DevId,
        block_size: u32,
        root_inode: Arc<Inode>,
        root_dentry: Arc<Dentry>,
        inode_cache: InodeCache,
        ops: Arc<dyn SuperblockOps>,
    }

```

代码 4-1 索引节点与超级块的职责边界

这种拆分背后的原因,是文件对象的身份和文件系统实例的身份需要分别维护。同一个超级块内部的索引节点编号可以由驱动决定。tmpfs 内存文件系统可以使用自己的分配器,磁盘文件系统可以沿用磁盘上的编号,devtmpfs 文件系统可以根据投影对象生成编号。VFS 只要求 fs_id 与 ino 的组合全局唯一。这样既避免全局索引节点号分配器成为竞争点,也允许驱动保留自己的内部结构。

索引节点缓存保存在超级块中,并使用弱引用。弱引用可以加速同一编号的重复查找,却不阻止对象回收。这个选择与生命周期语义有关。目录项、文件对象和正在执行的文件系统操作都可能持有索引节点的强引用。超级块的缓存若也持有强引用,就会把所有访问过的索引节点永久保活。弱引用让缓存成为索引,而不是所有权来源。查找时弱引用仍然有效则升级为强引用,失效时清理缓存并重新交给驱动创建对象。

4.4 目录项与路径缓存

目录项 (Dentry) 表达命名空间中的一个位置。它由父目录、分量名称和可选索引节点组成。正向目录项表示名称存在,负向目录项表示名称不存在,失效目录项表示缓存结论需要重新验证。这个结构让路径解析可以缓存成功查找,也可以缓存失败查找。对于 shell 按 PATH 搜索命令这种场景,负向缓存能减少大量重复失败查询。

```

enum DentryState {
    Positive,
    Negative,
    Invalid,
}

struct DentryMeta {
    name: SmallStr,
    parent: Option<Arc<Dentry>>,
}

struct Dentry {
    inode: Option<Arc<Inode>>,
    state: AtomicU8,
    meta: Spinlock<DentryMeta>,
}

struct DentryCache {
    shards: [Spinlock<DentryShard>; 16],
}

fn lookup_cached(parent: &Arc<Dentry>, name: &str) -> Option<Arc<Dentry>> {
    let shard = hash(parent.as_ptr(), name) % 16;
    let guard = DCACHE.shards[shard].lock();
    guard.get(parent, name).filter(|d| !d.is_invalid())
}

```

代码 4-2 目录项状态与分片缓存

父目录项与名称共同构成缓存键。只用字符串作为键是不够的, 因为不同目录下可以有相同名称。只用索引节点作为键也不够, 因为同一个索引节点可以通过多个硬链接出现在不同路径。目录项的键恰好对应路径解析的局部步骤, 即在某个父目录下寻找一个名称。分片哈希表降低了并发查找时的锁竞争。名称使用小字符串 (SmallStr) 保存, 短路径分量直接内联在对象中, 避免为常见短名称进入堆分配路径。

负向缓存的失效策略需要谨慎。父目录发生创建、链接或重命名后, 旧的“不存在”结论可能失效。VFS 在这些写操作成功后, 会让相关目录下的负向条目失效。这样做没有追求全局精确失效, 因为精确追踪会让每次目录写入扫描更多结构。当前策略把代价放在下一次查找上。若缓存已失效, 路径解析重新进入驱动。若目录没有变化, 负向结论可以直接复用。

4.5 路径解析与挂载跨越

路径解析从 VFS 上下文 (VfsContext) 中取得起点。绝对路径从进程可见根开始, 相对路径从当前工作目录开始。解析器逐个分量前进, 并在每一步处理目录项缓存、底层 lookup 操作、符号链接、挂载跨越和 .. 回退。最后一个分量还需要结合打开标志决定是否允许缺失、是否跟随符号链接, 以及是否必须是目录。

```

fn lookup(ctx: &VfsContext, dirfd: Dirfd, path: UserPath, flags:
LookupFlags)

```

```

-> Result<LookupResult, Errno>
{
    let mut cursor = choose_start(ctx, dirfd, path)?;
    let mut symlink_depth = 0;

    for component in path.components() {
        if component == "." {
            continue;
        }

        if component == ".." {
            cursor = step_parent_without_crossing_root(ctx, cursor);
            continue;
        }

        let child = match lookup_cached(&cursor.dentry, component) {
            Some(dentry) => dentry,
            None => cursor.inode().ops.lookup(&cursor.dentry, component)?,
        };

        let mut next = enter_mount_if_needed(cursor.mount_ns(), child);

        if next.is_symlink() && should_follow(component, flags) {
            symlink_depth += 1;
            if symlink_depth > MAX_SYMLINKS {
                return Err(ELOOP);
            }
            next = resolve_symlink_target(ctx, next, flags)?;
        }

        cursor = next;
    }

    Ok(cursor)
}

```

代码 4-3 路径解析的核心控制流

.. 的处理是路径解析中最容易出错的部分。普通目录树中，.. 只需要回到父目录项。挂载树中，如果当前目录项是某个挂载对象的根，.. 需要先退出当前挂载对象，再回到挂载点所在的父挂载对象。进程可见根又给这个过程加了一层限制。解析器到达进程根后必须停止回退，不能越过 chroot 或容器视图的边界。我们使用目录项指针和挂载对象指针判断边界，避免用字符串比较路径，后者既慢也难以处理符号链接和重复斜杠。

符号链接展开同样需要上下文。绝对目标从进程可见根重新解析。相对目标从符号链接所在目录解析。解析器维护深度计数，超过阈值后返回 ELOOP。这个限制看似简单，却是抵御恶意循环链接的必要条件。没有深度限制时，一个普通 stat 就可能把内核困在无限递归中。

4.6 文件对象与能力冻结

文件对象(`File`)表达一次已经打开的文件能力。它保存不可变的打开选项、可变的标志、调用者凭据快照、共享文件偏移、数据操作接口、打开时的目录项和所在挂载对象。路径解析和权限检查在创建文件对象之前完成。文件对象创建以后, 后续系统调用通过文件描述符找到它, 并根据冻结的访问能力调用文件操作集(`FileOps`)。

```
struct OpenOptions {
    readable: bool,
    writable: bool,
    append: bool,
    directory: bool,
    nofollow: bool,
}

struct File {
    options: OpenOptions,
    status_flags: AtomicU32,
    cred: Arc<Credentials>,
    pos: AtomicU64,
    pos_lock: Spinlock<>,
    ops: Box<dyn FileOps>,
    dentry: Arc<Dentry>,
    mount: Arc<Mount>,
}

fn read(file: &Arc<File>, buf: UserBuf) -> Result<usize, Errno> {
    if !file.options.readable {
        return Err(EBADF);
    }

    let _pos_guard = file.pos_lock.lock();
    let old = file.pos.load(Ordering::Relaxed);
    let done = file.ops.read_at(old, buf)?;
    file.pos.store(old + done as u64, Ordering::Relaxed);
    Ok(done)
}
```

代码 4-4 文件对象与能力冻结

能力冻结解决的是路径 TOCTOU 问题。攻击者可以在权限检查后替换路径指向, 也可以重命名上级目录。若读写操作每次都重新走路径, 文件描述符就无法稳定代表打开时的对象。文件对象保存目录项、挂载对象和凭据快照, 后续操作只面对这个对象。可变的 `O_NONBLOCK`、`O_APPEND` 等状态位则保存在 `status_flags` 中, 由 `fcntl` 调整。访问模式不可变, 状态标志可变, 这个划分符合 POSIX 对打开文件描述的语义。

共享偏移使用位置锁串行化。单纯把偏移放进原子变量不足以满足语义, 因为一次普通 `read` 包含读取旧偏移、执行 I/O、根据实际长度推进偏移三个步骤。多个线程共享同一个文件描述符时, 这三个步骤必须整体有序。显式偏移接口如 `pread` 和 `pwrite` 不修改共享偏移, 因此可以绕过位置锁。

4.7 文件描述符表与进程文件视图

文件描述符表 (FdTable) 把整数编号映射为文件对象。编号分配遵守最小可用规则。描述符级标志如 CLOEXEC 保存在表项中, 而不是保存在文件对象中。这样同一个文件对象可以被 dup 复制成多个描述符, 每个描述符拥有独立的关闭时继承语义, 但共享文件偏移和文件状态。

```
struct FdEntry {
    file: Arc<File>,
    flags: FdFlags,
}

struct FdTable {
    inner: Spinlock<FdTableInner>,
}

struct FdTableInner {
    entries: Vec<Option<FdEntry>>,
    bitmap: Vec<u64>,
    soft_limit: u32,
    hard_limit: u32,
}

fn alloc_fd(table: &FdTableInner, start: u32) -> Result<u32, Errno> {
    for word_index in word_range_from(start) {
        let free = !table.bitmap[word_index];
        if free != 0 {
            let bit = free.trailing_zeros();
            let fd = word_index as u32 * 64 + bit;
            if fd < table.soft_limit {
                return Ok(fd);
            }
        }
    }
    Err(EMFILE)
}

fn close_on_exec(table: &FdTable) -> Vec<FdEntry> {
    let mut guard = table.inner.lock();
    let files = guard.take_entries_with(FdFlags::CLOEXEC);
    drop(guard);
    files
}
```

代码 4-5 文件描述符表的分配与批量关闭

关闭路径采用锁内摘取、锁外释放。FileOps::release 可能进入设备驱动, 也可能触发等待队列或回写。若在持有文件描述符表锁时执行这些动作, 其他线程的 open、dup 和 close 都会被长时间阻塞。锁内只修改表结构并取出文件对象强引用 (Arc<File>), 真正的释放发生在锁外。这个原则也用于 execve 的 CLOEXEC 批量关闭。

fork 和 clone 对文件描述符表的处理由第五章的侧表钩子完成。普通 fork 复制描述符表, 但每个条目仍然引用同一个文件对象, 所以父子共享文件偏移。CLONE_FILES

则让两个任务直接共享同一个文件描述符表，任一方的打开和关闭对另一方可见。VFS 不需要理解任务创建细节，只提供可复制和可共享的对象边界。

4.8 挂载命名空间与超级块实例

挂载把一个超级块的根目录叠加到命名空间中的某个目录项上。挂载对象（Mount）保存超级块、挂载根、挂载位置、子挂载列表、挂载标志和打开计数。挂载命名空间（MountNamespace）保存一组挂载对象，并维护从挂载点到子挂载对象的索引。路径解析跨越挂载点时，通过这个索引从父挂载对象切换到子挂载对象。

```
struct Mount {
    superblock: Arc<Superblock>,
    mount_root: Arc<Dentry>,
    location: Spinlock<MountLocation>,
    children: Spinlock<Vec<Arc<Mount>>>,
    flags: AtomicU32,
    open_count: AtomicUsize,
}

struct MountData {
    root: Arc<Mount>,
    mounts: Vec<Arc<Mount>>,
    by_mountpoint: Map<MountpointKey, Vec<Arc<Mount>>>,
    by_root: Map<DentryKey, Arc<Mount>>,
}

struct MountNamespace {
    data: Spinlock<MountData>,
}

fn mount_at(ns: &MountNamespace, at: LookupResult, sb: Arc<Superblock>,
flags: MountFlags)
-> Result<(), Errno>
{
    let mount = Mount::new(sb, at.dentry, flags);
    let mut data = ns.data.lock();
    data.attach(at.mount, mount)?;
    data.rebuild_indexes_for_new_mount();
    Ok(())
}
```

代码 4-6 挂载命名空间的索引结构

挂载标志属于挂载实例，而不属于超级块。同一个文件系统对象可以在不同位置以不同标志挂载。一个位置可以只读，另一个位置可以允许执行。若把这些标志放进超级块，就无法表达同一实例多次挂载的差异。VFS 在写入、执行和设备访问检查时读取当前挂载对象的标志，从而把文件系统内容和命名空间策略分离。

挂载命名空间使用单锁保护挂载集合。挂载和卸载并非热路径，路径解析中的挂载点查询只短暂持锁。单锁使 pivot_root、重新挂载和卸载的状态变化更容易保持原子。这里没有追求细粒度锁，因为挂载树操作天然跨越多个挂载对象。拆成多把锁后，正确性证明会显著变难，死锁风险也会增加。

卸载前需要繁忙检测。文件对象保存所在挂载对象, 打开时递增挂载对象的打开计数, 释放时递减。卸载读取计数即可发现仍有活跃文件。它不需要扫描所有进程的文件描述符表。这个设计把全局搜索转化为局部引用计数, 代价是在打开和关闭路径增加一次原子操作。

4.9 文件系统驱动与特殊文件系统

具体文件系统通过文件系统驱动 (FsDriver) 注册到全局注册表。挂载时, VFS 根据类型名找到驱动, 由驱动创建超级块、根索引节点和根目录项。索引节点上的索引节点操作集负责 lookup、create、mkdir、unlink、rmdir、symlink、link 和 open 等操作。打开成功后, 驱动返回文件操作集, 后续读写和控制操作就进入文件数据路径。

tmpfs 内存文件系统用内存中的目录映射保存名称关系, 用页或缓冲区保存文件数据。它适合作为临时目录和共享内存后端。procfs 文件系统和 sysfs 文件系统的重点是按需生成。它们的文件内容来自读取时的内核状态格式化结果, 而非持久化存储。pipefs 管道文件系统则主要服务于内核内部对象建模。管道不需要挂载到用户可见目录, 但它仍然需要索引节点、文件对象和文件操作集, 以便系统调用层用同一条路径处理读写、poll 和 close。

devtmpfs 文件系统延续第三章的投影设计。它不通过主次设备号反查驱动, 而是在节点载荷中保存类型化设备能力或适配器所需的对象引用。打开 /dev/null、/dev/tty、块设备节点或 RTC 节点时, devtmpfs 文件系统根据节点载荷构造对应的文件操作集。设备号仍然存在, 但它主要服务于 POSIX 的 stat 结果和用户态兼容, 不再是内核内部寻找设备的主路径。

表 4-2 特殊文件系统的对象来源

文件系统	对象来源	VFS 边界
tmpfs 内存文件系统	内存中的目录项、元数据和文件数据。	提供普通文件、目录和共享内存对象。
devtmpfs 文件系统	第三章设备能力注册表的投影事件和设备节点规范 (DevNodeSpec)。	把类型化设备能力落实为可打开的设备节点。
procfs 文件系统	任务表、内存统计、挂载状态和内核诊断信息。	读取时动态生成内容, 目录项可随状态变化。
pipefs 管道文件系统	管道对象的环形缓冲区和读写端引用计数。	不依赖用户可见挂载点, 但使用统一文件对象模型。

4.10 VFS 上下文与跨子系统交接

VFS 上下文 (vfsContext) 是进程可见文件系统状态的集合。它保存当前目录、进程根目录、挂载命名空间、凭据、umask 和资源限制。第五章中的 clone 根据标志决定共享或复制它。CLONE_FS 共享当前目录和根目录, 普通 fork 则复制上下文。凭据更新时, 调度层会同步替换 VFS 上下文中的凭据快照, 使权限检查消费稳定对象。

VFS 与内存管理的交接主要发生在 `mmap` 和页故障。`mmap` 把文件对象转化为进程地址空间中的 VMA。页故障到来时，内存管理根据 VMA 找回文件和偏移，再请求文件系统提供对应内容。VFS 不直接操作用户页表，内存管理也不直接修改文件系统命名空间。二者通过文件对象和页故障回调交接。

VFS 与设备模型的交接发生在 `devtmpfs` 文件系统和设备文件投影器。设备模型发布设备能力。投影层生成设备节点规范。`devtmpfs` 文件系统创建节点并在打开时构造文件操作集。这个过程保持单向。设备移除时，设备能力先进入失效状态。已打开文件的后续 I/O 会在适配器中看到设备不可用。新打开则会被节点状态或适配器拒绝。这样可以保证热插拔时用户态名字空间和底层设备生命周期不会互相撕裂。

4.11 工程设计总结

虚拟文件系统子系统的设计集中处理了三个长期存在的工程矛盾。第一个矛盾是路径名的灵活性与对象身份的稳定性。路径可以移动、重命名和被多个硬链接引用，但已打开文件必须继续指向同一个对象。第二个矛盾是统一 ABI 与资源异构性。用户态希望所有资源都能通过文件描述符访问，内核内部却需要保留普通文件、设备、管道和动态状态文件的差异。第三个矛盾是缓存性能与一致性。目录项和索引节点缓存必须加速热路径，同时不能让 `rename`、`unlink`、`mount` 和设备移除留下错误结论。

虚拟文件系统子系统具备以下创新：

第一是把命名关系、对象身份和打开能力拆成三个层次。这个拆分来源于本内核的并发和生命周期问题，并在传统 VFS 概念之上重新校准了边界。目录项只描述父目录下的名称关系，因此可以安全缓存正向和负向查找。索引节点只描述文件对象本体，因此可以在多个目录项和多个文件对象之间共享。文件对象则描述一次已经完成授权的打开能力，因此可以在路径变化后继续稳定工作。早期若把路径和文件对象直接绑定，`rename` 与 `unlink-but-open` 的语义会变得难以实现。若把所有状态都放进索引节点，描述符级标志和共享偏移又会污染对象本体。当前分层使每个对象只承担一种生命周期。命名空间变化只影响目录项。对象回收只影响索引节点。读写能力只影响文件对象。这个边界让复杂 POSIX 语义可以通过组合得到，无需在每个系统调用里重新判断特殊情形。

第二是缓存和回收围绕引用关系设计，而不是围绕全局扫描设计。目录项缓存使用分片降低锁竞争。索引节点缓存使用弱引用避免把历史对象永久保活。挂载对象的繁忙检测通过打开计数完成，不扫描所有任务的文件描述符表。`unlink` 后的索引节点先从命名空间消失，再等最后一个强引用释放后调用驱动的驱逐操作。这个设计把大范围遍历转化为局部引用变化，热路径开销更可控。事后回顾，这种引用关系也让 `open`、`close`、`rename` 和 `unlink` 等高频组合路径更容易分析，因为每个对象的所有权来源是清楚的。

VFS 向上可以提供稳定的 POSIX 兼容层，向下可以容纳不同文件系统和设备投影，向侧面与进程、内存和设备管理保持清晰交接。路径解析、打开能力、挂载命名空间和对象回收分别处在各自边界内。新增文件系统时，只需要实现驱动接口。新增设备节点时，只需要补充投影器和适配器。新增进程隔离语义时，可以从 VFS 上下文和挂载命名空间扩展。VFS 因此成为用户态观察内核资源的结构化边界与整个操作系统的 POSIX 兼容层，而不仅仅是一个虚拟“文件系统”。

第五章 进程与线程管理

在第四章中，VFS 通过 VFS 上下文和文件描述符表为用户程序提供了稳定的文件系统视图。文件描述符、当前目录、挂载命名空间和凭据都需要依附在某个执行实体上。内存管理中的用户虚拟地址空间、信号处理中的阻塞掩码、调度器中的运行状态也有同样需求。本章讨论的进程与线程管理子系统，正是这些资源的组织中心。它需要回答三个问题。谁在运行。它拥有哪些资源。它如何创建、退出、等待和接收异步事件。

传统叙述常把进程和线程分成两套对象。我们在实现中采用统一的任务对象（Task）抽象。任务对象是调度器管理的最小实体，也是 POSIX 线程语义的落点。它可以代表一个拥有独立地址空间和文件表的进程，也可以代表共享地址空间和信号处理表的线程。二者的区别不由结构体类型决定，而由 clone 时选择共享哪些资源决定。这个设计使调度、等待、信号和退出都围绕同一个生命周期状态机展开。

统一任务对象以后，还需要处理 POSIX ABI 对整数 PID 的要求。内核内部不把 PID 作为任务主身份。任务身份由任务强引用（Arc<Task>）表达，父子关系、等待队列、信号投递和调度队列都围绕引用对象运行。PID 命名层只在系统调用和 procfs 文件系统等 ABI 边界提供整数名字。这个分层让热路径避开全局 PID 查找，也为后续 PID 命名空间留出空间。

5.1 设计目标与约束

进程与线程管理的约束来自三条路径。第一条是调度路径。调度器需要快速读取状态、切换内核上下文并唤醒任务，不能被 VFS、虚拟内存或 PID 命名细节拖慢。第二条是 POSIX 兼容路径。fork、clone、waitpid、kill、setuid 和信号处理都要求稳定的 ABI 语义。第三条是资源生命周期路径。任务退出时，虚拟内存、文件表、健壮 futex 机制、clear_child_tid 地址、PID、线程组和父子列表必须按固定顺序收束。

表 5-1 进程与线程管理的设计目标

目标	设计含义	关键约束
统一执行实体	任务对象同时承载进程和线程语义。	调度器只面对任务对象，资源共享由 clone 标志和侧表钩子决定。
PID 边界后移	整数 PID 是 ABI 名字，内部身份由任务强引用表达。	PID 登记表保存弱引用，不能反向保活任务。
生命周期分阶段	退出、僵尸状态、wait 回收和最终释放分开处理。	退出前钩子必须早于虚拟内存和文件描述符表清理，wait 可见状态必须保留到父回收。

续表 5-1 进程与线程管理的设计目标

目标	设计含义	关键约束
跨子系统解耦	VFS、文件描述符表、用户虚拟地址空间和架构状态通过任务扩展接入。	调度核心不直接依赖 VFS 和内存管理的内部结构。

这些目标决定了任务对象不能成为所有子系统字段的简单堆叠。若把 VFS、虚拟内存、套接字、ptrace 和架构扩展全部写成固定字段，调度 crate 会依赖大量上层模块。若完全使用动态表，系统调用热路径又会因为频繁查找扩展项而变慢。当前实现采用折中方案。固定热槽位保存高频扩展，低频扩展继续使用类型擦除表。这样既保留了可扩展性，也避免 read、write、mmap 等热路径每次都锁住扩展向量。

5.2 任务对象核心身份模型

任务对象的主身份是引用计数对象。父任务的子列表持有子任务强引用，运行队列和当前 CPU 也会在调度期间持有强引用。等待队列只保存弱引用。任务退出后进入僵尸状态，仍留在父任务的子列表中，直到父任务执行 wait 回收。父任务先退出时，子任务被移交给 init 任务。这个关系使僵尸状态可观察，也避免等待队列保活已经退出的任务。

```
enum TaskState {
    New,
    Runnable,
    Running,
    Sleeping,
    Uninterruptible,
    Stopped,
    Continued,
    Zombie,
    Dead,
}

enum TaskKind {
    User,
    KernelThread,
    Idle,
}

struct Relations {
    parent: Weak<Task>,
    children: Vec<Arc<Task>>,
    thread_group: Arc<ThreadGroup>,
    process_group: Arc<ProcessGroup>,
    pid_in_ns: Vec<(Arc<PidNamespace>, PidT)>,
}

struct Task {
    sched: SchedEntity,
    kind: AtomicU8,
```

```

state: AtomicU8,
rel: Spinlock<Relations>,
exit_waiters: WaitQueue,
signal: SignalState,
shared_signal: Spinlock<Arc<SharedSignal>>,
creds: Spinlock<Arc<Credentials>>,
hot_ext: HotTaskExt,
ext: Spinlock<Vec<TaskExt>>,
}

```

代码 5-1 任务对象的核心字段

亲缘关系集中在一把锁下。父指针、子列表、线程组、进程组和 PID 登记副本同时受到保护。这个粒度看起来偏粗，但它避免了父任务锁、子任务锁、线程组锁和 PID 锁之间的反序风险。亲缘关系操作本身并非系统调用热路径。`fork`、`wait`、`setpgid` 和重新收养的频率远低于调度节拍和文件 I/O。我们更重视这一部分的可推理性。

任务状态由原子值保存。唤醒路径可以通过比较交换操作把睡眠状态或不可中断睡眠状态转为可运行状态，避免每次读取状态都进入运行队列锁。状态机限制了不合法转换。僵尸任务不会被唤醒回可运行状态。死亡任务只等待最后引用释放。停止状态和继续状态则服务于作业控制和 `wait` 可观察事件。

5.3 PID 命名层、线程组与进程组

PID 命名层把任务强引用映射为整数 PID。每个 PID 命名空间 (`PidNamespace`) 维护自己的登记表，槽位中保存弱引用。弱引用失效后，后续查找会自然失败，父进程回收僵尸任务时再归还 PID。任务内部保存自己在各命名空间中的 PID 副本，根命名空间的 PID 对应当前系统调用可见的 `getpid`、`kill` 和 `waitpid` 语义。

```

struct PidNamespace {
    parent: Option<Arc<PidNamespace>>,
    level: u32,
    registry: PidRegistry,
}

struct PidSlot {
    task: Weak<Task>,
    generation: u32,
}

fn resolve_pid(ns: &PidNamespace, pid: PidT) -> Option<Arc<Task>> {
    let weak = ns.registry.lookup(pid)?;
    weak.upgrade()
}

fn task_pid_in_ns(task: &Task, ns: &Arc<PidNamespace>) -> Option<PidT> {
    let rel = task.rel.lock();
    rel.pid_in_ns
        .iter()
        .find(|(entry_ns, _)| Arc::ptr_eq(entry_ns, ns))
        .map(|(_, pid)| *pid)
}

```

代码 5-2 PID 命名层的边界

线程组和进程组是独立对象。线程组表达共享信号处理语义, 组内成员共享共享信号状态 (SharedSignal), 包括信号动作表和共享待处理队列。进程组服务于作业控制。终端向前台进程组发送信号时, 内核可以遍历进程组成员并投递。会话再组织多个进程组, 并保存控制终端等状态。成员索引使用弱引用, 避免组对象反向保活任务。

这个结构让 PID、线程组、进程组和会话各自承担一类 ABI 语义。PID 解决命名。线程组解决同一进程内部线程之间的信号共享。进程组解决 shell 作业控制。会话解决登录和控制终端边界。它们关联在任务对象的亲缘锁下, 但生命周期不混成同一个概念。

5.4 子任务与资源共享

clone 是创建新任务的统一入口, fork 和 vfork 都是它的特例。克隆参数 (CloneArgs) 中的标志与 Linux UAPI 对齐。系统调用层解析用户 ABI 后, 把标志、用户栈、TLS、tid 地址、pidfd 和请求 PID 交给调度层。调度层再根据标志决定父子之间共享哪些资源。

表 5-2 常用 clone 标志与资源语义		
标志	语义	交给谁落实
CLONE_VM	父子共享用户地址空间。	虚拟内存扩展钩子决定共享或 fork 写时复制。
CLONE_FS	共享 VFS 上下文。	VFS 扩展钩子共享或复制 VFS 上下文。
CLONE_FILES	共享文件描述符表。	文件描述符表扩展钩子共享或深拷贝描述符表。
CLONE_SIGHAND	共享信号动作表和共享待处理队列。	任务创建路径安装共享信号状态。
CLONE_THREAD	加入父线程组。	线程组对象和 PID/TGID 规则共同约束。
CLONE_VFORK	父任务阻塞到子任务 exec 或 exit。	子任务的 vfork_done 等待队列负责唤醒父任务。

创建流程可以拆成四步。第一步创建任务对象, 建立父子关系、线程组和进程组关系, 并在 PID 命名层登记整数名字。第二步复制或共享扩展状态。调度核心不理解 VFS 上下文、文件描述符表和用户虚拟地址空间的内部结构, 而是调用内核在启动期注册的任务扩展克隆钩子 (TaskExtCloneHook)。第三步构造用户上下文。架构层复制或调整陷阱帧, 处理子栈、TLS 和子线程标识地址。第四步安装内核执行体, 把子任务放入运行队列。

```
fn do_clone(parent: &Arc<Task>, args: CloneArgs) -> Result<PidT, Errno> {
    validate_clone_flags(args.flags)?;

    let groups = choose_groups(parent, args.flags);
    let child = Task::new(default_sched_params(), Arc::downgrade(parent),
        groups.tg, groups.pg);

    parent.add_child(Arc::clone(&child));
    register_pid_for_child(&child, args.requested_pid)?;

    for ext in parent.snapshot_exts() {
        let cloned = EXT_CLONE_HOOK.clone_for(parent, &child, ext.key,
            &ext.payload, args.flags);
        child.ext_install(ext.key, cloned);
    }

    process_clone_user_context(parent, &child, args)?;
    child.into_kernel_thread(user_clone_entry, 0);
    enqueue_task(Arc::clone(&child), now_ns_public());

    if args.flags.has(CLONE_VFORK) {
        parent_sleep_until_child_exec_or_exit(&child);
    }

    Ok(child.pid_root().unwrap_or(0))
}
```

代码 5-3 clone 的分阶段控制流

这里最重要的设计点是拷贝策略外置。VFS 知道 `CLONE_FS` 对当前目录和根目录意味着共享还是复制。文件描述符表知道 `CLONE_FILES` 下描述符项如何保留共享文件对象。内存管理知道 `CLONE_VM` 与 `fork` 写时复制的差异。调度器只负责按标志调用钩子。这个边界避免了调度器随着子系统增加而持续膨胀。

`vfork` 的时序需要单独说明。它让子任务临时共享父地址空间，并让父任务阻塞到子任务 `exec` 或 `exit`。这个阻塞不是普通 `wait`。父任务等待的是子任务释放地址空间共享风险的时刻，而不是等待子任务结束。我们用 `vfork_done` 等待队列表达这个事件。`exec` 和 `exit` 路径都会唤醒父任务，避免父子同时修改共享地址空间。

5.5 执行上下文与架构交接

任务对象本身保持架构无关。内核栈由通用层分配，架构上下文缓冲区的大小和对齐由架构上下文接口 (`ArchContextOps`) 注入。新建内核线程时，通用层分配内核栈 (`KernelStack`) 和架构上下文槽 (`ArchContextSlot`)，再调用架构层的 `init_kernel_context` 设置首次恢复入口。上下文切换时，调度器只把两个上下文指针交给 `switch_context` 汇编例程。

```
struct ArchContextOps {
    context_size: usize,
    context_align: usize,
    init_kernel_context: unsafe fn(ctx: NonNull<u8>, stack_top: usize,
        entry: KernelEntry, arg: usize),
```

```

        switch_context: unsafe fn(prev: NonNull<u8>, next: NonNull<u8>),
    }

    fn into_kernel_thread(task: &Arc<Task>, entry: KernelEntry, arg: usize) {
        let stack = KernelStack::new();
        let ctx = ArchContextSlot::new_from_registered_ops();
        let stack_top = stack.top();
        unsafe {
            ARCH_OPS.init_kernel_context(ctx.as_nonnull(), stack_top, entry,
arg);
        }
        task.install_execution(stack, ctx);
    }
}

```

代码 5-4 架构上下文注入

这种注入方式与第一章的启动上下文和第二章的架构内存接口保持一致。平台相关层负责寄存器布局和汇编切换。平台无关层负责生命周期、状态机和资源关系。这样可以让 RISC-V64 与 LoongArch64 共用任务对象结构和调度逻辑，同时保留各自异常返回、陷阱帧和上下文切换的实现差异。

5.6 退出、僵尸状态与 wait 回收

任务退出被拆成多个阶段。第一阶段运行退出前钩子。这个阶段仍然保留用户地址空间和文件表，因此可以处理 CLONE_CHILD_CLEARTID、健壮 futex 机制和其它必须访问用户地址的清理动作。第二阶段标记退出码和退出原因，把任务状态改为僵尸状态，唤醒等待者，并按退出信号规则通知父任务。第三阶段释放重量级扩展状态，例如用户虚拟地址空间、文件描述符表和 VFS 上下文。第四阶段由父任务执行 wait 回收，从子列表中取走僵尸任务，释放 PID，并把任务状态改为死亡状态。

```

fn exit_task(task: &Arc<Task>, code: ExitCode) -> ! {
    task.cleanup_before_exit();
    reparent_children_to_init(task);

    mark_task_exited(task, code);
    notify_parent_if_needed(task);
    task.vfork_done.wake_all();

    schedule_away_forever();
}

fn reap_child(parent: &Arc<Task>, selector: WaitSelector) ->
Result<WaitResult, Errno> {
    loop {
        if let Some(child) = parent.reap_matching(|task|
selector.matches(task)) {
            let status = child.exit_wait_status().unwrap();
            release_all_pids(&child);
            child.cleanup_exit_extensions();
            child.retire_execution();
            child.set_state(TaskState::Dead);
            return Ok(WaitResult { child, status });
        }
    }
}

```



```

    }

    if selector.nohang {
        return Err(EAGAIN);
    }

    parent.exit_waiters.wait_event(parent, || has_waitable_child(parent,
selector));
    }
}

```

代码 5-5 退出和 wait 的阶段边界

僵尸状态的价值在于保留 wait 可见信息。父任务可能在子任务退出很久以后才调用 wait。若 exit 立即释放任务对象，退出码、终止信号、资源用量和 procfs 文件系统可见状态都会丢失。若僵尸任务继续保留完整虚拟内存、文件表和内核栈，短生命周期进程密集创建和退出时会快速积累内存压力。当前实现把轻量状态留在任务对象本体中，把重量级状态交给幂等的退出钩子清理。这样既满足 wait 语义，也控制了僵尸任务的资源占用。

父任务先退出时，子任务会被移交给 init 任务。移交过程更新子任务的父弱引用，并把子任务移动到 init 任务的子列表中。这个操作保证所有僵尸任务最终都有可回收者。init 任务的 wait 循环因此是进程系统的最后兜底。

5.7 等待队列

等待队列是任务阻塞和唤醒的通用原语。它保存等待者的弱引用。准备睡眠时，任务先进入等待队列，再把状态切为睡眠状态或不可中断睡眠状态。真正让出 CPU 前，调用方必须重新检查条件。事件发生时，唤醒者从队列中取出弱引用，升级成功后把任务状态切回可运行状态，并在锁外调用调度器入队。

```

struct WaitQueue {
    waiters: Spinlock<VecDeque<Weak<Task>>>,
}

fn wait_event(queue: &WaitQueue, task: &Arc<Task>, condition: impl Fn() ->
bool) {
    while !condition() {
        queue.enqueue(task);
        task.set_state(TaskState::Sleeping);

        if condition() {
            queue.finish_wait(task);
            return;
        }

        schedule_once(now_ns_public());
        queue.finish_wait(task);
    }
}

fn wake_one(queue: &WaitQueue) {

```

```

let picked = queue.pop_upgradeable_waiter();
if let Some(task) = picked {
    task.cas_state(TaskState::Sleeping, TaskState::Runnable);
    enqueue_task(task, now_ns_public());
}
}

```

代码 5-6 等待队列协议

弱引用是这里的关键。等待队列不能成为任务生命周期的所有者。否则任务等待自身退出、poll 多个对象或超时取消时，队列中的强引用可能让任务迟迟无法释放。弱引用还让过期等待者的清理变得自然。升级失败就跳过。唤醒回调在锁外执行，避免等待队列锁和运行队列锁形成反向依赖。

先入队再切状态的顺序也有明确原因。若先把任务切为睡眠状态，再把它放入队列，事件可能在两步之间发生。唤醒者看到队列为空后返回，任务随后入队并睡眠，唤醒就丢失了。当前协议把登记放在前面，并在让出 CPU 前再次检查条件，从而覆盖事件与睡眠之间的竞态窗口。

5.8 信号机制

信号是异步通知机制。发送阶段只负责权限检查和入队。接收阶段发生在目标任务返回用户态前，由调度和陷入返回路径检查待处理信号，再决定默认动作、忽略或构造用户态信号帧。每个任务有私有待处理队列和阻塞掩码。线程组共享共享信号状态，其中保存信号动作表和共享待处理队列。

```

struct SignalState {
    pending_bits: AtomicU64,
    pending_infos: Spinlock<Vec<SigInfo>>,
    blocked: AtomicU64,
    saved_blocked: AtomicU64,
    sigtimedwait_mask: AtomicU64,
}

struct SharedSignal {
    actions: Spinlock<[SigAction; NSIG]>,
    pending_bits: AtomicU64,
    pending_infos: Spinlock<Vec<SigInfo>>,
}

fn send_signal(target: &Arc<Task>, info: SigInfo) -> Result<(), Errno> {
    check_signal_permission(current_task(), target, info.sig)?;
    target.signal.deliver(info);
    wake_if_interruptible_sleep(target);
    Ok(())
}

```

代码 5-7 信号状态结构

SIGKILL 和 SIGSTOP 有硬性语义，不能被捕获、忽略或屏蔽。普通信号进入待处理队列后，会受到阻塞掩码影响。实时信号和带信号信息的信号需要保存额外载荷，

因此实现同时维护位图和信号信息队列(SigInfo)。位图用于快速判断是否存在信号, 队列用于保存发送者 PID、UID 和用户态信号信息。

线程组共享信号的处理需要结合每个线程的阻塞掩码。一个发送到线程组的信号可以由组内任意未屏蔽该信号的线程处理。共享待处理队列因此不能绑定到某个固定任务。任务返回用户态前, 会同时检查私有待处理队列和共享待处理队列, 并按自身屏蔽字选择可处理信号。这个设计保持了 POSIX 线程语义, 也避免在发送阶段做复杂的线程选择。

5.9 凭据与能力

凭据描述任务的权限身份, 包括 UID、GID、fsuid、fsgid、附加组和 Linux capability 能力集合。调度层的凭据对象(Credentials)与 VFS 凭据类型保持分离, 跨层转换由内核胶合层完成。这样调度 crate 不需要依赖 VFS。VFS 在打开文件时使用同步过去的凭据快照完成权限检查, 信号权限检查则使用调度层凭据。

```
struct Credentials {
    uid: Uid,
    euid: Uid,
    suid: Uid,
    fsuid: Uid,
    gid: Gid,
    egid: Gid,
    sgid: Gid,
    fsgid: Gid,
    groups: Vec<Gid>,
    caps: CapSet,
    cap_permitted: CapSet,
    cap_inheritable: CapSet,
    cap_bset: CapSet,
}

fn replace_credentials(task: &Arc<Task>, update: impl FnOnce(&Credentials) -
> Credentials) {
    let old = task.credentials();
    let new = Arc::new(update(&old));
    task.set_credentials(Arc::clone(&new));
    sync_vfs_credentials(task, new);
}
```

代码 5-8 凭据快照与整体替换

凭据采用不可变快照和整体替换。setuid、setgid 和 capset 不在原对象上逐字段修改, 而是构造新对象后替换凭据强引用(Arc<Credentials>)。这样读者要么看到旧凭据, 要么看到新凭据, 不会看到 UID 已变但 fsuid 尚未变的中间状态。能力集使用位图表达, 权限检查可以通过一次位测试完成。root 凭据携带完整 Linux capability 能力集, 普通用户默认没有 Linux capability 能力。

5.10 工程设计总结

进程与线程管理子系统把执行身份、资源集合和 POSIX ABI 语义放在同一个生命周期框架内处理。它既要支撑调度器的高频状态切换，也要满足 `fork`、`clone`、`exec`、`wait` 和信号等复杂接口的兼容要求。我们没有把所有资源都写进一个巨大结构，也没有把 PID 作为内部唯一身份，而是把任务对象、PID 命名层、任务扩展、线程组和等待队列拆成可以独立推理的对象。

进程与线程管理子系统具备以下创新：

第一是以任务对象统一进程和线程的执行抽象。这个设计让调度器、等待机制、信号机制和退出机制都只面对一种执行实体。进程和线程的差异被推迟到资源共享层，由 `clone` 标志和扩展钩子决定。早期若采用进程结构体包线程结构体的模型，调度器需要选择线程，`wait` 需要回到进程，信号又要在二者之间转发。当前结构避免了这种双层身份转换。一个任务是否与父共享地址空间、文件表和信号处理表，由创建时的标志决定。任务状态机本身不关心这些差异。这个统一性降低了系统调用路径的分支数量，也让内核线程和空闲任务能够通过任务类型（`TaskKind`）纳入同一调度框架，同时隔离出 POSIX 信号和 `wait` 语义。

第二是把 PID 从内部身份降级为 ABI 兼容层。PID 对用户程序不可或缺，但它不适合作为内核内部的主引用。整数会复用，查找需要全局表，生命周期也容易与任务对象脱节。我们让内部路径传递任务强引用，PID 登记表只保存弱引用。`getpid`、`kill`、`waitpid` 和 `procfs` 文件系统在边界上把整数 PID 翻译为任务引用。这个设计减少了 PID 表锁对内部路径的影响，也避免登记表保活已经退出的任务。更重要的是，它为 PID 命名空间保留了自然结构。任务可以在多层命名空间中拥有不同 PID，调度核心仍然只处理同一个任务对象。

第三是任务扩展钩子把调度核心和资源子系统解耦。VFS、文件描述符表、用户虚拟地址空间、RISC-V 向量状态和信号栈扩展都可以挂在任务上，但调度 `crate` 不需要知道它们的内部布局。`clone` 时，扩展钩子根据标志决定共享、深拷贝或写时复制。`exit` 时，退出前钩子先处理必须依赖用户地址空间的清理，扩展退出钩子再释放重量级资源。为了避免动态扩展表拖慢系统调用热路径，我们又为 VFS、文件描述符表和用户虚拟地址空间等高频项设置热路径扩展槽位。这个组合同时满足了解耦和性能两个目标。

这些创新共同支撑了进程管理子系统的长期演化能力。新增资源类型可以通过任务扩展接入。新增命名空间语义可以沿 PID 和组对象扩展。新增架构上下文可以通过架构钩子注入。用户态 ABI 仍然看到熟悉的 PID、`wait` 状态、信号和凭据，而内核内部保持引用对象、单向依赖和分阶段生命周期。这个结构把复杂性放在明确的交接点上，使进程与线程管理能够承接 VFS、内存管理和调度系统的共同需求。

第六章 调度系统

在第五章中，任务对象被定义为统一的执行实体，并通过状态机、亲缘关系和扩展侧表承载进程与线程语义。本章继续向下推进，讨论这些任务对象如何获得 CPU 时间。进程管理回答谁存在，调度系统回答谁运行。这个问题看似只涉及一个选择动作，但它实际连接了公平性、响应性、上下文切换成本、CPU 亲和性和实时策略。

调度系统面对的核心矛盾，是目标之间存在天然冲突。公平性要求每个任务按权重获得 CPU。响应性要求刚被唤醒的交互任务尽快运行。吞吐量要求减少无意义的上下文切换。实时策略又要求某些任务绕过普通公平竞争，优先获得确定的执行机会。若调度器只追求公平，短等待任务可能在唤醒后等待过久。若调度器过度偏向唤醒任务，长期运行任务会失去稳定份额。我们采用分层调度类和 EEVDF 公平调度，把不同目标放在不同层次处理。

当前实现的调度系统以运行队列（Runqueue）为核心。每个 CPU 拥有一个运行队列。运行队列内部按截止时间类、实时类、公平类和空闲类分层。公平类使用 EEVDF 调度算法。实时类提供 FIFO 和轮转语义。截止时间类保留 EDF、预算和补充时间点的框架。空闲类作为兜底，只在没有其它任务可运行时使用。这样的结构把策略选择、队列排序、抢占判断和架构上下文切换分开，使调度系统可以随平台能力逐步扩展。

6.1 设计目标与约束

调度系统的设计目标可以概括为四项。第一，常规任务需要权重公平。nice 值应当稳定影响 CPU 份额，任务不能因为一次睡眠获得无限补偿。第二，短等待任务需要及时响应。futex 机制、管道、套接字和 wait 这类场景中，唤醒后的交接不能总是等到下一个调度节拍。第三，策略边界必须清晰。实时任务、截止时间任务和普通任务不应混在一棵含义不明的队列中。第四，调度器不能过度依赖平台细节。寄存器保存、CPU 标识、时间读取和空闲指令由架构层注入。

表 6-1 调度系统的设计目标

目标	设计含义	实现边界
公平性	公平类通过权重、虚拟时间和 EEVDF 截止点分配 CPU。	nice 到权重的映射沿用 Linux 权重表，虚拟运行时间推进与权重成反比。
响应性	唤醒路径可请求重调度，futex 机制热路径可设置一次性优先候选。	优先候选不改变长期优先级，选择时仍复查调度类和亲和性。
策略分层	截止时间类、实时类、公平类和空闲类按固定顺序协作。	不同策略拥有独立队列和判断规则，避免公平类逻辑污染实时类。

续表 6-1 调度系统的设计目标

目标	设计含义	实现边界
平台解耦	时间、CPU 标识、空闲放松动作和上下文切换通过架构钩子注入。	调度核心只处理任务对象、运行队列和抽象时间戳。

这里还有一个现实约束。当前代码已经把每 CPU 数组、CPU 位图、调度拓扑和跨 CPU 迁移接口准备好，但启动路径仍以 CPU0 为主要运行环境。文档在描述多 CPU 机制时，只把它作为数据结构和接口层面的能力，不把它写成完整应用处理器启动后的运行状态。这个边界有助于避免把预留设计误读为已经完成的 SMP 调度。

6.2 总体结构

调度系统分为四层。最上层是调度策略模型，把用户态传入的调度属性转换为调度策略（SchedPolicy）和调度类（SchedClass）。第二层是每个任务的调度实体，保存权重、虚拟时间、截止点、实时优先级和截止时间预算等信息。第三层是每 CPU 运行队列，按调度类保存有序队列。第四层是全局调度入口，处理当前任务、定时器节拍、重调度标志、空闲任务和跨 CPU 均衡请求。

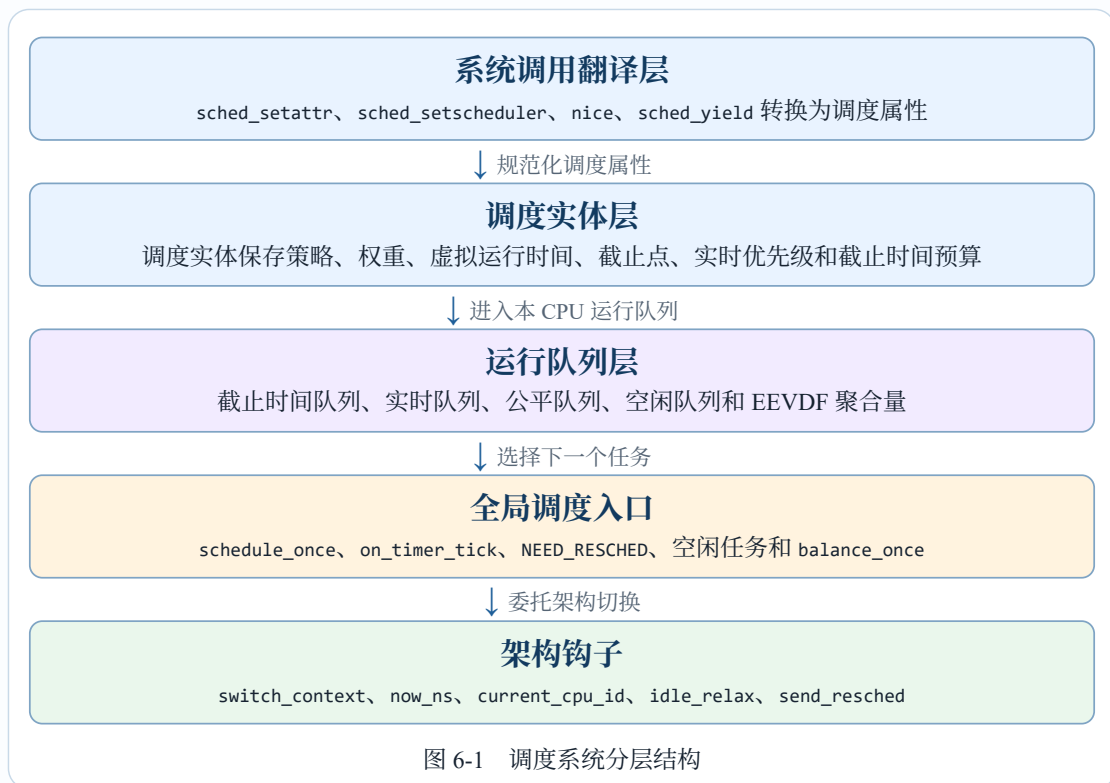


图 6-1 调度系统分层结构

这种分层的关键收益是策略和机制分开。调度属性（SchedAttr）负责规范化用户请求。调度实体（SchedEntity）负责保存单任务状态。运行队列负责排序和选择。scheduler.rs 文件负责系统级入口和每 CPU 槽位。架构层只处理硬件相关动作。每

层都可以在不重写其它层的前提下演化。例如未来把公平队列内部从有序映射 (BTreeMap) 换成更专用的数据结构, 不需要改变任务对象身份模型和系统调用 ABI。

6.3 调度策略与调度类

调度策略使用稳定枚举表达。调度策略描述用户可见策略, 包括公平策略、实时 FIFO 策略、实时轮转策略、截止时间策略和空闲策略。调度类描述运行队列选择顺序。截止时间类优先于实时类, 实时类优先于公平类, 公平类优先于空闲类。运行队列选择任务时按这个顺序查找可运行任务。

```
enum SchedClass {
    Deadline,
    Realtime,
    Fair,
    Idle,
}

enum SchedPolicy {
    Fair,
    RtFifo,
    RtRoundRobin,
    Deadline,
    Idle,
}

struct SchedAttr {
    policy: SchedPolicy,
    nice: i8,
    slice_ns: u64,
    priority: u8,
    runtime_ns: u64,
    deadline_ns: u64,
    period_ns: u64,
}

fn normalize(attr: SchedAttr) -> Result<SchedAttr, Errno> {
    match attr.policy {
        SchedPolicy::Fair => normalize_fair(attr),
        SchedPolicy::RtFifo => validate_rt_priority(attr),
        SchedPolicy::RtRoundRobin => validate_rr_slice(attr),
        SchedPolicy::Deadline => validate_deadline_tuple(attr),
        SchedPolicy::Idle => Ok(idle_attr()),
    }
}
```

代码 6-1 调度策略与属性

我们没有在调度热路径上使用运行时虚表来表示调度类。策略被规约为枚举, 运行队列根据枚举进入不同子队列。这减少了间接调用, 也让选择顺序在代码中保持显式。代价是新增调度类需要修改枚举和运行队列结构。考虑到调度类数量很少, 并且每类策略都需要特殊的系统调用校验和统计语义, 这个代价可以接受。

实时类采用静态优先级。FIFO 任务在同优先级内按入队顺序运行，轮转任务在同优先级内消耗时间片后重新入队。截止时间类保存运行时间、截止时间和周期，并用绝对截止时间排序，同时维护预算和补充时间点。当前实现尚未提供完整的全局准入控制，所以截止时间类更接近调度框架和 ABI 边界，不能被描述为完整硬实时保证。

6.4 公平类与 EEVDF

公平类负责普通任务。它使用 EEVDF 调度算法。EEVDF 的核心是三个值。vruntime 表示任务在虚拟时间轴上的运行进度。权重越大，物理运行时间转换成虚拟运行时间的增量越小。avg_vruntime 表示运行队列上所有公平任务的加权平均进度。只有 vruntime ≤ avg_vruntime 的任务是合格任务。deadline 表示任务当前时间片在虚拟时间轴上的截止点，合格任务中截止点最小者优先运行。

```
const NICE_0_WEIGHT: u64 = 1024;

fn scale_delta(delta_exec_ns: u64, weight: u64) -> u64 {
    ((delta_exec_ns as u128 * NICE_0_WEIGHT as u128) / weight.max(1) as
    u128) as u64
}

fn update_current(task: &Arc<Task>, delta_exec_ns: u64) {
    let delta_vruntime = scale_delta(delta_exec_ns, task.sched.weight());
    task.sched.vruntime += delta_vruntime;
    task.sched.deadline = task.sched.vruntime +
    scale_delta(task.sched.slice_ns(), task.sched.weight());
}

fn eligible(task: &Arc<Task>, avg_vruntime: u64) -> bool {
    task.sched.vruntime() <= avg_vruntime
}
```

代码 6-2 EEVDF 的基本计算

EEVDF 对 CFS 的一个重要修正，是避免睡眠任务在唤醒后获得过量奖励。任务离开运行队列时，调度器保存 lag = avg_vruntime - vruntime。任务重新入队时，调度器按新的平均虚拟运行时间恢复这个滞后量。这样任务睡眠前的领先或落后状态可以跨睡眠周期保留。长期睡眠不会直接变成无限优先级，短等待任务又能保留应该获得的补偿。

```
fn dequeue_fair(task: &Arc<Task>, rq: &mut RqInner) {
    let avg = avg_vruntime(rq);
    let lag = avg as i64 - task.sched.vruntime() as i64;
    task.sched.store_lag(lag);
    remove_from_fair_tree(task, rq);
    subtract_rq_account(task, rq);
}

fn enqueue_fair(task: Arc<Task>, rq: &mut RqInner, now_ns: u64) {
    let avg = avg_vruntime(rq);
    let lag = task.sched.lag();
```

```

if lag != 0 {
    task.sched.store_vruntime((avg as i64 - lag).max(0) as u64);
    task.sched.store_lag(0);
} else {
task.sched.store_vruntime(task.sched.vruntime().max(rq.min_vruntime));
}

let deadline = task.sched.recalc_deadline();
task.sched.store_deadline(deadline);
insert_into_fair_tree(task, rq);
}

```

代码 6-3 滞后量保存与恢复

运行队列维护 `total_weight` 和 `weighted_vruntime_sum`，用增量方式计算平均虚拟运行时间。任务入队和离队时更新聚合量。当前任务运行时，调度节拍推进它的虚拟运行时间，并同步更新聚合量。这样平均虚拟运行时间的维护不需要每次遍历所有任务。选择路径仍会扫描有序候选，因为它还要复查任务状态、CPU 亲和性、优先候选和合格条件。文档因此只说平均虚拟时间的维护是增量的，不把整个选择路径说成纯 $O(1)$ 。

6.5 运行队列与多调度类队列

每个运行队列内部有五棵有序树。公平树按 EEVDF 截止点和任务地址排序。实时树按倒序优先级、入队序号和任务地址排序。截止时间树按绝对截止时间排序。截止时间限流树保存预算耗尽后等待补充的任务。空闲树保存兜底任务。运行队列用一把锁保护所有子队列和当前任务指针。

```

struct Runqueue {
    inner: Spinlock<RqInner>,
}

struct RqInner {
    fair_tree: BTreeMap<FairKey, Arc<Task>>,
    rt_tree: BTreeMap<RtKey, Arc<Task>>,
    deadline_tree: BTreeMap<DeadlineKey, Arc<Task>>,
    deadline_throttled: BTreeMap<DeadlineThrottleKey, Arc<Task>>,
    idle_tree: BTreeMap<RtKey, Arc<Task>>,
    total_weight: u128,
    weighted_vruntime_sum: u128,
    min_vruntime: u64,
    current: Option<Arc<Task>>,
    enqueue_seq: u64,
    preferred_fair_addr: Option<usize>,
}

```

代码 6-4 运行队列结构

单锁保护整个运行队列，是一个有意识的取舍。调度操作需要同时观察当前任务、多个子队列和 EEVDF 聚合量。若给每个子队列单独加锁，跨调度类抢占和任务选择就需要固定锁序，错误空间明显增大。调度锁确实是热路径锁，但每 CPU 运

行队列把竞争限制在本 CPU 内。当前核心数和任务规模下，单锁的可调试性收益更重要。

有序映射的选择也偏向工程稳定。堆在取最小值上很便宜，但删除任意任务和重排任务不方便。链表插入简单，但查找最早截止点或最高优先级需要扫描。有序映射让插入、删除和取最小都保持对数复杂度，并且便于调试时观察键的顺序。后续若某类任务数量增长，可以在子队列内部替换数据结构，不影响调度类边界。

6.6 重调度时机与上下文切换

定时器中断调用 `on_timer_tick`。它唤醒超时睡眠者，处理实时计时器，调用当前 CPU 的运行队列节拍函数，并在需要抢占时设置 `NEED_RESCHED`。定时器中断本身不直接执行完整上下文切换。真正的切换发生在系统调用返回、中断返回、主动让出或空闲循环等安全点。

```
fn on_timer_tick(now_ns: u64) {
    wake_expired_sleepers(now_ns);
    fire_expired_realtime_itimers(now_ns);

    let cpu_id = current_cpu_id();
    if RUNQUEUES[cpu_id].tick(now_ns) {
        NEED_RESCHED[cpu_id].store(true, Release);
    }
}

fn run_if_needed() {
    let cpu_id = current_cpu_id();
    if NEED_RESCHED[cpu_id].swap(false, AcqRel) {
        if NEED_BALANCE[cpu_id].swap(false, AcqRel) {
            balance_once(cpu_id);
        }
        schedule_once(now_ns_public());
    }
}
```

代码 6-5 调度节拍与延迟重调度

延迟重调度的原因来自内核关键区。定时器中断可能打断任意位置。若中断处理函数直接切换任务，当前任务可能正在持有自旋锁，新的任务随后等待同一把锁，就会出现不可恢复的阻塞。`NEED_RESCHED` 把异步事件变成同步决策。切换只在已知安全的位置发生。这一原则与第五章中等待队列的锁外唤醒保持一致。

`schedule_once` 的主要流程是取得当前 CPU 的运行队列，选择下一个可运行任务，必要时回退到空闲任务，然后通过架构钩子切换上下文。退出任务在最终脱离 CPU 后会进入延迟释放列表，后续调度边界再在活任务栈上释放它的虚拟内存、文件表和执行上下文。这样可以避免在已经离开的内核栈上运行析构。

```
fn schedule_once(now_ns: u64) {
    let cpu_id = current_cpu_id();
    cleanup_retired_tasks(cpu_id);
```

```

let prev = CURRENT_TASKS[cpu_id].clone().unwrap();
deliver_pending_signals_if_needed(&prev);

let next = RUNQUEUES[cpu_id]
    .pick_next_on(now_ns, cpu_mask(cpu_id))
    .or_else(|| idle_task(cpu_id))
    .unwrap_or_else(|| Arc::clone(&prev));

if Arc::ptr_eq(&prev, &next) {
    return;
}

CURRENT_TASKS[cpu_id] = Some(Arc::clone(&next));
arch_switch_context(prev.arch_context(), next.arch_context());
}

```

代码 6-6 schedule_once 的关键路径

上下文切换的直接成本包括保存寄存器、恢复寄存器、设置当前任务指针和切换内核陷入栈。间接成本来自 TLB、缓存和分支预测状态。地址空间切换由第二章中的用户页表接口和架构层处理。线程之间共享用户虚拟地址空间时，可以避免一部分页表和 TLB 成本。调度系统只要求任务在切换前拥有有效架构上下文，不直接解析页表细节。

6.7 唤醒、优先候选与系统调用后交接

唤醒路径通过 `enqueue_task` 把任务放入目标 CPU 的运行队列，并请求该 CPU 重新调度。普通唤醒只改变任务状态和队列归属。`futex` 这类短等待热路径可以调用 `enqueue_task_preferred`，在公平队列中记录一次性优先候选。选择任务时仍然会复查任务状态、调度类和 CPU 亲和性。优先候选只是减少短等待任务等到下一个调度节拍的概率，不改变长期公平性。

`clone` 后还有一个专门的系统调用后交接。新任务创建成功时，父任务的系统调用返回值和程序计数器仍在收尾阶段。若在 `clone` 内部立即切换，父任务的陷阱帧可能尚未写好。我们把交接请求记录在 `POST_SYSCALL_HANDOFF` 中，等系统调用分发器完成收尾后再执行一次有界调度。这个设计解决了父子任务首轮运行顺序与陷阱帧一致性的冲突。

```

fn enqueue_task_preferred(task: Arc<Task>, now_ns: u64) {
    let cpu = select_task_cpu(&task);
    task.set_current_cpu(cpu);
    RUNQUEUES[cpu].enqueue_preferred(task, now_ns);
    request_resched(cpu);
}

fn request_post_syscall_handoff() {
    let cpu = current_cpu_id();
    POST_SYSCALL_HANDOFF[cpu].store(1, Release);
    request_resched(cpu);
}

```

```
fn run_post_syscall_handoff(now_ns: u64) {
    let cpu = current_cpu_id();
    if POST_SYSCALL_HANDOFF[cpu].swap(0, AcqRel) != 0 {
        schedule_once(now_ns);
    }
}
```

代码 6-7 唤醒与一次性交接

这里的共同原则是把异步性限制在安全边界。唤醒可以发生在等待队列、设备中断、套接字状态变化或定时器到期时。它们只负责把任务变为可运行状态，并请求调度。是否立刻切换，由当前 CPU 在安全点决定。

6.8 CPU 亲和性、拓扑与负载均衡

调度器使用固定容量 CPU 位图表示亲和性和在线状态。CPU 位图（CpuMask）始终被截断到当前构建支持的 CPU 范围内，空亲和性会退回启动 CPU。调度拓扑（SchedTopology）保存调度域。默认拓扑只有覆盖所有支持 CPU 的根域，平台可以在启动期安装更细的域层级。任务放置时，调度器在亲和性与在线 CPU 的交集中选择负载较低的 CPU，并尽量保留当前 CPU。

```
fn select_task_cpu(task: &Arc<Task>) -> usize {
    let affinity = CpuMask::from_bits_or_boot(task.cpu_affinity());
    let online = online_cpu_set();
    let current = CpuId::new(task.current_cpu());
    let prefer_current = task.state() != TaskState::New;
    let snapshot = collect_rq_load_snapshot(affinity.intersection(online));

    sched_topology()
        .select_cpu(affinity, online, current, prefer_current, |cpu|
            snapshot.load_of(cpu))
        .unwrap_or_else(CpuId::boot)
        .get()
}
```

代码 6-8 CPU 选择与负载快照

负载均衡通过 `balance_once` 从较忙 CPU 拉取一个可迁移任务到本 CPU。它不会直接摘走远端当前任务，只从公平、实时和截止时间就绪队列中取可迁移任务。截止时间限流队列和空闲队列不参与迁移。选择源 CPU 时，调度器使用一次负载快照，避免在拓扑遍历过程中反复持有多个运行队列锁。

```
fn balance_once(cpu_id: usize) -> bool {
    let local = CpuId::new(cpu_id).unwrap_or_else(CpuId::boot);
    let online = online_cpu_set();
    let local_load = RUNQUEUES[cpu_id].migratable_load();

    let source = select_balance_source(sched_topology(), local, online,
        local_load, |cpu| {
            RUNQUEUES[cpu.get()].migratable_load_for(local.mask().bits())
        });
}
```



```

    let task = RUNQUEUES[source.get()].take_migratable(local.mask().bits(),
now_ns_public());
    task.set_current_cpu(cpu_id);
    RUNQUEUES[cpu_id].enqueue(task, now_ns_public());
    true
}

```

代码 6-9 一次负载均衡

这个均衡策略偏保守。它只在明显不均衡时迁移，避免任务在两个负载接近的 CPU 之间来回移动。迁移会带来缓存冷启动和 TLB 局部性损失。对于当前单核为主的运行环境，均衡逻辑主要是为多 CPU 启动准备好数据结构和锁顺序。未来接入完整应用处理器后，可以在调度域中表达共享缓存、NUMA 或其它拓扑信息。

6.9 空闲任务与初始化顺序

调度器初始化依赖架构上下文钩子。启动期先注册架构上下文接口和时间接口，再创建 init 任务，建立根 PID 命名空间、线程组、进程组和会话。init 任务接管当前 CPU 的执行上下文。随后内核注册任务扩展克隆钩子、退出钩子和退出前钩子，并为 CPU0 派生空闲任务。

空闲任务是每个 CPU 的兜底执行体。它不参与普通公平权重竞争，调度类被设置为空闲类。运行队列没有其它可运行任务时，schedule_once 才会回退到空闲任务。空闲循环会尝试一次负载均衡，然后调用 schedule_once。仍然没有任务可运行时，架构空闲钩子可以执行 wfi 一类放松指令。未注入时退化为自旋提示。

```

fn sched_boot_init() {
    register_arch_context_ops();
    register_arch_time_ops();
    let init = sched::init();

    register_ext_clone_hook(&KERNEL_EXT_CLONE_HOOK);
    register_ext_exit_hook(&KERNEL_EXT_EXIT_HOOK);
    register_pre_exit_hook(&KERNEL_PRE_EXIT_HOOK);

    spawn_idle_for_cpu(0);
    start_user_init_from(init);
}

fn idle_loop() -> ! {
    loop {
        balance_once(current_cpu_id());
        schedule_once(now_ns_public());
        arch_idle_relax_or_spin();
    }
}

```

代码 6-10 调度器初始化与空闲任务

空闲任务被设计成独立调度类, 而非一个极低权重的普通任务。任何有限权重都意味着它会在公平竞争中获得份额。空闲类的真实语义是没有其它任务时才运行。独立调度类能直接表达这个语义, 也让运行队列在队列为空时拥有安全落点。

6.10 工程设计总结

调度系统把高频执行路径和策略扩展路径分开。高频路径只处理每 CPU 运行队列、原子状态、虚拟时间和架构上下文。策略扩展通过调度策略、调度类、调度属性和子队列完成。这个结构让常规任务、实时任务、截止时间任务和空闲任务都能进入同一调度框架, 同时保留各自语义。

调度系统具备以下创新:

第一是以 EEVDF 作为普通任务的公平调度基础。它把权重公平和延迟敏感放在同一个虚拟时间模型中处理。虚拟运行时间保证任务按权重推进。合格条件防止已经领先平均进度的任务继续占用 CPU。截止点选择让更紧迫的任务更早运行。滞后量保存和恢复解决了睡眠任务跨周期的公平性问题。与简单的最小虚拟运行时间选择相比, EEVDF 更适合处理大量短等待任务和长期运行任务混合的场景。我们在实现中还保留了增量平均虚拟运行时间维护, 避免每次调度重新遍历整个公平队列。

第二是调度类分层让策略冲突保持在明确边界内。截止时间类、实时类、公平类和空闲类按固定顺序协作。实时任务不参与 nice 权重竞争, 普通任务也不能因为滞后量补偿越过实时任务。截止时间类保留 EDF 和预算框架, 但文档不把它描述为完整硬实时实现, 因为准入控制尚未落地。这样的写法看似保守, 却符合工程实际。一个调度器最危险的问题之一, 是对外承诺强于内部机制能够保证的语义。我们宁愿把边界写清楚, 也不让后续维护者误以为当前截止时间类已经提供完整实时保证。

第三是架构相关操作全部通过钩子注入。调度核心不读取具体 CSR, 不解释保存寄存器的布局, 也不直接执行某条空闲指令。它只知道如何取得当前 CPU 标识、读取时间戳、请求远端重调度和切换上下文。这个结构使 RISC-V64 与 LoongArch64 共用调度策略, 同时允许两者在汇编切换、陷入栈设置和空闲放松动作上各自优化。第一章启动层、第二章内存层和第五章任务执行体都遵循类似模式。调度系统在这里延续了全局的单向依赖原则。

这些创新共同形成了调度子系统的工程价值。它让普通任务获得可解释的权重公平, 让短等待路径具备及时交接能力, 让实时和截止时间策略拥有独立入口, 也让未来 SMP 扩展有稳定的数据结构基础。调度器越成熟, 用户越不应直接感知它。我们的目标并非让调度策略在正常负载下频繁暴露存在感, 而是让任务创建、阻塞、唤醒和退出都能自然落入同一套可分析的时间分配机制。

第七章 并发与同步机制

在第六章中, 调度系统已经把任务对象的运行状态、运行队列和安全调度点组织起来。调度系统能够决定谁运行, 但它不能替代同步机制。多个任务会同时访问文件描述符表、VFS 缓存、网络套接字、等待队列和设备状态。中断、定时器和内核线程也会在不同上下文中推进同一批对象。若没有稳定的同步原语, 前面章节中讨论的生命周期和所有权都无法成立。

本章讨论并发与同步机制。重点在于说明不同同步原语为什么存在, 以及它们如何与调度器的状态机配合。我们把同步需求分成三类。第一类是短临界区, 典型场景是运行队列、亲缘关系和小型缓存索引。第二类是可能阻塞的临界区, 典型场景是文件系统对象、套接字状态和设备控制路径。第三类是事件等待, 典型场景是管道读写、futex 机制、定时睡眠、网络收发和退出等待。三类需求分别对应自旋锁、可睡眠互斥锁和等待队列。

7.1 设计目标与约束

同步机制首先要保证正确性。锁必须给共享数据提供互斥访问, 等待队列必须避免丢失唤醒, 状态转换必须与调度器可见状态一致。其次要控制上下文边界。中断上下文不能进入可能睡眠的锁。持有运行队列锁时不能调用分配器或唤醒路径。可睡眠锁可以在普通任务上下文中等待, 但不能在中断入口中使用。最后还要保证可调试性。锁序复杂度过高时, 死锁会以系统停滞的形式出现, 定位成本远高于普通崩溃。

表 7-1 同步原语的使用边界

原语	适用场景	使用约束
自旋锁	短临界区、运行队列、亲缘关系、缓存索引。	持锁期间禁止睡眠, 禁止进入可能反向取锁的路径。
可睡眠互斥锁	普通任务上下文中的长临界区, 可包含阻塞 I/O。	不能在中断上下文使用, 同一任务重复获取会自锁。
等待队列	条件等待、超时睡眠、事件通知、退出等待队列。	登记后必须重新检查条件, 唤醒回调在锁外执行。
原子状态	任务状态、锁位、标志位和热路径观测字段。	只表达局部事实, 跨字段不变量仍需要锁或协议保护。

这些边界看起来朴素, 但它们构成了内核并发的基本纪律。自旋锁追求短时间独占。互斥锁允许任务睡眠。等待队列只表达事件发生和任务唤醒, 不拥有业务条件本身。原子变量负责无锁观测和状态转换。每一种机制都只解决一类问题, 避免把所有并发控制都压进同一种锁。

7.2 自旋锁

自旋锁使用原子布尔量（AtomicBool）实现先测后测并设置策略。获取锁时先尝试把 false 改为 true，失败后在只读循环中观察锁位，期间执行 spin_loop 自旋提示。释放锁时用释放语义写回 false。被保护的数据放在内部可变单元（UnsafeCell）中，只有持有自旋锁守卫（SpinlockGuard）时才能取得引用。

```
struct Spinlock<T> {
    locked: AtomicBool,
    data: UnsafeCell<T>,
}

impl<T> Spinlock<T> {
    fn lock(&self) -> SpinlockGuard<T> {
        while self.locked
            .compare_exchange_weak(false, true, Acquire, Relaxed)
            .is_err()
        {
            while self.locked.load(Relaxed) {
                spin_loop();
            }
        }
        SpinlockGuard { lock: self }
    }
}

impl<T> Drop for SpinlockGuard<T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Release);
    }
}
```

代码 7-1 自旋锁的核心结构

先测后测并设置策略的意义在于减少总线写竞争。若每个等待者都不断执行交换操作，锁位会在缓存一致性协议中频繁失效。先用只读循环等待锁位变化，再尝试写入，可以降低争用时的缓存抖动。当前内核的自旋锁主要保护短临界区。运行队列选择、等待队列列表、父子关系和小型注册表都符合这个条件。

自旋锁的风险同样明确。持锁路径不能睡眠。若任务持有自旋锁后被调度出去，其它任务可能永远等不到锁释放。第五章和第六章中的很多设计都在避免这个问题。等待队列唤醒在锁外执行。运行队列锁内不调用可能分配或阻塞的函数。调度器只在安全点切换任务。同步机制与调度机制因此形成相互约束。

7.3 可睡眠互斥锁

可睡眠互斥锁面向普通任务上下文。它仍然有一个原子锁位，用于快速路径比较交换。锁已被持有时，当前任务会挂入内部等待队列，切换到睡眠状态并让出 CPU。释放锁时先写回未锁定状态，再唤醒一个等待者。这个顺序保证被唤醒者重新检查锁位时能够看到释放结果。

```

struct Mutex<T> {
    locked: AtomicBool,
    waiters: WaitQueue,
    data: UnsafeCell<T>,
}

fn lock(&self) -> MutexGuard<T> {
    loop {
        if self.locked.compare_exchange(false, true, Acquire,
Relaxed).is_ok() {
            return MutexGuard { lock: self };
        }

        let current = current_task();
        self.waiters.prepare_to_wait(&current, TaskState::Sleeping);

        if self.locked.compare_exchange(false, true, Acquire,
Relaxed).is_ok() {
            self.waiters.finish_wait(&current);
            return MutexGuard { lock: self };
        }

        schedule_once(now_ns_public());
        self.waiters.finish_wait(&current);
    }
}

```

代码 7-2 可睡眠互斥锁的等待协议

这里有两个容易忽略的细节。第一，进入等待队列后要再次尝试获取锁。若锁在第一次失败和登记等待之间被释放，第二次比较交换可以直接成功，避免任务睡眠后无人唤醒。第二，`finish_wait` 必须在任务醒来后清理等待队列状态，并把当前任务恢复到运行状态或可运行状态。等待队列只保存弱引用，业务对象不能假设队列中的任务一定仍然存在。

可睡眠互斥锁适合 VFS、网络、设备控制等可能阻塞的路径。它不适合中断上下文，也不适合运行队列等调度热路径。这个区分保证了长操作不会在自旋锁下发生，同时也保证中断路径不会进入不可控睡眠。

7.4 等待队列与丢失唤醒

等待队列是所有阻塞操作的共同基础。它内部保存任务弱引用（`Weak<Task>`），不保活等待者。事件发生时，唤醒者把弱引用升级为强引用，成功后通过调度器统一入队。唤醒回调在队列锁外执行，避免等待队列锁和运行队列锁形成反向嵌套。

```

fn wait_event(queue: &WaitQueue, condition: impl Fn() -> bool) {
    let task = current_task();
    while !condition() {
        queue.prepare_to_wait(&task, TaskState::Sleeping);

        if condition() {
            queue.finish_wait(&task);
        }
    }
}

```



```

        return;
    }

    schedule_once(now_ns_public());
    queue.finish_wait(&task);
}

fn wake_all(queue: &WaitQueue) {
    let waiters = queue.drain_waiters();
    for weak in waiters {
        if let Some(task) = weak.upgrade() {
            task.cas_state(TaskState::Sleeping, TaskState::Runnable);
            enqueue_task(task, now_ns_public());
        }
    }
}

```

代码 7-3 等待队列的双重检查

双重检查协议用于处理事件和睡眠之间的竞态。第一次检查发现条件不满足后，任务登记到等待队列。登记后再次检查条件。若事件已经发生，任务立即退出等待，不会调用调度器睡眠。若条件仍不满足，任务才让出 CPU。这个顺序覆盖了最常见的丢失唤醒窗口。

等待队列不负责业务条件。管道可读、套接字可写、futex 用户值相等、子任务退出、定时器到期，这些条件都由调用方定义。等待队列只负责让任务在条件不满足时安全睡眠，并在事件可能改变条件时唤醒任务重新检查。这种分工让等待队列足够通用，也避免它理解每个子系统的内部状态。

7.5 原子状态与锁序控制

原子变量用于表达局部事实。任务状态、锁位、等待请求、重调度标志和设备失效标记都属于这类信息。原子读写让热路径可以无锁观察状态，也让比较交换成为状态机转换的边界。第五章中睡眠状态到可运行状态的唤醒转换，第六章中 `NEED_RESCHED` 的设置，都是这种模式。

原子变量不能替代所有锁。跨字段不变量仍然需要锁保护。例如父子关系中，父弱引用、子列表、线程组和 PID 副本必须一起变化。运行队列中，当前任务、多个子队列和 EEVDF 聚合量也必须一起变化。若把这些字段拆成多个原子值，读取者可能看到组合不一致的中间状态。我们的原则是，单个字段状态用原子，多字段关系用锁，跨模块交接用明确协议。

锁序控制则体现在多个子系统的边界上。等待队列不在持锁时调用调度器。文件描述符表关闭文件时锁内摘取、锁外释放。挂载命名空间用单锁完成跨挂载变化。运行队列用单锁保护跨调度类任务选择。我们在多个地方选择较粗的锁粒度，是为了减少隐式锁序。后续若某个锁成为性能瓶颈，可以在已有边界内拆分，而不是在一开始就引入难以验证的多锁结构。

7.6 上下文类型与睡眠边界

同步原语的选择不能只看共享数据本身, 还要看调用路径所处的上下文。内核中常见的上下文大致可以分为早期启动上下文、中断上下文、调度器上下文、普通任务上下文和退出回收上下文。它们能够执行的操作不同, 能够承受的延迟也不同。早期启动阶段尚未建立完整调度能力, 很多锁只能作为结构化互斥使用, 不能依赖睡眠等待。中断上下文没有当前任务可以安全阻塞, 必须使用短路径和有限循环。调度器上下文直接维护运行队列, 必须避免进入任何可能再次调度的路径。普通任务上下文拥有完整的阻塞能力, 可以使用可睡眠互斥锁和等待队列。退出回收上下文处于资源释放阶段, 必须额外考虑对象是否已经被其它路径摘除。

我们在同步设计中把“能否睡眠”作为第一层分类。这个分类看起来简单, 但它比“是否性能敏感”更基础。一个性能不敏感的路径若处于中断入口, 仍然不能睡眠。一个性能敏感的路径若需要等待用户缓冲区换页或块设备完成, 仍然必须让出 CPU。同步原语因此不能脱离上下文讨论。表 7-2 列出了主要上下文和允许使用的同步方式。

表 7-2 上下文类型与同步限制		
上下文	可用机制	设计理由
早期启动	原子状态、一次性初始化锁、短自旋锁。	调度器和等待队列尚未完全可用, 锁主要用于保护初始化顺序。
中断入口	原子标志、短自旋锁、延迟工作登记。	不能阻塞当前上下文, 复杂处理应交给工作线程或普通任务。
调度器内部	运行队列锁、原子重调度标志。	持锁路径必须可界定, 避免调度器递归进入自身。
普通任务	自旋锁、可睡眠互斥锁、等待队列、超时等待。	具备完整任务状态, 可以在条件不满足时安全让出 CPU。
退出回收	一次性状态转换、弱引用等待、锁外析构。	对象可能已经对新访问关闭, 回收路径要避免重新发布引用。

这个分类直接影响代码结构。以设备中断为例, VirtIO 或串口中断进入内核后, 理想路径是读取中断状态, 确认事件来源, 然后把需要耗时处理的部分交给对应队列或工作线程。若中断入口直接执行文件系统写回、套接字回调或用户缓冲区复制, 就会把不可预测的延迟带入中断屏蔽区。相反, 若所有中断都只设置一个原子标志, 普通任务迟迟没有机会观察, 也会造成吞吐下降。我们采用的折中方式是, 中断路径只完成硬件确认和事件发布, 真正需要锁住复杂对象的部分在可调度上下文中完成。

调度器内部的同步限制更严格。运行队列锁保护当前 CPU 的可运行任务集合, 也保护选择和入队过程中需要一致观察的聚合数据。持有这个锁时, 不能进入可睡眠锁, 也不能调用可能触发分配、文件访问或用户态复制的函数。运行队列锁的职责因此被控制在非常小的范围内。等待队列唤醒路径也遵循同样原则, 先在等待队列锁下取出等待者快照, 再在锁外调用调度器入队。这样可以避免一个事件源同时持有等待队列锁和运行队列锁。

普通任务上下文拥有最完整的同步能力。文件系统路径可以在索引节点锁内等待块设备 I/O, 套接字路径可以在接收缓冲区为空时睡眠, `futex` 机制可以在用户值未变化时阻塞当前任务。这里的关键是, 睡眠必须建立在可恢复的业务条件上。任务睡眠前要明确自己等待的条件, 登记等待队列后要再次检查条件, 醒来后要清理等待状态并重新判断。这样才能把调度器的状态机和业务对象的状态机连接起来。

退出回收上下文需要单独说明。任务进入退出流程后, 会逐步关闭文件、解除内存空间、从线程组摘除, 并向父任务发布 `wait` 事件。这个阶段很多对象已经被标记为失效或关闭中。同步路径要避免将这些对象重新放回全局结构, 也不能因为旧引用仍然存在就允许新请求进入。我们在多个对象中使用一次性状态位来表达关闭过程。例如设备能力的失效标志、任务对象的僵尸状态、文件对象的关闭路径, 都采用先阻止新访问、再排空旧访问、最后释放资源的顺序。这个顺序与同步机制密切相关, 因为每一步都对应不同的可见性边界。

7.7 锁粒度与所有权划分

锁粒度决定了并发路径的可读性和性能上限。粒度过粗会让无关操作相互阻塞, 粒度过细会增加锁序数量, 使死锁和一致性错误更难发现。我们在设计各子系统时采用了一个相对保守的原则。对象内部存在强不变量时, 先使用对象级锁保证正确性。只有当性能数据证明锁竞争已经成为瓶颈, 再沿着清晰的不变量边界拆分锁。这个原则贯穿文件描述符表、运行队列、等待队列、VFS 路径缓存和网络套接字状态。

文件描述符表是一个典型例子。文件描述符表同时维护 `fd` 到文件对象的映射、关闭时执行标志、可用编号扫描状态以及 `clone` 共享关系。若把这些字段拆成多个锁, `dup2`、`close_range` 和 `exec` 路径就需要跨锁更新多个集合。跨锁操作必须定义严格顺序, 否则 `close` 与 `dup` 之间会出现难以复现的竞态。我们选择在表级锁下完成映射变更, 锁外释放被摘除的文件对象。这样临界区内只处理表结构, 可能触发复杂析构的文件释放在锁外执行。

```
fn close_fd(table: &FdTable, fd: usize) -> Result<(), Errno> {
    let file = {
        let mut guard = table.lock();
        guard.remove(fd)?
    };

    drop(file);
    Ok(())
}
```

代码 7-4 文件描述符关闭的锁内摘取与锁外释放

这个模式看起来只是移动了 `drop` 的位置, 但它对锁序很重要。文件对象的析构可能减少索引节点引用, 索引节点引用减少可能触发文件系统回收, 文件系统回收又可能访问块设备和等待队列。若这些动作发生在文件描述符表锁内, 文件描述符表就会被迫参与 VFS、块设备和调度器的锁序。我们把析构推到锁外, 使文件描述符表的锁序只覆盖自身。

运行队列的粒度选择有另一种理由。调度器热路径确实性能敏感, 但任务选择、入队、出队、虚拟运行时间维护和优先候选之间存在密切关系。若每个调度类拥有独

立锁，跨类抢占和负载迁移就需要组合多个锁。当前实现使用每 CPU 运行队列锁作为核心互斥点，配合较短临界区和明确的调用边界。多核扩展时，跨 CPU 迁移采用固定顺序获取目标队列，避免形成循环等待。这个设计牺牲了一部分理论并行度，但换来了更稳定的调度语义。

等待队列的锁粒度则围绕等待者列表展开。等待队列锁只保护列表，不保护业务条件。管道是否可读由管道自己的锁保护，套接字是否可写由套接字状态保护，任务对象是否已经退出由任务状态保护。等待队列只负责把当前任务登记为“条件变化时需要唤醒的观察者”。这个划分避免等待队列锁变成全局业务锁。若等待队列试图同时保护业务条件，它就必须理解每个子系统的数据结构，最终会破坏分层。

网络套接字状态采用组合策略。发送缓冲区、接收缓冲区、连接状态和等待队列之间存在耦合，但并非所有操作都需要同时修改所有字段。轻量状态查询使用原子或短锁。涉及缓冲区移动和唤醒的路径先在套接字锁内修改状态，再在锁外唤醒等待者。协议栈轮询路径尽量按接口和协议层划分锁，避免单个全局网络锁控制所有包处理。这个策略保持了套接字层与协议栈层的边界，使网络章节可以独立讨论数据包处理，而不需要把所有同步细节合并到套接字对象中。

锁粒度最终服务于所有权划分。一个锁应当保护一个对象的内部不变量，或者保护一组对象之间必须同时变化的关系。它不应当因为调用方便而横跨多个子系统。我们在代码审查中通常会问三个问题。第一，持锁期间是否会调用外部回调。第二，持锁期间是否可能分配或等待。第三，锁保护的不变量能否用一句话描述清楚。若三个问题中任何一个回答含糊，就说明锁边界需要重新审视。

7.8 内存序与可见性

原子操作的内存序是同步机制中最容易被低估的部分。Rust 类型系统能够防止很多数据竞争，但它不能自动决定 CPU 和编译器应当如何重排原子读写。内核运行在多核和弱内存序平台上，RISC-V 与 LoongArch64 都要求我们明确表达获取和释放语义。若锁位已经释放，但被保护数据的写入尚未对其它 CPU 可见，等待者就可能在获得锁后读到旧状态。若等待者先读取业务条件，再看到唤醒标志，条件与标志之间也可能出现时序错位。

我们在同步原语中使用一个相对简单的规则。获取锁使用获取语义，释放锁使用释放语义。状态位的纯观察可以使用宽松语义，但只要观察结果决定是否访问被保护数据，就必须通过锁或其它获取语义建立同步关系。比较交换成功后拥有获取语义，表示后续读写不能越过锁获取。释放语义写入表示临界区内的写入必须先于锁释放对其它获取者可见。这个规则覆盖了自旋锁、互斥锁和部分状态机转换。

```
fn acquire(lock: &AtomicBool) {
    while lock.compare_exchange(false, true, Acquire, Relaxed).is_err() {
        while lock.load(Relaxed) {
            spin_loop();
        }
    }
}

fn release(lock: &AtomicBool) {
```

```
lock.store(false, Release);
}
```

代码 7-5 锁获取和释放的可见性契约

等待队列的可见性更复杂。业务对象通常先修改条件，再唤醒等待者。等待者醒来后重新检查条件。这里唤醒动作本身不携带全部业务语义，真正的可见性由业务对象的锁或原子状态保证。例如管道写入数据后，在管道锁内更新缓冲区计数，释放管道锁后唤醒读等待队列。读者醒来后再次获取管道锁，并读取缓冲区计数。管道锁的释放语义和获取语义形成同步，等待队列只负责让读者有机会重新检查。

对于任务状态，我们使用比较交换表达有限状态转换。睡眠状态到可运行状态的转换不能简单写入可运行状态，因为任务可能已经被超时路径、自身信号或退出路径改变状态。比较交换失败并不一定表示错误，它说明另一个合法路径先完成了状态转移。唤醒者在比较交换失败时通常不再入队，因为任务已经不处于可唤醒睡眠状态。这个设计避免了同一任务重复进入运行队列。

```
fn try_wake(task: &Task) {
    if task.state.compare_exchange(Sleeping, Runnable, AcqRel,
    Acquire).is_ok() {
        enqueue_task(task.clone(), now_ns_public());
    }
}
```

代码 7-6 睡眠任务的条件唤醒

内存序还影响设备状态。设备能力被标记失效后，新 I/O 应当尽快观察到这一事实。失效标志可以使用释放语义写入，I/O 路径使用获取语义读取。这样热插拔路径在标记之前完成的状态更新，对观察到失效的访问方可见。对于只用于统计的计数器，我们通常使用宽松语义，因为它们不承载生命周期判断。这个区分能减少不必要的同步成本，也能避免把统计读数误认为强一致状态。

我们没有在每个原子变量上追求最弱内存序。过度精细的内存序优化会降低可维护性。当前原则是，锁和生命周期状态使用获取、释放或获取释放语义，统计和单纯提示使用宽松语义。若未来某条路径被证明是性能瓶颈，再基于具体访存关系进一步收窄。这样做的收益在于，代码读者可以从内存序直接看出变量是否参与同步协议。

7.9 超时等待、信号打断与取消路径

等待并不总是无限期进行。poll、select、futex 机制、nanosleep、套接字接收、waitpid 等接口都需要支持超时或信号打断。超时等待比普通等待多出一条控制流。任务登记到等待队列后，定时器可能先到期，事件也可能先发生，信号也可能要求提前返回。三条路径都可能试图让任务离开睡眠状态。若没有统一协议，就会出现重复唤醒、遗漏清理或返回值错误。

我们把超时等待拆成三个阶段。第一阶段是准备等待，任务把自己登记到等待队列，并把状态切换为可打断或不可打断睡眠。第二阶段是让出 CPU，调度器负责在事件、超时或信号发生后重新运行该任务。第三阶段是收尾，任务从等待队列中移除自己，重新检查业务条件，并根据原因生成返回值。这个顺序与普通等待保持一致，只是在调度点外增加了截止时间和信号检查。


```

fn wait_event_timeout(queue: &WaitQueue, deadline: Time, condition: impl
Fn() -> bool)
    -> WaitOutcome
{
    let task = current_task();

    loop {
        if condition() {
            return WaitOutcome::Ready;
        }
        if now_ns_public() >= deadline {
            return WaitOutcome::Timeout;
        }
        if task.has_pending_signal() {
            return WaitOutcome::Interrupted;
        }

        queue.prepare_to_wait(&task, TaskState::InterruptibleSleep);
        if condition() {
            queue.finish_wait(&task);
            return WaitOutcome::Ready;
        }

        schedule_until(deadline);
        queue.finish_wait(&task);
    }
}

```

代码 7-7 带超时的等待模式

这里仍然保留登记后的二次条件检查。若事件在登记等待者之后立即发生，任务可以直接返回就绪。若定时器到期和事件同时发生，返回值通常由业务接口定义。poll 类接口更关心最终可见事件，nanosleep 类接口更关心剩余时间，futex 机制则需要检查用户值是否变化。等待队列不参与这些语义判断，它只保证任务不会永久睡眠。

信号打断需要与第十章的信号状态配合。可打断睡眠允许待处理信号使任务提前返回，系统调用层再根据接口语义返回 EINTR、ERESTARTSYS 或重新启动系统调用。不可打断睡眠则只响应真正的事件和超时。我们在同步层不直接决定系统调用返回码，而是把等待结果上交给调用方。这样信号、futex 机制、套接字和文件系统可以在同一等待协议之上表达不同 POSIX 语义。

取消路径同样重要。任务可能在等待期间被退出流程回收，设备可能在等待期间失效，文件可能被另一线程关闭。等待队列保存弱引用，使事件源不会因为等待者存在而延长任务生命周期。业务对象则通过自身状态阻止新请求继续进入。例如套接字关闭会标记关闭状态并唤醒读写等待者。等待者醒来后重新检查套接字状态，并返回连接关闭或文件描述符失效。这个过程没有要求关闭路径直接修改等待者的用户栈，也没有要求等待队列理解套接字的关闭原因。

超时、信号和取消共同说明一个事实。等待队列只提供调度上的睡眠与唤醒能力，业务层必须把等待条件设计成可重试、可取消和可重新检查。若某个等待条件只能检查一次，或者醒来后无法确认事件是否仍然成立，那么这个等待设计本身就不稳

定。我们在多个系统调用中采用循环检查，就是为了把所有竞态收束到同一个模式中。

7.10 典型路径中的同步组合

同步机制只有放回具体路径中，才能看出它的边界是否合理。本节选取几个贯穿内核的路径，说明自旋锁、互斥锁、等待队列和原子状态如何组合使用。

第一个路径是管道读写。管道内部有环形缓冲区、读端计数、写端计数、读等待队列和写等待队列。写入时先获取管道锁，检查缓冲区是否有空间。若空间不足且文件处于阻塞模式，写任务登记到写等待队列，释放锁并睡眠。读端读取数据后，更新缓冲区计数，再唤醒写等待者。这里管道锁保护缓冲区不变量，等待队列表达“空间可能出现”这个事件。写者醒来后仍然需要重新获取管道锁并检查空间，因为多个写者可能同时被唤醒。

第二个路径是 `waitpid`。父任务等待子任务退出时，业务条件是子任务进入僵尸状态或停止状态等可收集状态。父任务不能直接持有子列表锁睡眠，否则子任务退出时需要同一把锁发布状态，会形成阻塞。正确路径是在父任务的等待队列上登记自己，释放相关锁，然后调度出去。子任务退出时修改自身状态，通知父任务等待队列。父任务醒来后重新扫描子列表，确认是否存在可回收对象。这个模式把“子任务状态变化”转化为一次可能唤醒，而不是把等待队列绑定到某个具体子任务上。

第三个路径是套接字接收。接收方调用 `recvmsg` 时，先检查接收缓冲区是否有数据。若没有数据，且套接字处于阻塞模式，任务登记到套接字的读等待队列。协议栈收到数据包后，把载荷放入接收队列，再唤醒读等待者。读者醒来后重新检查接收队列、连接状态和错误状态。若连接已经关闭，`recvmsg` 可能返回 0 或错误。若队列仍为空，说明被其它读者抢先消费，需要继续等待。这里等待队列同样不保证醒来后一定可读，只保证状态发生过可能相关的变化。

第四个路径是 `futex` 机制。`futex` 机制等待的业务条件位于用户地址空间。内核在进入等待前需要读取用户值，并确认它等于期望值。随后把当前任务登记到由用户地址映射出的 `futex` 桶中。登记后需要再次读取用户值，防止用户态在检查和值登记之间完成唤醒。唤醒路径根据同一个键找到等待者并调用调度器入队。`futex` 机制的特殊之处在于业务条件不是内核对象字段，因此 `copy_from_user` 的错误、地址空间解除映射和进程退出都必须成为等待路径的返回条件。

第五个路径是 `close_range`。这个系统调用可能一次关闭大量文件描述符。若在文件描述符表锁内逐个释放文件对象，系统调用时间会被每个文件的析构路径放大，并可能把 VFS 或套接字锁序带入文件描述符表。我们采用锁内批量摘取、锁外批量释放的方式。这样文件描述符表锁只保护数组变更，复杂释放逻辑在表锁释放后进行。这个模式对批量资源回收尤其重要，因为大量文件描述符的集中释放会迅速放大锁内析构带来的尾部延迟。

第六个路径是设备移除。设备能力进入失效状态后，新请求应当失败，旧请求需要排空。块设备会继续推进已经提交的 `Bio` 请求，字符设备会唤醒可能阻塞的读写者，网络设备会通知协议栈接口下线。同步上，这需要一个有序过程。先设置失效标志，再从注册表中移除可发现入口，然后唤醒等待者并排空 I/O，最后释放硬件资源。若先释放硬件资源，再设置失效标志，旧句柄可能继续访问已经失效的 MMIO 或队列。若只从注册表移除，不唤醒等待者，阻塞在设备上的任务可能无法返回。

这些路径没有使用同一种锁解决所有问题。它们共同遵守的是同一套组合规则。对象锁保护对象内部状态, 等待队列表达条件变化, 原子状态处理快速观察和一次性转换, 锁外回调避免跨子系统锁序扩散。这样的组合方式比某个单独原语更重要, 因为内核中的复杂竞态通常发生在两个子系统交接处, 而不是单个锁的实现内部。

7.11 调试、验证与故障收束

同步错误常常表现为卡死、偶发超时或资源泄漏。它们很少稳定地变成同一个内核崩溃。为了降低定位成本, 我们在同步设计中尽量让错误收束到可观察的状态。等待队列使用弱引用, 可以通过队列长度和成功升级数量判断是否存在陈旧等待者。任务状态使用有限枚举, 可以在非法转换时打印当前状态和目标状态。锁内不执行复杂回调, 可以通过审查持锁范围推断可能的死锁边界。

调试同步问题时, 第一步应当明确卡住的任务等待什么条件, 而不是立即增加日志。若任务卡在互斥锁获取路径, 需要确认持锁者是谁, 以及持锁者是否可能睡眠。若任务卡在等待队列, 需要确认业务条件是否可能再发生变化, 以及唤醒路径是否真的调用了唤醒函数。若任务卡在运行队列之外, 需要确认任务状态是否仍为睡眠状态, 还是已经变成可运行状态但没有入队。三类问题对应不同层级, 混在一起分析会浪费大量时间。

我们在设计等待协议时有意避免在热路径打印日志。系统调用、文件 I/O 和网络收发都会高频触发等待事件, 热路径日志会显著改变调度时序。更可取的方式是记录低成本计数器, 或者在异常分支输出一性诊断。例如等待超时后记录队列长度、当前任务状态和业务对象状态。设备失效后输出设备名和未完成请求数。调度器发现任务状态不一致时输出任务 PID、命令名和状态字段。这些信息足以定位大部分同步错误, 同时不会让正常路径变慢。

```
fn wait_with_diagnostics(queue: &WaitQueue, deadline: Time) -> WaitOutcome {
    let outcome = wait_event_timeout(queue, deadline, condition_ready);
    if outcome == WaitOutcome::Timeout {
        trace_once("wait timeout", queue.debug_len(),
current_task().state());
    }
    outcome
}
```

代码 7-8 等待超时后的低成本诊断

死锁验证依赖锁序文档。我们没有为每一把锁建立庞大的形式化证明, 但核心路径保持了几个固定规则。运行队列锁不嵌套可睡眠锁。等待队列锁不调用调度器入队。文件描述符表锁内不释放文件对象。设备注册表锁外发布事件。VFS 路径解析锁不反向调用挂载注册。网络套接字唤醒在释放套接字锁后进行。这些规则的价值在于可审查。只要新代码违反其中一条, 审查者就能立即要求调整。

资源泄漏也是同步错误的一种表现。等待队列保存强引用会保活任务, 任务保存强引用又可能保活文件和地址空间, 最终使退出回收无法完成。我们使用弱引用避免这类循环。弱引用带来的代价是唤醒时需要处理升级失败, 这要求唤醒路径天然容忍等待者已经消失。这个代价是可接受的, 因为唤醒本来就只表示条件可能变化。升级失败说明等待者已经不再需要唤醒, 不影响业务正确性。

同步验证还需要结合真实并发负载。单线程启动路径很难覆盖等待竞态。fork、wait、futex 机制、管道、poll、套接字、文件写回和设备移除等场景会让睡眠、唤醒和回收路径同时活动。我们在分析这些负载时，通常关注两类现象。第一是任务进入可打断睡眠后能否被异步事件唤醒。第二是大量短操作后是否出现资源滞留，这检验锁外释放和引用回收是否及时。同步机制的设计如果能在这些负载下保持可收束，后续扩展才有可靠基础。

7.12 工程取舍与演进边界

同步机制的难点不只在于写出正确的锁实现，还在于为后续维护留下清晰边界。内核的子系统会持续扩展，文件系统会增加更多缓存层，网络栈会增加更多协议状态，设备模型也会接入更多类型的设备能力。若同步原语本身过于专门化，新功能就会绕过它自行实现等待和唤醒。若同步原语过于庞大，所有子系统又会被迫理解一套复杂的全局机制。我们采用较小的核心原语，再通过固定使用模式组合它们，正是为了在这两个方向之间保持可演进空间。

第一个取舍是性能与可审查性的关系。细粒度锁和无锁结构在理论上可以提高并行度，但它们要求维护者同时理解内存序、生命周期和业务不变量。系统负载覆盖面很宽，很多错误只会在特定时序下出现。我们宁愿在早期使用较粗但边界明确的锁，也不把多个字段拆成大量原子状态。这样做可能让某些极端微基准不够理想，但可以使网络、文件系统和进程管理中的状态变化更稳定。等某条路径的真实竞争被识别出来，再沿着对象边界拆锁，风险会低很多。

第二个取舍是通用等待协议与接口特殊语义之间的关系。poll、futex 机制、套接字、waitpid 和 nanosleep 都可以使用等待队列，但它们的返回语义不同。poll 需要返回事件集合，futex 机制需要重新比较用户值，套接字需要区分正常关闭和错误，nanosleep 还要计算剩余时间。我们没有把这些语义放进等待队列，而是让等待队列只负责睡眠和唤醒。调用方在醒来后重新解释业务状态。这个取舍使等待队列足够简单，也让各系统调用保留自身的 POSIX 兼容细节。

第三个取舍是热路径日志与可诊断性的关系。同步错误需要诊断信息，但热路径日志会改变时序。尤其在系统调用和网络收发路径中，额外输出会让本来偶发的竞态消失，或者把一个延迟问题放大成新的性能问题。我们更倾向于在异常分支、超时分支和状态不一致分支输出结构化信息。正常路径只维护低成本计数器。这样在问题发生时仍然能看到任务状态、队列长度和对象状态，平时不会让同步路径承受额外 I/O 成本。

第四个取舍是跨架构一致性与平台优化之间的关系。RISC-V 和 LoongArch64 的内存模型、异常返回和中断屏蔽细节并不完全相同，但高层同步原语不能让调用方感知这些差异。自旋锁、互斥锁和等待队列都提供统一语义，架构差异被限制在原子指令、屏障和调度入口附近。这样做使 VFS、套接字和设备模型可以共享同一套同步协议。若未来某个平台需要更强的屏障或更高效的等待提示，也应当在原语内部调整，而不是让每个子系统分支处理。

第五个取舍是失败路径的处理方式。内核不能假设每次分配、每次唤醒和每次队列登记都成功。等待队列登记失败时，调用方应当返回错误或短暂重试，不能把任务状态改成睡眠状态后失去唤醒来源。唤醒时弱引用升级失败应当被视为正常情况，因为等待者可能已经超时或退出。锁获取路径如果检测到非法上下文，应当尽早暴露，

而不是继续执行到更深处才出现不可解释的卡死。同步机制的鲁棒性很大程度体现在失败路径能否收束。

这些取舍形成了第七章后续章节的基础。异常和系统调用章节会使用任务状态与返回前收尾。用户态执行环境章节会在 `exec` 和地址空间切换中依赖锁外释放。信号章节会使用可打断睡眠和系统调用重启。终端章节会使用等待队列连接输入事件和读系统调用。网络章节会大量使用套接字等待和协议栈轮询。同步机制因此是后续章节共同使用的工程语言。我们在这里把边界讲清楚, 后文就可以专注于每个子系统自己的状态机, 而不必反复解释睡眠、唤醒和锁序的基础规则。

7.13 工程设计总结

并发与同步机制贯穿整个内核。它不像 VFS 或网络那样有单一用户可见接口, 却决定了所有接口能否可靠运行。本章把同步机制分成短临界区、可睡眠临界区和事件等待三类, 并说明它们与调度器状态机的关系。这些机制共同支撑了内核在高并发负载下的稳定性。同步机制的价值来自每个原语清晰的上下文限制和交接规则, 也来自这些规则在不同子系统的一致复用。只要这些规则保持稳定, 后续扩展新的套接字等待、设备事件或文件系统缓存时, 就可以沿用同一套并发模型。同步层因此成为连接调度、资源管理和用户态可观察行为的一条基础线索。它把局部互斥、条件等待和生命周期收束组织在同一套工程纪律之下, 使复杂路径仍然能够被逐段推理, 也使故障定位能够从任务状态、等待条件和锁边界三个方向同时展开。这也是后续章节能够减少重复解释的重要原因之一, 并能提升审查效率。实际维护时, 开发者只要先确认当前路径所在上下文, 再确认业务条件由谁保护, 最后确认唤醒是否在锁外发生, 就能排除大部分同步设计错误。

第八章 异常处理与系统调用接口

在第七章中，同步机制给内核内部的并发访问提供了基本纪律。本章转向用户态和内核态之间的入口。用户程序无法直接调用内核函数。它只能通过异常、系统调用、缺页和中断等硬件入口进入内核。异常处理子系统负责保存现场、判断来源、规范化硬件状态，并把控制权交给平台无关的内核逻辑。系统调用子系统则在此基础上提供稳定的 POSIX ABI。

异常和系统调用必须同时满足两个目标。第一，架构差异必须被限制在清晰边界内。RISC-V64 使用 `ecall`、`sepc` 和 `sstatus`。LoongArch64 使用 `syscall`、`ERA` 和 `PRMD`。寄存器编号、程序计数器推进方式和返回指令都不同。第二，系统调用实现不能被架构细节污染。`openat`、`clone`、`mmap` 和 `rt_sigaction` 等实现只应看到统一的参数数组、当前任务和陷阱帧引用。

8.1 异常入口的职责

异常入口的第一职责是保存现场。来自用户态的异常必须切换到内核栈，并保存用户寄存器、程序计数器、状态寄存器和异常原因。来自内核态的异常可以沿用当前内核栈，但仍需要构造统一的陷阱帧。第二职责是区分异常类型。系统调用可以走更短路径，缺页需要进入内存管理，设备中断需要进入中断分发，非法指令和访问错误需要转化为信号或内核故障。第三职责是返回用户态前恢复现场，并在需要时投递信号或执行调度。



RISC-V64 的异常入口包含系统调用快速路径。来自用户态的 `ecall` 可以跳过 FPU 保存，直接保存系统调用所需的通用寄存器并调用快速处理函数。若返回路径

需要完整恢复，例如 FPU 处于活动状态、信号投递或调度发生，入口会转入完整恢复路径。LoongArch64 则有普通异常入口、TLB 重填快路径和机器错误入口。TLB 重填尽量依赖硬件页表遍历快速填充，机器错误采用停止运行策略。

8.2 系统调用帧接口

平台无关的系统调用分发通过系统调用帧接口（SyscallFrameOps）访问陷阱帧。架构层提供四个回调。取系统调用号、取六个参数、写返回值和推进程序计数器。这个接口把 ABI 寄存器约定留在架构层，把分发表和系统调用实现留在 general 与 kernel 层。

```
struct SyscallFrameOps {
    sys_nr: fn(TrapFramePtr) -> usize,
    sys_args: fn(TrapFramePtr) -> [usize; 6],
    set_sys_ret: fn(TrapFramePtr, isize),
    advance_pc: fn(TrapFramePtr),
}

fn dispatch(tf: TrapFramePtr) {
    let ops = frame_ops().unwrap();
    let nr = (ops.sys_nr)(tf);
    let args = (ops.sys_args)(tf);
    let task = current_task();

    let mut ctx = SyscallContext::new(nr, args, tf, task);
    let ret = call_registered_syscall(&mut ctx);

    if !ctx.frame_finalized() {
        (ops.set_sys_ret)(tf, ret);
        (ops.advance_pc)(tf);
    }
}
```

代码 8-1 SyscallFrameOps 契约

系统调用上下文（SyscallContext）把系统调用号、参数、陷阱帧和当前任务打包。普通系统调用只需要参数和当前任务。execve 和 sigreturn 这类特殊调用会重写完整陷阱帧，因此可以调用 finalize_frame 阻止默认写返回值和推进程序计数器。这个设计避免每个系统调用都理解异常返回细节，同时保留少数特例的控制能力。

8.3 表驱动分发

系统调用表是固定长度数组，条目为函数指针。启动期由 kernel::syscalls::register_all 注册，运行期只做无锁读取。重复注册会触发崩溃处理，避免表项被静默覆盖。未注册的系统调用返回 ENOSYS。这种方式把兼容范围和实现入口显式列出，也让系统调用覆盖范围能够通过表项注册情况直接定位。

```
type SyscallFn = fn(&mut SyscallContext) -> Result<usize, Errno>;
const SYSCALL_TABLE_LEN: usize = 512;
static SYSCALL_TABLE: [AtomicUsize; SYSCALL_TABLE_LEN] = [...];
```



```
fn register_syscall(nr: usize, f: SyscallFn) {
    let old = SYSCALL_TABLE[nr].compare_exchange(0, f as usize, AcqRel,
    Acquire);
    assert!(old.is_ok());
}

fn call_registered_syscall(ctx: &mut SyscallContext) -> isize {
    let ptr = SYSCALL_TABLE[ctx.nr].load(Acquire);
    if ptr == 0 {
        return -ENOSYS;
    }
    let f = unsafe { transmute::<usize, SyscallFn>(ptr) };
    match f(ctx) {
        Ok(value) => value as isize,
        Err(errno) => -(errno.as_i32() as isize),
    }
}
```

代码 8-2 系统调用表驱动分发

表驱动分发的好处是热路径短。系统调用是用户态进入内核最频繁的路径之一。若每次调用都进入全局锁或字符串匹配,简单系统调用的成本会明显上升。固定数组配合启动期注册,让运行期只需要边界检查、原子读和一次函数调用。系统调用实现按文件系统、进程、内存、进程间通信、信号和系统日志分类,便于维护。

8.4 返回前收尾

系统调用返回前有三类收尾。第一类是普通返回,写入返回值并推进程序计数器。第二类是信号处理。若当前任务有待处理信号,分发器会调用调度层操作,在用户栈上构造信号帧,或者执行默认动作。第三类是调度交接。任务可能在系统调用期间进入僵尸状态、停止状态或继续状态。clone 后也可能设置系统调用后交接。分发器必须在返回用户态前处理这些状态。

这里有一个重要边界。系统调用实现返回 Err(EINTR) 时,分发器会尝试消费可重启信号,并在需要时构造信号帧。若信号帧已经建立,分发器不再写普通返回值。这个行为使信号机制能介入阻塞系统调用,同时不要求每个系统调用都手写信号投递逻辑。

8.5 异常、缺页与信号

系统调用只是异常处理的一类。用户态缺页会进入内存管理的缺页路径。非法指令、访问错误和断点等用户异常会转换为对应信号,例如 SIGILL、SIGSEGV、SIGBUS 或 SIGTRAP。内核态异常则需要区分可恢复用户访问错误和致命内核故障。第二章提到的用户访问接口 (UserAccessOps) 和缺页解码,正是为了把用户指针访问错误转化为系统调用可返回的 EFAULT,而不是让内核直接崩溃。

异常处理的关键在于来源判断。用户态错误通常应该反馈给用户进程。内核态错误若发生在显式用户访问窗口,可以转化为错误码。若发生在普通内核代码中,说明

内核不变量被破坏，应当停止运行。这个边界保护了内核自身的可信状态，也让用户态恶意指针无法扩大为内核崩溃。

8.6 入口分层与状态规范化

异常入口必须在极短时间内完成状态规范化。硬件把控制权交给内核时，提供的是架构相关现场。RISC-V64 的异常原因来自 `scause`，异常地址来自 `stval`，返回地址保存在 `sepc`。LoongArch64 的异常原因和子原因来自控制寄存器，返回地址位于 `ERA`，旧特权级信息位于 `PRMD`。如果平台无关层直接读取这些寄存器，系统调用、缺页和信号处理就会到处出现架构分支。我们采用的方式是，在入口层尽快把硬件现场转化为陷阱帧（`TrapFrame`）和规范化的异常枚举，再向上层交接。

规范化并不意味着抹平所有差异。不同架构的程序计数器推进规则、异常嵌套规则、页表错误编码和返回指令仍然不同。入口层只把上层需要消费的信息整理出来。系统调用层需要系统调用号、参数、返回值写入位置和程序计数器推进动作。缺页层需要访问地址、访问类型、用户态或内核态来源。信号层需要产生信号的原因、用户程序计数器和可恢复上下文。调度层需要知道当前任务是否来自用户态，是否可以在返回前执行抢占。每一类信息都在入口层整理为稳定接口。

表 8-1 异常入口向上层交付的信息

消费方	需要的信息	入口层职责
系统调用分发	系统调用号、六个参数、返回寄存器、程序计数器推进方式。	通过 <code>SyscallFrameOps</code> 提供统一访问，不暴露寄存器编号。
内存缺页处理	缺页地址、访问类型、用户态来源、可恢复标记。	解析架构异常码，构造平台无关缺页信息。
信号投递	用户程序计数器、栈指针、异常原因和信号码。	保留可重建用户上下文，支持在用户栈上构造信号帧。
调度收尾	是否来自用户态、是否需要返回前检查重调度和待处理信号。	把返回用户态作为统一安全点，避免异步处理打断关键区。

入口层还需要保证内核栈选择正确。用户态进入内核时，不能继续使用用户栈，因为用户栈可被用户程序任意修改，也可能尚未映射。入口汇编必须切换到当前任务的内核栈，随后再保存寄存器。内核态异常则已经处于内核栈上，处理路径可以复用当前栈，但要小心嵌套异常。我们在入口层保留来源判断，使上层能够区分用户异常、内核异常和显式用户访问窗口中的异常。这个来源判断直接决定后续是给用户进程发信号、返回 `EFAULT`，还是触发内核故障处理。

规范化还包括错误码归一。硬件异常码通常无法直接作为 POSIX 错误码返回。页错误可能对应 `EFAULT`、`ENOMEM` 或 `SIGSEGV`。非法指令可能对应 `SIGILL`。未实现系统调用对应 `ENOSYS`。系统调用层以系统调用结果（`Result<usize, Errno>`）作为内部返回类

型，分发器统一把错误码取负写入返回寄存器。这使每个系统调用实现可以按 Rust 错误类型表达失败原因，而不需要手动处理 ABI 负返回值规则。

8.7 快速路径与完整路径

系统调用是最频繁的异常类型之一。getpid、clock_gettime、read、write、futex 机制和 poll 都会为用户程序中高频出现。异常入口如果总是保存完整上下文，简单系统调用就会承担不必要成本。我们因此区分快速路径和完整路径。快速路径只保存系统调用分发和返回所需的通用寄存器，尽量减少额外状态处理。完整路径保存更完整的现场，处理信号、调度、FPU 状态、调试异常和复杂恢复。

快速路径不能破坏正确性。它只适用于来源明确、上下文简单、返回前不需要额外处理的场景。一旦系统调用过程中发现待处理信号、需要调度、需要重写陷阱帧，或者当前任务处于特殊状态，就必须转入完整返回路径。这个设计类似一个短判定链。常见路径走短分支，异常情况回到通用处理。

```
fn syscall_fast_entry(tf: TrapFramePtr) -> TrapExit {
    let ret = dispatch_fast(tf);

    if current_task().has_pending_signal() {
        return TrapExit::FullReturn;
    }
    if current_task().need_resched() {
        return TrapExit::FullReturn;
    }
    if frame_was_finalized(tf) {
        return TrapExit::FullReturn;
    }

    write_ret_and_advance_pc(tf, ret);
    TrapExit::FastReturn
}
```

代码 8-3 系统调用快速路径的控制结构

RISC-V64 的优化重点在于减少入口保存和恢复的寄存器数量。系统调用 ABI 只需要保存返回后用户态仍需保留的寄存器，以及系统调用参数和返回值寄存器。若每次都保存全部通用寄存器，简单系统调用的成本会被入口代码放大。LoongArch64 也有类似问题，只是寄存器命名和异常返回机制不同。我们把优化点放在架构入口，而不是在某几个系统调用内部硬编码特例。这保证了所有简单系统调用都能受益。

完整路径的价值在于提供统一收尾。信号投递需要在用户栈上写入信号帧，并把用户程序计数器修改为处理函数地址。调度让出需要保存当前任务状态，并在重新运行后继续返回。execve 需要替换地址空间和用户上下文。rt_sigreturn 需要从用户栈恢复旧上下文。所有这些操作都超出了简单返回值写入的范围。快速路径只负责识别自己不能处理的情况，然后交给完整路径。

快速路径和完整路径之间的边界必须可证明。我们不把“当前系统调用号是否简单”作为唯一依据，因为简单系统调用也可能遇到待处理信号或调度请求。真正的判断条件来自当前任务和陷阱帧状态。若任务仍然运行，未请求调度，没有待处理信号，

陷阱帧未被特殊系统调用接管，入口可以直接返回。否则走完整路径。这个判定方式比按系统调用白名单更稳健，也避免后续新增系统调用时忘记维护快速路径列表。

8.8 参数、返回值与错误码语义

系统调用 ABI 的表面形式很简单。用户态把系统调用号和参数放入约定寄存器，执行陷入指令，内核写回返回值。实际实现中，返回值语义需要同时兼容 POSIX、libc 和架构 ABI。成功返回通常是非负值，失败返回是负错误码。用户态 libc 再把负返回值转化为 `errno` 并返回 `-1`。少数系统调用使用特殊返回约定，例如 `clone` 在父子任务中返回不同值，`execve` 成功后不返回旧上下文，`rt_sigreturn` 恢复旧返回值。

我们在系统调用实现中使用系统调用结果类型。成功值保持 `usize`，错误值保持强类型错误码。分发器负责把它编码为 ABI 返回值。这样可以避免每个系统调用手动写 `-(errno as isize)`，也能减少正负值混用带来的错误。对于需要返回负数作为合法值的接口，系统调用实现必须在语义层明确处理，不能绕过统一编码。

```
fn encode_ret(result: Result<usize, Errno>) -> isize {
    match result {
        Ok(value) => value as isize,
        Err(errno) => -(errno.as_i32() as isize),
    }
}
```

代码 8-4 系统调用返回值编码

参数读取也需要规范化。六个参数在不同架构上位于不同寄存器。系统调用实现拿到的是 `[usize; 6]`。指针参数不会在入口层自动检查，因为入口层无法知道它是只读缓冲区、可写缓冲区、字符串、数组还是结构体。具体系统调用通过用户指针 (`UserPtr`)、用户切片 (`UserSlice`) 或对应辅助函数执行访问。这样指针检查靠近语义使用点，错误码也更准确。`read` 的缓冲区写入失败返回 `EFAULT`，`execve` 的 `argv` 读取失败也返回 `EFAULT`，但它们对最大长度、空指针和字符串终止的要求不同。

系统调用号越界和未注册统一返回 `ENOSYS`。这个行为对兼容性很重要。libc 会根据 `ENOSYS` 判断某些新接口是否由内核支持，并可能回退到旧接口。例如 `clone3` 不存在时，用户态可能尝试 `clone`。若内核对未知系统调用返回其它错误，libc 的探测逻辑会受到影响。固定分发表和统一未实现返回，使兼容层行为更可预测。

错误码的稳定性还影响用户态兼容。很多程序不仅关心系统调用是否失败，也依赖错误码的具体含义。`accept` 在普通文件上应返回 `ENOTSOCK`，无效指针应返回 `EFAULT`，无效标志应返回 `EINVAL`。系统调用框架本身不能替业务实现决定这些细节，但它必须保证错误码能无损传递到用户态。统一负返回编码正是这个基础。

8.9 用户访问窗口与可恢复异常

系统调用大量处理用户指针。读取路径包括路径字符串、I/O 向量、套接字地址、信号集、时间结构和 `futex` 地址。写入路径包括 `read` 缓冲区、`stat` 结构、`gettimeofday` 输出、`recvmsg` 控制消息和 `wait` 状态。用户指针可能指向未映射页面，可能跨越权限边界，也可能在检查后被其它线程修改。内核不能把用户指针当成可信内核地址。

我们把用户访问放在显式窗口中处理。进入用户访问辅助函数前, 内核知道当前正在执行可恢复用户访问。若该窗口内触发缺页或访问错误, 缺页处理器可以把错误转换为 EFAULT, 而不是认为内核出现致命错误。窗口之外的内核缺页仍然按内核故障处理。这个边界让用户态恶意地址无法导致内核崩溃, 也避免把真正的内核缺陷隐藏成普通错误码。

```
fn copy_from_user(dst: &mut [u8], src: UserPtr<u8>) -> Result<(), Errno> {
    user_access_begin();
    let ret = raw_copy(dst.as_mut_ptr(), src.addr(), dst.len());
    user_access_end();

    if ret.faulted {
        return Err(Errno::EFAULT);
    }
    Ok(())
}
```

代码 8-5 用户访问窗口的错误恢复

用户访问窗口还需要与调度和信号保持边界。copy_from_user 本身不应持有运行队列锁, 也不应在不可恢复的自旋锁临界区内执行。若用户页缺失, 缺页路径可能需要分配物理页或加载文件映射, 这些操作可能阻塞。即使当前实现暂时没有完整换页, 也应保持接口边界。调用方应先把需要保护的内核状态整理好, 再执行用户复制。若复制失败, 调用方根据业务语义回滚或返回错误。

字符串读取尤其容易出错。openat、execve、chdir 等接口需要从用户态读取以 NUL 结尾的字符串。内核必须限制最大长度, 防止用户传入没有终止符的巨大地址范围。读取 argv 和 envp 时, 还需要先读取指针数组, 再逐个读取字符串。每一步都可能失败, 失败后已经分配的中间对象必须释放。用户访问 helper 因此不能只提供裸复制, 还要提供按语义组织的上层工具。

用户访问窗口对性能也有影响。小块复制是系统调用热路径。read 和 write 经常复制几百字节到几千字节, 套接字控制消息和 stat 结构也会频繁复制。若每次复制都进入复杂动态分派, 系统调用成本会明显增加。我们在架构层保留可优化的原始复制实现, 高层通过统一辅助函数调用。这样既能在 RISC-V64 和 LoongArch64 上分别优化指令序列, 又能让系统调用实现保持相同接口。

8.10 系统调用注册与分类组织

系统调用数量多, 且涉及多个子系统。若所有实现都堆在一个文件中, 维护成本会随接口数量快速上升。我们按当前代码中的功能域组织注册。文件、目录、描述符、事件文件和 socket 入口目前集中在 fs.rs, 其中 socket 相关系统调用只承担用户指针拷贝、文件描述符查找和 VFS socket 转发, 真正的网络语义继续下沉到 VFS socket 层和 libs/net。进程和线程相关接口在 process.rs, 内存接口在 mm.rs, 进程间通信接口在 ipc.rs, 信号接口在 signal.rs, 系统日志接口在 syslog.rs。启动期 register_all 负责把这些实现绑定到统一系统调用表。

表 8-2 系统调用分类与依赖关系

类别	典型接口	主要依赖
文件与目录	openat、read、write、getdents64、fcntl。	VFS、文件描述符表、用户指针复制、权限检查。
进程与线程	clone、fork、execve、wait4、exit_group。	任务对象、调度器、地址空间、信号和父子关系。
内存管理	mmap、munmap、mprotect、brk。	用户虚拟地址空间、页表、用户访问窗口、文件映射。
信号	rt_sigaction、rt_sigprocmask、rt_sigsuspend。	信号状态、返回前投递、用户栈信号帧。
网络与 IPC	socket、bind、connect、sendmsg、recvmsg、pipe。	套接字层、等待队列、协议栈、VFS 文件对象。

分类组织的核心价值是控制依赖方向。系统调用分发器不理解 VFS、网络和内存管理的具体逻辑。它只负责从表中找到函数并调用。各功能模块在注册时声明自己的系统调用号。这样新增一个系统调用时，开发者需要同时明确它属于哪个功能域、依赖哪些子系统、如何处理用户指针和错误码。注册表成为 ABI 覆盖范围的集中索引。

启动期注册还有一个好处。系统调用表在运行期基本只读，避免热路径加锁。重复注册使用 CAS 检测并触发崩溃处理，可以在启动阶段暴露编号冲突。若用运行期可变哈希表，冲突可能在第一次调用时才暴露，甚至被后注册的实现静默覆盖。系统调用 ABI 不应在运行中变化，启动期固定化更符合内核接口性质。

分类组织也有助于维护定位。出现某个系统调用语义问题时，可以直接从系统调用号定位到模块。例如 fcntl 失败通常进入文件描述符和文件标志语义，ptrace 失败进入进程控制和信号停止语义，mmap 失败进入地址空间管理。注册表清晰后，用户态观察到的系统调用名能够快速映射到源码区域，减少排查成本。

8.11 返回前安全点与异步事件

用户态返回前是内核处理异步事件的天然安全点。系统调用执行过程中，内核可能持有锁、正在修改对象、处于不可睡眠上下文，或者正在复制用户缓冲区。若异步信号在任意位置打断执行，就会破坏这些临界区。我们把信号投递、调度让出和部分任务状态收尾放到返回用户态之前处理。此时系统调用主体已经结束，大部分锁已经释放，陷阱帧仍然可修改，用户态还没有恢复执行。

待处理信号的处理需要修改用户上下文。若信号有用户处理函数，内核要在用户栈上写入信号帧，保存旧程序计数器、旧寄存器、旧信号屏蔽字和返回跳板，然后把陷阱帧中的程序计数器改为处理函数地址。若信号默认动作为终止，当前任务进入退出流程。若信号被屏蔽，则暂不投递。所有这些动作都需要访问当前任务的信号状态和陷阱帧，因此放在返回前最合适。

调度让出也适合在返回前处理。系统调用期间可能设置重调度标志 `need_resched`。例如时间片耗尽、唤醒了更高优先级任务，或者 `clone` 后需要让新任务尽快运行。若当前任务即将返回用户态，调度器可以在内核态完成一次安全切换。等任务重新获得 CPU 后，再继续异常返回。这样用户态观察到的是系统调用正常返回，只是返回时间稍晚。

`execve` 和 `rt_sigreturn` 是特殊情况。`execve` 成功后，旧用户上下文已经失效。内核必须安装新地址空间、新用户栈和新入口地址，然后直接返回到新程序。`rt_sigreturn` 则从用户栈取回旧信号帧，恢复到信号发生前的上下文。这两个系统调用都会接管陷阱帧，分发器不能再写普通返回值。`finalize_frame` 正是为这类场景提供显式标记。

```
fn finish_trap_return(tf: TrapFramePtr) {
    if current_task().need_resched() {
        schedule_once(now_ns_public());
    }

    if current_task().has_pending_signal() {
        deliver_signal_or_default(tf);
    }

    if current_task().is_zombie() {
        enter_exit_tail();
    }
}
```

代码 8-6 返回前收尾的分层顺序

实际顺序需要结合任务状态细化。若任务已经进入退出流程，信号投递不应再构造用户处理函数。若信号处理导致任务终止，也不应继续普通返回。若调度后任务收到新信号，返回前还要重新检查待处理信号。返回前安全点因此通常采用循环或多次检查，而不是单次 `if` 链。重要的是所有异步事件都在用户态边界集中处理，避免分散到每个系统调用实现中。

8.12 性能拆解与实现边界

系统调用性能不能只看某个系统调用实现。一次简单系统调用的开销包括用户态陷入指令、硬件特权级切换、入口汇编保存现场、栈切换、系统调用号和参数读取、分发表查找、函数调用、返回值写入、程序计数器推进、返回前状态检查和异常返回指令。若开启更多调试或信号检查，还会增加额外分支。我们在优化时必须先把这些组成部分拆开，否则很容易把总体差异误归因到某个子系统。

极短系统调用通常不会触发 VFS、内存分配或复杂等待，因此主要反映异常入口、分发器和返回路径成本。RISC-V64 与 LoongArch64 的差异需要从汇编入口、寄存器保存数量、内存访问次数、页表和 TLB 行为等多个角度分析。若只观察 Rust 层系统调用实现，很可能看不到真正瓶颈。

表 8-3 简单系统调用的主要成本段

成本段	内容	优化方向
入口硬件成本	陷入指令、特权级切换、异常寄存器读取。	无法由 Rust 层消除, 只能减少入口后续工作。
现场保存	保存通用寄存器、切换栈、构造陷阱帧。	区分快速路径和完整路径, 减少不必要保存。
分发成本	读取系统调用号、参数数组、查表、间接调用。	固定数组、启动期注册、运行期无锁读取。
返回收尾	写返回值、推进程序计数器、检查信号和调度。	把常见无事件路径保持为短分支。
异常返回	恢复寄存器、返回用户态、恢复状态寄存器。	架构汇编优化, 避免冗余内存访问。

分析系统调用路径时, 我们避免在热路径打印日志。插桩应当只记录时间戳或计数器, 并在需要时汇总。若每次系统调用都输出文本, 结果就会主要反映控制台和锁竞争成本。更稳妥的方法是在入口和出口记录周期数或纳秒差值, 分段存入每 CPU 缓冲区。每段只执行简单加法和计数, 不分配内存, 不格式化字符串。需要观察时再通过调试接口读取统计。

优化也要避免过早特化。若直接为 `getpid` 写一条特殊返回路径, 微基准可能变快, 但 `read`、`write`、`clock_gettime` 和其它短系统调用不会受益。我们更关注总体入口路径, 例如减少通用寄存器保存、缩短返回前检查、让分发表查找保持无锁。这样的优化对所有系统调用有效, 也更符合内核 ABI 层的职责。

性能分析还需要区分架构设计成本和具体运行环境成本。不同平台对异常入口、原子指令和寄存器保存的成本不同。我们在分析结果时更关注可解释的组成部分, 而不是简单归结为某个平台慢。

8.13 故障路径与内核可信边界

异常处理还承担内核可信边界的维护。用户态可以传入任意系统调用号、任意参数和任意指针。它可以制造非法指令、访问未映射地址, 也可以反复触发缺页。内核必须把这些行为约束在当前进程范围内。用户错误通常产生错误码或信号。内核错误则需要停止运行, 以避免破坏全局状态后继续执行。

可恢复内核异常只允许出现在明确标记的用户访问窗口中。例如 `copy_from_user` 期间发生用户页访问错误, 可以返回 `EFAULT`。若同样的缺页发生在普通内核指针访问中, 就说明内核自身使用了非法地址。把普通内核缺页也吞掉会掩盖严重缺陷。我们因此把恢复能力绑定到显式窗口, 而不是在缺页处理器中无条件尝试继续执行。

内核态系统调用实现还必须处理分配失败。分配失败不应默认触发内核崩溃。很多系统调用可以把 `ENOMEM` 返回用户态, 例如创建套接字、扩展文件描述符表、分配路径缓冲或构造 I/O 向量。只有破坏内核基础不变量的分配失败才可能触发不可恢

复处理。系统调用层本身不决定是否停止运行,而是让具体子系统根据对象生命周期判断。这个原则能避免用户态通过资源压力把普通资源不足放大为内核崩溃。

异常入口本身的故障则更严重。若入口无法切换到内核栈,无法保存最小现场,或者发现当前任务结构损坏,内核很难继续提供隔离。此时停止运行比带着损坏状态继续执行更安全。文档中需要区分这两类失败。用户请求导致的资源不足应尽量返回错误。入口基础设施损坏或内核不变量破坏应停止运行。这个边界保持清晰后,普通资源不足和真正内核故障才能被区分。

故障路径还涉及审计信息。用户态异常转信号时,应记录足够的 `si_code`、缺页地址和信号编号。系统调用返回错误时,错误码本身就是用户可见诊断。致命内核故障则需要打印程序计数器、返回地址、栈指针、缺页地址、异常码、当前进程号和任务名。输出信息不需要过度冗长,但必须能支持定位到入口路径、异常类型和当前任务。我们在异常处理设计中保留这些字段,正是为了让故障不只表现为静默停机。

8.14 兼容性与 ABI 视角

系统调用接口是用户态程序和内核之间最稳定的契约。用户态程序通常不关心内核内部使用哪种调度器、哪种 VFS 缓存,也不关心设备模型如何组织设备能力。它关心的是系统调用号是否存在,参数解释是否符合 ABI,返回值和错误码是否稳定,信号是否在正确的边界投递。异常和系统调用章节因此不能只描述入口机制,还必须说明这套机制如何支撑兼容性。

用户态可以传入非法 `fd`、非法指针、非法 `flag`、边界长度和不完整结构体。若系统调用实现直接解引用用户指针,内核会崩溃。若错误码选择不对,用户程序会走入错误分支。若系统调用表漏注册,用户态会收到 `ENOSYS`。这些现象分别对应入口恢复、业务语义和 ABI 覆盖三个层面。我们把系统调用框架设计成表驱动和强类型错误返回,就是为了让这三个层面可以分别定位。

运行库兼容也依赖稳定的返回边界。用户态运行库会根据错误码决定回退路径。例如某些接口在新内核上使用新系统调用,在旧内核上回退到旧组合。若未知系统调用没有返回 `ENOSYS`,用户态探测就可能走错分支。信号相关接口对 `EINTR` 和重启语义的要求也很严格。一个阻塞系统调用被信号打断后,是否自动重启取决于信号动作、系统调用类型和返回前收尾。把这些处理集中在系统调用边界,可以避免每个阻塞接口各自实现一套不一致的重启逻辑。

兼容性还要求异常处理对用户态错误保持隔离。用户程序执行非法指令应收到 `SIGILL`,访问非法地址应收到 `SIGSEGV` 或 `SIGBUS`,断点相关异常应转为 `SIGTRAP`。这些信号需要携带合理的缺页地址和信号码。若所有用户异常都简单终止进程,很多调试器和运行库会失去必要信息。若用户异常被当成内核缺页处理,系统稳定性又会受到影响。入口层的来源判断和信号映射因此是兼容性的一部分。

从工程顺序看,系统调用框架应当先保证语义稳定,再优化入口成本。错误码、信号、程序计数器推进和特殊陷阱帧接管是正确性条件。快速路径、减少保存寄存器 and 无锁查表是性能条件。性能优化不能改变正确性条件。我们在文档中把这两类问题分开,是为了后续调优时有明确约束。任何优化只要改变了返回值写入、信号投递或可恢复异常边界,都必须重新审查对应 ABI 语义。

8.15 工程设计总结

异常处理与系统调用接口处在硬件和内核策略之间。它既要面对寄存器、控制状态寄存器和返回指令，又要为上层提供稳定、简洁的 ABI 分发模型。我们把它拆成异常入口、陷阱帧访问契约、表驱动系统调用、返回前收尾和信号调度交接几个部分。

我们通过系统调用帧接口（`SyscallFrameOps`）把架构 ABI 收束为少量稳定回调。系统调用实现不需要知道参数来自哪个寄存器，也不需要知道程序计数器应当推进多少字节。Linux 的系统调用入口在各架构下也会保留独立汇编入口和通用 C 分发，但它的工程背景是庞大的成熟 ABI 体系。我们的实现规模更小，因此把差异明确压缩到 `sys_nr`、`sys_args`、`set_sys_ret` 和 `advance_pc` 这几个动作上。相比 xv6 直接在单一架构里读取陷阱帧并进入系统调用分派，这个设计更适合同时维护 RISC-V64 和 LoongArch64。

其次是系统调用表采用启动期注册和运行期无锁读取，并通过 `finalize_frame` 处理少数接管返回路径的系统调用。固定表让系统调用号到实现函数的关系集中可查，也避免运行期查找锁。`execve`、`rt_sigreturn` 这类调用需要重写用户上下文，普通返回流程不能再写返回值或推进程序计数器。我们用显式陷阱帧接管标记处理这些特例，使特殊控制流只在分发器边界出现，不扩散到所有系统调用实现中。

我们还把快速路径优化放在通用入口链路上。我们没有为某几个简单系统调用硬编码捷径，而是缩短异常入口、现场保存、分发表读取和返回前检查这些公共段。这样所有短系统调用都能共享收益。这个取舍和 Linux 的 vDSO 机制、系统调用入口优化方向相近，但我们当前更关注内核入口本身的统一性，避免在 ABI 层积累大量难以维护的例外。

最后我们对用户访问窗口和内核可信边界实现清晰分离。系统调用实现通过统一 helper 读取用户指针，访问错误转化为 `EFAULT`。普通内核路径中的缺页仍然被视为内核故障。这个边界使用户态恶意地址不能扩大为内核崩溃，也防止真正的内核 bug 被错误吞掉。FreeBSD 和 Linux 都有类似的 `copyin/copyout` 边界，我们在 Rust 内核中把它与异常恢复路径直接绑定，使错误来源更容易推理。

这些设计让异常入口和系统调用接口既贴近硬件，又不把硬件差异扩散到内核主体。平台相关层负责保存和恢复现场。平台无关层负责分发表、错误码和 POSIX 语义。二者在陷阱帧契约处交接，形成了清晰的 ABI 边界。对后续章节来说，这个边界还有另一层意义。用户态执行环境、信号、终端和网络都会通过系统调用暴露给用户程序。只要系统调用边界稳定，这些子系统就可以分别演进，而不会把架构 ABI 细节带入自身设计。入口层因此承担了硬件事实和用户态契约之间的长期稳定接口，也承担了性能优化与兼容性之间的协调责任。这个责任贯穿后续所有用户态可见功能，并决定了整套 ABI 的可维护性和问题可解释性，也保证问题能够分层定位和持续演进。后续若继续扩展审计、`seccomp` 过滤机制或追踪能力，也应当接在这条边界上，而不应侵入具体系统调用实现。

第九章 用户态执行环境

在第八章中, 系统调用接口建立了用户态进入内核的受控入口。本章讨论反方向的问题, 即内核如何把一个可执行文件变成真正运行的用户进程。用户态执行环境包括可执行与可链接格式(ELF)装载、解释器处理、地址空间构建、用户栈布置、辅助向量(auxv)、虚拟动态共享对象(vDSO)、初始文件描述符和陷阱帧。它们共同决定了动态链接程序、shell 程序、服务进程和普通命令能否按 Linux 兼容预期启动。

用户态支持的难点在于它横跨多个子系统。VFS 负责打开可执行文件和解释器。内存管理负责创建用户虚拟地址空间(VmSpace)、提交 PT_LOAD 段和映射用户栈。调度层负责把新的地址空间和陷阱帧挂到当前任务对象。架构层负责设置入口程序计数器、用户栈指针和返回用户态的状态寄存器。若这些步骤耦合在一个函数里, 任何 ABI 细节变化都会牵动多个子系统。我们把装载流程拆成可验证的阶段, 让每个阶段只处理自己的对象。

9.1 ELF 装载流程

用户态镜像的入口是 `load_user_image_from_path`。它从当前任务的 VFS 上下文打开路径, 读取文件前缀, 先处理 shebang 脚本头, 再解析 ELF 元数据。对于动态链接程序, 它会读取 PT_INTERP 指定的解释器, 并把主程序和解释器分别映射到地址空间中。主程序如果是位置无关可执行文件(PIE), 会使用平台提供的主程序 PIE 基址。解释器使用独立基址。两者最终共同生成入口程序计数器、用户栈和辅助向量。



图 9-1 用户镜像装载流程

ELF 解析阶段做了多项前置检查。文件头必须是 64 位 ELF，机器类型必须匹配当前架构，程序头大小不能超过内核设置的上限。解释器路径长度、动态段长度和 shebang 脚本头递归深度也有上限。这样的限制并非只为防御恶意输入，也用于避免错误镜像把内核带入大规模分配或无限递归。

9.2 地址空间构建

装载器为每次执行映像替换创建新的用户虚拟地址空间。主程序的每个可加载段通过 `VmSpace::commit_file_segment` 提交，装载器只保存程序头中的文件偏移和长度，由地址空间层按页从文件填充段内容。解释器当前先读入内核内存并解析为镜像对象，再通过 `VmSpace::commit_segment` 提交。段权限由 ELF 权限标志转换为 `VmFlags`，例如读、写、执行和用户可访问。文件大小小于内存大小的部分表示未初始化数据段（BSS），需要映射为零填充内存。用户栈则以固定栈顶和默认大小提交，带有 `GROWS_DOWN` 标志。

```
fn load_exec_image(vm: &Arc<VmSpace>, image: &ExecImage, file: &Arc<File>,
bias: usize)
-> Result<LoadedImage, Errno>
{
    for segment in image.segments.iter() {
        let va = bias + segment.vaddr;
        let flags = flags_from_elf_perms(segment.perms)
            .with(VmFlags::USER);

        vm.commit_file_segment(
            va,
            segment.memsz,
            segment.file_offset,
            segment.file_size,
            file,
            flags,
        );
    }

    Ok(LoadedImage {
        entry: bias + image.entry,
        base: bias,
        phdr: compute_phdr_addr(image, bias),
        phent: image.phent,
        phnum: image.phnum,
    })
}
```

代码 9-1 ELF 段提交

地址空间构建的关键是让装载器只表达虚拟内存区域（VMA）语义，而不直接操作页表。VmSpace 决定如何分配常驻页、如何设置用户权限、如何处理文件内容和零填充。架构层只通过第二章中的 `UserPgdOps` 安装页表项。这样，用户镜像装载在 RISC-V64 与 LoongArch64 上共享同一套策略。

9.3 用户栈与辅助向量

用户栈需要满足 C 运行库 (libc) 启动约定。内核在栈上写入参数数量 `argc`、参数指针数组 `argv`、环境指针数组 `envp` 和辅助向量。字符串区域保存参数、环境变量和执行路径。辅助向量提供页面大小、程序头地址、入口地址、用户 ID、随机数、vDSO 地址和其它启动信息。动态链接器和 C 运行库都会读取这些值。

```
struct UserStackBuilder {
    sp: usize,
    strings: Vec<UserString>,
    auxv: Vec<(usize, usize)>,
}

fn build_initial_stack(builder: &mut UserStackBuilder, image: &LoadedImage)
-> Result<usize, Errno> {
    let execfn = builder.push_string(image.exec_path)?;
    let random = builder.push_random_bytes(16)?;

    builder.push_aux(AT_PHDR, image.phdr);
    builder.push_aux(AT_PHENT, image.phent);
    builder.push_aux(AT_PHNUM, image.phnum);
    builder.push_aux(AT_PAGESZ, PAGE_SIZE);
    builder.push_aux(AT_ENTRY, image.entry);
    builder.push_aux(AT_RANDOM, random);
    builder.push_aux(AT_EXECFN, execfn);
    builder.push_aux(AT_NULL, 0);

    builder.finish_with_argc_argv_envp()
}
```

代码 9-2 用户栈布局

栈布局的顺序不能随意改变。字符串需要先复制到用户栈，随后才能把指针写入 `argv`、`envp` 和辅助向量。栈指针需要按 ABI 对齐。AT_RANDOM 必须指向用户栈中 16 字节随机区域。AT_EXECFN 应当指向执行路径字符串。若这些细节错误，程序可能在内核返回用户态之后才崩溃，排查会比装载阶段直接返回错误更困难。

9.4 动态链接器与 vDSO

动态程序通过 `PT_INTERP` 指定解释器。装载器读取解释器文件，映射其 ELF 段，并把最终入口设置为解释器入口。主程序的入口通过辅助向量中的 `AT_ENTRY` 交给解释器。若解释器缺失，只有经过验证可以无解释器运行的镜像才允许继续。这个判断避免把动态程序错误当作静态程序启动。

vDSO 是内核向用户态提供的只读快速路径。装载器把合成的 vDSO ELF 映射到用户地址空间，并把共享数据页作为只读用户映射附加进去。AT_SYSINFO_EHDR 指向 vDSO ELF 头。用户态可以通过它获取时间等信息，减少进入内核的系统调用次数。vDSO 的存在也要求装载器和架构层共享一组稳定的用户地址布局参数。

9.5 执行映像替换与任务状态交接

装载完成后, 调度层把新的用户虚拟地址空间安装到当前任务扩展槽中, 保存执行路径、`argv` 和 `envp` 诊断信息, 并构造新的用户陷阱帧。旧地址空间和旧用户栈不再参与运行。若执行映像替换发生在 `vfork` 子任务中, 替换成功后还需要唤醒被阻塞的父任务, 因为子任务已经不再共享父地址空间。

```
fn install_exec_image(task: &Arc<Task>, loaded: LoadedUserImage) ->
Result<(), Errno> {
    task.ext_install(TASKEXT_VM_SPACE, Arc::clone(&loaded.vm));
    task.ext_install(TASKEXT_EXEC_PATH, Arc::new(loaded.exec_path.clone()));

    let stack_top = task.ensure_kernel_stack();
    let mut frame = UserTrapFrame::new();
    frame.set_pc(loaded.entry_pc);
    frame.set_sp(loaded.user_sp);
    frame.set_kernel_stack_top(stack_top);
    task.ext_install(TASKEXT_TRAP_FRAME, Arc::new(frame));

    wake_vfork_parent_if_needed(task);
    Ok(())
}
```

代码 9-3 执行映像替换成功后的任务安装

执行映像替换是进程生命周期中的替换操作, 而非新建操作。PID、父子关系、打开文件描述符和许多调度属性保留。地址空间、用户栈和执行入口被替换。带执行时关闭标志的描述符会在执行映像替换边界关闭。信号处理方式也需要根据 POSIX 规则重置部分状态。这种保留与替换的组合, 使执行映像替换成为进程管理、VFS、内存管理和信号机制之间的重要交接点。

9.6 初始用户进程

第一章已经说明, 内核最终会尝试 `/init`、`/sbin/init` 和 `/bin/init`。第一个可加载镜像会成为 PID 1 的用户态执行体。启动阶段准备好的 VFS 上下文、文件描述符表和标准输入输出会安装到 `init` 任务上。此后, 系统进入普通调度和系统调用循环。

PID 1 的特殊性在于它是孤儿任务的收养者, 也是用户态服务树的根。若 `init` 进程无法启动, 内核没有可继续运行的用户态管理者。装载错误必须清楚返回和记录, 不能在半安装状态进入用户态。我们把装载结果封装为 `LoadedUserImage`, 只有在所有段、栈和 `vDSO` 映射都准备好后, 才把它安装到任务上。

9.7 装载请求与阶段化数据结构

用户态装载流程不应直接在一个长函数里修改任务状态。它涉及路径、文件对象、ELF 元数据、解释器、地址空间、用户栈和陷阱帧。任何一步失败, 都需要回滚已经分配但尚未提交的中间资源。我们因此把执行映像替换分成输入请求、中间镜像和最终装载结果三类对象。输入请求保存路径、`argv`、`envp` 和标志。中间镜像保存解

析出的 ELF 元数据、程序头和解释器信息。最终装载结果保存新的用户虚拟地址空间、入口地址、用户栈指针、辅助向量和诊断信息。

这样的阶段化设计有两个目的。第一，失败路径可以局部处理。解析 ELF 失败时，只需要释放文件读取缓冲。提交地址空间失败时，只释放新建的 `VmSpace`，不影响当前任务仍在运行的旧地址空间。构造用户栈失败时，可以丢弃尚未安装的装载结果。第二，提交点清晰。只有 `LoadedUserImage` 完整构造后，执行映像替换才进入任务安装阶段。安装之前，当前任务的用户态状态保持不变。安装之后，旧地址空间才被引用计数释放。

```
struct ExecRequest {
    path: String,
    argv: Vec<String>,
    envp: Vec<String>,
    flags: ExecFlags,
}

struct ParsedImage {
    file: Arc<File>,
    elf: ElfMetadata,
    interp: Option<InterpreterRequest>,
}

struct LoadedUserImage {
    vm: Arc<VmSpace>,
    entry_pc: usize,
    user_sp: usize,
    exec_path: String,
    auxv: Vec<AuxEntry>,
}
```

代码 9-4 执行映像替换装载的阶段化对象

`ExecRequest` 的内容来自用户态，因此构造时必须限制 `argv` 和 `envp` 数量、单个字符串长度以及总字节数。若不设上限，用户程序可以通过执行映像替换构造极大的参数集合，使内核在启动新程序前耗尽内存。`ParsedImage` 的内容来自 VFS 读取的文件，也必须经过 ELF 文件头、程序头和解释器路径校验。`LoadedUserImage` 是内核即将安装的可信结果，它只在所有构造步骤完成后进入任务状态。

阶段化对象还方便记录诊断信息。用户程序启动失败时，经常需要知道当前任务名、执行路径、`argv` 前几项和入口地址。若这些信息只存在于临时栈变量中，替换成功后就难以观察。我们在安装阶段把稳定的执行路径和必要诊断复制到任务扩展槽中。这样 `procfs` 文件系统、日志和崩溃日志都能看到当前用户程序身份。

9.8 ELF 校验与段权限

ELF 文件是用户态执行环境的核心输入。内核不能假设文件格式一定正确。ELF 文件头中的类别、端序、机器类型、镜像类型、入口地址、程序头偏移和数量都需要检查。程序头还需要检查对齐、虚拟地址、文件范围、内存大小和权限。若这些检查不足，错误镜像可能导致地址溢出、段重叠、可写可执行区域扩大，或者读取文件越界。

我们在校验中区分格式错误和资源不足。格式错误返回 `ENOEXEC` 或对应的执行错误，表示文件不能作为当前架构可执行镜像。资源不足返回 `ENOMEM`，表示镜像本身可能合法，但内核当前无法分配所需结构或物理页。文件读取失败保留 VFS 错误。这个区分对 shell 程序和 C 运行库很重要。shell 程序看到 `ENOEXEC` 可能尝试按脚本方式执行，看到 `EACCES` 会报告权限问题，看到 `ENOMEM` 则说明系统资源不足。

表 9-1 ELF 校验项目与失败含义

校验项	检查内容	失败处理
文件头	魔数、类别、端序、机器类型、版本和镜像类型。	返回执行格式错误，避免继续解析错误结构。
程序头	条目大小、数量上限、文件范围和读取长度。	返回格式错误或文件读取错误。
PT_LOAD 段	虚拟地址范围、对齐、 <code>filesz</code> 与 <code>memsz</code> 关系、段重叠。	拒绝越界或重叠映射，防止地址空间不一致。
权限	<code>PF_R</code> 、 <code>PF_W</code> 、 <code>PF_X</code> 到 <code>VmFlags</code> 的转换。	按 ELF 权限建立 VMA，保留 <code>W^X</code> 审查空间。
解释器	<code>PT_INTERP</code> 是否唯一、路径长度、解释器自身格式。	解释器非法时执行失败，不退化为静态启动。

段权限转换需要克制。ELF 的 `PF_R`、`PF_W` 和 `PF_X` 表示用户程序期望的访问能力，内核还要额外添加 `USER` 标志。内核不应因为实现方便把所有段都映射为可写可执行。代码段通常是只读可执行，数据段通常是读写不可执行，只读数据段（`rodata`）是只读不可执行。BSS 部分使用零填充页，并继承所在段权限。若未来加入更严格的 `W^X` 策略，这个转换点就是统一入口。

段范围也需要处理页对齐。ELF 段的虚拟地址和文件偏移通常满足页内偏移一致，但映射时要把起始地址向下对齐，把结束地址向上对齐。文件内容只覆盖 `filesz`，`memsz` 超出的部分需要清零。若起始页包含文件内容前的页内空洞，内核要确保用户不可读到未初始化内核数据。主程序路径通过 `VmSpace::commit_file_segment` 集中处理这些细节，解释器和 `vDSO` 等已经位于内核内存中的镜像仍通过 `VmSpace::commit_segment` 处理。装载器只传入段语义，不直接操作页表。

PIE 镜像和普通 `ET_EXEC` 镜像的处理也不同。`ET_EXEC` 通常使用固定虚拟地址。PIE 可以加载到平台布局给出的随机或固定基址。解释器本身也常是 PIE，需要独立基址。我们保留主程序 PIE 基址和解释器基址两组参数，避免主程序和解释器地址相互覆盖。事后回顾，这种地址空间布局也为后续地址空间布局随机化（ASLR）预留了空间。即使当前随机化程度有限，布局边界已经被显式建模。

9.9 用户地址布局与保护区

用户地址空间不能只看 ELF 段和栈。它还要容纳堆、`mmap` 区、`vDSO` 映射、共享数据页、保护页和未来可能加入的线程栈。布局若过于紧凑，后续 `brk`、`mmap` 和动

态链接器装载共享库时会频繁冲突。布局若过于分散, 又会浪费页表和虚拟地址空间管理成本。64 位平台虚拟地址空间较大, 我们更关心布局清晰和错误隔离。

典型用户地址空间可以分为低地址保留区、程序映像区、堆增长区、mmap 区、vDSO 区和用户栈区。低地址保留区用于捕获空指针附近访问。程序映像区放置主程序和解释器。堆从 brk 起点向上增长。mmap 区用于匿名映射、文件映射和共享库。vDSO 放在高地址附近但不与栈重叠。用户栈从固定高地址向下增长, 并在下方保留保护区。



保护页的作用是尽早暴露栈溢出。若用户栈下方立即连接其它有效映射, 栈向下越界可能静默覆盖其它区域。保留未映射区后, 越界访问会触发用户缺页并转化为 SIGSEGV。同样, 空指针附近保留未映射区, 可以让空指针解引用及时失败。用户态程序可能仍然通过 mmap 请求低地址映射, 但内核可以根据策略拒绝或限制这类映射。

地址布局还影响信号。信号处理需要在用户栈上构造信号帧, 若当前用户栈已经接近保护区, 投递信号可能失败。后续支持 sigaltstack 后, 内核需要根据任务的信号栈状态选择普通栈或备用栈。第十章会展开信号栈细节。本章只需要说明, 执行映像替换构造的初始栈必须为后续信号帧预留合理空间, 不能只刚好容纳 argv 和 envp。

vDSO 布局则要求内核和用户态共享少量只读事实。AT_SYSINFO_EHDR 指向 vDSO ELF 头, 用户态动态链接器通过辅助向量找到它。vDSO 内部函数需要访问共享数据页, 例如时钟基准和更新序列。共享数据页应映射为用户只读, 内核更新时通过同步协议保证一致性。这样用户态能够在不陷入内核的情况下读取常用时间信息, 同时仍然由内核控制数据来源。

9.10 用户栈 ABI

用户栈是执行映像替换最容易出现兼容性细节错误的地方。内核返回用户态后, 第一段运行的通常是动态链接器或 C 运行库启动代码。它们按照 ABI 从栈上读取 argc、argv、envp 和辅助向量。若指针顺序、终止空指针、对齐或辅助向量项错误,

程序可能在还没有调用任何系统调用之前崩溃。因为崩溃发生在用户态早期，日志往往只显示非法访问或段错误，难以直接指向栈布局错误。

我们按从高地址向低地址构造栈。先复制字符串内容，包括 `argv`、`envp`、执行文件名和随机字节。然后写入辅助向量数组。再写入 `envp` 指针数组和 `argv` 指针数组。最后写入 `argc`，并把最终栈指针对齐到架构要求。字符串先写的原因很直接。指针数组必须指向最终用户地址，只有字符串位置确定后，指针值才稳定。辅助向量中的 `AT_RANDOM` 和 `AT_EXECFN` 也依赖前面复制出的用户地址。

```
fn build_stack(req: &ExecRequest, image: &LoadedImage) -> Result<usize,
Errno> {
    let mut stack = StackWriter::new(USER_STACK_TOP);

    let argv_ptrs = stack.push_strings(&req.argv)?;
    let envp_ptrs = stack.push_strings(&req.envp)?;
    let execfn = stack.push_string(&req.path)?;
    let random = stack.push_random(16)?;

    stack.align(16);
    stack.push_auxv(make_auxv(image, execfn, random)?);
    stack.push_null();
    stack.push_ptrs(&envp_ptrs);
    stack.push_null();
    stack.push_ptrs(&argv_ptrs);
    stack.push_usize(req.argv.len());

    Ok(stack.sp())
}
```

代码 9-5 用户栈构造的顺序

辅助向量项需要兼容 Linux 习惯。`AT_PHDR`、`AT_PHENT` 和 `AT_PHNUM` 让动态链接器找到主程序程序头。`AT_PAGESZ` 提供页大小。`AT_ENTRY` 提供主程序入口，解释器启动后会跳转到它。`AT_RANDOM` 提供栈上随机字节，C 运行库用于栈保护等初始化。`AT_EXECFN` 提供执行路径。`AT_SYSINFO_EHDR` 提供 vDSO ELF 头地址。`AT_NULL` 终止辅助向量。缺少关键项可能导致动态链接器行为异常。

对齐需要遵循架构 ABI。64 位平台通常要求栈指针按 16 字节对齐。动态链接器和编译器生成的函数序言可能假设这个对齐。如果内核返回时栈指针不满足要求，用户态可能在保存寄存器或访问栈对象时出现未对齐访问。某些架构允许未对齐但性能差，某些场景会直接异常。我们在构造栈时把对齐作为最终步骤，并避免后续再写入破坏对齐的数据。

参数上限不仅是安全问题，也是兼容性问题。Linux 对 `argv` 和 `envp` 总长度有上限，用户态程序可能依赖过大参数返回 `E2BIG`。我们不需要完全复制每一个历史细节，但必须避免无界增长。当前实现对字符串长度、数量和总字节数都进行限制。达到上限时返回明确错误，而不是在内核堆分配失败后出现不可解释的 `ENOMEM` 或内核崩溃。

9.11 执行映像替换事务与失败回滚

执行映像替换的语义是用新程序替换当前进程映像。这个替换必须呈现事务性。成功后，用户态从新入口运行。失败后，原进程仍然应该能够继续处理错误返回。若

内核在装载到一半时已经替换地址空间, 随后又发现栈构造失败, 当前进程就会失去旧上下文, 也无法进入新程序。这类半安装状态非常危险。

我们把执行映像替换提交点放在最后。路径打开、shebang 脚本头解析、ELF 读取、解释器读取、VmSpace 创建、段提交、栈构造和 vDSO 映射都在新对象上完成。当前任务对象仍然指向旧地址空间。只有所有步骤成功后, `install_exec_image` 才获取必要锁, 安装新虚拟内存对象、新陷阱帧和诊断信息。旧虚拟内存对象通过引用计数释放。如果安装失败, 当前任务仍保留旧地址空间, 可以向用户态返回错误。

```
fn do_execve(req: ExecRequest) -> Result<usize, Errno> {
    let parsed = parse_exec_image(&req)?;
    let loaded = load_into_new_vm(&req, &parsed)?;

    prepare_close_on_exec(current_task());
    install_exec_image(current_task(), loaded)?;
    finish_exec_state(current_task());

    current_syscall_context().finalize_frame();
    Ok(0)
}
```

代码 9-6 执行映像替换的事务性提交

CLOEXEC 文件描述符处理处在提交边界附近。若执行映像替换最终失败, 带执行时关闭标志的文件描述符不应被关闭。若替换成功, 它们必须在新程序运行前关闭。我们因此先构造完整装载结果, 再执行执行时关闭处理。关闭文件描述符可能触发复杂析构, 所以需要遵守第七章的锁外释放原则。批量摘取文件描述符后再释放文件对象, 可以避免文件描述符表锁参与 VFS 或套接字的锁序。

信号状态也要按 POSIX 语义调整。捕获处理器通常在执行映像替换后重置为默认, 忽略信号可能保留, 待处理信号和信号屏蔽字的处理需要遵循进程语义。多线程进程执行映像替换时, 除调用线程外的其它线程需要被终止或收束。当前实现围绕单任务或受控线程组场景处理, 但接口设计保留了线程组执行映像替换的扩展空间。关键是替换成功后, 旧用户处理函数地址已经无效, 不能继续使用旧地址空间中的处理函数。

`vfork` 是执行映像替换提交点的另一个特殊场景。`vfork` 子任务在执行映像替换或退出前会让父任务阻塞, 避免父子同时使用同一地址空间。替换成功意味着子任务已经拥有新地址空间, 可以唤醒父任务。若替换失败, 子任务仍然处在旧共享状态, 需要按 `vfork` 语义继续处理。我们把 `vfork` 父任务唤醒放在安装成功后, 避免父任务过早恢复运行。

失败回滚还要处理资源释放顺序。新的 VmSpace 未安装前只被装载流程持有。失败时释放它会回收段映射、页和临时对象。已打开的解释器文件和主程序文件由临时对象引用计数释放。已经复制到新用户栈的字符串随着新虚拟内存对象一起释放。这个结构避免了手写复杂回滚列表。阶段化对象配合引用计数, 使失败路径和成功路径使用同一套生命周期机制。

9.12 文件描述符、凭据与进程身份

执行映像替换会替换用户映像，但进程身份并非全部重置。PID、父子关系、进程组、会话、当前工作目录、根目录、打开文件表和部分资源限制都会保留。文件描述符表只关闭带 `FD_CLOEXEC` 标志的条目。标准输入、输出和错误通常由 `init` 进程或 `shell` 程序提前准备。动态链接器和新程序从一开始就依赖这些文件描述符。若执行映像替换错误地重建文件描述符表，`shell` 管道、重定向和服务进程继承语义都会失效。

凭据处理同样重要。传统 Unix 执行映像替换可能触发 `setuid`、`setgid` 和 `capability` 权限变化，也可能清理可转储标志。当前实现以已有凭据模型为基础，保留后续扩展空间。即使暂未完整实现所有安全规则，文档中也需要明确执行映像替换是凭据重新评估的边界。打开文件的访问权限通常在打开时冻结，替换后进程凭据变化不应反向改变已打开文件对象的能力。这个原则与第四章的文件对象能力冻结相呼应。

进程名和诊断身份需要更新。用户程序执行后，`procfs` 文件系统、日志和崩溃日志应显示新的任务名或执行路径。这个信息不参与权限判断，但对调试和运维非常重要。出现崩溃或异常退出时，能够看到当前执行的程序名，有助于定位问题。我们在执行映像替换安装阶段更新任务对象的诊断字段，而不是在 ELF 解析阶段提前更新。若替换失败，旧进程名仍然正确。

初始文件描述符对 PID 1 尤其关键。`init` 进程通常需要可用的 `/dev/console` 作为标准输入输出。若启动时没有正确安装标准输入、标准输出和标准错误，用户态脚本可能启动但无法输出错误信息。第一章和第三章已经讨论设备和 `devtmpfs` 文件系统的启动顺序。本章强调的是，用户态执行环境要把这些准备好的文件描述符作为进程状态的一部分保留下来，不能在执行映像替换边界意外丢失。

9.13 ELF 与缺页、`mmap` 的关系

当前 ELF 装载把 `PT_LOAD` 段提交到 `VmSpace`。这可以采用立即分配物理页的方式，也可以采用延迟缺页加载。无论策略如何，装载器应当只表达映射语义。具体页面何时分配、文件内容何时复制、BSS 如何清零，由内存管理决定。这样后续引入按需分页（demand paging）或文件页缓存时，不需要重写 ELF 解析层。

`mmap` 区与执行映像替换初始布局也有关。动态链接器启动后，会通过 `mmap` 加载 C 运行库和其它共享库。若初始布局把主程序、解释器、vDSO 和栈放得过近，动态链接器的 `mmap` 请求容易失败。我们使用明确的用户布局参数分配这些区域，使初始执行映像替换和后续 `mmap` 不互相干扰。`brk` 起点也根据主程序段尾部确定，避免堆覆盖已加载段。

缺页处理需要知道 VMA 来源。代码段缺页、数据段缺页、BSS 缺页、栈增长缺页和非法访问应产生不同结果。装载器提交段时保留 VMA 标志，内存管理根据标志决定是否允许读写执行、是否允许向下增长、是否可以从文件内容填充。用户态执行环境只负责初始化这些元信息。这样第八章中的缺页处理器才能根据 VMA 信息决定返回用户态、发信号或执行内核故障处理。

vDSO 和共享数据页也参与 `mmap` 语义。它们由内核在执行映像替换时主动映射，但用户态可以通过辅助向量发现。它们通常不应被普通用户写入，也不应出现在可执行文件的程序头中。把 vDSO 映射作为执行映像替换布局的一部分，可以保证每个新

程序启动时都有一致视图。若某个平台暂不支持 vDSO, 也可以省略 `AT_SYSINFO_EHDR`, 用户态会回退到普通系统调用。

9.14 兼容性与启动语义

用户态执行环境的错误往往在启动早期暴露。动态程序找不到解释器, 会在执行映像替换阶段失败。辅助向量错误会让动态链接器崩溃。栈对齐错误可能导致运行库启动代码异常。执行时关闭错误会让新程序继承不该继承的文件描述符。`argv` 和 `envp` 上限错误会影响 `shell` 程序和服务管理程序。我们在设计中把这些观察点全部归入执行映像替换阶段, 而不是让它们散落在后续系统调用中。

执行映像替换相关语义覆盖范围很广。`execve` 需要处理非法路径、非法指针、脚本解释器、参数过长、权限错误和文件格式错误。`fcntl` 和 `close_range` 会间接依赖 `CLOEXEC`。信号语义要求替换后处理函数按规则重置。`shell` 脚本依赖 `shebang` 脚本头。动态链接程序依赖 `PT_INTERP` 和辅助向量。每个失败都可能指向不同阶段, 因此阶段化设计可以直接缩短排查路径。

普通用户态程序也依赖正确装载。动态链接命令、`shell` 小程序和长期运行的服务都需要正确的解释器、环境变量、标准文件描述符和时间相关辅助向量。若这些基础不稳定, 后续性能优化和功能扩展都缺少可靠前提。我们在执行环境章节强调 ABI 细节, 正是因为所有用户态能力都建立在稳定启动之上。一个程序能够跑起来, 只是最低标准。它还需要在运行库预期的环境中跑起来。

兼容性还包括错误返回的一致性。执行映像替换失败应当返回到旧程序, 使 `shell` 程序可以打印错误并继续执行下一条命令。若替换失败后破坏了旧地址空间, 用户态会表现为 `shell` 程序自身崩溃。若错误码不准确, 脚本可能做出错误分支判断。事务性提交、阶段化错误和清晰错误码共同保证了这一点。

9.15 安全性与资源约束

执行映像替换是用户态能够主动触发的大规模内核操作。一次替换可能读取多个文件、分配新的地址空间、分配页、构造用户栈、复制大量字符串, 并更新任务状态。若缺少资源约束, 普通用户程序就可以通过反复执行映像替换或构造极端参数把内核推入内存压力。我们在装载路径中把约束放在尽可能早的位置。`argv` 和 `envp` 在从用户态复制时检查数量和总长度。ELF 程序头在读取前检查数量和大小。`shebang` 脚本头递归有深度限制。解释器路径长度也有上限。早失败比晚失败更安全, 因为越早失败, 已经分配的内核资源越少。

安全性还体现在权限和可执行性检查。路径打开必须经过 VFS 权限检查。文件必须具有可执行语义, 目录不能作为普通 ELF 执行。解释器文件也要经过同样检查, 不能因为它来自 `shebang` 脚本头或 `PT_INTERP` 就绕过权限。若未来接入 `mount noexec`、`setuid`、`capability` 权限和 LSM 安全模块类策略, 执行映像替换路径就是统一检查点。把这些检查集中在装载前半段, 可以避免已经映射一部分地址空间后才发现权限不满足。

资源失败应当优先返回错误。构造 `argv` 缓冲失败、分配 `vmSpace` 失败、提交段失败、创建用户栈失败, 都可以返回 `ENOMEM` 或对应 VFS 错误。用户态压力不应直接导致内核崩溃。只有当内核已经发现自己的基础结构损坏, 例如当前任务缺少必须存在

的内核栈、陷阱帧安装不变量被破坏，才应进入不可恢复处理。这个原则与第八章的异常边界一致。用户输入导致的失败可恢复，内核不变量破坏不可伪装。

半安装状态是执行映像替换安全性中最需要避免的部分。新虚拟内存对象未完成时不能替换旧虚拟内存对象。执行时关闭标志未确认替换成功前不能生效。信号处理函数未确认新地址空间安装前不能重置。`vfork` 父任务不能在子任务尚未真正脱离旧地址空间时被唤醒。每一个步骤都有对应的提交条件。我们把这些条件组织在装载结果和安装函数之间，使成功路径只有一个方向，失败路径只释放未提交对象。

执行环境还需要防止用户态借助地址布局攻击内核假设。用户栈、`vDSO` 和程序段都应位于用户地址范围内，不能越过内核空间边界。地址计算必须检查溢出。段起止地址对齐后仍要验证范围。辅助向量中写给用户态的地址必须是用户可访问地址，不能泄露内核指针。随机字节和共享数据页也应按用户权限映射。这样即使用户提供畸形 ELF，最终也只能得到执行映像替换失败，而不能让内核建立越权映射。

这些约束使执行映像替换能够承受恶意输入和资源压力。它们不会让装载器变得更轻，但能把复杂性限制在明确阶段。对用户态而言，结果是成功启动或明确错误码。对内核而言，结果是旧状态保留或新状态完整安装。中间状态不向用户态暴露，也不留在任务结构中。这一点是用户态执行环境可靠性的核心。

9.16 工程设计总结

用户态执行环境把文件系统、内存管理、调度和架构返回路径连接在一起。它需要处理 ELF 细节，也需要维持进程生命周期语义。我们把这个过程拆成路径打开、镜像解析、地址空间构建、栈布局 and 任务安装几个阶段，保证了用户态程序看到稳定的 Linux 兼容启动环境。内核内部则保持分层。VFS 提供文件，内存管理提供地址空间，调度层保存任务状态，架构层执行最终返回。用户态支持因此成为多个子系统的交汇点，而不是破坏分层的例外路径。它把一个磁盘上的文件转化为可调度、可进入系统调用、可接收信号、可访问文件描述符的进程实体。这个转化过程越清晰，后续用户态兼容和性能优化就越有稳定基础。对于整个内核而言，执行映像替换的可靠性直接决定了用户态生态是否拥有可信起点。它也是从内核初始化走向完整用户态生态的最后一段桥接路径，并为后续信号、终端和网络程序运行提供统一前提，也让用户态问题可以从启动边界开始追踪和复现。稳定的执行环境最终会反过来降低用户态问题的解释成本，并为后续迭代提供直接依据。

第十章 信号与异步事件

在第九章中，用户态执行环境把 ELF 镜像、用户栈和陷阱帧准备好，使任务能够进入用户态运行。本章讨论用户态运行过程中最重要的异步机制，信号。信号允许内核或其它任务打断目标任务的正常控制流，要求它终止、停止、继续，或者在用户态执行一个处理函数。它既属于进程管理，也属于系统调用返回路径，还与等待队列和用户栈布局密切相关。

信号机制的难点在于它不是普通函数调用。发送者不能假设接收者正在内核态，也不能等待接收者立即处理。接收者可能屏蔽某个信号，可能正在不可中断睡眠，也可能正在从系统调用返回用户态。线程组又引入共享信号处理动作和共享待处理队列。我们把信号拆成发送、排队、选择、投递和返回五个阶段，让异步事件在可控边界内落地。

10.1 信号状态模型

每个任务对象拥有私有信号状态（SignalState）。其中包含待处理位图、带负载数据的 SigInfo 队列、信号屏蔽字、临时保存的信号屏蔽字，以及 sigtimedwait 使用的等待屏蔽字。线程组共享信号状态（SharedSignal）保存信号动作表和共享待处理队列。这个分层对应 POSIX 线程语义。信号处理动作属于线程组，屏蔽字属于单个线程。

```
struct SignalState {
    pending_bits: AtomicU64,
    pending_infos: Spinlock<Vec<SigInfo>>,
    blocked: AtomicU64,
    saved_blocked: AtomicU64,
    sigtimedwait_mask: AtomicU64,
}

struct SharedSignal {
    actions: Spinlock<[SigAction; NSIG]>,
    pending_bits: AtomicU64,
    pending_infos: Spinlock<Vec<SigInfo>>,
}

struct SigAction {
    handler: SigHandler,
    mask: SigSet,
    flags: SigActionFlags,
    restorer: usize,
}
```

代码 10-1 信号状态结构

待处理位图用于快速判断是否有信号。SigInfo 队列保存发送者 PID、UID、信号码和用户提供的原始负载。标准信号和实时信号在语义上有不同要求，sigqueueinfo

还需要保留用户提供的信号信息。当前实现没有把待处理状态简化成单个位图, 而是使用队列保存具体 `SigInfo`, 位图作为快速索引。取出信号时会跳过被信号屏蔽字屏蔽的条目。

10.2 发送与权限检查

信号发送入口包括 `kill`、`tkill`、`tgkill`、`rt_sigqueueinfo` 和 `pidfd_send_signal`。目标可以由 PID、线程 ID 或 `pidfd` 文件描述符表示。发送阶段首先解析目标任务或线程组, 然后检查权限, 再把 `SigInfo` 放入目标的私有待处理队列或共享待处理队列。若目标处于可中断睡眠, 发送路径会尝试唤醒它。

表 10-1 信号发送入口

入口	目标选择	特点
<code>kill</code>	按进程或进程组发送。	适合 <code>shell</code> 作业控制和普通进程终止。
<code>tkill</code> / <code>tgkill</code>	按线程 ID 或线程组加线程 ID 精确发送。	用于 POSIX 线程和调试相关场景。
<code>rt_sigqueueinfo</code>	按 PID 发送带信号信息的信号。	需要从用户态复制 128 字节信号信息负载。
<code>pidfd_send_signal</code>	通过 <code>pidfd</code> 文件对象找到目标任务。	避免 PID 复用带来的名字竞态。

权限检查依赖发送者和接收者的凭据。第五章已经说明, 调度层凭据与 VFS 凭据分离。当前信号权限使用调度层凭据中的 `UID`、`EUID`、目标保存 `UID` 和 `CAP_KILL` 判断访问关系。`SIGKILL` 和 `SIGSTOP` 拥有特殊语义, 不能被捕获、忽略或屏蔽。发送阶段会按信号类型选择共享待处理队列、线程私有待处理队列, 或者对 `SIGKILL` 触发线程组收束, 再由目标任务在安全边界执行后续动作。

10.3 屏蔽、等待与信号动作

`rt_sigaction` 修改线程组共享的动作表。`rt_sigprocmask` 修改当前任务的信号屏蔽字, 并自动剥离不可屏蔽的 `SIGKILL` 与 `SIGSTOP`。`rt_sigpending` 返回当前待处理信号与屏蔽信号的组合视图。`rt_sigsuspend` 临时替换信号屏蔽字, 睡眠直到某个信号到来, 然后恢复旧屏蔽字并返回 `EINTR`。`rt_sigtimedwait` 则允许用户态同步等待一组信号。

```
fn rt_sigsuspend(mask: SigSet) -> Result<usize, Errno> {
    let task = current_task();
    task.signal.save_blocked(mask);

    loop {
        if sigpending().raw() != 0 {
            break;
        }
    }
}
```

```

    }
    task.cas_state(TaskState::Running, TaskState::Sleeping);
    sched_yield();
}

task.signal.restore_blocked();
Err(EINTR)
}

```

代码 10-2 sigsuspend 的控制流

这类调用体现了信号的双重性质。信号既可以异步打断任务,也可以被任务同步等待。同步等待不能绕过待处理队列,因为信号仍然需要保留发送者信息和屏蔽语义。sigtimedwait 先非阻塞轮询,未命中时登记等待屏蔽字并让出 CPU,到期后返回 EAGAIN。这种做法避免忙等,也使信号等待与第七章的等待队列模型一致。

10.4 返回用户态前的投递

真正执行信号动作发生在任务返回用户态前。系统调用分发器、异常返回路径和调度边界都会检查待处理信号。若默认动作是终止,调度层调用退出路径。若默认动作是停止,任务进入停止状态并通知父任务。若动作是用户处理函数,内核在用户栈或备用信号栈上构造信号帧,修改陷阱帧,使用户态返回到处理函数。

```

fn deliver_pending_signals(task: &Arc<Task>, ctx: UserContextRef) ->
Result<(), Errno> {
    while let Some(info) = dequeue_unblocked_signal(task) {
        let action = task.shared_signal().get_action(info.sig);
        match action.handler {
            SigHandler::Default => run_default_action(task, info.sig),
            SigHandler::Ignore => continue,
            SigHandler::Handler(entry) => {
                setup_signal_frame(task, info, action, ctx)?;
                return Ok(());
            }
        }
    }
    Ok(())
}

```

代码 10-3 信号投递阶段

用户态处理函数的关键是信号帧。信号帧保存原陷阱帧、旧信号屏蔽字、信号信息和返回跳板。处理函数执行完成后调用 rt_sigreturn, 内核从用户栈恢复原上下文。sigaltstack 允许任务指定备用信号栈。当前实现会检查当前栈指针是否已经在备用栈内,避免在处理信号时非法切换备用栈。

10.5 信号与系统调用重启

阻塞系统调用被信号打断时,用户态通常看到 EINTR, 某些带 SA_RESTART 的信号可以触发重启语义。当前系统调用分发器在看到 EINTR 返回时,会尝试消费一条带 SA_RESTART 的用户处理信号,并在写回返回值和推进程序计数器之前构造信号帧。

`restart_syscall` 作为兼容入口保留，但当前实现没有在这里维护通用的系统调用重启状态。这个设计使大多数系统调用实现只需返回错误码，不需要直接管理信号帧。

这里的边界很重要。系统调用实现负责自己的业务状态。信号投递负责改变用户态控制流。分发器负责在返回前把两者接起来。若每个阻塞系统调用都自行处理信号，管道、套接字、`futex` 机制、`nanosleep` 和等待接口都会重复一套复杂逻辑。集中处理降低了错误概率。

10.6 默认动作与任务状态转换

信号的默认动作不只是终止进程。POSIX 信号可以终止、产生核心转储语义、停止、继续或忽略。`SIGKILL` 必须终止，`SIGSTOP` 必须停止，二者不能被用户处理函数捕获，也不能被信号屏蔽字屏蔽。`SIGCHLD` 默认忽略，但它仍然承担父子进程状态通知功能。`SIGCONT` 会让已停止任务继续运行，并可能清理相反方向的停止信号。默认动作因此需要和任务状态机、父子等待和进程组控制结合。

任务终止动作进入退出路径。信号投递阶段发现默认动作为终止时，不能简单设置一个标志后继续返回用户态。它必须调用进程退出流程，关闭资源，通知父任务，唤醒等待队列，并根据线程组语义决定是否退出整个进程。停止动作则把任务或线程组置为停止状态，并向父任务发布可等待状态。继续动作把停止状态恢复为可运行状态，并通知等待者。每个动作都需要穿过第七章的等待队列和第六章的调度状态。

表 10-2 典型默认动作与内核处理		
动作	典型信号	处理要点
终止	<code>SIGKILL</code> 、 <code>SIGTERM</code> 、 <code>SIGSEGV</code> 。	进入退出路径，通知父任务，释放资源并停止返回用户态。
停止	<code>SIGSTOP</code> 、 <code>SIGTSTP</code> 、 <code>SIGTTIN</code> 。	设置停止状态，触发等待接口可观察事件，等待继续信号。
继续	<code>SIGCONT</code> 。	清理停止状态，唤醒可运行任务，并通知父任务。
忽略	部分 <code>SIGCHLD</code> 和用户设置忽略的信号。	从待处理队列中消费，不构造用户信号帧。
捕获	用户通过 <code>rt_sigaction</code> 注册处理函数的信号。	构造信号帧，修改陷阱帧，返回用户处理函数。

默认动作与父子等待之间存在细节。父任务调用 `wait4` 时，可能等待子任务退出，也可能在指定标志下观察停止或继续状态。子任务收到停止信号后，需要把状态变化发布给父任务。收到继续信号后，也可能发布继续事件。若这些事件只修改任务状态而不唤醒父任务，等待接口会长时间阻塞。若重复发布，父任务可能看到重复状态。我们在任务状态中保留可消费的等待事件，使状态变化和等待语义能够对齐。

默认动作还要考虑线程组。向进程发送的终止信号通常影响整个线程组。向特定线程发送的信号可能只投递给该线程，但如果默认动作是进程终止，最终效果仍然是

整个进程退出。当前实现以线程组共享信号状态为基础, 把进程级动作放在共享层收束。这样后续扩展完整 POSIX 线程语义时, 不需要重新设计状态划分。

10.7 线程组选路与待处理队列

信号发送的目标可能是一个线程, 也可能是一个线程组。tgkill 可以精确指定线程组和线程 ID。kill 面向进程或进程组。终端作业控制会向前台进程组发送信号。pidfd 文件描述符则通过文件对象引用具体任务, 避免 PID 复用问题。目标选择阶段必须把这些形式转化为任务对象或共享信号状态上的待处理条目。

线程组信号需要选择一个可投递线程。最理想的目标是未屏蔽该信号、处于可唤醒状态的线程。若当前没有线程可投递, 信号应留在共享待处理队列中, 等某个线程解除屏蔽或返回用户态时再处理。线程私有信号则放入目标任务的私有待处理队列。投递阶段先检查私有待处理队列, 再检查共享待处理队列, 或者按内核定义的优先级合并选择。这个过程不能丢失信号信息, 也不能让屏蔽信号越过信号屏蔽字。

```
fn pick_pending_signal(task: &Task) -> Option<SigInfo> {
    let blocked = task.signal.blocked.load(Acquire);

    if let Some(info) = task.signal.take_unblocked(blocked) {
        return Some(info);
    }

    let shared = task.thread_group.shared_signal();
    shared.take_unblocked_for_task(blocked)
}
```

代码 10-4 线程组待处理信号的投递选择

待处理位图和信号信息队列需要保持一致。位图用于快速判断某个信号是否可能存在。队列保存具体负载。取出最后一个对应信号后, 需要清理位图。若位图提前清理, 而队列中仍有条目, 信号可能迟迟不投递。若队列已经空, 位图仍然保留, 返回用户态前会反复进入无效检查。我们把队列修改放在锁内, 位图作为加速索引同步更新。这样既保留了快速检测路径, 也避免把信号信息负载丢失在位图状态之外。

实时信号对队列要求更高。完整语义需要保留用户负载, 并进一步处理排队顺序、资源限制和标准信号合并规则。rt_sigqueueinfo 和 pidfd_send_signal 都可能携带用户提供的信号信息。当前队列模型已经能够保存 128 字节 siginfo 负载, 并把队列与位图分开维护。某些实时细节仍然需要继续完善, 但结构上已经避免把待处理状态简化为单个位图。这个选择的成本是队列管理稍复杂, 收益是后续兼容扩展不需要重建信号状态。

10.8 信号动作语义与用户处理函数

rt_sigaction 不是简单设置函数指针。它还包含处理函数类型、附加屏蔽字、标志和恢复入口。处理函数可以是默认、忽略或用户地址。标志会影响投递行为, 例如是否传递信号信息、处理函数执行期间是否自动屏蔽当前信号、系统调用是否尝试重启、是否使用备用信号栈。恢复入口通常由 C 运行库提供, 用于处理函数返回后执行 rt_sigreturn。

修改信号动作时使用线程组共享锁。读取动作时也要在稳定边界上取得快照。投递阶段不能在持有动作表锁时写用户栈, 因为用户栈写入可能触发用户访问错误, 也可能需要复杂处理。正确方式是先复制出信号动作, 再释放锁, 然后构造信号帧。这样动作表锁只保护动作表, 不参与用户内存访问和调度路径。

```
fn deliver_one(task: &Task, info: SigInfo, ctx: UserContextRef) ->
Result<(), Errno> {
    let action = {
        let actions = task.shared_signal().actions.lock();
        actions[info.sig as usize].clone()
    };

    match action.handler {
        SigHandler::Default => run_default_action(task, info.sig),
        SigHandler::Ignore => Ok(()),
        SigHandler::Handler(entry) => setup_signal_frame(task, info, action,
entry, ctx),
    }
}
```

代码 10-5 信号动作快照与投递分离

处理函数执行期间的屏蔽字处理需要精确。投递时, 内核通常把当前信号和动作附加屏蔽字加入信号屏蔽字, 除非设置了对应标志。这样可以避免处理函数被同一信号无限递归打断。旧屏蔽字保存在信号帧中。rt_sigreturn 从用户栈恢复旧屏蔽字。若恢复失败, 例如用户栈上的信号帧被破坏, 内核应向任务发送致命信号或终止进程, 因为继续运行的上下文已经不可信。

用户处理函数的入口参数也受 ABI 影响。普通处理函数可能只接收信号编号。带 SA_SIGINFO 的处理函数接收信号编号、信号信息指针和用户上下文指针。内核需要在用户栈上按 ABI 布置这些对象, 并设置对应寄存器。架构层通过进程映像接口或陷阱帧辅助函数完成寄存器设置。信号核心层只表达投递这个信号到这个处理函数, 不直接编码每个架构的调用约定。

10.9 sigaltstack 与嵌套投递

备用信号栈用于处理普通用户栈不可用或接近溢出的场景。用户通过 sigaltstack 注册一段内存, 设置启用或禁用状态。信号动作若带 SA_ONSTACK, 且当前不在备用栈上, 内核应在备用栈上构造信号帧。若当前已经在备用栈上, 再次投递通常继续使用当前栈, 避免递归切换导致状态混乱。

备用栈检查依赖当前用户栈指针。内核需要判断栈指针是否落在已注册的备用栈区间内。若已经在区间内, SS_ONSTACK 状态应反映出来。若信号动作请求备用栈但备用栈未启用或大小不足, 内核需要按普通栈或错误策略处理。信号栈本质上仍然是用户内存, 写入信号帧时可能失败。失败通常意味着无法安全进入处理函数, 任务应按致命信号处理。

```
fn choose_signal_stack(task: &Task, action: &SigAction, user_sp: usize) ->
usize {
    let alt = task.signal.altstack.load();
```

```

    if action.flags.contains(SA_ONSTACK) && alt.enabled && !
alt.contains(user_sp) {
        return alt.top_aligned();
    }
    user_sp
}

```

代码 10-6 信号栈选择

嵌套投递需要控制递归深度。用户处理函数执行期间，若又有未屏蔽信号到来，返回用户态前可能再次构造信号帧。合法嵌套是 POSIX 允许的，但无限嵌套会耗尽用户栈。内核不能完全阻止用户通过屏蔽字允许递归信号，但需要确保每次信号帧构造都检查用户地址范围和栈边界。若写入信号帧失败，不能继续假装投递成功。

备用栈还影响调试。若程序栈溢出导致 SIGSEGV，普通栈可能已经不可用。启用备用栈后，处理函数有机会运行并输出诊断。内核正确支持 sigaltstack，有助于用户态运行库和异常处理框架处理严重错误。它也是动态语言运行时和 sanitizer 检查工具依赖的基础能力之一。

10.10 信号等待与第七章的同步模型

信号等待建立在第七章的等待队列模型之上，但它多了屏蔽字和待处理队列语义。rt_sigsuspend 会临时替换信号屏蔽字，然后睡眠到有未屏蔽信号到来。rt_sigtimedwait 等待指定屏蔽字中的信号，并把信号信息复制给用户态。两者都需要避免丢失唤醒。任务改变屏蔽字后必须重新检查待处理信号，登记等待后也要再次检查。

sigtimedwait 的返回语义与普通异步投递不同。它同步消费信号，并把信息返回给调用者，不构造用户处理函数。若等待超时，返回 EAGAIN。若用户信号信息指针不可写，返回 EFAULT。若等待期间收到不在等待屏蔽字中但未屏蔽的其它信号，可能需要按普通信号投递处理。这要求等待路径和返回前投递路径共享同一套待处理信号选择逻辑。

```

fn rt_sigtimedwait(mask: SigSet, timeout: Option<Time>) -> Result<SigInfo,
Errno> {
    let task = current_task();
    loop {
        if let Some(info) = task.signal.take_matching(mask) {
            return Ok(info);
        }
        if timeout_expired(timeout) {
            return Err(Errno::EAGAIN);
        }

        task.signal.set_wait_mask(mask);
        task.signal_wait.prepare_to_wait(&task,
TaskState::InterruptibleSleep);
        if let Some(info) = task.signal.take_matching(mask) {
            task.signal_wait.finish_wait(&task);
            return Ok(info);
        }
    }
}

```



```

    schedule_until(timeout);
    task.signal_wait.finish_wait(&task);
}
}

```

代码 10-7 sigtimedwait 的等待和消费

发送信号时, 如果目标任务正在等待相关屏蔽字, 发送路径需要唤醒它。若目标任务正在不可中断睡眠, 待处理状态仍然记录, 但任务可能要等到睡眠结束才处理。这个行为符合信号语义。信号并不保证立即执行处理函数, 它保证事件被记录, 并在合适边界被观察。等待队列只提供有信号可能可消费的通知, 具体是否匹配屏蔽字仍由醒来后的检查决定。

这个模式也解释了为什么信号不能简单作为普通回调执行。发送者和接收者之间没有直接函数调用关系。发送者只能提交事件并唤醒。接收者在自己的上下文中检查、消费和投递。这样可以避免发送者在持有其它锁时进入目标任务的用户栈构造, 也避免跨任务执行流难以推理。

10.11 系统调用重启的语义边界

系统调用重启是信号机制中最容易产生兼容性差异的部分。阻塞系统调用被信号打断后, 用户态可能看到 `EINTR`, 也可能在处理函数返回后自动重启。是否重启取决于系统调用类型、信号动作中的 `SA_RESTART`、已经完成的业务进度以及内核返回码。一个已经部分写入数据的 `write` 不应简单重启为全量写。一个尚未发生任何副作用的等待操作可以更容易重启。

我们把系统调用实现和信号投递框架分开。系统调用实现遇到可打断等待时返回内部错误码或 `EINTR`。分发器在返回前检查待处理信号。若命中带 `SA_RESTART` 的用户处理信号, 分发器会在普通返回写回之前构造信号帧, 并保留原始参数寄存器和陷阱帧中的用户态上下文。处理函数返回后, `rt_sigreturn` 恢复被打断位置的现场。当前实现把 `restart_syscall` 保留为兼容入口, 尚未把所有可重启系统调用抽象成独立的重启状态机。

重启语义必须谨慎处理副作用。`nanosleep` 被打断时需要返回剩余时间。`poll` 被打断时通常返回 `EINTR`。`read` 若已经读到数据, 应返回数据长度。`futex` 等待如果被信号打断, 需要区分用户值变化、超时和信号。我们不能把所有 `EINTR` 都机械重启。系统调用实现应当在业务层决定当前点是否可重启, 分发器只执行统一框架。

第八章已经说明, 返回前收尾是处理信号和重启的安全点。第十章补充的是, 信号动作标志和当前系统调用的可重启属性要在这里汇合。这样做能避免每个阻塞系统调用都直接修改用户陷阱帧, 也能让信号章节统一描述用户处理函数和信号返回的控制流。

10.12 作业控制与终端信号

终端子系统会产生信号。用户在终端输入中断字符, 通常向前台进程组发送 `SIGINT`。输入停止字符会发送 `SIGTSTP`。后台进程读写终端可能收到 `SIGTTIN` 或 `SIGTTOU`。这些信号由 TTY 层根据会话、进程组和控制终端状态生成, 并不经过普通 `kill` 入口。第十一章会讨论终端输入泵, 本章只说明信号层需要承接这些来源。

作业控制要求信号能够按进程组发送。shell 程序启动管道时，会把多个进程放入同一进程组。终端前台进程组收到控制字符后，整组任务都应收到信号。父 shell 程序通过等待接口观察子进程停止或继续状态，再决定是否恢复提示符。若信号层只支持单个 PID，终端作业控制就无法正常工作。我们在发送入口中保留进程组目标，正是为了支持这一类语义。

停止和继续信号还影响等待接口。父进程可以使用 `WUNTRACED` 观察停止状态，用 `WCONTINUED` 观察继续状态。信号默认动作执行时，必须把这些状态转换转化为等待接口可见事件。否则 shell 程序无法知道前台作业已经暂停，也无法正确实现前台和后台切换。这个交互说明信号不是孤立子系统，它连接终端、调度、进程组和等待语义。

作业控制还要求某些信号具有不可屏蔽或特殊合并行为。`SIGKILL` 与 `SIGSTOP` 不受用户动作控制。`SIGCONT` 到达时会使已停止任务继续，并可能清除待处理的停止类信号。停止类信号到达时，也可能与待处理的继续信号交互。我们在状态模型中把默认动作和待处理队列处理分开，使这些规则可以在投递阶段集中实现。

10.13 兼容性与运行时语义

信号 ABI 的细节非常密集。`rt_sigaction` 需要正确保存旧动作，`rt_sigprocmask` 需要剥离不可屏蔽信号，`sigsuspend` 应按语义返回 `EINTR`，`sigtimedwait` 需要取出信号信息，`sigaltstack` 需要报告正确状态，`kill`、`tkill` 和 `tgkill` 需要按目标发送。任何一个细节错误都会影响用户态运行库和 shell 程序的行为。

信号语义的困难在于时序。发送信号和接收信号之间可能经历调度。阻塞系统调用可能在信号到来前或到来后进入睡眠。父进程可能在子进程状态变化前调用等待接口，也可能在变化后调用。我们使用待处理队列、等待队列和返回前投递，把这些时序统一成可重复协议。若某个路径记录了待处理状态但没有唤醒，或者唤醒后没有重新检查条件，任务就可能停留在错误状态。

用户态运行库对信号有自己的包装。运行库通常提供恢复入口、信号集封装和 POSIX 线程相关语义。内核需要接受用户传入的动作结构布局，并按 ABI 复制。若结构大小、屏蔽字节数或标志解释错误，用户态包装层会出现异常。我们在系统调用层使用明确的结构转换，避免把用户结构直接当作内核结构长期保存。

信号兼容性还关系到进程收束能力。若任务处于可打断睡眠却不能被信号唤醒，发送者会看到目标长期不退出。若 `SIGKILL` 被错误屏蔽，系统无法可靠终止异常任务。我们在信号发送阶段对强制信号特殊处理，并在等待路径中让可打断睡眠响应待处理信号。这个能力对系统长期运行非常关键。

10.14 信号帧与 `rt_sigreturn`

用户处理函数的执行需要一个可恢复现场。内核投递信号时，不能只把程序计数器改成处理函数地址。处理函数返回后，用户程序应回到信号发生前的位置，寄存器、栈指针和信号屏蔽字都要恢复。这个恢复由信号帧和 `rt_sigreturn` 配合完成。信号帧位于用户栈或备用栈上，保存信号信息、用户上下文、旧屏蔽字和返回跳板需要的信息。处理函数执行完成后跳到恢复入口，恢复入口发起 `rt_sigreturn` 系统调用，内核再从用户栈读取信号帧并恢复陷阱帧。

信号帧是内核写入用户内存的数据结构, 因此必须经过用户访问辅助函数。写入信号帧时可能失败。失败说明用户栈不可写, 或者用户提供的备用栈无效。此时内核无法安全进入处理函数, 通常应执行默认致命处理。rt_sigreturn 读取信号帧时也可能失败。当前恢复路径会校验帧头魔数、长度、偏移和上下文区域大小, 并通过架构上下文转换恢复寄存器状态。若这些检查失败, 内核拒绝恢复原现场, 避免在已经无法信任的用户帧上继续执行。

```
fn setup_signal_frame(task: &Task, info: SigInfo, action: SigAction, ctx:
UserContextRef)
-> Result<(), Errno>
{
    let sp = choose_signal_stack(task, &action, ctx.user_sp());
    let frame_addr = allocate_frame_on_user_stack(sp,
size_of::<SignalFrame>());
    let frame = SignalFrame::from_context(info, action, ctx,
task.signal.blocked());

    copy_to_user(frame_addr, &frame)?;
    task.signal.set_blocked(action.mask_with_current(info.sig));
    ctx.set_user_arg0(info.sig as usize);
    ctx.set_user_pc(action.handler_addr());
    ctx.set_user_sp(frame_addr);
    Ok(())
}

fn rt_sigreturn(ctx: UserContextRef) -> Result<usize, Errno> {
    let frame = copy_frame_from_user(ctx.user_sp());
    validate_sigframe_layout(&frame)?;
    restore_context(ctx, frame.saved_context);
    current_task().signal.set_blocked(frame.old_mask);
    current_syscall_context().finalize_frame();
    Ok(0)
}
```

代码 10-8 信号帧的构造与恢复

rt_sigreturn 是少数会接管陷阱帧的系统调用。它成功后不应按普通系统调用规则写返回值和推进程序计数器, 而是恢复信号发生前的上下文。第八章中的 finalize_frame 正是为这类系统调用准备的。若分发器在 rt_sigreturn 后继续写返回值, 就会破坏用户程序原本的寄存器状态。这个细节是信号机制和系统调用框架之间的重要接口。

信号帧也影响调试和安全。用户态可以篡改信号帧, 内核必须验证恢复路径中最基本的布局和上下文一致性。当前实现把帧格式检查放在信号返回路径, 把寄存器编码和 mcontext 应用放在架构用户上下文层, 信号核心层只管理信号帧生命周期和屏蔽字语义。验证策略不能过度严格, 否则会破坏用户态运行库的合法行为。也不能过度宽松, 否则用户可以借助 sigreturn 构造异常控制流。后续若继续收紧返回地址、栈指针和处理器状态校验, 也应放在架构上下文边界内完成。

10.15 权限、凭据与 pidfd 文件描述符

信号发送涉及权限。一个进程不能任意向其它用户的进程发送信号。通常规则依赖真实 UID、有效 UID、保存 UID 和 capability 权限。特权任务可以发送更多信号。发送给自己或同一用户进程通常允许。具体规则需要和凭据模型对齐。第五章已经讨论任务凭据，第十章在此基础上使用调度层凭据进行权限判断。

权限检查应在目标解析后、待处理队列入队前完成。目标解析可能失败，返回 ESRCH。权限不足返回 EPERM。信号号非法返回 EINVAL。若信号号为 0，kill 用于探测目标是否存在和是否有权限，不真正入队信号。这个语义被很多用户态程序用于进程探测。把这些错误码区分清楚，是 POSIX 兼容的一部分。

pidfd 文件描述符提供了比 PID 数字更稳定的目标引用。PID 可能复用。用户态拿到一个 PID 后，目标进程可能退出，另一个进程复用同一 PID。pidfd 文件对象持有对目标任务或进程的引用，可以避免名字竞态。pidfd_send_signal 通过文件描述符解析目标，然后执行同样权限检查和入队逻辑。这样 pidfd 只改变目标定位方式，不改变信号核心语义。

进程组发送还涉及权限遍历。向一个进程组发送信号时，可能部分目标存在，部分目标无权限。内核需要按语义决定是否对允许的目标发送，并返回合适结果。这个细节对 shell 作业控制很重要。我们在发送路径中把目标集合构造、权限检查和入队动作分开，使后续完善进程组语义时有清晰落点。

10.16 信号动作与进程接口的关系

信号默认动作经常触发进程退出或状态变化。退出后，父进程需要通过等待接口观察。若子进程因信号终止，等待状态应包含终止信号。若子进程停止，等待状态应包含停止信号。若子进程继续，等待状态应表达继续状态。信号层因此需要把默认动作的原因传给进程管理层，而不是只设置任务状态。

退出路径也会产生信号。子进程退出时，父进程通常收到 SIGCHLD。若父进程忽略 SIGCHLD 或设置特定标志，子进程回收语义会受影响。这个方向说明信号和等待接口是双向耦合。信号可以导致退出，退出也可以产生信号。我们把实际资源回收放在进程管理层，信号层只提交事件和默认动作原因，使职责边界保持清晰。

线程组退出更复杂。一个线程收到致命信号后，整个线程组可能需要进入退出状态。其它线程如果正在睡眠，需要被唤醒并收束。若只终止当前线程，用户态可能看到违反进程级信号语义的状态。当前设计通过共享信号状态和线程组状态为这一点预留结构。即使实现逐步完善，文档中的模型也明确了目标方向。

等待侧的同步遵循第七章原则。父任务等待子状态变化时登记等待队列。子任务因信号退出或停止时修改状态并唤醒父任务。父任务醒来后重新扫描子列表，消费状态事件。信号层不会直接把返回值写入父任务用户栈，也不会直接完成等待系统调用。这样异步信号和同步等待之间保持清晰交接。

10.17 工程设计总结

信号机制把异步事件转化为用户态可见控制流。它不能在任意位置直接打断内核执行，只能在待处理队列、返回用户态和用户信号帧之间传递状态。信号机制的价

值在于把异步性变成有序协议。发送者只提交事件，接收者在安全边界处理事件。任务可以屏蔽、等待、捕获或默认处理信号，但这些动作都围绕同一组状态结构和返回路径展开。它连接进程管理、系统调用返回、终端控制和运行库语义，是用户态兼容性中最能体现细节密度的子系统之一。信号层越稳定，用户态程序在异常、超时和交互场景下的行为就越可预测。对内核而言，信号层也是检验睡眠唤醒、任务状态、用户栈访问和系统调用返回是否协调一致的集中场景，并为后续终端章节提供控制事件基础，也为异常进程收束提供可靠工具。这种能力会直接影响整机长期运行稳定性，也影响用户态交互体验。

第十一章 日志、控制台与终端

在第十章中，信号机制说明了异步事件如何影响用户态控制流。本章讨论另一类同样横跨内核与用户态的设施，日志、控制台和终端。它们看起来只是输入输出通道，实际承担启动诊断、崩溃报告、用户交互、TTY 控制字符和标准输入输出绑定等多重职责。若设计不当，最早期的启动日志会丢失，用户态 shell 程序无法交互，Ctrl-C 也无法及时转化为信号。

控制台路径需要跨越多个阶段。早期启动时，内核可能只有固件串口或平台提供的字符输出。设备模型初始化后，真实字符设备被注册为全局控制台。VFS 和 devtmpfs 文件系统就绪后，用户态通过 /dev/console、/dev/tty 或具体串口节点访问同一个底层设备。TTY 行规程又把字符输入解释为回显、换行和控制字符。我们把这些阶段分开，是为了让早期日志、设备生命周期和用户态终端语义互不阻塞。

11.1 全局控制台

全局控制台本质上是一个字符设备句柄。内核只保存字符设备 (CharDevice)，不再额外定义一套控制台接口。写日志时，console_write 取出当前注册的字符设备并写入字节。控制台未注册时，写入为空操作。设备移除后，字符设备内部活跃或移除状态会让后续访问自然失败。

```
static CONSOLE: Mutex<Option<CharDevice>> = Mutex::new(None);

fn register_console(dev: CharDevice) {
    *CONSOLE.lock() = Some(dev);
}

fn console_write(buf: &[u8]) {
    if let Some(dev) = CONSOLE.lock().as_ref().cloned() {
        let _ = dev.write_all(buf);
    }
}

fn console_read(buf: &mut [u8]) -> usize {
    CONSOLE.lock()
        .as_ref()
        .cloned()
        .and_then(|dev| dev.read(buf).ok())
        .unwrap_or(0)
}
```

代码 11-1 全局控制台结构

这个设计让控制台与第三章的设备模型保持一致。UART 串口、virtio 控制台或其它字符设备都可以注册为控制台。devtmpfs 文件系统在启动阶段把选定的字符设备直接绑定为 /dev/console。内核日志和用户态文件接口因此可以共享底层字符设

备，但两者通过不同入口访问。日志路径不需要进入 VFS，用户态路径也不需要知道全局日志模块。

11.2 日志输出与早期诊断

日志路径必须在系统最脆弱的阶段可用。内存管理、设备扫描、VFS 挂载和用户态启动都有可能失败。若日志依赖完整文件系统或复杂分配器，很多故障会发生在日志可用之前。当前控制台写入路径尽量薄，只通过字符设备批量写字节。格式化发生在调用侧，控制台写入器只实现 `fmt::Write`。

内核崩溃和致命故障路径对日志提出更高要求。发生内核错误时，系统状态可能已经不可信。日志路径不能再依赖长链路锁和阻塞操作。我们没有把崩溃报告写入普通文件，也没有要求 `procfs` 文件系统或 `syslog` 日志接口参与早期错误输出。控制台因此是最基础的诊断出口。后续 `syslog` 系统调用可以读取或控制日志策略，但它不是早期诊断的前提。

11.3 devtmpfs 文件系统与用户态终端入口

用户态看到的是文件节点。`/dev/console`、`/dev/tty`、串口设备，以及后续可能扩展的伪终端，都需要通过 VFS 的文件操作进入。第四章已经说明 `devtmpfs` 文件系统通过设备文件投影器把设备能力投影为节点。对于控制台，`devtmpfs` 使用当前选定的字符设备创建固定入口。对于 TTY，节点打开后会构造带行规程的文件操作，使 `read`、`write`、`poll` 和 `ioctl` 都进入终端兼容层。

表 11-1 终端相关入口		
入口	内核对象	语义
内核日志	全局字符设备控制台。	不依赖 VFS，用于启动和故障诊断。
<code>/dev/console</code>	<code>devtmpfs</code> 文件系统控制台节点。	用户态固定控制台入口。
<code>/dev/tty</code>	当前任务控制终端或 TTY 兼容层。	支持行规程、控制字符和终端控制接口。
具体串口节点	字符设备能力投影。	直接面向底层设备的字节流访问。

这种分层避免用户态 ABI 反向决定日志系统。即使 `/dev` 尚未挂载，内核仍然能输出日志。即使控制台节点创建失败，底层字符设备也可以服务其它内核路径。反过来，用户态终端需要的终端控制接口和行规程也不会污染早期日志路径。

11.4 TTY 输入泵

TTY 输入需要处理底层 FIFO 队列和行规程之间的时序。字符设备驱动只报告字节可读。终端控制字符属于 `devtmpfs` 文件系统和 TTY 兼容层。若直接在中断上下

文中执行行规程，就可能触碰 VFS 或 TTY 锁。当前设计把定时器和中断钩子做得很薄，只设置原子请求位。低优先级内核线程在进程上下文中执行 `poll_tty_input`。

```
static POLL_REQUESTED: AtomicBool = AtomicBool::new(false);
const FALLBACK_INTERVAL_NS: u64 = 10_000_000;

fn tick_tty_poll(now_ns: u64) {
    POLL_REQUESTED.store(true, Release);
}

fn tty_poll_worker() -> ! {
    loop {
        wait_for_request_or_timeout(FALLBACK_INTERVAL_NS);
        POLL_REQUESTED.swap(false, AcqRel);
        devtmpfs::poll_tty_input();
    }
}
```

代码 11-2 TTY 输入泵

兜底线程最多 10ms 醒一次。这个间隔是交互延迟和后台负载之间的折中。正常情况下，定时器或中断钩子会尽快置位请求。若某些虚拟串口后端没有可靠外部中断，兜底超时仍能推进输入。线程睡眠前会二次检查请求位，避免请求发生在检查和睡眠之间时被遗漏。这个协议与第七章等待队列的双重检查思路一致。

11.5 控制字符与信号边界

终端不仅传输字节，还解释控制字符。用户按下 Ctrl-C 时，TTY 行规程应向前台进程组发送 SIGINT。Ctrl-Z 对应停止信号。回车、退格和回显也属于行规程语义。信号发送本身由第十章的信号机制处理，TTY 只负责把输入字节翻译为进程组信号或普通输入。

这个边界使终端不会直接修改任务状态。TTY 不需要知道任务如何进入停止状态，也不需要自己构造信号帧。它只通过进程组和信号发送接口提交事件。这样，控制终端、作业控制和信号投递之间的关系保持清楚。

11.6 启动阶段的输出链路

控制台设计首先要服务启动诊断。内核启动早期还没有完整设备模型，也没有 VFS，更没有用户态 syslog 日志服务。若此时出现页表、内存分配或设备发现错误，唯一可靠的观察窗口就是早期字符输出。我们把启动输出链路分成早期原始写入器、正式控制台和用户态终端三个阶段。早期原始写入器面向架构或平台提供的最小输出能力。正式控制台面向第三章的字符设备。用户态终端面向 VFS 文件节点和 TTY 语义。

阶段之间需要可切换。早期阶段可能使用固件串口或平台固定 MMIO。设备模型初始化后，真实 UART 串口或 virtio 控制台被枚举并注册为 CharDevice。此时内核可以把全局控制台切换到正式设备。VFS 挂载和 devtmpfs 文件系统完成后，用户态再通过启动时绑定的 `/dev/console` 使用同一底层设备。这个顺序使日志不会等待完整用户态环境，也使用户态终端不必关心早期输出细节。

表 11-2 控制台输出阶段

阶段	能力	约束
早期输出	平台固定字符写入，通常只支持逐字节写入。	不能依赖堆、VFS、设备注册表和复杂锁。
正式控制台	注册后的 CharDevice，支持批量写和可选读。	需要处理设备移除和写入失败，日志路径不能阻塞过久。
用户态终端	devtmpfs 节点、TTY 行规程、轮询和终端控制接口。	需要遵守 VFS、文件描述符和信号语义。

早期输出的实现应当非常克制。它可以牺牲格式化能力，但不能因为等待设备初始化而丢失关键信息。正式控制台注册后，后续日志进入字符设备路径。我们没有要求早期写入器回放所有历史日志，因为这会引入早期缓冲和分配问题。更合理的做法是在关键阶段输出阶段性标记，使启动日志即使分段，也能帮助定位失败点。

崩溃路径与早期输出类似。系统进入致命故障后，很多锁可能已经处于未知状态。此时日志路径应尽量直接写控制台，避免等待可睡眠锁或进入 VFS。若控制台已经移除或写入失败，崩溃路径仍然要继续执行关机或停机流程。诊断输出很重要，但不能让崩溃路径因为输出失败再次卡死。

11.7 控制台注册、替换与生命周期

全局控制台保存字符设备句柄，但注册和替换仍然需要生命周期规则。设备驱动探测成功后可以注册控制台。若另一个更合适的设备出现，例如从早期串口切换到正式 virtio 控制台，注册逻辑需要决定是否替换。替换时不能让正在写日志的路径持有旧设备锁太久，也不能在旧设备移除后继续访问底层硬件。我们使用 Arc 句柄和字符设备内部活跃状态处理这个问题。

写日志时，控制台路径先克隆当前字符设备句柄，再释放全局锁并执行写入。这样全局控制台锁只保护指针替换，不保护底层设备写操作。若写操作在持有全局锁时执行，慢速串口会阻塞所有其它控制台访问，也可能与设备移除路径形成锁序问题。克隆 Arc 后写入，使旧设备即使被替换，也能在当前写入结束后自然释放。

```
fn console_write(buf: &[u8]) {
    let dev = {
        let guard = CONSOLE.lock();
        guard.as_ref().cloned()
    };

    if let Some(dev) = dev {
        let _ = dev.write_all(buf);
    }
}
```

代码 11-3 控制台写入的锁外执行

设备移除时，字符设备会被标记为已移除。全局控制台若仍然指向它，后续写入会得到设备不可用错误。注册新控制台后，全局指针可以替换。这个过程不要求所有

旧引用立即消失。需要注意的是，当前 `/dev/console` 是 `devtmpfs` 中的字符设备节点，代表绑定时的设备对象，并不会因为全局控制台指针替换而自动改写。内核日志路径也不应因为控制台不可用而崩溃。控制台是诊断出口，不是内核运行的必要不变量。这个设计让热插拔和错误设备不会扩大为全局故障。

读取控制台的语义更谨慎。内核内部很少需要从全局控制台读取字节，用户态读取应通过 `/dev/console` 或 TTY 节点进入 VFS。全局控制台的读取能力只作为简单后备能力存在，不承载完整终端语义。这样日志系统不会被输入处理和控制字符污染。

11.8 日志缓冲、级别与输出成本

日志输出有两个目标。第一，提供足够诊断信息。第二，不显著扰动被诊断的路径。系统调用热路径、调度器、网络收发和块设备输入输出都可能高频执行。若每次事件都直接格式化并写串口，系统行为会主要反映日志成本。我们因此区分关键日志、一次性诊断和可选追踪日志。文档中的日志章节重点说明控制台路径，而不是鼓励在所有热路径打印。

日志级别可以表达信息重要性。启动进度、设备注册、文件系统挂载和用户态入口属于信息级。异常恢复、资源不足和兼容性回退属于警告级。致命故障、内核崩溃和无法启动 `init` 进程属于错误级或致命级。调试某个性能问题时才启用追踪日志。不同级别可以走同一控制台写入器，但调用点应控制输出频率。

缓冲策略需要权衡。完全同步输出简单可靠，但慢速串口会拖慢系统。异步缓冲可以降低热路径延迟，但在内核崩溃时可能丢失尚未刷出的日志。我们可以为普通日志使用环形缓冲区，并在控制台可用时刷出，同时让崩溃路径直接同步输出关键现场。这样正常运行不被串口过度拖慢，故障时仍有最重要信息。当前实现保持简化路径，但接口边界已经允许后续加入环形缓冲区。

```
struct LogRecord {
    level: LogLevel,
    timestamp_ns: u64,
    cpu: usize,
    text: ArrayString<LOG_LINE_MAX>,
}

struct LogBuffer {
    records: RingBuffer<LogRecord>,
    dropped: AtomicUsize,
}
```

代码 11-4 日志缓冲的可扩展结构

输出成本还影响并发。日志路径不能在持有运行队列锁、等待队列锁或 VFS 核心锁时执行慢速写入。若需要记录这些路径的信息，应先保存低成本状态，再在锁外输出。第七章已经讨论锁外回调。日志系统也遵守同一原则。诊断信息再重要，也不能把锁序变得不可预测。

11.9 devtmpfs 投影与标准输入输出

用户态通过文件描述符访问终端。init 进程启动前，内核通常会为它准备标准输入、标准输出和标准错误。最常见的绑定是 `/dev/console`。这要求 devtmpfs 文件系统能够提供控制台节点，VFS 能够打开它，文件描述符表能够把三个标准文件描述符指向对应文件对象。若其中任一步失败，用户态仍可能运行，但调试和交互会非常困难。

devtmpfs 投影不应决定底层控制台身份。第三章已经说明，用户态投影层观察设备能力注册表并创建节点。对于控制台，节点绑定启动阶段选定的字符设备。对于具体串口，节点可以指向对应字符设备能力。这样 `/dev/console` 保持固定语义，其它串口节点保留设备身份。用户程序可以选择固定控制台，也可以选择具体设备。

标准输入输出的继承依赖执行映像替换语义。第九章说明执行映像替换保留文件描述符，除非带 `FD_CLOEXEC`。因此 shell 程序打开的终端文件描述符会被子进程继承，管道和重定向也能工作。终端章节的职责是提供可用文件操作，进程章节和 VFS 章节负责文件描述符生命周期。这个分工保证终端不会直接管理进程文件表。

devtmpfs 节点还需要支持轮询。shell 程序和交互程序可能使用 `poll` 或 `select` 等待输入。TTY 文件的 `poll` 实现需要同时观察行缓冲和底层字符设备 FIFO。行缓冲已有完整数据时可以直接报告可读。底层 FIFO 有字节时也要报告可读，使阻塞 `read` 醒来后继续把字节拉进行程处理。等待者登记落到底层字符设备的等待队列上，输入泵则负责在没有用户态读调用栈时推进控制字符和缓冲状态。若 `poll` 只看行缓冲，规范模式下新到达的串口字节可能没有机会被重新拉进行程。若 `poll` 只看底层 FIFO，又会绕过回显、控制字符和规范模式缓冲。终端语义必须在 TTY 层完成。

11.10 TTY 缓冲与控制接口

TTY 行规程把底层字节流转化为用户态可读数据。最简单的原始模式直接交付字节。规范模式则按行缓冲，处理回车、退格、文件结束、回显和控制字符。即使当前实现只覆盖核心语义，也需要把底层字符设备和行规程分开。底层驱动只知道字节到达，行规程才知道终端模式和进程组。

输入缓冲通常分为原始输入缓冲和规范化行缓冲。底层设备读到字节后放入原始缓冲，行规程消费字节并生成可读行。控制字符在行规程阶段被截获。普通字符可能回显到输出。回车可能转换为换行。退格可能删除缓冲中最后一个字符并回显擦除序列。文件结束字符可能使当前行提前完成。这些行为都属于用户态可见终端语义。

```
fn process_tty_char(tty: &Tty, byte: u8) {
    match byte {
        CTRL_C => tty.send_signal_to_foreground(SIGINT),
        CTRL_Z => tty.send_signal_to_foreground(SIGTSTP),
        b'\r' | b'\n' => {
            tty.line_buffer.push(b'\n');
            tty.echo(b"\r\n");
            tty.read_waiters.wake_all();
        }
        BACKSPACE => tty.erase_last_char(),
        ch => {
            tty.line_buffer.push(ch);
        }
    }
}
```



```
        if tty.echo_enabled() {
            tty.echo(&[ch]);
        }
    }
}
```

代码 11-5 TTY 行规程处理输入字符

终端控制接口是 POSIX 兼容中很细的一部分。用户态程序会通过 TCGETS、TCSETS、TIOCGPGRP、TIOCSPGRP 等命令读取和设置终端属性。shell 程序、交互程序和 readline 类库都可能依赖这些接口。终端层需要维护 termios 终端属性状态，包括输入标志、输出标志、本地标志、控制字符数组和波特率等字段。并非所有字段都必须立即完整生效，但读写结构体和核心标志应保持稳定。

termios 终端属性状态影响行规程。例如 ECHO 控制是否回显，ICANON 控制规范模式，ISIG 控制控制字符是否生成信号。若 ISIG 关闭，Ctrl-C 应作为普通字符进入缓冲，而不是发送 SIGINT。若 ECHO 关闭，输入字符不应回显。终端层因此必须在处理字符时读取当前 termios 快照。修改 termios 时需要加锁，避免输入泵和终端控制接口同时更新状态。

表 11-3 终端控制接口与状态影响

接口	状态	影响
TCGETS	读取 termios 终端属性。	用户态库判断终端模式。
TCSETS	写入 termios 终端属性。	影响回显、规范模式、控制字符和输出转换。
TIOCGPGRP	读取前台进程组。	shell 程序判断当前控制终端归属。
TIOCSPGRP	设置前台进程组。	作业控制决定信号发送目标。

终端控制接口还要处理用户结构复制。读取 termios 终端属性时，内核把状态复制到用户指针。设置 termios 终端属性时，内核从用户指针读取结构并校验。用户指针错误返回 EFAULT。命令不支持返回 ENOTTY 或 EINVAL，具体取决于语义。用户态运行库会依赖这些错误码。终端控制接口因此也依赖第八章的系统调用错误传递和用户访问窗口。

终端的控制字符通常发送给前台进程组，而不是单个进程。shell 程序创建作业后，将作业进程放入进程组，再把该进程组设置为终端前台组。用户按 Ctrl-C 时，终端向前台组发送 SIGINT。作业停止后，shell 程序重新获得前台终端。这个机制需要终端层保存前台进程组 ID，并通过信号层按组发送信号。

后台进程访问终端也有特殊规则。后台进程读控制终端可能收到 SIGTTIN，后台进程写控制终端可能根据 termios 终端属性设置收到 SIGTTOU。完整实现需要结合会话、控制终端和进程组。即使当前只实现核心控制字符，我们也把前台进程组作为 TTY 状态建模，避免后续扩展时重写终端对象。

作业控制的关键边界是，TTY 只决定目标进程组和信号类型，不直接修改任务状态。停止、继续和等待接口可见事件都由第十章信号和第五章进程管理处理。这样终端层不会持有 TTY 锁进入复杂进程状态修改，也不会把作业控制逻辑散落到输入处理里。

11.11 故障诊断与分层定位

控制台和终端问题很容易被误判为其它子系统问题。用户态没有输出，可能是 init 进程没启动，也可能是标准输出没绑定，也可能是控制台节点不存在，也可能是底层字符设备已经移除。Ctrl-C 无效，可能是输入泵没运行，也可能是 ISIG 关闭，也可能是前台进程组错误，也可能是信号层没有唤醒任务。我们在文档中拆分这些层次，是为了让排查有明确路径。

诊断终端问题时，可以按顺序检查。第一，早期内核日志是否能输出。第二，正式控制台是否注册。第三，devtmpfs 文件系统是否创建 `/dev/console`。第四，init 进程的标准文件描述符是否指向可写文件。第五，TTY 输入泵是否能读取底层字符。第六，行规程是否把普通字符放入缓冲。第七，控制字符是否调用信号发送。这个顺序从底层到用户态，能快速定位断点。

脚本程序和交互程序对终端有不同要求。脚本通常需要控制台和文件描述符继承稳定。交互 shell 程序需要回显、控制字符和部分 termios 终端属性。后台服务可能只需要标准输出输出结果。我们没有把所有终端兼容细节一次性做成庞大模块，而是先保证关键链路稳定，再围绕 termios 终端属性和作业控制扩展。这样可以避免终端层过早膨胀。

崩溃诊断也依赖控制台。若内核发生致命故障时无法输出程序计数器、进程号和任务名，后续排查会非常困难。控制台路径越简单，崩溃输出越可靠。我们把崩溃输出与用户态终端分开，正是为了让复杂 TTY 状态不会影响内核最后的诊断机会。

11.12 虚拟化环境中的控制台约束

虚拟控制台和真实硬件串口的行为不完全相同。有些后端提供稳定中断，有些后端更接近轮询设备。有些输出路径很快，有些则因为同步刷新而非常慢。控制台设计不能假设每个字符写入都有相同成本，也不能假设输入中断一定可靠。第十一章前面提到的 TTY 输入泵，正是为了兼容这类环境差异。

LoongArch64 和 RISC-V64 平台上的控制台设备接入方式也不同。某些平台使用 PCI virtio 控制台或串口，某些平台使用 MMIO 设备。第三章的设备能力模型把这些差异收束为字符设备。终端层只看到 CharDevice。这样虚拟化平台的差异不会扩散到 TTY 行规程。若某个平台暂时只有最小串口能力，也可以先注册为控制台，再逐步补齐轮询和终端控制接口。

虚拟环境还会放大日志成本。大量同步输出会拖慢系统，改变调度时序。我们通常只保留阶段性日志和异常日志，避免在每次 read、write、poll 或中断中打印。文档中强调低扰动日志，就是基于这种经验。

输入侧也需要容忍丢事件。某些虚拟串口不会在每个输入字节到来时可靠触发中断。若内核只依赖中断，shell 可能无法及时读到输入。兜底轮询线程提供了一个

保守保障。它不追求最低输入延迟，但能保证没有中断时仍然推进终端缓冲。对终端层而言，这种稳定性比极致交互延迟更重要。

11.13 锁序与输出重入

日志和终端都容易出现重入。一个路径持有锁时打印日志，日志路径又尝试获取控制台锁或字符设备锁。字符设备写入失败时又打印错误。崩溃路径在故障中再次进入日志。这些情况如果没有约束，会形成递归输出或死锁。我们把日志路径设计得尽量短，并要求热路径和持锁路径控制打印频率。

全局控制台锁只保护句柄替换。底层字符设备写入在锁外执行。TTY 锁保护行规程状态和缓冲，不应在持有 TTY 锁时调用可能阻塞的 VFS 操作。输入泵从底层设备读字节后，应按小批量处理，避免长期占用 TTY 锁。控制字符触发信号时，也应在合适边界调用信号发送，不能持有过多终端内部锁进入进程组遍历。

```
fn handle_input_batch(tty: &Arc<Tty>, bytes: &[u8]) {
    let mut signals = Vec::new();

    {
        let mut state = tty.state.lock();
        for byte in bytes {
            if let Some(sig) = state.consume_control_char(*byte) {
                signals.push((state.foreground_pgrp, sig));
            } else {
                state.push_input(*byte);
            }
        }
    }

    for (pgrp, sig) in signals {
        signal_process_group(pgrp, sig);
    }
}
```

代码 11-6 TTY 控制字符的锁外信号发送

这个模式和第七章的锁外回调一致。TTY 锁内只更新终端缓冲和收集事件。锁外再发送信号。这样信号层遍历任务、检查权限和唤醒等待者时，不会反向依赖 TTY 锁。输出方向也类似。行规程决定需要写出的字节后，可以把字节批量提交给底层字符设备，避免每个字符都持锁往返。

重入防护还涉及内核崩溃。崩溃期间如果再次崩溃，日志路径应尽量直接输出简短信息，避免进入复杂格式化。可以使用原子标志记录崩溃已经发生。后续崩溃只输出最小上下文。这样至少保留最后诊断信息，而不是因为递归日志耗尽栈或卡在锁上。

11.14 用户态接口与运行组织

终端用户态接口不只服务人工交互。脚本通过标准输出输出结果，系统服务通过控制台打印日志，交互程序通过标准输入读取命令。若标准输出阻塞或丢失，程序可能已经完成但操作者无法看到结果。若标准输入行规程错误，交互式 shell 程序无法

执行命令。终端章节因此需要把标准输入输出看作用户态运行基础设施,而不只是用户体验问题。

终端链路可以分层观察。内核早期日志验证原始输出。设备注册日志验证控制台切换。启动 `init` 进程后,用户态输出可以验证标准输出。运行 `echo` 可以验证写路径。运行 `cat` 或 `shell` 程序读取可以验证标准输入。发送 `Ctrl-C` 可以验证控制字符和信号。执行 `stty` 可以验证 `termios` 终端属性。每一层失败都指向不同模块。这样比单纯观察 `shell` 程序是否能用更可定位。

很多用户态程序间接依赖终端和标准文件描述符。脚本需要 `/bin/sh` 输出错误。守护进程需要日志通道。交互程序需要终端控制接口和设备节点。即使我们暂时不完整支持伪终端,也应保证 `/dev/console` 和基本 TTY 能稳定工作。这样其它子系统的问题不会被终端问题掩盖。

用户态接口还要处理非阻塞模式。`shell` 程序和库可能对终端文件描述符设置 `O_NONBLOCK`, `poll` 后再 `read`。非阻塞 `read` 在无数据时返回 `EAGAIN`, 阻塞 `read` 则睡眠。`write` 也可能短写。TTY 层需要把这些语义映射到缓冲状态和底层设备状态。否则用户态会出现忙等循环或长时间卡住。

11.15 工程设计总结

日志、控制台和终端同时服务内核诊断和用户交互。它们跨越启动早期、设备模型、VFS、信号和调度。我们把它们拆成全局控制台、`devtmpfs` 用户节点、TTY 行规程和输入泵几个层次。

日志、控制台与终端机制具备以下创新:

第一是控制台直接复用字符设备模型,同时把内核日志路径和用户态终端路径分开。内核不维护另一套控制台接口,任何能实现字符设备语义的底层对象都可以注册为控制台。日志可以在 VFS 尚未可用时输出,用户态则通过 `devtmpfs` 节点和 TTY 兼容层访问终端。`xv6` 的控制台路径非常直接,适合教学内核,但它不区分完整 TTY 语义和早期诊断路径。我们的设计在保持早期路径短小的同时,给用户态终端留下独立演进空间。

第二是 TTY 输入泵把中断提示和行规程处理分开。钩子只设置原子位,真正的行规程在内核线程中执行。这样避免在中断上下文中获取 VFS 或 TTY 锁,也保留了输入中断不可靠时的兜底推进。Linux 的 TTY 层拥有更成熟的行规程和翻转缓冲机制,我们没有复制它的复杂度,而是抽取其中最重要的工程边界,即底层输入事件和上层终端语义不能在中断上下文中混在一起。

第三是终端状态与行规程被建模为独立层。`termios` 终端属性、回显、规范模式、控制字符和前台进程组都属于 TTY 对象,不属于底层字符设备。底层设备只提供字节流。控制字符通过信号机制表达,TTY 不直接改变任务状态,而是向前台进程组发送信号。这个分层使 UART 串口、`virtio` 控制台或未来伪终端都能复用同一终端语义,也让作业控制、停止或继续状态和用户处理函数回到第十章的信号路径处理。

第四是 `devtmpfs` 投影和标准输入输出继承形成统一用户态入口。`/dev/console`、具体串口节点和 `/dev/tty` 分别表达固定控制台、设备身份和当前控制终端。执行映像替换保留文件描述符后,`shell` 程序、脚本和服务进程都能通过统一文件描述符语义使用终端。这个设计把设备模型、VFS 和用户态输入输出连接起来,同时避免让 `/dev` 节点反向决定底层控制台生命周期。

这些设计让控制台既能承担最早期的故障诊断，也能在用户态成为标准输入输出和交互终端。它是设备模型、VFS 和信号机制之间的桥，但不是任何一方的附属实现。控制台路径保持短，终端路径保留完整语义，二者共同支撑启动可观察性和用户态交互。对整个系统而言，终端层提供的是一种最低成本的可观察性。即使文件系统、网络或复杂用户态服务尚未完全稳定，控制台仍然能帮助开发者看到内核走到了哪里、用户程序输出了什么、系统为何停止。

终端层的另一项价值，是把人机交互和脚本执行统一在同一套文件语义上。人工输入、shell 脚本和服务输出都通过文件描述符、`read`、`write`、`poll`、`ioctl` 和信号这些机制交汇。只要这条路径稳定，用户态生态就有一个基本可靠的控制面。后续无论继续完善网络、文件系统还是调试设施，控制台和终端都会作为最基础的反馈通道存在。

第十二章 网络子系统

在第十一章中，终端把字符设备、VFS 和信号机制连接在一起。本章讨论网络子系统。网络的复杂性来自两个方向。向下，它依赖具体网卡驱动、DMA 或 virtqueue 队列、链路状态和 MTU。向上，它需要为套接字系统调用提供 TCP、UDP、原始套接字、地址绑定、监听、连接、收发和轮询语义。中间还需要协议栈状态机、路由表、接口配置和等待队列。

我们把网络子系统拆成两层。一层是 `libs/net`，负责网络设备抽象、接口管理、协议栈适配、路由和网络套接字句柄。另一层是 VFS 与套接字兼容层，负责把 POSIX 套接字文件描述符映射到网络栈或 Unix 套接字。第三章中的 `NetFunction` 把网络设备作为设备能力发布，协议栈再挂接对应 `NetDevice`。这样，驱动、协议栈和用户态套接字三者之间没有循环依赖。

12.1 分层结构

网络分层的底部是网络驱动接口（`NetDriver`）。具体驱动实现收包、发包、链路状态、MAC 地址和统计信息。`NetDevice` 保存接口身份、名称、MTU 和活跃或移除状态。`NetFunction` 把 `NetDevice` 适配为设备模型中的设备能力，并提供类型化控制请求。`NetStack` 管理所有已挂接的接口、路由表和套接字等待者。VFS 套接字层把系统调用转换为 `NetStack` 操作。



这个结构保留了替换协议栈的可能性。NetDriver 和公共配置类型不依赖 smoltcp。协议引擎适配集中在适配器、引擎和接口层。若未来替换协议引擎，驱动和 VFS 系统调用层不需要整体重写。当前公共 API 暴露的是 Endpoint、IfConfig、NetSocketHandle、SocketState 和错误类型，而不是 smoltcp 内部对象。

12.2 接口管理与路由

NetStack 使用读写锁保护接口注册表。挂接和分离需要写锁，轮询和套接字操作使用读锁。每个接口内部又有独立互斥锁，因为协议引擎的单个接口不可并发访问。这样不同接口可以并行，单个接口内部保持串行。接口挂接成功后，路由表会根据接口配置更新直连路由和网关路由。分离时移除接口，并清理对应路由。

```
struct NetStack {
    tuning: NetTuning,
    interfaces: RwLock<BTreeMap<InterfaceId, Arc<Mutex<ManagedInterface>>>>,
    routes: RwLock<RouteTable>,
    notify_waiters: WaitQueue,
}

fn attach(&self, dev: Arc<NetDevice>, config: IfConfig) -> Result<(),
NetError> {
    let id = dev.id();
    let managed = ManagedInterface::new(dev, config.clone(),
self.tuning.tcp, self.tuning.tcp_listen)?;

    self.interfaces.write().insert(id, Arc::new(Mutex::new(managed)));
    self.routes.write().replace_connected(id, &config.addresses);
    self.routes.write().replace_gateway(id, config.gateway);
    Ok(())
}
```

代码 12-1 NetStack 的核心结构

读写锁的选择来自网络输入输出的访问模式。轮询、发送、接收和状态查询远比挂接和分离高频。读锁允许多个读路径同时进入接口表，再在具体接口锁上串行化。若使用单一全局互斥锁，所有接口和套接字操作都会互相阻塞。若完全无锁，接口热移除时的一致性又难以保证。当前设计把注册表变化和单接口状态变化分开处理。

12.3 协议栈轮询与主动推进

协议栈需要周期性轮询。定时器或网络线程调用 poll_ns，它将调度器纳秒时间转换为网络层时间，然后驱动所有接口进行一轮收发。每个接口轮询后会产生结果，网络栈根据结果更新路由、状态或套接字可用性。轮询结束后，当前实现会唤醒全局套接字等待队列中的任务，让它们重新检查自身套接字状态。

```
fn poll(&self, timestamp: NetInstant) {
    let table = self.interfaces.read();
    for (id, iface_lock) in table.iter() {
        if let Some(mut iface) = iface_lock.try_lock() {
```

```

        let result = iface.poll(timestamp);
        drop(iface);
        self.apply_poll_result(*id, result);
    }
}
drop(table);
self.wake_socket_waiters();
}

fn poll_now(&self) {
    let timestamp = NetInstant::from_nanos(now_ns_public());
    for _ in 0..self.tuning.active_poll.max_rounds {
        if !self.poll_one_active_round(timestamp) {
            break;
        }
    }
}
}

```

代码 12-2 协议栈轮询

`poll_now` 用于套接字操作刚刚产生出站数据之后主动推进协议栈。回环场景尤其需要它。一次发送可能立即回灌为接收包，若只等待定时器节拍，`connect`、`accept` 或 `read` 会出现不必要延迟。主动轮询至少执行两轮，第一轮负责发送，第二轮处理回灌或同接口接收。之后若没有套接字状态变化，就停止空转。

当前全局唤醒策略有意保持简单。所有阻塞在网络套接字上的任务会被唤醒，然后各自重新检查就绪状态。这个策略会有惊群，但实现可靠。文档必须明确它的边界。高并发网络负载下，应当进一步按套接字句柄和事件类型精确唤醒。当前实现先换取兼容性和可调试性。

12.4 套接字与 VFS 边界

POSIX 套接字在用户态表现为文件描述符。VFS 套接字层负责创建 `FileOps`，把 `read`、`write`、`sendmsg`、`recvmsg`、`ioctl` 和 `poll` 转换为套接字操作。网络套接字使用 `NetSocketHandle` 指向协议栈中的套接字。Unix 套接字则由 `libs/socket` 在内核内存中实现，不经过网络协议栈。二者共享文件描述符模型，但底层数据路径不同。

表 12-1 Socket 类型与实现边界

类型	实现对象	语义
TCP 套接字	NetStack 中的协议套接字句柄。	连接、监听、 <code>accept</code> 、字节流收发和状态查询。
UDP 套接字	NetStack 中的数据报套接字。	保留消息边界， <code>sendto/recvfrom</code> 处理远端地址。
原始套接字	协议栈原始套接字或 ICMP 句柄。	调用方提供完整 IP 包或协议相关负载。
Unix 套接字	<code>libs/socket</code> 内存对象。	本机 IPC，不经过网络设备和 IP 路由。

这个边界让套接字系统调用层保持统一。socket 返回文件描述符。poll 和 epoll 观察就绪状态。close 释放文件对象。具体数据路径由文件操作内部选择。网络套接字可能触发协议栈轮询和等待队列。Unix 套接字则在内核内存队列中完成连接和收发。用户态不需要知道二者差异。

12.5 网络设备能力与控制路径

网络设备通过 NetFunction 进入设备模型。它暴露接口 ID、名称、链路介质、链路状态、MAC 地址、MTU 和统计信息。控制请求如设置 MTU 或管理启用状态会进入类型化控制，而不是通过无结构的全局设备表查找。设备移除时，NetFunction::mark_gone 会标记底层 NetDevice，正在进行的收发可以看到设备不可用。

```
enum NetControlRequest {
    GetInterfaceId,
    GetName,
    GetLinkState,
    GetMacAddress,
    GetMtu,
    SetMtu { mtu: usize },
    SetAdminUp { up: bool },
}

fn control_net_device(dev: &Arc<NetDevice>, req: NetControlRequest)
-> Result<NetControlResponse, ControlError>
{
    if !dev.is_active() {
        return Err(ControlError::NoDevice);
    }

    match req {
        NetControlRequest::SetAdminUp { up } => {
            stack().set_iface_admin_up(dev.id(), up)?;
            Ok(NetControlResponse::Done)
        }
        NetControlRequest::SetMtu { mtu } => {
            dev.set_mtu(mtu)?;
            Ok(NetControlResponse::Done)
        }
        _ => query_device_property(dev, req),
    }
}
```

代码 12-3 网络设备类型化控制

控制路径的设计延续了第三章的类型化设备能力思路。网络设备的自然消费者是协议栈，而不是字符设备读写。用户态控制接口兼容层可以把传统命令翻译为类型化请求。设备核心则只处理结构化请求。这样可以减少命令码与参数结构不匹配的风险。

12.6 套接字生命周期

套接字的生命周期从 `socket()` 系统调用开始。用户态指定地址族、类型和协议。内核创建对应套接字对象，并把它包装成 VFS 文件对象。之后用户态通过文件描述符执行 `bind`、`listen`、`connect`、`accept`、`sendmsg`、`recvmsg`、`shutdown`、`poll` 和 `close`。每个操作都要维护套接字状态机。TCP、UDP、原始套接字和 Unix 套接字的状态机不同，但它们都要进入统一文件描述符生命周期。

TCP 套接字的状态最复杂。主动连接从已创建或已绑定进入连接中，握手完成后进入已建立。被动监听套接字进入监听状态，收到连接后产生待接受子连接，`accept` 再把子连接交给用户态。关闭时需要处理半关闭、FIN、RST 和本地关闭。当前实现可以先覆盖核心状态，但状态机边界必须清楚。若 `connect` 尚未完成，非阻塞 `connect` 应返回 `EINPROGRESS`，后续 `poll` 写事件表示连接完成或失败。若 `accept` 队列为空，阻塞 `accept` 睡眠，非阻塞 `accept` 返回 `EAGAIN`。

UDP 套接字保留消息边界。`bind` 决定本地地址和端口。`sendto` 可以指定远端地址。`connect` 对 UDP 只是设置默认远端并过滤接收来源，不建立握手。`recvfrom` 每次返回一个数据报及其来源地址。若把 UDP 当作字节流处理，用户态会看到错误的边界语义。套接字层因此要在文件操作统一入口下保留类型差异。

表 12-2 套接字生命周期中的关键状态

阶段	主要操作	状态约束
创建	<code>socket</code> ，分配协议套接字和文件对象。	检查地址族、类型和协议，失败返回明确错误码。
绑定	<code>bind</code> ，设置本地地址和端口。	端口冲突返回 <code>EADDRINUSE</code> ，地址非法返回 <code>EADDRNOTAVAIL</code> 。
连接或监听	<code>connect</code> 、 <code>listen</code> 、 <code>accept</code> 。	TCP 进入握手或监听状态，UDP 记录默认远端。
收发	<code>sendmsg</code> 、 <code>recvmsg</code> 、 <code>read</code> 、 <code>write</code> 。	遵守阻塞、非阻塞、消息边界和错误状态。
关闭	<code>shutdown</code> 、 <code>close</code> 。	唤醒等待者，释放句柄，通知协议栈状态变化。

关闭需要和 VFS 文件生命周期协调。用户关闭文件描述符时，文件对象引用可能仍被其它线程或轮询结构持有。套接字对象要先阻止新的输入输出，再唤醒等待者，最后在引用释放时让协议栈关闭句柄。若直接释放协议套接字，仍在等待的任务可能访问悬空对象。我们沿用第七章的生命周期收束原则，用状态标记和引用计数分离用户不可再发现和对象内存可释放。

12.7 阻塞、非阻塞与就绪状态

套接字系统调用必须处理阻塞和非阻塞模式。阻塞 `read` 在无数据时睡眠。非阻塞 `read` 返回 `EAGAIN`。阻塞 `connect` 可以等待握手完成。非阻塞 `connect` 返回

EINPROGRESS。poll 和 epoll 不直接执行输入输出，只报告就绪状态。就绪状态表达协议套接字的用户可见状态，不能简单等同于底层设备可读写。例如 TCP 监听套接字的可读表示 accept 队列非空，TCP 连接套接字的可读表示接收缓冲有数据或对端关闭。

我们把等待语义放在套接字文件层和 NetStack 的等待队列之间。每个套接字操作先检查协议状态。若可立即完成，直接返回。若不可完成且文件描述符非阻塞，返回 EAGAIN 或对应错误。若可阻塞，任务登记到网络等待队列，触发轮询或等待协议栈推进，再睡眠。醒来后重新检查套接字状态。全局唤醒会带来惊群，但每个等待者都重新检查自己的句柄，因此不会破坏正确性。

```
fn recv_blocking(sock: &NetSocketFile, buf: UserSliceMut<u8>) ->
Result<usize, Errno> {
    loop {
        match sock.try_recv(buf) {
            Ok(n) => return Ok(n),
            Err(Errno::EAGAIN) if sock.nonblock() => return
Err(Errno::EAGAIN),
            Err(Errno::EAGAIN) => {
                stack().register_waiter(current_task());
                stack().poll_now();
                if sock.is_ready_for_read() {
                    stack().unregister_waiter(current_task());
                    continue;
                }
                schedule_interruptible()?;
                stack().unregister_waiter(current_task());
            }
            Err(e) => return Err(e),
        }
    }
}
```

代码 12-4 套接字阻塞读取模式

就绪状态还要处理错误。连接失败、对端关闭、套接字被 shutdown，都应使 poll 返回相应事件，使等待者醒来并读取错误或文件结束。若只在数据到来时唤醒，关闭事件可能被忽略，用户态会永久阻塞。我们在轮询结果应用阶段统一唤醒套接字等待者，保证状态变化能被观察。后续优化可以按句柄精确唤醒，但必须保留错误状态唤醒。

信号打断同样适用。阻塞套接字操作进入可打断睡眠。若收到待处理信号，睡眠返回 EINTR 或参与系统调用重启。套接字层不直接构造信号帧，只把等待结果上交给系统调用分发器。这个边界与第十章一致。

12.8 路由、地址与接口状态

网络接口挂接后，NetStack 根据配置生成直连路由。配置网关后，路由表生成默认或指定网关路由。发送数据包时，协议栈根据目标地址查询路由，选择接口和下一跳。若没有路由，connect 或 sendto 应返回 ENETUNREACH 或相关错误。若接口关闭，返回 ENETDOWN 或让套接字进入错误状态。路由错误不能表现为静默丢包，否则用户态很难诊断。

接口状态分为设备活跃、链路可用和管理启用。设备活跃表示驱动对象仍然有效。链路可用表示物理或虚拟链路可用。管理启用表示内核配置允许该接口参与收发。三者不应混为一谈。设备移除时，所有操作都应失败。链路断开时，设备仍然存在，但收发可能不可用。管理关闭是用户或配置主动关闭接口。类型化控制可以分别查询和设置这些状态。

表 12-3 接口状态层次		
状态	来源	影响
设备活跃	设备生命周期。	移除后阻止新输入输出，分离后清理路由和等待者。
链路可用	驱动或虚拟后端报告。	影响实际收发和用户态链路状态查询。
管理启用	内核配置或用户控制。	决定协议栈是否把接口作为可用出口。
路由就绪	地址和网关配置。	决定 <code>send/connect</code> 是否能找到路径。

地址配置也要分层。接口可以同时保存 IPv4 地址、IPv6 地址、前缀长度、广播地址和网关。IfConfig 统一描述这些配置，使协议栈挂接时一次性获得地址信息。当前 NetStack 已经提供 IPv4 与 IPv6 地址设置、默认路由和静态路由更新入口，路由表也能按照 IPv4 或 IPv6 目标地址选择接口。用户态兼容管理面仍不完全对称。传统 SIOC* 接口和当前 netlink 写路径主要覆盖 IPv4 地址、子网掩码、MTU、管理启停和 IPv4 路由，IPv6 的运行期配置更多停留在内核内部接口和查询视图上。这个边界需要在文档中明确，否则容易把底层协议能力误读成完整的用户态网络管理能力。

路由表更新需要和接口注册表保持一致。挂接先创建接口，再更新路由。分离先阻止接口继续参与路由，再移除接口。若路由指向已移除接口，发送路径可能访问无效句柄。我们用接口 ID 作为关联键，分离时清理对应路由。读路径通过锁保护看到一致快照。

12.9 回环与主动推进

本机通信大量依赖回环接口。发送和接收都发生在同一内核内。若协议栈只靠周期定时器轮询，发送方写入数据后要等下一个时钟节拍才能被接收方看到，吞吐和延迟都会变差。poll_now 的作用就是在套接字操作后主动推进协议栈，使本机数据尽快回灌。

主动推进需要限制轮数。若每次发送都无限轮询，用户态写入会被协议栈处理拖住，甚至在大量连接时长时间占用 CPU。当前调优参数中保留主动轮询参数，设置最大轮数。第一轮通常把出站数据交给设备或回环接口，第二轮处理回灌和状态变化。若没有更多变化，就停止。这个策略兼顾短连接延迟和 CPU 使用。

回环接口还暴露唤醒策略问题。发送方写入后，接收方可能正在 recv 阻塞。协议栈轮询处理到接收数据后，应唤醒等待者。全局唤醒会让所有网络等待者重新检查，

简单可靠但可能惊群。高并发 TCP 服务器中，精确唤醒更合适。我们先使用全局唤醒保证正确性，再把精确唤醒作为后续优化方向。这个取舍符合当前工程阶段。

主动轮询也不能在持有过多锁时执行。套接字操作可能持有套接字文件锁，NetStack 轮询需要接口锁。若锁序设计不清，会出现套接字锁和接口锁互相等待。我们尽量在短临界区内更新套接字状态，调用协议栈推进时避免持有不必要的 VFS 锁。这个边界对网络性能和死锁预防都很重要。

12.10 并发、锁序与资源回收

网络对象生命周期复杂。套接字文件对象被文件描述符表持有，协议栈套接字集合持有句柄，等待队列持有等待者弱引用，接口表持有 `ManagedInterface`，设备模型持有 `NetFunction`。关闭、分离、进程退出和设备移除都可能同时发生。设计目标是让每条路径都能收束。关闭阻止新的套接字操作并释放句柄。分离标记接口不可用并唤醒等待者。进程退出通过文件描述符关闭触发套接字释放。设备移除通过 `NetDevice` 活跃状态让协议栈停止使用驱动。

锁序需要特别控制。接口注册表锁保护接口 ID 到接口的映射。接口锁保护协议引擎对象。套接字文件锁保护文件级标志和局部状态。等待队列锁只保护等待者列表。调用顺序应避免在接口锁内执行用户复制，也避免在等待队列锁内进入协议栈轮询。唤醒操作尽量锁外进行。这个模型与第七章同步规则一致。

资源回收时还要唤醒阻塞任务。一个线程阻塞在 `recv`，另一个线程关闭同一个套接字。关闭路径应让 `recv` 返回错误或文件结束，而不是永远睡眠。接口分离时，所有依赖该接口的套接字也应观察到错误。当前全局唤醒策略虽然粗，但能保证状态变化被等待者重新检查。后续精确唤醒不能牺牲这一点。

网络缓冲区分配失败应返回错误或触发背压，不应导致内核崩溃。用户态可以通过高频发送造成内存压力。协议栈和套接字层应返回 `ENOBUFFS`、`ENOMEM` 或短写。只有内核基础结构损坏时才进入不可恢复路径。这个原则和前面章节对分配失败的处理一致。

12.11 性能与演进视角

网络性能主要受数据复制、协议栈推进、锁竞争、设备队列和唤醒延迟影响。连续 TCP/UDP 吞吐更关注批量收发能力，请求响应型负载更关注系统调用成本、主动轮询和等待者唤醒。边界参数和阻塞语义则更多依赖套接字系统调用兼容层。不同负载暴露的问题不同，因此分析网络性能时需要先判断瓶颈位于数据路径、控制路径还是 ABI 处理。

我们在网络设计中优先保证正确性和可收束。全局唤醒、主动轮询和清晰错误转换会带来一些额外成本，但能让状态变化可靠传递。性能优化可以沿三个方向推进。第一，精确套接字唤醒，减少惊群。第二，减少数据复制和锁持有时间。第三，根据网卡中断和队列能力改进轮询触发。每个方向都可以在现有分层中独立推进。

文件输入输出与网络看似无关，但它们共享用户复制和调度能力。若 `copy_from_user` 慢，网络 `sendmsg` 和 `recvmsg` 也会受影响。若调度唤醒慢，阻塞套接字的延迟也会增大。网络性能分析因此不能只看协议栈。它要结合第七章同步、第八章系统调用、第十二章轮询和驱动路径共同分析。

网络程序的诊断输出还依赖第十一章的 `console`。若终端输出慢，会影响日志观察和问题定位。文档把这些关系讲清楚，有助于后续解释网络路径和用户态可观察行为之间的差异。

■ 12.12 工程设计总结

网络子系统连接设备驱动、协议栈、VFS 和用户态套接字。它的核心挑战是保持这些层的边界。驱动只提供链路与包收发。协议栈处理接口、路由和协议状态。VFS 负责文件描述符和 POSIX 套接字语义。设备模型负责生命周期和类型化控制。这样的设计使网络子系统可以在当前规模下保持简洁，同时为后续精确套接字唤醒、更多驱动、更完整的 IPv6 用户态管理接口和更细粒度路由策略留下空间。网络不是单纯的设备驱动，也不是单纯的文件接口。它位于设备、协议和 POSIX ABI 的交界处，必须用清晰的分层来控制复杂度。

后记：工程不足与未来展望

前十二章已经从启动、内存、设备、文件系统、进程、调度、并发、系统调用、用户态、信号、终端和网络等方向说明了当前内核的主要设计。回到工程实现本身，还需要承认一个更基本的事实。当前系统已经形成了较完整的内核轮廓，也具备跨架构运行、用户态装载、文件系统访问、设备接入和网络通信等能力，但它仍然处在持续收束阶段。很多子系统的边界已经建立，机制也能支撑主要路径，距离长期稳定的通用内核仍有明显差距。

当前的内核设计与实现仍然存在以下不足。

在设备抽象管理方面，当前系统已经完成了以设备能力为中心的多轨结构。字符设备、块设备、网络设备和 RTC 设备能够沿着统一的注册表进入系统，驱动探测、设备能力发布、用户态投影之间也形成了较清楚的信息流。这个结构解决了早期设备模型只围绕字符设备和块设备展开的问题，使网络设备、时间设备和后续可能加入的控制型设备都有合适的接入位置。它的不足主要出现在生命周期的完整性上。设备移除路径已经具备 `mark_gone` 和 `drain_io` 这样的收束入口，但在更多真实设备场景中，移除并不只是停止新请求和等待旧请求完成。驱动可能持有中断处理器、DMA 缓冲区、总线资源、协议栈句柄和用户态投影节点。它们之间存在严格的释放顺序。现在的框架能够表达大致阶段，却还缺少足够细的状态记录。比如某个设备能力已经从全局注册表中注销，但对应 `/dev` 节点还被打开。再如块设备请求已经进入队列，但底层驱动正在失效。这样的边界需要更细的状态机和更多审计信息，而不能只依赖引用计数自然收束。

设备抽象管理的另一个不足，是设备发现和驱动绑定仍然偏静态。当前实现可以处理固件描述和主要虚拟设备，适合系统启动阶段完成设备枚举。对于复杂总线和热插拔设备，PnP 层还需要更完整的资源仲裁机制。中断号、MMIO 区间、DMA 地址窗口和设备层级关系应当统一进入资源图，并且能够在失败回滚时恢复到可审计状态。现在驱动绑定失败时可以回滚已注册能力，但资源分配的诊断信息还不够充分。用户态投影也还有继续完善的空间。`devtmpfs` 文件系统、`sysfs` 文件系统和 `procfs` 文件系统已经能观察设备能力并生成视图，但这些视图的语义深度仍然有限。很多设备属性目前只能通过内部结构查询，尚未形成稳定的用户态属性命名和权限模型。设备号、节点权限、符号链接、类别目录和热插拔事件之间也还没有完全成体系。后续需要让资源管理从驱动内部逻辑进一步上移，同时把投影层继续做厚，让用户态看到的是稳定 ABI，而不是内核内部对象的直接影子。

在内存分配器方面，当前内存管理已经具备启动分配器、伙伴页分配器、内核地址空间、Slab 分配器、内核堆和用户地址空间等主要部件。启动期到运行期的交接路径比较清楚，普通对象和页级对象也分别走不同的分配后端。这个结构使内核可以在较复杂的子系统组合下运行，也为后续审计和优化预留了边界。它的主要不足是内存压力下的退化策略仍然不足。分配失败在内核中不应当轻易演变成不可恢复的崩溃。很多路径可以返回错误码，延迟重试，或者通过回收机制释放缓存。现在部分分配路径已经能返回 `ENOMEM`，但仍然存在一些隐含假设，即某些分配在正常运行中必然成功。这种假设在小规模测试中不容易暴露，一旦进入文件系统、网络 and 进程密集场

景,就会导致错误路径过于脆弱。后续应当系统性梳理所有分配点,区分启动期必需分配、运行期可失败分配和必须提前预留的分配。

内存分配器还需要继续补强碎片控制、缓存回收和调试审计。伙伴页分配器能够管理物理页阶数, Slab 分配器也能处理小对象热路径,但二者之间缺少统一的内存压力反馈。某些对象缓存可能长期持有空闲页,导致页级分配看到的可用内存不足。反过来,页级分配紧张时也缺少对对象缓存的精确回收请求。当前实现更偏向静态分配策略,后续需要引入分级回收、缓存收缩器和更细的统计指标,让系统可以在压力上升时主动回收可丢弃对象。调试审计也应当下沉到热路径附近。对于重复释放、越界写、悬空引用和长期泄漏这类问题,开发阶段可以接受额外开销,以换取更强的错误定位能力。运行阶段则需要较低成本的采样和统计。后续可以把红区、对象标签、分配栈摘要、Slab 类别统计和页所有者追踪做成可开关能力,使调试强度能够根据构建配置调整。更长远看,文件页、匿名页、页缓存、共享映射和 COW 页最终都消耗物理页。没有统一的内存压力模型,系统在压力上升时只能较被动地失败。后续应当把页缓存、文件系统回写和匿名页回收纳入同一套回收框架。

在 VFS 与 POSIX 兼容方面,当前 VFS 已经把索引节点、目录项、超级块、文件对象、文件描述符表、挂载命名空间和特殊文件系统连接起来。它承担了大量 POSIX 兼容工作,也是用户态程序观察内核行为的主要入口。当前设计的价值在于把文件对象能力冻结、路径解析和文件描述符生命周期分开,使文件系统、设备文件和套接字都能进入同一套文件接口。它的不足首先是 POSIX 语义覆盖仍然不完整。许多常见系统调用已经可以工作,但 POSIX 的困难往往不在普通路径,而在标志位组合、错误码顺序、权限检查和并发边界。以路径解析为例,绝对路径、相对路径、`dirfd`、符号链接、挂载点跨越、根目录限制和 `..` 组件之间存在复杂关系。当前实现已经覆盖主要流程,但与成熟系统相比,边界条件还需要继续补齐。后续应当以语义矩阵的方式整理每类系统调用,不只记录是否实现,还要记录不同标志组合下的返回值和副作用。

VFS 与 POSIX 兼容还需要补齐权限模型、缓存失效和文件系统驱动契约。文件系统访问需要结合用户 ID、组 ID、补充组、文件模式位、挂载选项、能力位和特殊文件语义。当前实现可以支撑基本用户态运行,但还没有完整表达所有权限路径。某些操作目前更依赖调用路径的局部判断,没有形成统一的访问检查入口。后续应当把凭据、权限检查和命名空间边界整理成独立层,使 VFS 调用方不需要在每个系统调用里重复拼装权限逻辑。目录项缓存和文件对象缓存能减少路径查找成本,但缓存带来失效问题。创建、删除、重命名、挂载、卸载和符号链接更新都会影响目录项状态。后续应当明确目录项缓存的状态机,区分正缓存、负缓存、正在失效和不可缓存节点,使路径解析行为更可预测。文件系统驱动方面,普通文件读写、目录遍历和元数据查询已经具备基础能力,文件锁、扩展属性、稀疏文件、同步语义、权限继承和错误恢复仍然较弱。后续需要先把 VFS 与文件系统驱动之间的契约写清楚,再逐步补充文件系统特性。POSIX 兼容最终比拼的是细节稳定性,任何一个错误码和副作用顺序都可能影响用户态程序。

在网络与套接字方面,当前网络子系统已经具备网卡驱动抽象、接口注册表、协议栈适配、路由、TCP/UDP 套接字、Unix 套接字和 VFS 文件描述符入口。回环通信和基本网络收发能够工作,套接字也已经纳入 `poll` 和文件生命周期。这个基础使用户态网络程序可以在内核上运行,也证明了设备模型、VFS 和协议栈之间的连接是可行的。它的不足首先是网络推进机制仍然偏简单。周期轮询和主动轮询可以覆盖基本场景,但成熟网络栈通常需要更细的中断、软中断、工作队列和批处理机制。当前

实现为了可靠性选择了较直接的推进方式，代价是高并发连接和大吞吐路径下调度成本较高。后续应当把设备中断、接收批处理、发送完成、协议定时器和套接字唤醒拆成更明确的执行上下文，避免所有网络事件都集中到粗粒度轮询中。

网络与套接字还需要继续细化等待唤醒、协议覆盖和缓冲所有权。当前全局唤醒策略实现简单，也能保证等待者重新检查状态，但它会带来惊群。连接数增加后，许多无关任务会被唤醒再睡回去。后续需要按套接字句柄、事件类型和等待方向组织等待队列。可读、可写、连接完成、监听队列非空、错误状态变化等事件应当具备独立唤醒路径。协议语义覆盖也仍然有限。TCP 和 UDP 的基础收发已经存在，广播、组播、原始套接字、ICMP 套接字和 IPv6 地址路由也已经有底层入口，但这些能力还没有完全收束到稳定的用户态兼容语义。例如错误队列目前只能识别 MSG_ERRQUEUE 标志，ICMP 错误还不能按照套接字错误队列精确交付。IPv6 在底层地址和路由表中已经具备表达能力，但 ioctl 和部分 netlink 写入路径仍然偏向 IPv4。原始套接字也还缺少更完整的选项处理、协议过滤和错误传播。路径 MTU 发现、组播源过滤、广播权限细节、保活、延迟确认、地址复用和拥塞控制参数等能力，都需要继续补齐。缓冲路径方面，现在的实现更关注语义打通，数据包在驱动、协议栈、套接字缓冲和用户态之间可能经历多次复制。后续可以引入更清晰的缓冲所有权模型，减少短包频繁分配，优化批量发送和批量接收。对于本机回环路径，也可以建立更直接的数据转移机制。这里需要谨慎推进，因为零拷贝和缓冲复用会显著增加生命周期复杂度，必须先有可靠的引用和回收规则。

在 ELF 装载、内存映射与调度方面，三者共同决定用户程序能否稳定运行。ELF 装载负责把可执行文件和动态链接环境放入用户地址空间。内存映射层负责维护映射、缺页、COW 和用户指针访问。调度系统负责线程生命周期、阻塞唤醒和时间片推进。三者之间的耦合很强。装载阶段创建地址空间，缺页路径依赖调度上下文，阻塞系统调用又反过来依赖地址空间和文件对象稳定存在。当前 ELF 装载的不足主要是鲁棒性。主执行文件的装载已经把元数据解析和段内容填充分开处理，普通段内容不再需要整文件读入内核堆，而是按程序头定位并按页从文件填充。这个方向是正确的，但它还没有覆盖所有边界。解释器装载、复杂段布局、辅助向量、TLS、栈随机化和动态链接细节，还需要更严格的处理。对于大文件和异常文件，装载器仍然需要保持清晰的错误返回，并保证已经创建的映射能够回滚。后续还需要完善 ELF 校验，使地址溢出、段重叠、权限非法和解释器缺失都能进入可诊断路径。

内存映射和地址空间管理的不足集中在映射语义和压力处理。当前缺页处理、匿名映射、文件映射和 COW 已经有基础能力，但 mmap、munmap、mprotect、mremap 等调用的边界条件非常多。部分行为在普通程序中不明显，在数据库、语言运行时和测试工具中会集中暴露。后续需要细化 VMA 分裂、合并、权限变更和回滚逻辑。文件映射还需要与页缓存和回写机制更紧密地结合。没有稳定的页缓存和回收路径，内存映射层很难在长时间运行中保持一致的行为。调度系统的不足主要是策略和多核扩展。当前调度器能够支撑任务创建、阻塞、唤醒和时间片轮转，但策略仍然相对基础。优先级、交互性、负载均衡、实时任务、CPU 亲和性和 NUMA 感知都还没有形成完整体系。单核场景下，简单时间片调度可以支撑主要功能。多核场景下，还需要处理运行队列之间的迁移、唤醒目标选择、锁竞争和跨核中断。否则随着并发度上升，调度器会成为系统一致性和延迟的共同瓶颈。

未来,我们的工程会把重点从实现更多内核功能转向完善已有功能,继续改进当前内核仍然存在的设计缺陷。对于少数因为抽象过厚而影响性能的热路径,需要添加边界清晰的快速路径,从而提高内核的整体可用性与兼容性。

附录

附录只放查阅型材料。正文各章讨论设计目标、机制边界和工程取舍，附录则保存与源码路径、接口范围和代码句柄直接相关的信息。这样处理可以避免正文被实现细节打断，也能给后续维护者留下较明确的阅读入口。

A. 代码组织路径

表 附录-1 给出源码阅读时最常用的路径。这里列出的路径只用于帮助定位代码，不作为正文架构原则的替代说明。正文中的分层、信息流和子系统边界仍以各章论述为准。

表 附录-1 代码组织路径		
路径	主要内容	阅读场景
arch/	LoongArch64 与 RISC-V64 的入口、异常现场、页表细节、系统调用进入路径和平台指令封装。	排查启动入口、异常处理、架构寄存器、系统调用快路径和页表格式时优先阅读。
hal/	把架构侧能力整理为上层可消耗的运行期接口，包括用户上下文、调度钩子、用户访问和平台能力注册。	需要理解平台相关能力如何交给通用子系统时阅读。
general/	启动上下文、固件视图、系统调用分发、用户地址空间、设备能力、进程间通信和跨子系统公共对象。	阅读平台无关基础设施和跨子系统交接对象时使用。
kernel/	内核主入口、系统调用实现、用户程序装载、信号兼容、VFS 适配、设备挂接和运行期编排。	追踪用户态请求如何进入具体内核策略时阅读。
libs/allocator/	物理页、Slab 分配器、内核堆和受管堆相关的底层分配能力。	排查内存分配、对象回收和分配失败路径时阅读。
libs/vfs/	索引节点、目录项、文件对象、文件描述符表、挂载命名空间、设备文件和套接字文件适配。	追踪文件系统、设备文件、路径解析和文件描述符生命周期时阅读。
libs/sched/	任务对象、调度类、运行队列、等待队列、信号状态、凭据、资源限制和任务生命周期。	排查调度、阻塞唤醒、进程关系和信号状态时阅读。

续表 附录-1 代码组织路径		
路径	主要内容	阅读场景
libs/net/	网络设备对象、网络栈、接口管理、路由、协议栈适配和网络套接字句柄。	排查网络设备接入、协议推进、套接字状态和路由行为时阅读。
libs/socket/	Unix 套接字、内核内存队列、套接字状态和本机进程间通信对象。	排查 Unix 套接字和非网络协议栈的数据路径时阅读。
libs/errno/	统一错误码枚举以及错误码和整数返回值之间的转换。	核对系统调用错误返回、兼容层错误码和子系统错误映射时阅读。
userland/	用户态根文件系统、辅助脚本、场景文件和随镜像进入用户态的程序资源。	需要确认用户态程序、启动脚本和镜像内容时阅读。

B. 系统调用和 errno 表

系统调用实现分散在 kernel/src/syscalls/ 及其调用的通用子系统中。表 附录-2 按 Linux asm-generic 系统调用号列出当前已经接入实际语义路径的入口。这里的“已实现”指调用已经注册到分发表, 并且不会直接落入纯 ENOSYS 占位实现。对于带 flags、子命令或架构兼容差异的接口, 表中的实现范围描述的是当前内核已经支持的部分。

表 附录-2 已实现系统调用表			
编号	ABI 名称	内核入口	实现范围
17	getcwd	fs::sys_getcwd	读取当前工作目录路径。
19	eventfd2	fs::sys_eventfd2	创建事件计数文件描述符。
20	epoll_create1	fs::sys_epoll_create1	创建 epoll 实例。
21	epoll_ctl	fs::sys_epoll_ctl	添加、修改或删除 epoll 监听目标。
22	epoll_pwait	fs::sys_epoll_pwait	等待 epoll 就绪事件并可临时切换信号掩码。
23	dup	fs::sys_dup	复制文件描述符到最小可用编号。
24	dup3	fs::sys_dup3	复制文件描述符到指定编号并设置描述符标志。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
25	fcntl	fs::sys_fcntl	执行文件描述符复制、标志、锁和所有权控制。
29	ioctl	fs::sys_ioctl	向文件、设备或套接字发送控制命令。
30	ioprio_set	fs::sys_ioprio_set	设置任务或进程组 I/O 优先级。
31	ioprio_get	fs::sys_ioprio_get	读取任务或进程组 I/O 优先级。
32	flock	fs::sys_flock	处理 BSD 文件锁请求。
33	mknodat	fs::sys_mknodat	创建设备节点或特殊文件。
34	mkdirat	fs::sys_mkdirat	创建目录。
35	unlinkat	fs::sys_unlinkat	删除目录项或目录。
36	symlinkat	fs::sys_symlinkat	创建符号链接。
37	linkat	fs::sys_linkat	创建硬链接。
38	renameat	fs::sys_renameat	重命名路径。
39	umount2	fs::sys_umount2	卸载挂载点。
40	mount	fs::sys_mount	挂载文件系统。
41	pivot_root	fs::sys_pivot_root	切换根挂载布局。
43	statfs	fs::sys_statfs	读取文件系统统计信息。
44	fstatfs	fs::sys_fstatfs	通过文件描述符读取文件系统统计信息。
45	truncate	fs::sys_truncate	按路径截断文件。
46	ftruncate	fs::sys_ftruncate	按文件描述符截断文件。
47	fallocate	fs::sys_fallocate	预分配或调整文件存储范围。
48	faccessat	fs::sys_faccessat	按路径检查访问权限。
49	chdir	fs::sys_chdir	切换当前工作目录。
50	fchdir	fs::sys_fchdir	通过文件描述符切换当前工作目录。
51	chroot	fs::sys_chroot	切换当前根目录。
52	fchmod	fs::sys_fchmod	修改文件描述符目标权限。
53	fchmodat	fs::sys_fchmodat	按路径修改权限。
54	fchownat	fs::sys_fchownat	按路径修改属主。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
55	fchown	fs::sys_fchown	修改文件描述符目标属主。
56	openat	fs::sys_openat	相对目录文件描述符打开或创建路径。
57	close	fs::sys_close	关闭文件描述符。
59	pipe2	fs::sys_pipe2	创建管道读写端。
61	getdents64	fs::sys_getdents64	读取目录项序列。
62	lseek	fs::sys_lseek	读取或修改共享文件偏移。
63	read	fs::sys_read	从文件描述符读取数据。
64	write	fs::sys_write	向文件描述符写入数据。
65	readv	fs::sys_readv	按 iovec 分散读取数据。
66	writew	fs::sys_writew	按 iovec 聚集写入数据。
67	pread64	fs::sys_pread64	在指定偏移读取且不改变共享文件偏移。
68	pwrite64	fs::sys_pwrite64	在指定偏移写入且不改变共享文件偏移。
69	preadv	fs::sys_preadv	按 iovec 和指定偏移读取数据。
70	pwritev	fs::sys_pwritev	按 iovec 和指定偏移写入数据。
71	sendfile	fs::sys_sendfile	在两个文件描述符之间搬运数据。
72	pselect6	fs::sys_pselect6	等待 fdset 描述的就绪事件。
73	ppoll	fs::sys_ppoll	等待 poll 描述符集合并可临时切换信号掩码。
74	signalfd4	fs::sys_signalfd4	创建或更新信号文件描述符。
75	vmsplice	fs::sys_vmsplice	把用户 iovec 数据写入管道类目标。
76	splice	fs::sys_splice	在两个文件描述符之间移动数据并维护可选偏移。
77	tee	fs::sys_tee	复制管道类数据流。
78	readlinkat	fs::sys_readlinkat	读取符号链接目标。
79	fstatat	fs::sys_newfstatat	相对路径读取对象元数据。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
80	fstat	fs::sys_fstat	读取文件描述符对应对象元数据。
81	sync	fs::sys_sync	触发全局同步。
82	fsync	fs::sys_fsync	同步文件数据和元数据。
83	fdatasync	fs::sys_fdatasync	同步文件数据及必要元数据。
84	sync_file_range2	fs::sys_sync_file_range2	同步文件指定范围。
85	timerfd_create	fs::sys_timerfd_create	创建定时器文件描述符。
86	timerfd_settime	fs::sys_timerfd_settime	设置定时器文件描述符到期时间。
87	timerfd_gettime	fs::sys_timerfd_gettime	读取定时器文件描述符状态。
88	utimensat	fs::sys_utimensat	修改路径时间戳。
90	capget	process::sys_capget	读取当前任务能力集合。
91	capset	process::sys_capset	设置当前任务能力集合。
92	personality	process::sys_personality	读取或设置进程 personality 标志。
93	exit	process::sys_exit	终止当前任务并进入退出回收路径。
94	exit_group	process::sys_exit_group	终止当前线程组并唤醒等待者。
95	waitid	process::sys_waitid	按 waitid 语义等待子任务状态变化。
96	set_tid_address	process::sys_set_tid_address	在线程退出时清零并唤醒的用户地址。
98	futex	process::sys_futex	执行传统 futex 等待、唤醒和重排队操作。
99	set_robust_list	process::sys_set_robust_list	登记 robust futex 链表头。
100	get_robust_list	process::sys_get_robust_list	读取目标线程的 robust futex 链表头。
101	nanosleep	process::sys_nanosleep	按相对时间睡眠并在被中断时返回剩余时间。
102	getitimer	process::sys_getitimer	读取进程 interval timer 状态。
103	setitimer	process::sys_setitimer	设置进程 interval timer 状态。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
112	clock_settime	process::sys_clock_settime	设置本内核支持的墙钟时间。
113	clock_gettime	process::sys_clock_gettime	读取指定时钟的当前时间。
114	clock_getres	process::sys_clock_getres	读取指定时钟分辨率。
115	clock_nanosleep	process::sys_clock_nanosleep	按指定时钟执行相对或绝对睡眠。
116	syslog	syslog::sys_syslog	读取或控制内核日志缓冲区的兼容入口。
117	ptrace	process::sys_ptrace	支持追踪附加、继续、分离、中断等 ptrace 控制路径。
118	sched_setparam	process::sys_sched_setparam	设置调度参数。
119	sched_setscheduler	process::sys_sched_setscheduler	设置调度策略和参数。
120	sched_getscheduler	process::sys_sched_getscheduler	读取调度策略。
121	sched_getparam	process::sys_sched_getparam	读取调度参数。
122	sched_setaffinity	process::sys_sched_setaffinity	设置 CPU 亲和性掩码。
123	sched_getaffinity	process::sys_sched_getaffinity	读取 CPU 亲和性掩码。
124	sched_yield	process::sys_sched_yield	当前任务主动让出处理器。
125	sched_get_priority_max	process::sys_sched_get_priority_max	读取调度策略最大优先级。
126	sched_get_priority_min	process::sys_sched_get_priority_min	读取调度策略最小优先级。
127	sched_rr_get_interval	process::sys_sched_rr_get_interval	读取轮转调度时间片。
128	restart_syscall	signal::sys_restart_syscall	保留可重启系统调用兼容入口，当前返回 EINTR。
129	kill	process::sys_kill	向进程或进程组投递信号。
130	tkill	process::sys_tkill	向指定线程投递信号。
131	tgkill	process::sys_tgkill	向指定线程组中的线程投递信号。
132	sigaltstack	signal::sys_sigaltstack	设置或读取备用信号栈。
133	rt_sigsuspend	signal::sys_rt_sigsuspend	临时替换信号屏蔽字并等待信号。
134	rt_sigaction	signal::sys_rt_sigaction	安装或读取信号处理动作。
135	rt_sigprocmask	signal::sys_rt_sigprocmask	修改或读取线程信号屏蔽字。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
136	rt_sigpending	signal::sys_rt_sigpendi	读取当前挂起信号集合。
137	rt_sigtimedwait	signal::sys_rt_sigtimed	限时等待指定信号集合。
138	rt_sigqueueinfo	signal::sys_rt_sigqueue	携带 siginfo 投递信号。
139	rt_sigreturn	signal::sys_rt_sigretur	从用户信号帧恢复陷阱帧。
140	setpriority	process::sys_setpriorit	设置 nice 优先级。
141	getpriority	process::sys_getpriorit	读取 nice 优先级。
142	reboot	process::sys_reboot	处理关机、重启和重启命令的特权入口。
143	setregid	process::sys_setregid	设置真实和有效组标识。
144	setgid	process::sys_setgid	设置组标识。
145	setreuid	process::sys_setreuid	设置真实和有效用户标识。
146	setuid	process::sys_setuid	设置用户标识。
147	setresuid	process::sys_setresuid	设置真实、有效和保存用户标识。
148	getresuid	process::sys_getresuid	读取真实、有效和保存用户标识。
149	setresgid	process::sys_setresgid	设置真实、有效和保存组标识。
150	getresgid	process::sys_getresgid	读取真实、有效和保存组标识。
151	setfsuid	process::sys_setfsuid	设置文件系统用户标识。
152	setfsgid	process::sys_setfsgid	设置文件系统组标识。
153	times	process::sys_times	读取任务累计运行时间。
154	setpgid	process::sys_setpgid	设置进程组关系。
155	getpgid	process::sys_getpgid	读取进程组标识。
156	getsid	process::sys_getsid	读取会话标识。
157	setsid	process::sys_setsid	创建新会话并成为会话首进程。
158	getgroups	process::sys_getgroups	读取附属组列表。
159	setgroups	process::sys_setgroups	设置附属组列表。
160	uname	process::sys_uname	读取 UTS 系统信息。
161	sethostname	process::sys_sethostnam	设置 UTS 主机名。
162	setdomainname	process::sys_setdomainna	设置 UTS 域名。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
163	getrlimit	process::sys_getrlimit	读取资源限制。
164	setrlimit	process::sys_setrlimit	设置资源限制。
165	getrusage	process::sys_getrusage	读取任务或子任务资源用量。
166	umask	process::sys_umask	设置并返回旧文件创建权限屏蔽字。
167	prctl	process::sys_prctl	处理进程控制命令中已接入的兼容子集。
168	getcpu	process::sys_getcpu	读取当前 CPU 和 NUMA 节点兼容信息。
169	gettimeofday	process::sys_gettimeofday	读取兼容 timeval 形式的墙钟时间。
170	settimeofday	process::sys_settimeofday	设置兼容 timeval 形式的墙钟时间。
172	getpid	process::sys_getpid	读取进程标识。
173	getppid	process::sys_getppid	读取父进程标识。
174	getuid	process::sys_getuid	读取真实用户标识。
175	geteuid	process::sys_geteuid	读取有效用户标识。
176	getgid	process::sys_getgid	读取真实组标识。
177	getegid	process::sys_getegid	读取有效组标识。
178	gettid	process::sys_gettid	读取线程标识。
179	sysinfo	process::sys_sysinfo	读取系统内存和负载概要信息。
194	shmget	ipc::sys_shmget	创建或查找 System V 共享内存段。
195	shmctl	ipc::sys_shmctl	查询、修改或删除 System V 共享内存段。
196	shmat	ipc::sys_shmat	把共享内存段附加到当前地址空间。
197	shmdt	ipc::sys_shmdt	从当前地址空间分离共享内存段。
198	socket	fs::sys_socket	创建套接字文件描述符。
199	socketpair	fs::sys_socketpair	创建一对已连接本地套接字。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
200	bind	fs::sys_bind	绑定套接字本地地址。
201	listen	fs::sys_listen	把流式套接字切换到监听状态。
202	accept	fs::sys_accept	接受监听队列中的连接。
203	connect	fs::sys_connect	发起连接或设置默认远端地址。
204	getsockname	fs::sys_getsockname	读取套接字本地地址。
205	getpeername	fs::sys_getpeername	读取套接字对端地址。
206	sendto	fs::sys_sendto	向套接字发送数据并可指定目标地址。
207	recvfrom	fs::sys_recvfrom	从套接字接收数据并可返回来源地址。
208	setsockopt	fs::sys_setsockopt	设置套接字选项。
209	getsockopt	fs::sys_getsockopt	读取套接字选项。
210	shutdown	fs::sys_shutdown	关闭套接字读端、写端或双向通信。
211	sendmsg	fs::sys_sendmsg	按消息头发送分散缓冲区和控制信息。
212	recvmsg	fs::sys_recvmsg	按消息头接收数据和控制信息。
213	readahead	fs::sys_readahead	向文件对象提交预读提示。
214	brk	mm::sys_brk	调整传统用户堆顶。
215	munmap	mm::sys_munmap	解除用户虚拟内存映射。
216	mremap	mm::sys_mremap	调整或移动既有映射。
220	clone	process::sys_clone	按共享标志创建进程或线程。
221	execve	process::sys_execve	替换当前进程的用户态映像。
222	mmap	mm::sys_mmap	建立匿名或文件后备用户映射。
223	fadvise64	fs::sys_fadvise64	提交文件访问模式提示。
226	mprotect	mm::sys_mprotect	修改用户映射权限。
227	msync	mm::sys_msync	同步用户映射对应的后备对象。
228	mlock	mm::sys_mlock	锁定用户内存范围的兼容入口。
229	munlock	mm::sys_munlock	解除用户内存范围锁定的兼容入口。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
230	mlockall	mm::sys_mlockall	锁定当前地址空间映射的兼容入口。
231	munlockall	mm::sys_munlockall	解除地址空间映射锁定的兼容入口。
232	mincore	mm::sys_mincore	查询用户地址范围的驻留状态。
233	madvise	mm::sys_madvise	接收用户映射访问建议。
240	rt_tsigqueueinfo	process::sys_rt_tsigqueueinfo	向指定线程携带 siginfo 投递信号。
242	accept4	fs::sys_accept4	接受连接并设置新描述符标志。
243	recvmsg	fs::sys_recvmsg	批量接收多个消息。
260	wait4	process::sys_wait4	等待子任务状态变化并回收退出任务。
261	prlimit64	process::sys_prlimit64	读取或设置目标任务资源限制。
267	syncfs	fs::sys_syncfs	同步指定文件描述符所在文件系统。
269	sendmsg	fs::sys_sendmsg	批量发送多个消息。
272	kcmp	process::sys_kcmp	比较两个任务之间的文件、地址空间或共享对象关系。
274	sched_setattr	process::sys_sched_setattr	按 sched_attr 设置调度属性。
275	sched_getattr	process::sys_sched_getattr	按 sched_attr 读取调度属性。
276	renameat2	fs::sys_renameat2	带扩展标志的重命名路径。
278	getrandom	process::sys_getrandom	从内核随机源填充用户缓冲区。
279	memfd_create	fs::sys_memfd_create	创建匿名内存文件。
281	execveat	process::sys_execveat	相对目录文件描述符执行程序替换。
283	membarrier	process::sys_membarrier	处理内存屏障能力查询和全局屏障请求。
284	mlock2	mm::sys_mlock2	带 flags 的 mlock 兼容入口。
285	copy_file_range	fs::sys_copy_file_range	在两个文件描述符之间复制指定范围。
286	preadv2	fs::sys_preadv2	带 RWF 标志的向量读取入口。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
287	pwritev2	fs::sys_pwritev2	带 RWF 标志的向量写入入口。
291	statx	fs::sys_statx	读取扩展文件元数据。
293	rseq	process::sys_rseq	登记或注销 restartable sequences 控制块。
403	clock_gettime64	process::sys_clock_gettime64	time64 形式的 clock_gettime 入口。
404	clock_settime64	process::sys_clock_settime64	time64 形式的 clock_settime 入口。
406	clock_getres_time64	process::sys_clock_getres_time64	time64 形式的 clock_getres 入口。
407	clock_nanosleep_time64	process::sys_clock_nanosleep_time64	time64 形式的 clock_nanosleep 入口。
410	timerfd_gettime64	fs::sys_timerfd_gettime64	time64 形式的 timerfd_gettime 入口。
411	timerfd_settime64	fs::sys_timerfd_settime64	time64 形式的 timerfd_settime 入口。
412	utimensat_time64	fs::sys_utimensat_time64	time64 形式的 utimensat 入口。
413	pselect6_time64	fs::sys_pselect6_time64	time64 形式的 pselect6 入口。
414	ppoll_time64	fs::sys_ppoll_time64	time64 形式的 ppoll 入口。
417	recvmmsg_time64	fs::sys_recvmmsg_time64	time64 形式的 recvmmsg 入口。
421	rt_sigtimedwait_time64	process::sys_rt_sigtimedwait_time64	time64 形式的 rt_sigtimedwait 入口。
422	futex_time64	process::sys_futex	time64 形式的传统 futex 入口。
423	sched_rr_get_interval_time64	process::sys_sched_rr_get_interval_time64	time64 形式的轮转时间片查询入口。
424	pidfd_send_signal	signal::sys_pidfd_send_signal	通过 pid 文件描述符向目标任务投递信号。
434	pidfd_open	process::sys_pidfd_open	为目标任务创建 pid 文件描述符。
435	clone3	process::sys_clone3	按 clone3 参数块创建进程或线程。
436	close_range	fs::sys_close_range	批量关闭或设置一段文件描述符。

续表 附录-2 已实现系统调用表

编号	ABI 名称	内核入口	实现范围
437	openat2	fs::sys_openat2	按 open_how 参数块打开路径。
438	pidfd_getfd	fs::sys_pidfd_getfd	通过 pid 文件描述符复制目标任务文件描述符。
439	faccessat2	fs::sys_faccessat2	带 flags 的路径访问权限检查。
441	epoll_pwait2	fs::sys_epoll_pwait2	使用 timespec 超时的 epoll 等待入口。
449	futex_waitv	process::sys_futex_waitv	等待多个 futex 条件之一满足。
452	fchmodat2	fs::sys_fchmodat2	带 flags 的路径权限修改入口。
454	futex_wake	process::sys_futex_wake	futex2 唤醒入口。
455	futex_wait	process::sys_futex_wait	futex2 等待入口。
456	futex_requeue	process::sys_futex_requeue	futex2 重排队入口。
510	mygo_sched_info	process::sys_mygo_sched_info	读取内核私有调度诊断信息。

表 附录-3 整理 POSIX errno 名称。该表列名称和语义，不列数值，因为具体整数值可能受平台 ABI 影响。当前内核是否已经完整产生这些错误码，仍以 `libs/errno/src/lib.rs` 和具体系统调用实现为准。

表 附录-3 POSIX errno 表

错误码	POSIX 语义	中文说明
E2BIG	Argument list too long	参数列表过长。
EACCES	Permission denied	权限检查拒绝访问。
EADDRINUSE	Address in use	地址或端口已经被占用。
EADDRNOTAVAIL	Address not available	请求的本地地址不可用。
EAFNOSUPPORT	Address family not supported	地址族不被支持。
EAGAIN	Resource unavailable, try again	资源暂时不可用，调用方可以稍后重试。
EALREADY	Connection already in progress	连接操作已经在进行中。
EBADF	Bad file descriptor	文件描述符无效。
EBADMSG	Bad message	消息格式或内容无效。
EBUSY	Device or resource busy	设备或资源正忙。

续表 附录-3 POSIX errno 表

错误码	POSIX 语义	中文说明
ECANCELED	Operation canceled	操作被取消。
ECHILD	No child processes	没有可等待的子进程。
ECONNABORTED	Connection aborted	连接被中止。
ECONNREFUSED	Connection refused	连接请求被拒绝。
ECONNRESET	Connection reset	连接被重置。
EDEADLK	Resource deadlock would occur	操作会导致资源死锁。
EDESTADDRREQ	Destination address required	需要提供目标地址。
EDOM	Mathematics argument out of domain of function	数学函数参数超出定义域。
EDQUOT	Reserved	POSIX 保留错误名，传统语义通常表示磁盘配额耗尽。
EEXIST	File exists	目标文件、目录或命名对象已存在。
EFAULT	Bad address	用户地址或指针无效。
EFBIG	File too large	文件过大。
EHOSTUNREACH	Host is unreachable	目标主机不可达。
EIDRM	Identifier removed	IPC 标识符已经被移除。
EILSEQ	Illegal byte sequence	非法字节序列。
EINPROGRESS	Operation in progress	操作正在进行中。
EINTR	Interrupted function	函数调用被信号中断。
EINVAL	Invalid argument	参数无效。
EIO	I/O error	输入输出错误。
EISCONN	Socket is connected	套接字已经连接。
EISDIR	Is a directory	目标是目录。
ELOOP	Too many levels of symbolic links	符号链接层级过多。
EMFILE	File descriptor value too large	进程打开的文件描述符数量达到限制。
EMLINK	Too many links	链接数过多。

续表 附录-3 POSIX errno 表

错误码	POSIX 语义	中文说明
EMSGSIZE	Message too large	消息过大。
EMULTIHOP	Reserved	POSIX 保留错误名，传统语义通常表示多跳链路错误。
ENAMETOOLONG	Filename too long	文件名或路径名过长。
ENETDOWN	Network is down	网络不可用。
ENETRESET	Connection aborted by network	连接被网络侧中止。
ENETUNREACH	Network unreachable	网络不可达。
ENFILE	Too many files open in system	系统范围内打开文件数量达到限制。
ENOBUFS	No buffer space available	缓冲区空间不足。
ENODATA	No message is available on the stream head read queue	流首读队列中没有可用消息。
ENODEV	No such device	没有对应设备。
ENOENT	No such file or directory	没有对应文件或目录。
ENOEXEC	Executable file format error	可执行文件格式错误。
ENOLCK	No locks available	没有可用锁。
ENOLINK	Reserved	POSIX 保留错误名，传统语义通常表示链路断开。
ENOMEM	Not enough space	内存或地址空间不足。
ENOMSG	No message of the desired type	没有所需类型的消息。
ENOPROTOPT	Protocol not available	协议选项不可用。
ENOSPC	No space left on device	设备或文件系统空间不足。
ENOSR	No stream resources	流资源不足。
ENOSTR	Not a stream	对象不是流。
ENOSYS	Functionality not supported	功能不被支持。
ENOTCONN	The socket is not connected	套接字尚未连接。

续表 附录-3 POSIX errno 表

错误码	POSIX 语义	中文说明
ENOTDIR	Not a directory	路径组件不是目录。
ENOTEMPTY	Directory not empty	目录非空。
ENOTRECOVERABLE	State not recoverable	互斥量保护状态不可恢复。
ENOTSOCK	Not a socket	对象不是套接字。
ENOTSUP	Not supported	操作不被支持。
ENOTTY	Inappropriate I/O control operation	不适合的输入输出控制操作。
ENXIO	No such device or address	没有对应设备或地址。
EOPNOTSUPP	Operation not supported on socket	套接字不支持该操作。
EOVERFLOW	Value too large to be stored in data type	数值过大，无法存入目标数据类型。
EOWNERDEAD	Previous owner died	健壮互斥量的前一个拥有者已经终止。
EPERM	Operation not permitted	操作不被允许。
EPIPE	Broken pipe	管道或套接字对端已经关闭。
EPROTO	Protocol error	协议错误。
EPROTONOSUPPORT	Protocol not supported	协议不被支持。
EPROTOTYPE	Protocol wrong type for socket	套接字协议类型不匹配。
ERANGE	Result too large	结果超出可表示范围。
EROFS	Read-only file system	只读文件系统。
ESPIPE	Invalid seek	无效定位操作。
ESRCH	No such process	没有对应进程。
ESTALE	Reserved	POSIX 保留错误名，传统语义通常表示陈旧文件句柄。
ETIME	Stream ioctl() timeout	流控制操作超时。
ETIMEDOUT	Connection timed out	连接超时。
ETXTBSY	Text file busy	文本文件正忙。

续表 附录-3 POSIX errno 表

错误码	POSIX 语义	中文说明
EWOULDBLOCK	Operation would block	操作会阻塞。该错误码可以与 EAGAIN 取相同值。
EXDEV	Cross-device link	跨设备链接或重命名。

C. 核心概念与代码句柄对照

正文优先使用中文概念。需要回到源码时，表 附录-4 可以作为概念和代码句柄之间的索引。表中列出的代码句柄均应在正文中使用反引号表示。

表 附录-4 核心概念与代码句柄对照

中文概念	代码句柄	主要位置
启动上下文	StartContext	general/src/start.rs。启动阶段整理固件、内存、命令行和架构交接信息。
系统调用上下文	SyscallContext	general/src/syscall.rs。系统调用分发时保存任务、参数、返回值和陷阱帧访问入口。
陷阱帧	TrapFrame	arch/src/riscv64/trap_frame.rs 与 arch/src/loongarch64/specific.rs。保存用户态寄存器和异常现场。
设备能力	DeviceFunction	general/src/dev/function.rs。设备向内核发布的统一能力边界。
字符设备对象	CharDevice	general/src/dev/char.rs。字符流设备和终端后端的类型化对象。
块设备对象	BlockDevice	general/src/dev/block.rs。块请求提交、同步等待和设备几何信息的类型化对象。
网络设备对象	NetDevice	libs/net/src/device.rs。网络接口身份、链路状态、MTU 和收发统计的承载对象。
实时时钟设备	RtcDevice	general/src/dev/rtc.rs。时间读写、告警和周期中断相关能力。
文件操作接口	FileOps	libs/vfs/src/file.rs。读写、控制、轮询和关闭等文件行为入口。

续表 附录-4 核心概念与代码句柄对照

中文概念	代码句柄	主要位置
文件描述符表	FdTable	libs/vfs/src/fdtable.rs。文件描述符到文件对象的映射和执行时关闭标志管理。
用户虚拟地址空间	VmSpace	general/src/mm/vm_space.rs。VMA、缺页、映射、权限变更和执行装载相关地址空间操作。
任务对象	Task	libs/sched/src/task.rs。任务身份、调度状态、信号状态、凭据和资源视图。
网络栈	NetStack	libs/net/src/stack.rs。接口注册表、路由、协议栈推进和网络套接字管理。
网络套接字状态	SocketState	libs/net/src/socket.rs。网络套接字连接、监听、收发和关闭状态。
Unix 套接字对象	Socket	libs/socket/src/state.rs。本机 Unix 套接字的连接和内存队列状态。
统一错误码	Errno	libs/errno/src/lib.rs。内核内部错误和用户态负 errno 返回值之间的统一枚举。

名词索引

A

安全点 垃圾回收或运行时检查可以安全观察线程状态的位置。

按需分页 虚拟地址在首次访问时才建立物理映射的内存管理策略。

B

保护页 故意保留为不可访问的页面, 用于捕获越界访问或栈增长边界。

备用信号栈 信号处理函数可以使用的独立用户栈, 常用于处理普通用户栈不可用的情况。

比较并交换 一种原子读改写操作, 用于无锁数据结构和并发状态转换。

扁平设备树二进制 固件传递给内核的设备树二进制表示, 用于描述平台硬件。

标记-清除 垃圾回收中的基础算法, 先标记可达对象, 再清扫不可达对象。

标准错误 用户程序默认的错误输出流。

标准输出 用户程序默认的输出流。

标准输入 用户程序默认的输出流。

C

程序断点扩展 用户地址空间中堆顶扩展相关机制, 通常用于传统堆区域增长。

程序解释器 装载可执行文件时需要同时装入的用户态解释器, 常见于动态链接程序。

程序头 可执行文件中描述可装载段、解释器和权限等信息的结构。

D

打开文件描述 多个文件描述符可以共同引用的打开文件状态。

大小类

小对象分配器中按照对象尺寸划分的分配类别。

待处理信号 已经投递到任务或线程组但尚未被处理的信号。

单向依赖 上层只消费下层导出的稳定接口, 下层不反向调用上层策略的依赖关系。

等待队列 阻塞任务挂接的位置, 事件发生后由唤醒路径重新调度。

地址空间 进程或内核可见的一组虚拟地址范围, 以及这些范围到物理后备的映射关系。

地址空间标识符 标记地址空间身份的处理器标签, 用于减少地址空间切换时的缓存刷新成本。

地址空间布局随机化 通过随机化可执行文件、栈、堆和映射区域的位置, 降低地址预测攻击的效果。

地址转换后备缓冲 处理器缓存虚拟地址到物理地址翻译结果的结构。

调度类 调度器中负责某类策略和任务集合的策略单元。

调度实体 参与调度排序和记账的对象。

调度系统 负责任务选择、阻塞唤醒、抢占和运行队列维护的子系统。

调度域 描述处理器拓扑和任务迁移边界的结构。

定时器钩子 架构或平台向调度和时间子系统提供的定时事件入口。

动态链接器 装载动态链接程序时负责解析共享对象和重定位的用户态组件。

读写锁 允许多个读者并发进入、写者独占进入的同步原语。

短临界区	持锁时间较短且不应执行阻塞操作的关键区域。	根集合	垃圾回收从中开始可达性分析的一组根引用。
对称多处理	多个处理器以相同角色共享同一个操作系统实例的多处理结构。	根文件系统	启动完成后作为路径解析根的文件系统。
F		根系统描述指针	固件提供的系统描述入口指针，用于定位相关平台表。
非一致内存访问	不同处理器访问不同内存节点时延迟和带宽不完全一致的体系结构。	工作队列	把可延迟处理的工作项放入队列并由内核工作上下文执行的机制。
分层结构	把内核职责拆成若干层级，使每层只依赖下层稳定边界。	固件	启动前向内核提供平台事实的执行环境和数据来源。
分代回收	把对象按生命周期划分代际，以降低垃圾回收扫描成本的策略。	固件表	固件提供的结构化平台信息表。
分段感知伙伴分配器	保留固件内存段信息的伙伴页分配器。	挂载标志	描述挂载点属性和访问限制的标志集合。
分配器	负责向内核或用户地址空间提供内存对象、页或地址范围的组件。	挂载点	文件系统树中另一个文件系统实例接入的位置。
分配注册表	记录分配结果来源和尺寸的登记结构，用于释放检查和统计审计。	挂载命名空间	为进程提供独立文件系统挂载视图的命名空间。
H		后备区间	虚拟地址范围背后已经分配或可按需分配的内存区间。
浮点单元	处理浮点寄存器和浮点运算状态的处理器部件。	后台进程组	不在控制终端前台的进程组，终端读写可能触发作业控制信号。
符号链接	目录项中保存另一路径文本的文件类型，路径解析时需要继续展开。	缓存行	处理器缓存与内存之间搬运数据的基本粒度。
负向缓存	记录某个名称不存在的目录项缓存，用于避免重复查找。	缓存一致性	多个处理器或设备观察同一内存位置时保持一致结果的约束。
负载均衡	调度器在处理器或运行队列之间迁移任务以平衡执行压力的机制。	回环接口	本机网络通信使用的虚拟网络接口。
辅助向量	内核在执行新程序时压入用户栈的键值数组，用于传递页面大小、入口信息等启动参数。	伙伴系统	按二次幂大小拆分与合并空闲块的物理页分配算法。
G		J	
高级配置与电源接口	固件向操作系统提供平台配置、电源管理和设备描述的规范。	即时探测	设备发现后立即尝试匹配驱动并发布能力的过程。

基址寄存器	外设总线中描述设备资源窗口起始地址和类型的寄存器。	可移植操作系统接口	用户态程序与类 Unix 操作系统之间的一组可移植接口规范。
记住对象	分代垃圾回收中记录跨代引用的对象。	可执行与可链接格式	用户程序和共享对象使用的二进制文件格式。
交接对象	某一阶段稳定化后传给下一阶段的一次性上下文对象。	空闲调度类	没有普通任务可运行时选择空闲任务的调度类。
阶段性交接	控制权和上下文从一个阶段无返回地移交给下一个阶段的过程。	空闲链表	保存当前可复用对象、页或块的链表。
截止时间	调度实体在截止时间类或公平调度模型中用于排序的时间点。	控制端点	面向设备或子系统控制命令的用户态或内核态入口。
截止时间调度类	按照截止时间和预算约束选择任务的调度类。	控制请求	类型化控制路径中表达操作意图和参数的请求对象。
进程	拥有地址空间、文件描述符表和资源视图的执行容器。	控制台	内核早期和运行期用于输出日志或交互字符的设备入口。
进程标识符	内核和用户态用于指代进程的整数名称。	控制与状态寄存器	处理器中保存控制状态和异常信息的寄存器。
进程间通信	不同进程之间交换数据、事件或共享内存的机制。	快速路径	常见场景下尽量减少分支、锁和额外检查的执行路径。
进程文件系统	把进程和内核状态投影为文件树的特殊文件系统。	L	
进程组	用于作业控制和信号投递的一组进程。	垃圾回收	自动回收不再可达对象的内存管理技术。
进程组标识符	用户态和内核用于指代进程组的整数名称。	懒加载	对象或页面在首次访问时才真正装入或建立映射的策略。
精确根槽	受管堆中记录精确根引用位置的槽位。	老年代	分代回收中保存生命周期较长对象的区域。
就绪状态	对象当前可以执行读、写或状态推进的用户可见状态。	类型化对象	保留具体设备、文件或协议语义的结构化对象。
K		类型化控制	用结构化请求和响应替代无类型命令码的控制方式。
可达性分析	从根集合出发判断对象是否仍可被访问的分析过程。	类型化请求	与具体设备或子系统绑定的结构化控制请求。
可扩展固件接口	固件向内核提供启动服务和运行时服务的接口规范。	类型化载荷	投影或控制路径中携带具体类型信息的载荷。
可睡眠锁	允许持锁路径进入睡眠状态的同步原语。	临界区	需要同步保护、不能被并发破坏的一段代码或状态访问范围。

临时文件系统	以内存对象为主要后备的临时文件系统。	内存映射输入输出	把设备寄存器映射到内存地址空间后进行访问的方式。
流式套接字	面向字节流传输的套接字类型。	内核地址空间	内核可见和管理的虚拟地址空间。
路径解析	根据路径字符串逐级查找目录项、跨越挂载点并处理符号链接的过程。	内核堆	运行期用于分配内核对象和大对象的堆空间。
轮询	反复检查设备、套接字或事件状态以推动系统前进的机制。	内核态	处理器以特权级执行内核代码的状态。
M		内核栈	内核执行任务上下文时使用的栈。
每处理器缓存	按处理器分片的小对象缓存, 用于减少全局锁竞争。	P	
命令行	固件或启动器传给内核的参数字符串。	凭据	任务身份和权限的快照, 包含用户、组和能力相关状态。
命名空间	为进程提供隔离视图的内核对象集合。	凭据快照	以不可变方式保存的一组身份和权限字段。
默认动作	信号未被用户处理时采用的内核预设处理方式。	平台设备	由固件或平台代码描述、并非通过可枚举总线发现的设备。
目录项	文件系统路径解析中把名称关联到索引节点的对象。	平台无关	不依赖具体指令集、固件入口或平台寄存器的通用逻辑。
目录项缓存	缓存名称查找结果以加速路径解析的结构。	平台资源	固件或总线为设备描述的地址窗口、中断和其他依赖。
N		普通信号栈	用户程序常规执行路径使用的信号处理栈。
内部碎片	分配块内部未被实际使用的空间。	Q	
内存保护	改变虚拟内存区域访问权限的机制。	启动参数	固件或启动器传给内核的初始参数集合。
内存分配器	管理内核堆、页帧、小对象或受管对象的分配组件。	启动阶段	进入主入口之前完成基础设施建立和上下文交接的阶段。
内存管理子系统	负责物理页、虚拟地址空间、堆和缺页处理的子系统。	启动控制流	从固件入口到内核主入口之间的控制权转移过程。
内存区域	物理内存或虚拟地址空间中一段可描述范围。	启动上下文	架构相关启动阶段整理后传给平台无关层的稳定上下文。
内存碎片	可用内存因分布或分配粒度原因难以满足后续请求的现象。	启动协议	固件、启动器和内核之间约定启动参数与入口方式的协议。
内存锁定	阻止指定用户内存范围被换出或回收的机制。	启动映射	启动早期为访问内核镜像、栈、固件表和设备窗口建立的地址映射。

前台进程组	拥有控制终端前台访问权的进程组。	设备模型	管理设备发现、驱动绑定、能力发布和用户态投影的整体结构。
强引用	保持对象存活的引用关系。	设备树	描述平台硬件层次、资源和属性的树形数据结构。
全局回收	跨缓存、页和文件后备协调释放内存的回收过程。	设备文件	把设备能力投影到文件系统名字空间后的文件节点。
权限检查	根据凭据、模式位、能力和上下文判断操作是否允许的过程。	设备文件投影器	把设备能力转换为用户可见设备节点的组件。
缺页	访问尚未建立有效映射的虚拟地址时产生的异常。	设备移除	设备退出系统时阻止新访问、排空旧请求并释放资源的过程。
缺页处理	根据异常地址和访问类型建立映射或向上返回错误的过程。	实时调度类	按照实时优先级选择任务的调度类。
缺页处理器	执行缺页修复或错误返回的处理逻辑。	实时时钟	提供日历时间、告警或周期中断能力的硬件或虚拟设备。
R			
热插拔	设备在运行期加入或移除系统的能力。	受管堆	提供受管对象分配和垃圾回收能力的堆。
任务	调度器管理的执行实体。	受管对象	由受管堆跟踪并参与自动回收的对象。
任务状态	任务在运行、可运行、睡眠、停止或退出等阶段中的状态。	数据包	网络设备和协议栈传递的包级数据单位。
日志接收端	接收并输出内核日志的抽象目标。	数据报套接字	保持消息边界的数据报通信端点。
软中断让步	在可延后处理或调度点上主动让出执行机会的行为。	输入输出	系统在设备、文件、套接字或用户缓冲区之间传递数据的操作。
S			
设备抽象	把异构硬件能力整理成内核可消费对象的设计边界。	双端队列	允许在两端插入和移除元素的队列结构。
设备发现	从固件、总线或平台信息中发现设备身份和资源的过程。	睡眠状态	任务因等待事件而暂时不可运行的状态。
设备功能单元	驱动向内核开放出来的设备能力对象。	私有待处理信号	只投递给某个具体任务的待处理信号。
设备功能单元注册表	保存所有已发布设备能力并广播生命周期事件的注册表。	私有文件映射	文件内容以私有方式映射到地址空间, 写入时通常触发复制。
设备号	用户态兼容视图中用于标识设备文件的编号。		

松弛语义	不建立额外同步顺序的原子内存序。	文件页	由文件内容填充或回写的页。
随机访问内存	可按地址随机读写的主存。	物理地址	处理器和设备访问物理内存或设备窗口使用的机器地址。
索引节点	表示文件元数据的文件系统对象，与文件名分离。	物理页	物理内存管理中的页粒度单位。
T			
特殊设备文件系统	自动投影设备节点的特殊文件系统。	物理页帧	页大小对齐的物理内存帧。
提升	分代回收中把长期存活对象移入更老代的过程。	无锁读取	不通过传统互斥锁完成读取的并发访问方式。
条件变量	线程等待某个条件变化并由其他线程唤醒的同步原语。	X	
同步等待	发起请求后等待完成结果的执行模式。	系统调用	用户态进入内核请求服务的接口。
通用异步收发器	串口控制器常见硬件类型。	系统调用表	根据系统调用号分发到具体处理函数的表。
图形设备	面向显示输出的设备类别。	系统调用入口	处理器从用户态进入内核执行系统调用的入口路径。
W			
外部碎片	空闲空间被分割成不连续小块后难以满足大块请求的现象。	系统调用上下文	系统调用路径传递参数、返回值和任务状态的上下文。
完全公平调度器	以虚拟运行时间为核心的公平调度算法。	系统文件系统	向用户态投影内核对象属性的特殊文件系统。
位置无关可执行文件	装载位置可以变化的可执行文件形式。	线程	与同组线程共享部分资源的可调度执行实体。
文件后备	虚拟内存区域背后由文件内容提供数据的后备关系。	线程标识符	内核和用户态用于指代线程的整数名称。
文件描述符	用户态通过整数句柄引用打开文件对象的方式。	线程局部存储	每个线程独立保存的数据区域。
文件描述符表	进程或线程组保存文件描述符到文件对象映射的表。	线程组	共享进程级资源的一组线程。
文件偏移	文件对象中记录顺序读写当前位置的值。	写时复制	多个对象共享同一物理后备，首次写入时再复制的机制。
文件系统用户标识符	文件系统权限检查时使用的用户身份。	信号	内核向任务或线程组传递异步事件的机制。
文件系统用户组标识符	文件系统权限检查时使用的组身份。	信号动作	指定信号处理函数、掩码和标志的配置。
		信号返回	用户态信号处理结束后恢复原上下文的过程。
		信号屏蔽字	控制哪些信号暂时不被递送的掩码。

信号帧	内核在用户栈上构造的信号处理返回上下文。	用户复制	内核与用户地址空间之间复制数据的过程。
虚拟地址	程序或内核通过地址空间看到的地址。	用户上下文	用户态寄存器、栈和信号恢复所需的上下文信息。
虚拟地址空间	由多个虚拟地址范围组成的地址视图。	用户态	处理器以非特权级执行用户程序的状态。
虚拟动态共享对象	内核映射到用户态的只读辅助代码或数据对象。	用户态投影	把内核对象映射为用户可见文件、属性或节点的过程。
虚拟队列	虚拟设备规范中用于前后端传递描述符的队列。	用户指针	用户态传入内核、需要安全检查的地址。
虚拟截止时间	公平调度中用于排序任务的虚拟时间目标。	原始套接字	允许用户态直接构造或接收低层协议负载的套接字。
虚拟内存	通过页表把虚拟地址映射到物理后备的内存机制。	运行队列	调度器保存可运行任务并执行选择的队列结构。
虚拟内存区域	用户地址空间中一段权限、后备和语义一致的区域。	运行期	内核基础能力建立之后的长期执行阶段。
虚拟文件系统	把不同文件系统、设备文件和套接字统一为文件对象接口的抽象层。	运行时前固件服务	固件退出启动服务前提供的一组服务。
Y		运行状态	任务当前正在处理器上执行的状态。
页表	保存虚拟地址到物理地址映射关系的数据结构。	运行资源限制	约束进程可使用文件数、内存或其他资源的限制。
页表项	页表中描述某一级映射、权限和物理地址的条目。	运行资源用量	进程或线程累计消耗的时间和资源统计。
页缓存	缓存文件页内容以加速文件访问和内存映射的结构。	Z	
硬件抽象层	把架构相关能力整理为上层可消费接口的抽象层。	脏卡	垃圾回收中记录某个内存卡片可能包含跨代引用的标记。
应用程序接口	程序通过函数、系统调用或协议消费服务的接口。	早期入口	处理器刚进入内核镜像时执行的最早代码路径。
应用二进制接口	规定二进制程序与内核、运行库和处理器约定之间交互方式的接口边界。	粘滞位	文件系统权限中的特殊模式位。
用户地址	用户地址空间中的虚拟地址。	栈指针	指向当前栈顶或栈操作位置的寄存器值。
用户地址空间	用户进程可见的虚拟地址空间。	直接内存访问	设备在不经处理器逐字节搬运的情况下访问内存的机制。

直接映射	将物理内存按固定偏移映射到内核虚拟地址空间的方式。
直接映射窗口	部分架构中用于建立固定地址映射的硬件窗口。
执行时关闭	程序执行替换时自动关闭指定文件描述符的标志。
执行替换	用新程序映像替换当前进程地址空间和用户态入口的过程。
终端行规程	在底层字符设备与用户读写之间处理回显、规范模式和控制字符的层。
中断控制器	接收、仲裁并向处理器投递中断的硬件或虚拟组件。
中断请求	设备或控制器向处理器请求中断处理的信号。
终端设备	面向字符交互、控制字符和作业控制的设备抽象。
中央处理器	执行指令和处理异常的处理器核心。
注册表	保存对象集合并提供查找、注册和注销能力的数据结构。
自旋锁	在等待期间忙等的同步原语。
资源所有权	内核对象或驱动对设备、内存或中断资源的占有关系。
资源限制	限制进程可使用资源数量或规模的机制。
阻塞模式	操作无法立即完成时允许任务睡眠等待的模式。
最大传输单元	网络接口一次可传输的最大数据单元。
最早合格虚拟截止时间优先	结合合格性和虚拟截止时间的公平调度算法。
最早截止时间优先	每次选择截止时间最早任务的调度算法。
作业控制	终端、进程组和信号协同管理前后台作业的机制。

缩写词表

英文缩写	英文全称	中文名称
ABI	Application Binary Interface	应用二进制接口
ACPI	Advanced Configuration and Power Interface	高级配置与电源接口
API	Application Programming Interface	应用程序接口
ASID	Address Space Identifier	地址空间标识符
ASLR	Address Space Layout Randomization	地址空间布局随机化
BAR	Base Address Register	基址寄存器
BSS	Block Started by Symbol	未初始化静态区
CAS	Compare-and-Swap	比较并交换
CFS	Completely Fair Scheduler	完全公平调度器
COW	Copy-on-Write	写时复制
CPU	Central Processing Unit	中央处理器
CSR	Control and Status Register	控制与状态寄存器
DL	Deadline	截止时间调度类
DMA	Direct Memory Access	直接内存访问
DMW	Direct Mapping Window	直接映射窗口
DTB	Device Tree Blob	扁平设备树二进制
EDF	Earliest Deadline First	最早截止时间优先
EEVDF	Earliest Eligible Virtual Deadline First	最早合格虚拟截止时间优先
EFI	Extensible Firmware Interface	可扩展固件接口
ELF	Executable and Linkable Format	可执行与可链接格式
EOF	End of File	文件结束标记
FDT	Flattened Device Tree	扁平设备树
FIFO	First In, First Out	先进先出
FPU	Floating-Point Unit	浮点单元
GC	Garbage Collection	垃圾回收
GID	Group Identifier	组标识符
HAL	Hardware Abstraction Layer	硬件抽象层
ICMP	Internet Control Message Protocol	互联网控制消息协议
IPC	Inter-Process Communication	进程间通信
I/O	Input/Output	输入输出
IP	Internet Protocol	互联网协议

英文缩写	英文全称	中文名称
IPv4	Internet Protocol Version 4	互联网协议第四版
IPv6	Internet Protocol Version 6	互联网协议第六版
IRQ	Interrupt Request	中断请求
LSM	Linux Security Module	Linux 安全模块
MAC	Media Access Control	介质访问控制
MADT	Multiple APIC Description Table	多 APIC 描述表
MMIO	Memory-Mapped Input/Output	内存映射输入输出
MSI	Message Signaled Interrupts	消息信号中断
MTU	Maximum Transmission Unit	最大传输单元
NAPI	New API	网络轮询接口
NUMA	Non-Uniform Memory Access	非一致内存访问
NUL	Null Character	空字符
PC	Program Counter	程序计数器
PCI	Peripheral Component Interconnect	外设组件互连
PCIe	PCI Express	高速外设组件互连
PGID	Process Group Identifier	进程组标识符
PID	Process Identifier	进程标识符
PIE	Position-Independent Executable	位置无关可执行文件
POSIX	Portable Operating System Interface	可移植操作系统接口
PTY	Pseudo Terminal	伪终端
RAM	Random Access Memory	随机访问内存
RSDP	Root System Description Pointer	根系统描述指针
RT	Real-Time	实时调度类
RTC	Real-Time Clock	实时时钟
SMP	Symmetric Multiprocessing	对称多处理
SP	Stack Pointer	栈指针
SPCR	Serial Port Console Redirection Table	串口控制台重定向表
TCP	Transmission Control Protocol	传输控制协议
TGID	Thread Group Identifier	线程组标识符
TID	Thread Identifier	线程标识符
TLB	Translation Lookaside Buffer	地址转换后备缓冲
TLS	Thread Local Storage	线程局部存储
TOCTOU	Time-of-Check to Time-of-Use	检查时到使用时竞态

英文缩写	英文全称	中文名称
TTY	Teletype	终端设备
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器
UDP	User Datagram Protocol	用户数据报协议
UAPI	User API	用户态接口
UID	User Identifier	用户标识符
VFS	Virtual File System	虚拟文件系统
VM	Virtual Memory	虚拟内存
VMA	Virtual Memory Area	虚拟内存区域
W^X	Write XOR Execute	写异或执行

参考项目与致谢

参考项目

我们的内存分配器结构参考了 FreeBSD 的 UMA 设计，包括内存分配器的伙伴系统、Slab 分配器、内核堆等子系统；进程调度的 EEVDF 算法、用户态的信号机制参考了 Linux 6.10 以上版本内核。同时，代码中的 `extfs` 和 `fatfs` 由 LLM 实现，使用 GPT-5.5 模型生成；代码中的注释部分由 Deepseek v4 模型生成；架构实现的 RISC-V64 的部分代码由 Claude Opus 4.6 模型生成；其余部分无过多 AI 生成内容。

致谢

本项目的顺利完成，离不开李欣老师的悉心指导。从框架设计到最终的修改发布，李老师始终以严谨的治学态度给予我们关键点拨。每当研究陷入瓶颈，他总能一针见血地指出问题所在，并鼓励我们大胆尝试不同思路。在此，谨向李欣老师致以最诚挚的敬意与感谢。

同时，感谢太原理工大学为我们提供了良好的学术资源与环境。